

REST Service API

Informatica MDM - Product 360

Version: 10.5 HotFix 5

Table of Contents

1 Available APIs 17
1.1 List API 17
1.2 Management API 17
1.3 Enum API 17
1.4 Meta API 18
1.5 Media API 18
1.6 Security API 18
1.7 Object API 18
2 Getting Started 18
2.1 Authentication 19
3 General Information 19
3.1 Data Representation 19
3.2 Entity Item Reference 19
3.3 Field Qualification 19
3.4 Search Query Language 19
3.5 Rest Java Client 20
3.6 Rest API Versions 20
3.6.1 List API changes in V2.0 20
4 REST General Information 21
4.1 Monitoring 21
4.1.1 Monitor all REST requests in the log file 21
4.2 REST Service Authentication 23
4.2.1 Basic Authentication 23
4.2.1.1 Simple example 23
4.2.1.2 Username / Password Basic Authentication 23
4.2.1.3 Token Basic Authentication 24
4.2.2 OAuth 24
4.3 REST Datatypes 25
4.3.1 Datatypes 25

4.3.1.1 STRING 25
4.3.1.2 INTEGER 25
4.3.1.3 DECIMAL 26
4.3.1.4 DATE 26
4.3.1.5 TIME 26
4.3.1.6 DATETIME 27
4.3.1.7 BOOLEAN 27
4.3.1.8 ENTITY_ITEM 28
4.3.1.9 MIME_VALUE 29
4.3.1.10 ANY 30
4.4 REST Field Qualification 31
4.4.1 Syntax 31
4.4.2 Logical Key Value Definition 32
4.4.2.1 With Enumeration 32
4.4.2.2 Without Enumeration 33
4.4.2.3 Using default values 34
4.4.2.4 Using wildcard matching 35
4.5 REST Transition Fields 36
4.5.1 Syntax 36
4.5.2 Examples 37
4.6 REST Search Query Language 38
4.6.1 Field Qualification 38
4.6.2 Characteristic 38
4.6.3 Operators 38
4.6.4 Value Literal 40
4.6.4.1 Dynamic Value 40
4.7 REST Java Client 42
4.7.1 Example 42
4.8 REST Characteristic 44
4.8.1 Characteristic Operand 44
4.8.1.1 Syntax 44
4.8.1.2 Usage examples 45
4.8.2 DESCENDANT Operator 49
4.8.2.1 Syntax 50

4.8.2.2 Limitations 50
4.8.2.3 Usage Examples 51
4.8.3 Limitation 54
5 REST List API 55
5.1 Overview 55
5.2 List-Based Mass Data Access 55
5.3 Root Entity vs. Sub Entity Access 55
5.4 Report vs. Search 55
5.5 Report Services 56
5.6 Search Services 56
5.7 Examples 56
5.8 REST List API Info 57
5.8.1 List all entities with reports 57
5.8.1.1 Result 57
5.8.2 List all available reports for an entity 58
5.8.2.1 Result 58
5.8.3 List report specific parameters 59
5.8.3.1 Result 60
5.8.4 Examples 60
5.8.4.1 List all entities with reports 60
5.8.4.2 Retrieving all reports for the entity Article 61
5.8.4.3 Retrieving all report specific parameters for the report byCatalog 62
5.9 REST List API Read 63
5.9.1 Caching 63
5.9.1.1 Use Caching with Paging 63
5.9.1.2 Disable Caching 63
5.9.2 REST List API Read for Root Entities 65
5.9.2.1 Executing a specific report 65
5.9.2.2 Execute a specific report for root entities for MIME values 78
5.9.3 REST List API Read for Sub Entities 81
5.9.3.1 Execute a specific report for Read 81
5.9.3.2 Execute a specific report of sub entities for MIME values 95
5.9.4 REST List API Read MIME files of Root and Sub Entities 99
5.9.4.1 Root Entity 99

5.9.4.2 Sub Entity 101
5.10 REST List API Write 106
5.10.1 REST List API Write for Root Entities 107
5.10.1.1 Write values into qualified fields 107
5.10.1.2 Examples 111
5.10.2 REST List API Write for Sub Entities 120
5.10.2.1 Write values into fields of a certain sub entity 120
5.10.2.2 Examples 122
5.11 REST List API Delete 132
5.11.1 Delete all objects returned by a report 132
5.11.1.1 Result 132
5.11.2 Delete sub entity records of objects returned by a report 133
5.11.2.1 Result 134
5.11.3 Examples 135
5.11.3.1 Delete all item in the supplier catalog TOOLS 135
5.11.3.2 Delete all sales prices with currency USD in the supplier catalog TOOLS 136
5.11.3.3 Delete a Characteristic QUALITY in a revision DAILY_VERSION 137
5.11.3.4 Delete all Characteristic Values of an Item in a revision DAILY_VERSION 137
5.12 REST List API Errors 138
5.12.1 General List API Errors 138
5.12.2 Report Specific Errors 138
5.12.3 Search Specific Errors 139
5.13 REST List API Tutorial 140
5.13.1 Loading Data For Examples 140
5.13.2 Tools 140
5.13.3 REST API Tutorial - Data Model 141
5.13.3.1 Data Model 141
5.13.4 REST API Tutorial - List Read for Root Entities 143
5.13.4.1 1. Which objects should be included (rows) -> Report 143
5.13.4.2 2. Which fields are required (columns) 143
5.13.4.3 3. Options 143
5.13.4.4 Examples 143
5.13.4.5 Transition field access 144
5.13.4.6 Paged access 145
5.13.5 REST API Tutorial - List Write for Root Entities 146

5.13.5.1 Update....146
5.13.5.2 Create....148
5.13.6 REST API Tutorial - List Read for Sub Entities....149
5.13.6.1 Motivation....149
5.13.6.2 Query....149
5.13.6.3 Qualification filter....150
5.13.7 REST API Tutorial - List Write for Sub Entities....151
5.13.7.1 Example....151
5.14 REST List API Troubleshooting and How To's....153
5.14.1 User and User Groups....153
5.14.1.1 Assigning User Groups to a User....153
5.14.2 Characteristics....153
5.14.3 Structure Groups....154
5.14.3.1 Query the list of items assigned to multiple structure groups of a structure....154
5.14.3.2 Updating Product's Structure Group with Rest API....154
5.14.3.3 Access to structure group fails....155
5.14.3.4 StructureGroup levels....155
5.14.3.5 Filtering on structure group attribute....155
5.14.3.6 Not possible to search by Structure Group Attribute Value....156
5.14.4 Media Assets....157
5.14.4.1 Get media asset document with attributes fails....157
5.14.5 How to read and write values for characteristics....158
5.14.5.1 Definitions....158
5.14.5.2 Qualifications....159
5.14.5.3 Datatypes....160
5.14.5.4 Fields....161
5.14.5.5 Read Access....161
5.14.5.6 Write Access....180
6 REST Object API....189
6.1 Overview Presentation....189
6.2 Examples....189
6.3 OpenAPI Support....189
6.3.1 Permissions....190
6.3.1.1 Entity Permissions....190

6.3.1.2 Field Permissions....191
6.4 REST Object API Read V1....192
6.4.1 Read Requests....192
6.4.1.1 Single Item....192
6.4.1.2 Multiple Items (since 10.1.0.02)....193
6.4.1.3 Query Parameters....195
6.4.1.4 Permissions....197
6.4.1.5 Result....198
6.4.1.6 Examples....229
6.5 REST Object API Read V2....230
6.5.1 Read Requests....230
6.5.1.1 Single Item....230
6.5.1.2 Multiple Items....231
6.5.1.3 Query Parameters....233
6.5.1.4 Permissions....237
6.5.1.5 Result....237
6.5.1.6 Examples....270
6.6 REST Object API Write....272
6.6.1 Create/Update Object....272
6.6.1.1 Permissions....272
6.6.1.2 Body....272
6.6.1.3 Response....278
6.6.1.4 Request Syntax....282
6.6.2 Examples....289
6.6.2.1 Create an Item in the Master catalog....289
6.6.3 Payload Examples....291
6.6.3.1 Create or Update....291
6.6.3.2 General Examples....291
6.6.3.3 Full Example....301
7 REST Management API....302
7.1 REST Assortment API....303
7.1.1 Get assortment content....303
7.2 REST Data Quality API....304
7.2.1 Rule Execution (since 8.0.03)....304

7.2.1.1 Content 305
7.2.1.2 Result....305
7.2.1.3 Examples....307
7.2.2 Schedule Rule Execution....310
7.2.2.1 Query Parameters....311
7.2.2.2 Content 312
7.2.2.3 Result....313
7.2.2.4 Workflow Callback (since 8.0.03) 313
7.2.2.5 Workflow Callback (since 10.0) 313
7.2.3 Examples....315
7.2.3.1 Executing a DQ checks for the catalog TOOLS....315
7.2.4 Deprecated....317
7.2.4.1 Execute a data quality rule configuration group immediately 317
7.2.4.2 Execute a set of data quality rule configurations 318
7.2.4.3 Execute channel's set of data quality rule configurations 319
7.2.4.4 Examples....320
7.3 REST Environment API 321
7.3.1 REST Environment APIs used for Extraction, Download and Integration of Environment transfer....321
7.3.2 Extraction....321
7.3.2.1 Query Parameters....321
7.3.2.2 Result....322
7.3.3 Download....322
7.3.3.1 URL Parameters....323
7.3.3.2 Result....323
7.3.4 Integration 323
7.3.4.1 Query Parameters....324
7.3.4.2 Form Data Parameters 324
7.3.4.3 Result....324
7.3.5 Checking the Progress of the Extraction/Integration Job 325
7.3.5.1 URL Parameters....326
7.3.5.2 Result....326
7.3.5.3 Possible Values of "currentState"....327
7.4 REST Export API 330
7.4.1 Schedule Export Job 330
7.4.1.1 Query Parameters....331

7.4.1.2 Content 331
7.4.1.3 Result....335
7.4.1.4 Complete example 335
7.4.2 Information about an export profile....336
7.4.2.1 Result....337
7.4.2.2 Example....337
7.4.3 Cancel Export Job....338
7.4.3.1 URL Parameters....338
7.4.3.2 Result....338
7.5 REST File API 339
7.5.1 Downloading a File 339
7.5.2 Uploading a File 339
7.5.2.1 Result....340
7.5.2.2 Examples....340
7.5.3 Retention of uploaded files....341
7.6 REST Import API....342
7.6.1 Schedule Import Job 342
7.6.1.1 Query Parameters....343
7.6.1.2 Content 343
7.6.1.3 Result....346
7.6.2 Cancel Import Job 347
7.6.2.1 Result....347
7.6.3 Examples....348
7.6.3.1 Scheduling an import for a certain date and time....348
7.6.3.2 Cancel a running import job 349
7.7 REST KPI API....350
7.7.1 Schedule calculation of key performance indicator (KPI) values 350
7.7.1.1 Content 350
7.7.1.2 Result....351
7.7.1.3 Examples....352
7.7.2 Remove persisted results of key performance indicator (KPI) calculations....353
7.7.2.1 Content 354
7.7.2.2 Result....354
7.7.2.3 Examples....355
7.8 REST Merge API 358

7.8.1 Schedule Merge Job 358
7.8.1.1 Query Parameters....359
7.8.1.2 Content 360
7.8.1.3 Result....366
7.8.2 Workflow Callback....367
7.8.2.1 Workflow Callback (since 10.0) 367
7.8.2.2 Workflow Callback Example....367
7.8.3 Examples....368
7.8.3.1 Schedule merge of the full "Apparel" catalog....368
7.9 REST Revision API 370
7.9.1 Schedule Revision Release Job....370
7.9.1.1 Query Parameters....371
7.9.1.2 Content 372
7.9.1.3 Result....375
7.9.2 Get Revision Job Information 375
7.9.2.1 Result....376
7.10 REST System API 378
7.10.1 Create a thread dump 378
7.10.2 Create thread dumps for all servers 378
7.10.3 Echo Text....379
7.10.3.1 Example:....379
7.10.4 Create token for Token Basic Authentication 380
7.10.4.1 Example:....380
7.10.5 Create token for Token Basic Authentication, based on a SAML Response 381
7.10.5.1 Example:....381
7.10.6 REST System API for License....383
7.10.6.1 Retrieve License Information....383
7.11 REST Task API 385
7.11.1 Create Task 385
7.11.1.1 Content 386
7.11.1.2 Result....393
7.11.2 Update Task....393
7.11.2.1 Content 393
7.11.3 Get Task....393
7.11.4 Delete Task....394

7.11.5 Update Task Content....394
7.11.5.1 Content....395
7.11.6 Get Task Content....395
7.11.7 Workflow callback....396
7.11.8 Information Page....396
7.11.9 Examples....396
7.11.9.1 Create a task for all items in the catalog TOOLS....396
7.11.9.2 Create a task for supplier....397
7.11.9.3 Update a task: Change name and description....398
7.11.9.4 Update a task's content: Add another item....398
7.12 REST Workflow API....399
7.12.1 Create / Update Workflow Details....399
7.12.1.1 General Info....399
7.12.1.2 Content....400
7.12.1.3 Example....404
7.12.2 Get Workflow Details....405
7.12.2.1 General Info....405
7.12.2.2 Parameters....406
7.12.3 Enter Workflow Process Status....407
7.12.3.1 General Info....407
7.12.3.2 Content....407
7.12.3.3 Example....408
7.12.4 Leave Workflow Process Status....409
7.12.4.1 General Info....409
7.12.4.2 Content....410
7.12.4.3 Example....410
7.12.5 End Workflow Process....411
7.12.5.1 General Info....411
7.12.5.2 Parameters....412
7.12.5.3 Example....412
7.12.6 Get Workflow Process Status of an entity item per process....412
7.12.6.1 General Info....412
7.12.6.2 Parameters....413
7.12.6.3 Example....413
7.12.7 Add error message to the BPM perspective protocoll view....414

7.12.7.1 General Info....414
7.12.7.2 Parameters....414
7.12.8 REST Workflow Task....416
7.12.8.1 Example....416
7.12.8.2 WorkflowTask....417
7.12.8.3 Task Callback....419
8 REST Enumeration API....426
8.1 All Enumerations....426
8.1.1 Result....426
8.2 Meta Data and Enumeration Entries....427
8.2.1 Result....428
8.3 Examples....429
8.3.1 Retrieving all enumerations....429
8.3.2 Retrieving the entries of enumeration Enum.Languages....430
8.3.3 Retrieving the entries of enumeration Enum.SupplierWithMainSupplier....431
9 REST Media API....433
9.1 Single Media Asset File....433
9.2 Preview of Media Asset File....433
9.3 Media Asset File Location....434
9.4 Upload single Media Asset File....434
9.5 Examples....435
9.5.1 Retrieve single media asset file in 36dpi JPEG quality with identifier D123456....435
9.5.2 Retrieve media asset file preview....436
9.5.3 Retrieve media asset file location....436
9.5.4 Upload media asset file to media asset server....436
9.6 REST List API for MediaAssetFile entity....439
9.6.1 Adjustments for MediaAssetFile entity....439
9.6.1.1 Internal id....439
9.6.1.2 Special reports....440
9.6.2 Examples....441
9.6.2.1 Retrieve media asset file with "byIdentifiers" report for external id....441
9.6.2.2 Retrieve media asset file with "byItems" report for internal id....442
9.6.2.3 Retrieve media asset files which belong(be assigned) to a category and are used in PIM core....443

9.6.2.4 Retrieve all media asset files which belong to a category or its child categories 444
9.6.2.5 Update media asset file(Root Entity)....445
9.6.2.6 Update media asset file language special attribute(Sub Entity)....446
9.6.2.7 Delete media asset file with identifier D11000100112532....448
9.6.2.8 Create media asset file 448
9.6.3 Media search query for MediaAssetFile entity....449
9.6.3.1 Syntax for mediaQuery....449
9.6.3.2 Examples....454
9.6.4 Search query for MediaAssetFile entity 455
9.6.4.1 Syntax for query....455
9.6.4.2 Examples....460
9.7 REST Media Asset Category API....469
9.7.1 Get Media Asset Category....469
9.7.2 Get top Media Asset Categories 469
9.7.3 Get direct child Media Asset Categories 470
9.7.4 Add Media Asset Category....470
9.7.5 Update Media Asset Category 471
9.7.6 Delete Media Asset Category....471
9.7.7 Examples....472
9.7.7.1 Retrieve single media asset category with identifier....472
9.7.7.2 Retrieve all top media asset categories....472
9.7.7.3 Retrieve all direct child media asset categories of the category with identifier 473
9.7.7.4 Add a top media asset category....474
9.7.7.5 Add a child media asset category under the category with identifier 475
9.7.7.6 Update the name of the media asset category with identifier....475
9.7.7.7 Delete a media asset category with identifier....476
10 REST Meta API....477
10.1 All Root Entities....477
10.1.1 Result....478
10.2 Metadata for a Root Entity 478
10.2.1 Parameters....479
10.2.2 Result....479
10.3 All Fields of a Root Entity....480
10.3.1 Result....480

10.4 Metadata for a Sub Entity 481
10.4.1 Parameters 482
10.4.2 Result 482
10.5 Metadata for a Field 482
10.5.1 Parameters 483
10.5.2 Result 483
10.6 Metadata for a Qualifier 485
10.6.1 Result 486
10.7 Examples 487
10.7.1 Retrieving all root entities 487
10.7.2 Retrieving meta data of the root entity Article 487
10.7.3 Retrieving all fields of the root entity Article 489
10.7.4 Retrieving meta data of the sub entity ArticlePriceValuePurchase 489
10.7.5 Retrieving metadata for the field ArticleLang.DescriptionLong 490
10.7.6 Retrieving meta data of the field ArticlePriceVa.DescriptionLong with qualification 491
10.7.7 Retrieving meta data of the qualifier Language for entity ArticleLang 492
10.8 REST Meta Media API 493
10.8.1 Derivative definition 493
10.8.1.1 Get single derivative definition 494
10.8.1.2 Get all derivative definitions 495
10.8.2 Media Asset File Property field definition 495
10.8.2.1 Get single property field definition for one supported language 495
10.8.2.2 Get single property field definition for all supported languages 499
10.8.2.3 Get all property field definitions for all supported languages 499
10.8.3 Examples 500
10.8.3.1 Retrieve single derivative definition with identifier 500
10.8.3.2 Retrieve all derivative definitions 501
10.8.3.3 Retrieve single property field definition for one supported language 503
10.8.3.4 Retrieve single property field definition for all supported languages 505
10.8.3.5 Retrieve property field definitions for all supported languages 507
11 REST Security API 510
11.1 List all Security ACL APIs 510
11.1.1 Result 510
11.2 Read ACL object 511

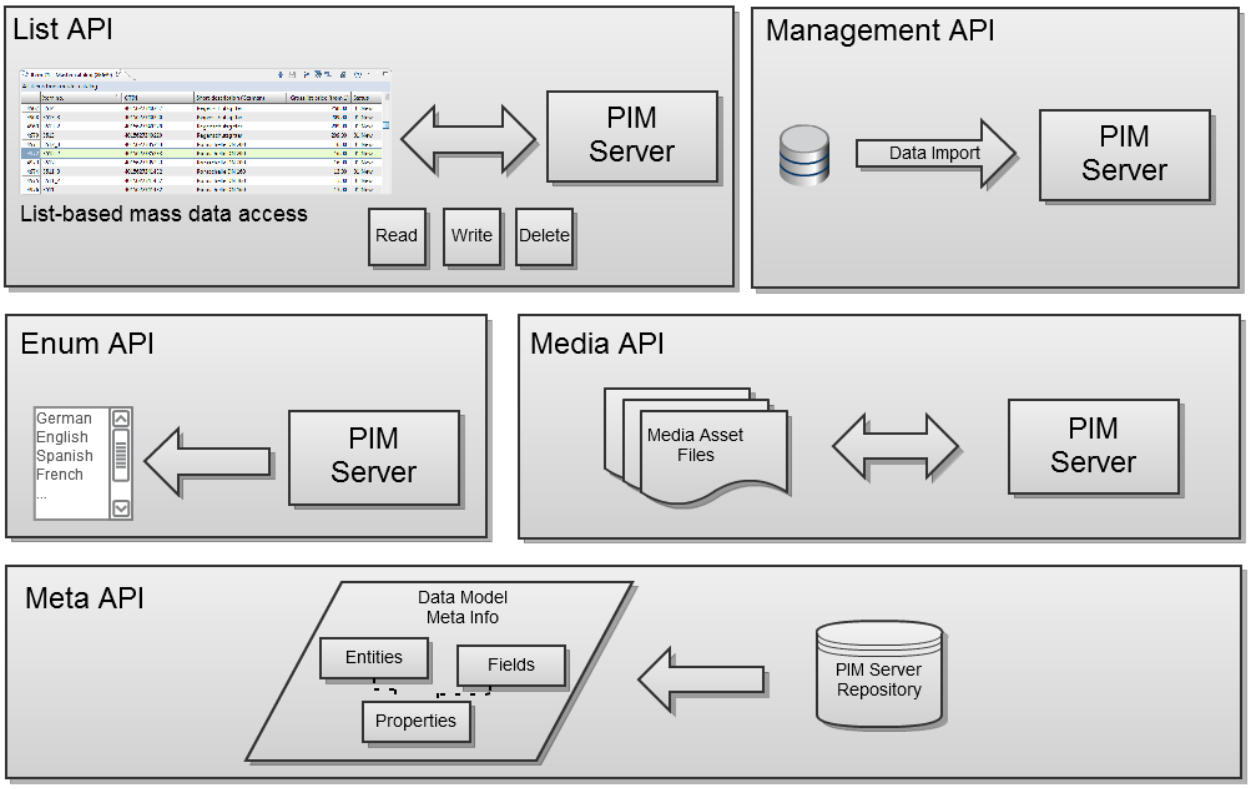
11.2.1 Content 511
11.3 Create ACL object 513
11.4 Examples 515
11.4.1 Read ACL object for an id 515
11.4.1.1 REST call for JSON 515
11.4.1.2 REST call for XML 517
11.4.1.3 REST Client Java 518
11.4.2 Create ACL object 518
11.4.2.1 REST call for JSON 518
11.4.2.2 REST call for XML 519
11.4.2.3 REST Client Java 520
12 PUBLIC Server Health Status 522
12.1 Overview 522
12.1.1 Health Status Page 522
12.1.1.1 Example 522
12.1.2 Info REST API 523
12.1.2.1 Example 523

Based on the REST Service Infrastructure introduced in Version 6 we created a set of services which allows you to read and write almost any data which is described by the repository. In general we provide services to create, read, update, and delete lists of entity items with a configurable set of field values.

If you would like to know more about REST in general, start with the Dare Obasanjo's blog post, Explaining REST to Damien Katz.

- Available APIs (see page 17)
  - List API (see page 17)
  - Management API (see page 17)
  - Enum API (see page 17)
  - Meta API (see page 18)
  - Media API (see page 18)
  - Security API (see page 18)
  - Object API (see page 18)
- Getting Started (see page 18)
  - Authentication (see page 19)
- General Information (see page 19)
  - Data Representation (see page 19)
  - Entity Item Reference (see page 19)
  - Field Qualification (see page 19)
  - Search Query Language (see page 19)
  - Rest Java Client (see page 20)
  - Rest API Versions (see page 20)

1 Available APIs



1.1 List API

The REST List API (see page 55) provides searching, reporting, navigation and finally result listing of nearly all objects in the system. Based on a query a homogeneous list of objects of a certain type is returned. The queries are parameterized. The resulting list is a two dimensional table with a given set of columns to control the informational character of the result.

1.2 Management API

The REST Management API (see page 302) provides access to control the server side management processes like Import, Merge, Export, and others. Typically these processes will be able to be started or scheduled, the process status can be evaluated and the process results can be requested and accessed.

1.3 Enum API

The Enumeration API provides read-only access to all enumerations in Product 360. The enum entry keys are used in field values as well as for the qualification of fields.

1.4 Meta API

The REST Meta API (see page 477)REST Meta API (see page 477) provides meta-information on all available Product Manager objects. It provides access not only to the objects, but also to their field including names, data types, constraints, etc. Using this API you can build powerful generic clients which may work with different installations of the Product Manager with different Repository configurations.

1.5 Media API

1.6 Security API

The REST Security API provides access to securities of the Product Manager.

1.7 Object API

With Version 10.5 we introduce an optimized version of the Read Object API (see page 230) and a new Write Object API (see page 272). The usage of Version 1 of the Read Object Api (see page 192) is supported, but discouraged.

With this all typical CRUD (Create, Read, Update and Delete) operations can be performed for all repository based entities which support the object API.

The REST Media API provides access to all kinds of media assets in the Product Manager.

2 Getting Started

Product Manager's REST APIs provide access to resources (data entities) via URI paths. To use a REST API, your application will make an HTTP/HTTPS request and parse the response. The Product Manager REST API uses JSON as its communication format, and the standard HTTP methods like GET, PUT, POST and DELETE. URIs for Product Manager's REST API resource have the following structure:

```
http://<host>:<port>/rest/<api-version>/<api>/<entity-identifier&gt;
```

- `<host>` : The Product Manager application server host.
- `<port>` : The http port which is configured in the server.properties (see page 16) file (see property http.port there), usually it's 1501.
- `<api-version>` : The version of the API. The current API version is V2.0.
- `<api>` : The specific api to call. See below for a list of the currently available APIs.
- `<entity-identifier>` : The unique identifier of the Product Manager's resource (also known as entity).

Most API's have a dynamic character since the actual resources which are available for each of those API's depend on the actual repository configuration of the Product Manager installation. To make life easier for

everyone we implemented some dynamic documentation lookup which provides all the needed parts. The documentation is provided as rich html, or as machine readable JSON depending on the requested format (html/json). The first "level" of documentation is the list of available api's which can be obtained by http://<host>:<port>/rest/V1.0/info

2.1 Authentication

Every request to the REST API must be authenticated, anonymous access to the Product Managers resources is not possible. Additionally to that, the user which is used for the authentication must have the Service Logon action right assigned. The currently available authentication method is HTTP Basic. Most client software provides a simple mechanism for supplying a user name and password and will build the required authentication headers automatically. More details can be found at REST Service Authentication (see page 23).

3 General Information

3.1 Data Representation

The .REST Service API v10.5 (see page 16) transfers all data physically as UTF-8 strings, either as JSON or XML representation. How numbers, dates, times and so on are formatted is described in REST Datatypes (see page 25).

3.2 Entity Item Reference

Links to other objects, like other products or structure groups, are represented as entity item references. An entity item reference is a structured object containing the ID and the label.

3.3 Field Qualification

Qualified fields can be specified on different levels, for example when reading and writing data. Always the same syntax is used which is described in REST Field Qualification (see page 31).

3.4 Search Query Language

We have defined a language for search expressions. The language is currently used in the *bySearch* report. The syntax of the search language is described in REST Search Query Language (see page 38).

3.5 Rest Java Client

The Rest Service API of the Heiler Product Manager can be from every client technology which is able to handle HTTP requests. However, we expect that a lot of clients will be built based on some Java technology, and therefore we provide also a REST Java Client (see page 42) for the Rest Service API. The Java client is part of the PIM 7 distribution. Rest Client Java sources are also available. Therefor also Javadoc can be viewed or created.

3.6 Rest API Versions

All API endpoints contain the Rest API Version. There are two versions available:

- V1.0 is the first iteration and has been extended with several new endpoint over time
- V2.0 provides the same functionality as V1.0 but comes with different defaults and semantics. Changes have been made based on experiences in the field.

In general, clients should always use the latest available version. Older versions might become deprecated and be removed in the future.

3.6.1 List API changes in V2.0

Change	Description
formatData	In V1.0, setting formatData to false didn't have an impact on fields with enumeration, like Article.CurrentStatus. When formatData is set to **false**, ▪ In V1.0 the enumeration label is returned ▪ In V2.0 the enumeration key is returned.
includeLabels	▪ New default value is false ▪ If set to false, root objects in the result also don't have a label anymore. In V1.0 this only affected fields with object references.
cacheId	New default value in responses is **no-cache** for the case, the client doesn't request using caching.

4 REST General Information

4.1 Monitoring

4.1.1 Monitor all REST requests in the log file

Add the following lines to the log4j.xml:

```
<Logger name="com.heiler.ppm.webservice" level="TRACE" />
```

After Server restart REST requests will be written to the server log file. Please use with care as the log file can grow very fast. Example output:

```
015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource] GET: http://
localhost:1501/rest/V1.0/list/Product2G/byCatalog?
fields=Product2G.AclProxy,Product2G.CurrentStatus,Product2G.Id,Product2G.Manufacturer
AID,Product2G.ManufacturerName,Product2GLang.DescriptionLong(eng),Product2GLang.Descr
iptionShort(eng),Product2GPriceValueSales.Amount('Public',net_customer,USD,US,2014-07
-01,1.0),Product2G.ProductNo&orderBy&formatData=false&metaData=true&startIndex=0&page
Size=100&includeLabels=true
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]   Request headers:
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]      Host:
localhost:1501
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]      Accept:
application/json
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]      Authorization:
Basic cmVzdDpoZWlsZXI=
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]      Accept-
Language: en-US
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]      User-Agent:
Apache-HttpClient/4.3.3 (java 1.5)
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]      Connection:
keep-alive
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]      Accept-
Encoding: gzip,deflate
2015-06-22 17:49:56,386 DEBUG [qtp1453109885-134] [ListResource]   Query parameters:
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]     startIndex: 0
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]     orderBy:
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]     formatData:
false
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]     includeLabels:
true
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]     pageSize: 100
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]     metaData: true
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]     fields:
Product2G.AclProxy,Product2G.CurrentStatus,Product2G.Id,Product2G.ManufacturerAID,Pro
```

```
duct2G.ManufacturerName,Product2GLang.DescriptionLong(eng),Product2GLang.DescriptionS
hort(eng),Product2GPriceValueSales.Amount('Public',net_customer,USD,US,2014-07-01,1.0
),Product2G.ProductNo
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]   User: rest
2015-06-22 17:49:56,387 DEBUG [qtp1453109885-134] [ListResource]   Locale: en_US
2
```

4.2 REST Service Authentication

An overview with examples on the various authentication methods supported by the REST Infrastructure of the Product Manager. All authentication methods are implemented as core functionality of the REST/WebService infrastructure. This means that this authentication will automatically be active for your own REST Services as well as for the REST Service API (see page 16) which comes out of the box.

- Basic Authentication (see page 23)
  - Simple example (see page 23)
  - Username / Password Basic Authentication (see page 23)
  - Token Basic Authentication (see page 24)
- OAuth (see page 24)

4.2.1 Basic Authentication

Product Manager allows REST clients to authenticate themselves with a user name and password using basic authentication .



Basic authentication is unsafe as long as it's not combined with the SSL protocol, since the username and password are transmitted more or less in plain text!

4.2.1.1 Simple example

Most client software provides a simple mechanism for supplying a user name and password and will build the required authentication headers automatically. For example you can specify the -u argument with curl as follows

```
curl -D- -u myUsername:myPassword -X GET -H "Content-Type: application/json" http://
hpm.heiler.com:1501/rest/V1.0/list/suppliercatalog/all
```

4.2.1.2 Username / Password Basic Authentication

If you need to you may construct and send basic auth headers yourself. To do this you need to perform the following steps:

1. Build a string of the form username:password
2. Base64 encode the string
3. Supply an "Authorization" header with content "Basic " followed by the encoded string, e.g. "Basic YWRtaW46YWRtaW4="

```
curl -D- -X GET -H "Authorization: Basic bXlVc2VybmFtZTpteVBhc3N3b3Jk" -H "Content-
Type: application/json" http://hpm.heiler.com:1501/rest/V1.0/list/suppliercatalog/all
```

4.2.1.3 Token Basic Authentication

Instead of supplying username/password clients can also use a token to authenticate calls. This token can be retrieved the following ways:

- When running inside the Product 360 Server / Client by using the API: `com.heiler.ppm.security.core.auth.Authenticator.createTokenAsString()`. This token is valid for one hour.
- When already authenticated, a token can also be retrieved via REST by sending a POST to `/rest/V1.0/manage/system/security/token`. For further details, see chapter REST System API (see page 378). This token is valid for one hour.
- When using SAML for authentication (see chapter SAML Configuration), a token can by retrieved for the user contained in a SAML Assertion resp. SAML Response. This is done via REST by sending a POST to `/rest/V1.0/manage/system/security/tokenForSaml`. For further details, see chapter REST System API (see page 378). The token validity period can be configured, and is one day by default.

The token then can be passed base64-encoded as Basic Authentication header. Example:

```
GET /rest/V1.0/list/Article/byCatalog
Authentication: Token
MkZDRDVDMkMyRjEwQTk0OTBFMTAwOEVDNjhFNTREOEEyREMyMUMzQzNGN0I5Q0RGMDVBNkE1MzVBODlFMkFCR
jA1MzFGMzE5Q0E0QkM5OTczRjAyM0QyQzc2RUEwQTNGODQ2ODA3MTQyMjM5NjdBOTIyM0UyNTlCQzNDOTI1Q0
MwQkM2MEIyRjk5RTg5MTBDRDFFNERERUMzNENGM0MzNUY2OEY3NTM0MTdGMUM5MzU1NzQwRjJFMUVDMkJDNTJ
CMUE5RTg4OUFFMjczRUM0MDc0NURDNDJCQTMyOERGRjVGRDNGRjY1ODk3RDk2NEVDMkE2MEI5RTcwOTI4ODlG
MERCQTM2MzdFNEIzOTAxMDYzOTQ4MjM4RjkxODcwQjk3RTgwOEZDQzZGMTIwMkIxNkIzQThCRkZERjIwMDkxO
UI5MDZBNzAwRUM3MzM1QjQ1RTY5QkM4NUM3MEEzMDY2N0U4OTkxRTQxQ0YwMUREN0FFOUY5QjJEMjk0NUE2Mj
Q2RjgyN0IyNUUzRUQ2MEQxNjVDN0U1OUQyOERENDcwNUQzRkU1QTMyMjlGRUZCQUQzNjRDNThGNjg2NTQ2QjA
1OEQzQTdEMzE1QkQ4M0QzRjlFQUFCQUVENTM0RDE0RTFGMzdCNTkxRDQ5REVEMTM1NjM2RkQyNTFGQ0NGNDRG
NUY=
```

4.2.2 OAuth

Not yet supported.

4.3 REST Datatypes

4.3.1 Datatypes

The REST Datatypes (see page 25) transfers all data physically as UTF-8 strings, either as JSON or XML representation. The following list gives an overview on the data types which are used throughout the whole REST Service API.

4.3.1.1 STRING

Definition	Unicode Characters
Pattern	
Example	"Hello World!", "Heute ist ein schöner Tag", "世界您好!"
Corresponding Java Datatype	java.lang.String

4.3.1.2 INTEGER

Definition	Integer Numbers (-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)
Pattern	[+/-] 0-9*
Example	4711
Corresponding Java Datatype	java.lang.Long

4.3.1.3 DECIMAL

Definition	Signed decimal number. Please see the properties of the parameter/field for details on the precision and the allowed number of fraction digits. The string representation consists of an optional sign,'+' or '-', followed by a sequence of zero or more decimal digits ("the integer"), optionally followed by a fraction The fraction consists of a decimal point followed by zero or more decimal digits. The string must contain at least one digit in either the integer or the fraction.
Pattern	[+/-] 0-9*[.0-9*]
Example	100.90
Corresponding Java Datatype	java.math.BigDecimal

4.3.1.4 DATE

Definition	Calendar Date (12/30/1899 to 12/31/9999)
Pattern	yyyy-MM-dd
Example	2012-05-01 (first of May in 2012)
Corresponding Java Datatype	java.sql.Date

4.3.1.5 TIME

Definition	Time (00:00:00 to 24:00:00)
Pattern	HH:mmZ

Example	12:05 (five minutes after 12 o'clock noon)
Corresponding Java Datatype	java.sql.Time

4.3.1.6 DATETIME

Definition	Date and Time in one value
Pattern	**Input** / **Output** yyyy-MM-dd'T'HH:mm:ss / yyyy-MM-dd'T'HH:mm:ss:SSSZ yyyy-MM-dd'T'HH:mm:ssZ yyyy-MM-dd'T'HH:mm:ss:SSS yyyy-MM-dd'T'HH:mm:ss:SSSZ
Example	2012-05-01T12:00:00 2012-05-01T12:00:00-0700 2012-05-01T12:00:00:000 2012-05-01T12:00:00:000-0700
Corresponding Java Datatype	java.sql.Timestamp

4.3.1.7 BOOLEAN

Definition	Boolean value - either true or false. Note: Fields which are not mandatory might also have an empty string which means no value which should not be interpreted as "false" or "true". It's just, "not defined".

Pattern	true / false
Example	true, false
Corresponding Java Datatype	java.lang.Boolean

4.3.1.8 ENTITY_ITEM

Definition	A Product Manager object which is described in the Meta API (see page 477). In case entity item's are part of a result object from a service call, they will be serialized as separate object structure. In case you need to provide an entity item as a parameter, a simpler string representation is mostly enough. As part of a result object an entity item is rendered like this: `"object": { "id": "86@1064", "label": "A1" }` If you do not need the human readable label of the entity items, you can disable the label generation by specifying `includeLabels` with `false`. See also the REST List API Read for Root Entities (see page 65) page.
Pattern	Entity Item ID Syntax: Entity Item without a container item: '<external_identifier>'|<internal_id> Entity Item with a container item: '<external_identifier>'|<internal_id>@'<container_external_identifier>'|<container_internal_id> *Please note that you need to escape single quotes in case they are part of the external identifier or container external identifier with a backslash*

Example	'SomeItem'@'SupplierCat1' 12345@5 'SomeSupplier' 42 'SomeItem\'WithASingleQuote'@'MASTER'
Corresponding Java Datatype	com.heiler.pim.webservice.client.EntityItemReference

4.3.1.9 MIME_VALUE

Definition	A reference to a mime file. In case mime values are part of a result object from a service call, they will be serialized as separate object structure. A mime value has a label. The label is shown in the UI in case a preview picture is not used there. The relative file path defines the location of the file inside of the zip archive when you download or upload the mime values The label and mime type are optional in case you want to write a mime value. They will be extracted from the `relativeFilePath` if omitted. Please note that the relative file path of a mime value will change once it's uploaded and saved as Product 360 makes sure that every mime value is unique in the system. If you retrieve the mime value again after the upload, the `relativeFilePath` will be different. It should always be treated as a black box.
Pattern	*No Simple string representation, it's always rendered as JSON/XML structure*
Example	`{ "label": "examplePicture.jpg", "mimeType": "image/jpg", "relativeFilePath": "dir1\\dir2\\dir3\\examplePicture.[sequenceToMakeFileNameUnique].jpg" }`

Corresponding Java Datatype	`com.heiler.pim.webservice.client.MIMEValueReference`

4.3.1.10 ANY

Definition	ANY can be one specific datatype like STRING,DATETIME, DECIMAL, ENTITY_ITEM or BOOLEAN. A field with ANY as datatype must have a specific implementation to figure out which of the specific datatypes mentioned before is actually taking effect.
Pattern	see at the specific datatype
Example	see at the specific datatype
Corresponding Java Datatype	java.lang.Object

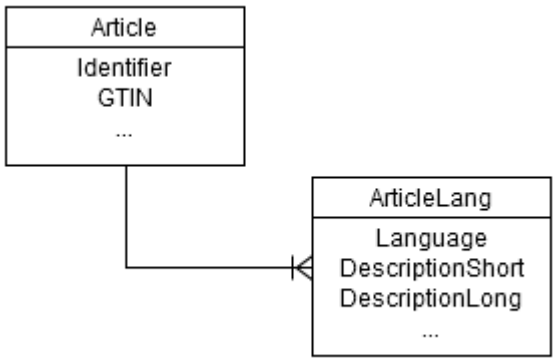
4.4 REST Field Qualification

Several services need fully qualified fields in one way or the other. The syntax to qualify fields is documented on this page

- Syntax (see page 31)
- Logical Key Value Definition (see page 32)
  - With Enumeration (see page 32)
  - Without Enumeration (see page 33)
  - Using default values (see page 34)
  - Using wildcard matching (see page 35)

Product Manager defines many business objects which are described in the Product Manager repository. The repository can be seen as a storage for all meta-data regarding the business models. All objects and all attributes of those objects are described in there. Additionally to that, also the relation between an object and it's child objects is described in the repository. You can find a detailed description about the Repository here.

An example would be the object "Article" and it's child object "ArticleLang". An Article can have multiple "ArticleLang" records, one for each language. Therefore, the language is the attribute which makes the "ArticleLang" records unique for a single item.



To "fully qualify" a field means, that you not only must define which field (by the field's identifier), but also all values for all attributes which identify the children uniquely. In this example, just the language.

4.4.1 Syntax

Fully qualified field: `<Field Identifier>[(<first logical key value>,..,<n-logical key value>)]`

Multiple Field Qualification: `<Fully qualified field>[, <Fully qualified field>]`

Element	Definition
Field Identifier	Basically the unique field identifier as given in the custom section of the repository. These identifiers typically but not necessarily start with the entity identifier separated by a "." from the filed's name. This makes the field unique in case of the same field supported by different data entities. Samples are: • Article.SupplierAID • Article.EAN • Article.ManufacturerName
Logical Key Value	The qualification value for the first, second, third and so on logical key. Which logical key is on which position is defined by the hierarchy and the order attribute of the repository logical key type. In case no order attribute is defined in the repository, the order is defined by the relative location in the repository xml file. <Parent LK 1, ParentLK 2, Own LK 1, Own LK 2>

4.4.2 Logical Key Value Definition

The definition of the logical key value depends on the datatype of the logical key (or it's respective field) and whether the logical key has an enumeration or not.

4.4.2.1 With Enumeration

You can define everything which can be interpreted by the enumeration as a synonym to one of it's keys. This can be done either as a constant value, or as a string (constants are not allowed to have spaces in it, use strings in this case!)

For example:
`ArticleLang.DescriptionShort(en)`, `ArticleLang.DescriptionShort(eng)`, `ArticleLang.DescriptionShort(English)` would be just the same.

Furthermore the key of an enumeration can be used if it's data type is not an entity item, like buyers, suppliers, units etc.

For example: `ArticleLang.DescriptionShort(9)`

A special situation would be a logical key which has an enumeration and it's data type is entity item. In this case you can either specify the actual entity item with the entity item syntax, as well as

a string or constant which is then parsed with the enumerations synonyms.

> Some enumerations are case sensitive regarding their synonyms. Which one those are is defined in the enumeration itself. You're always on the safe side if you treat all of them as case-sensitive.

4.4.2.2 Without Enumeration

Logical keys without enumeration must be defined depending on their datatype. Please see the REST Datatypes (see page 25) overview for details.

Datatype	Format	Example
Date (without time!)	YYYY-MM-DD	2012-06-22 which is the 22nd of June in 2012
Time (without date!)	HH:mm:SS	13:15:05 which is quarter past one, pm and five seconds
Date and Time	YYYY-MM-DD HH:mm:SS	2012-06-22 13:15:05
Integer	+#* or -#*	+1247 or -1247
Decimal	+#*.#* or -#*.#*	+1.45 or -1.45 (no thousand seperator, use dot as comma seperator!)
Boolean	true or false	true or false
String	"any string" or "any \"important\" string"	"My Attribute Name" escape the quotation marks by a backslash! E.g. "Size in \"" (meaning: size in inch)
Constant	any string a-Z followed by zero or more 0-9	eng, de, public

Datatype	Format	Example
ENTITY_ITEM	Entity Item without a container item: '<external_identifier>'|<internal_id> Entity Item with a container item: '<external_identifier>'|<internal_id>@'<container_external_identifier>'|<container_internal_id> **Please don't forget use single quotes when you want to specify the external identifier of the entity item!**	'SomeItem'@'SupplierCat1' 12345@5 'SomeSupplier' 42
Dynamic Value	A dynamic value is a placeholder which will be resolved while parsing the field qualification Syntax: `${Identifier}` Note: additional named values can be contributed to the PIM server using the extension point: `com.heiler.ppm.value.core.namedValues`	Currently available named values: • `${ CurrentDate}` = the current date (without time) • `${CurrentTime}` = the current time (without date) • `${CurrentTimestamp}` = the current date and time

Examples

The short description of an article in english: ArticleLang.DescriptionShort(eng)

The public customer price value, in Euro for Germany, Valid on the 06/22/2012, when I order one: ArticlePriceValueSales.Amount(public,net_customer,eur,de,2012-06-22,1.00)

4.4.2.3 Using default values

If the repository defines a default value for the logical key then the default value can be referenced in the field qualification using the following notation:

`${Default}`

Note that the resulting fully qualified field may change when the definitions in the repository change.

4.4.2.4 Using wildcard matching

If no value can be defined as qualification, it is possible to use the

`${Empty}`

qualifier.

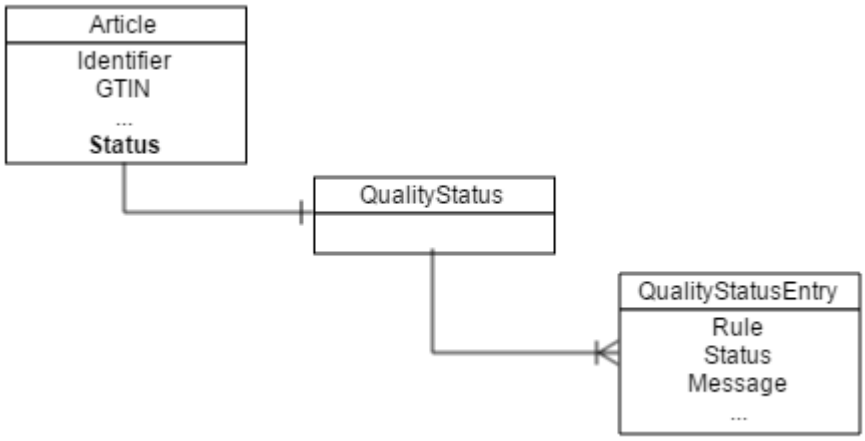
4.5 REST Transition Fields

The REST List Read API (see page 63) supports so-called transition fields. The syntax for such transition fields is documented on this page

- Syntax (see page 36)
- Examples (see page 37)

The PIM repository supports proxy fields which reference other repository entities. When fields of such referenced entities are accessed in the context of the referencing entity, they are called *transition fields*. Transition fields have a source and a target field, which can in turn be transition fields. Therefore it is possible to traverse recursively to a target field. All fields along this transition path need to be fully qualified (see page 31).

An example would be the object "Article" and it's field "Article.Status". Each item has a quality status, whose content resides in the "QualityStatus" object. If we want to obtain the quality status of a certain item for a specific data quality rule, we need to use a transition field with target "QualityStatusEntry.Status" and proper qualification for the "rule" logical key.



4.5.1 Syntax

Transition field: `<Fully qualified field> -> ... -> <n-Fully qualified field>`

Element	Definition
Fully qualified field	A fully qualified field as described in REST Field Qualification (see page 31). The field on the left of the arrow (`->`) must be a proxy transition field pointing to another repository object, as e.g. • Article.Status • ArticleStructureMap.StructureGroup • ArticleLogistic.PackageWeightUnitID ⚠ Make sure to have a space before and after the arrow, otherwise it will be interpreted as part of the field name, which most likely will not be correct.

4.5.2 Examples

- The quality status of an item of rule "CheckValidGTIN":
  Article.Status -> QualityStatusEntry.Status(CheckValidGTIN)

- The quality status of a structure group of rule "CheckValidName" where an item is assigned to:
  ArticleStructureGroupMap.StructureGroupProxy('ETIM3.0', 'shoes'@'ETIM3.0') -> StructureGroup.Status -> QualityStatusEntry.Status(CheckValidName)

4.6 REST Search Query Language

The HSQ syntax directly reflects the search expression API explained in Entity Search - so the same expressions are possible as if you build complex search queries the programmatic way using the corresponding API. It's main design goal was to provide an easy to understand and easy to use query syntax which can also be used by end users of the Product Manager.

Syntax

```
HSQ Query: [<CONDITION>]+
HSQ Condition: <TERM> [AND|OR <CONDITION>]* | [NOT]? (<CONDITION>)
HSQ Term: [NOT]? <FIELD QUALIFICATION | CHARACTERISTIC> <OPERATOR> <VALUE LITERAL> |
[NOT]? <FIELD QUALIFICATION | CHARACTERISTIC> IN (<VALUE LITERAL> [,<VALUE
LITERAL>]*)
```

Conditions/terms can be connected with AND / OR, whereas the AND has higher priority and thus is evaluated first. Evaluation priority can be influenced by building complex conditions by means of parenthesis (possibe around conditions as well as around terms). NOT is used for negating terms as well as complex conditions (where parenthesis are needed, of course).

See again example from code block above:

```
Article.EAN in ("123456789", "987654321", "abcdefghij") AND not (Article.Identifier =
"abc" OR Article.Identifier equals "xyz")

characteristic(9342) = "44"
characteristic(9347) = "22"&catalog=1011
```

4.6.1 Field Qualification

The HSQ expression is built with a fully qualified field. Please see the REST Field Qualification (see page 31) documentation on the used syntax here. Note, only a single field is allowed in the context of the search query.

4.6.2 Characteristic

Please see the REST Characteristic (see page 44) documentation chapter on the used syntax here. Note, only a single characteristic is allowed in the context of the search query.

4.6.3 Operators

Operators are interpreted case insensitive, thus " `equals` " and " `EQUALS` " and " `eQuAlS` " are, equal 🙂

Operator	Allowed Datatypes	Description
equals	String	Value must be **exactly** the same as the corresponding field content
equalsIC	String	Value must be the same as the corresponding field content, upper and lower case differences will be ignored
contains	String	Field content must contain the given string in the same case
containsIC	String	Field content must contain the given string, but the case of the content is ignored
startsWith	String	Field content must start with the given string in the same case
startsWithIC	String	Field content must start with the given string but the case of the content is ignored
wildcard	String	Value can consist wildcards like '**%**' (one or more characters) and '**_**' (single character)
wildcardIC	String	Value can consist wildcards but the case of the content is ignored
=	all	Value must be the same as the corresponding field content
~	String	Value can consist wildcards like '**%**' (one or more characters) and '**_**' (single character)
!~	String	Value can consist wildcards like '**%**' (one or more characters) and '**_**' (single character)
!= / <>	all	Field content must be unequal to the provided value

Operator	Allowed Datatypes	Description
>	String, Number, Datetime	The field content must be greater then the given value according to the "natural order" of the value
>=	String, Number, Datetime	The field content must be greater or equal as the given value according to the "natural order" of the value
<	String, Number, Datetime	The field content must be less then the given value according to the "natural order" of the value
<=	String, Number, Datetime	The field content must be less or equal to the given value
in	all	Field content must be equal to one of the provided values in parenthesis
is empty	all	Field content must be null or empty string

While using wildcard/ ~ operators make sure to escape any wildcard character using '\\'. For eg. "\\%" or "_"

4.6.4 Value Literal

The textual representation of the value is described in the REST Datatypes (see page 25). Multiple values (comma separated in parenthesis) can only be used for the "in" operator.

In general, the value literal's type must be compatible to the fields's data type where the field qualification points to. Implicit conversion is currently only performed when field destination type is of DECIMAL and provided value literal is of INTEGER - in all other cases the types have to be the same.

4.6.4.1 Dynamic Value

A value literal can also be a dynamic literal.

A dynamic value is a placeholder which will be resolved while parsing the field qualification
Syntax: `${Identifier}`

> Additional named values can be contributed to the PIM server using the extension point: `com.heiler.ppm.value.core.namedValues`

Currently available named values:

- `${ CurrentDate}` = the current date (without time)
- `${CurrentTime}` = the current time (without date)
- `${CurrentTimestamp}` = the current date and time

4.7 REST Java Client

The Rest Service API of the Heiler Product Manager can be from every client technology which is able to handle HTTP requests. However, we expect that a lot of clients will be built based on some Java technology, and therefore we provide also a REST Java Client (see page 42) for the Rest Service API. The Java client is part of the PIM 7 distribution. Rest Client Java sources are also available. Therefor also Javadoc can be viewed or created.

4.7.1 Example

Below you can find a short example on how to use the client. The `RestClientExample` class is also part of the rest client API package.

Usage example for the client API

```
 /**
   * Three arguments needed in the following order
   * <p>
   * - Basic Url, e.g.: "http://pim.heiler.com/rest"<br/>
   * - Username, e.g.: MyUserName <br/>
   * - Password, e.g.: MyPassword <br/>
   * </p>
   * <code> RestClientExample.main http://pim.heiler.com/rest MyUserName MyPassword </code>
   */
  public static void main( String[] args )
  {
    String basicUrl = extractBasicUrl( args );
    String userName = extractUserName( args );
    String password = extractPassword( args );
    try
    {
      //Create and login to the client...
      RestClient restClient = new RestClient();
      restClient.loginWithBasicAuth( basicUrl, userName, password, Locale.ENGLISH );
      //Create and parameterize the report query you want to execute...
      EntityItemReference masterCatalog = EntityItemReferenceFactory.createByIdentifier( "MASTER" ); //$NON-NLS-1$
      ReportQuery reportQuery = new ReportQuery( "byCatalog" ).addParameterValue( "catalog", masterCatalog ); //$NON-NLS-1$ //$NON-NLS-2$
      //Create and parameterize the list request
      ListReadRequest listReadRequest = restClient.createListReadRequest();
      EntityItemTable resultTable = listReadRequest.setFields( "Article.SupplierAID" ) //$NON-NLS-1$
```

```
26                                                  .setPageSize( 100 )
27                                                  .setStartIndex( 0 )
28                                                  .getRootItems( "Article
", reportQuery ); //$NON-NLS-1$
29        for ( EntityItemTableRow row : resultTable )
30        {
31          List< Object > values = row.getValues();
32          String currentSupplierAid = ( String ) values.get( 0 );
33          System.out.println( currentSupplierAid );
34        }
35      }
36      catch ( RestClientException e )
37      {
38        e.printStackTrace();
39      }
40    }
```

4.8 REST Characteristic

- Characteristic Operand (see page 44)
  - Syntax (see page 44)
  - Usage examples (see page 45)
- DESCENDANT Operator (see page 49)
  - Syntax (see page 50)
  - Limitations (see page 50)
  - Usage Examples (see page 51)
- Limitation (see page 54)

In this chapter describes how to use characteristics in the Service API. It introduces the characteristic operand and the DESCENDANT operator.

4.8.1 Characteristic Operand

4.8.1.1 Syntax

When searching for items using characteristic record values the term `characteristic` with parenthesis has to to used. Both qualifications for the characteristic are **optional**.

characteristic(`<Entity Item>,<Language>)`

Element	Definition
Entity Item	Characteristic entity item without a container item: '<external_identifier>'|<internal_id> **Please don't forget to use single quotes when you want to specify the external identifier of the characteristic!**
Language	The value for the language qualification can either be the key or a synonym as defined in the Enum.Language of the repository. Example: To qualify the German language the value 7 or "DE" can be used equally.

So valid examples for the characteristic term are:

```
characteristic()
```

```
characteristic('Ingredients')
```

```
characteristic(156335)
```

`characteristic(156335, "DE")`

`characteristic(156335, 7)`



It is not possible to search for any characteristic with a specific language.
~~`characteristic( "DE" )`~~

4.8.1.2 Usage examples

Searching for items which have a specific characteristic and a specific value

This call will return all items which have the string value "Bean" for the characteristic `Ingredient`. Note that the characteristic has the datatype `TEXT` and therefore the characteristic record values are of type `String`.

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic('Ingredient') = "Bean"
```

We have these items with characteristic records:

ItemIdentifier	Characteristic	Characteristic record value
Item1	`Ingredient`	"Salt"
Item2	`Ingredient`	"Bean"
Item3	`Ingredient`	"Egg"
Item4	`BakingMaterials`	"Flour"

The query above will only return **Item2**.

When searching for string values it is also possible to use specific string operators like : `equalsIC`, `equals`, `contains`, `startsWith`, `contains`, ...etc.

All possible operators can be found in the chapter REST Search Query Language (see page 38)

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic('Ingredient') startsWith "Bean"
```

We have these items with characteristic records:

ItemIdentifier	Characteristic	Characteristic record value
Item1	`Ingredient`	"Salt"
Item2	`Ingredient`	"Beans"
Item3	`Ingredient`	"Egg"
Item4	`BakingMaterials`	"Flour"

The query above will only return **Item2**.

When searching for all items which have a characteristic record for the characteristic `Ingredient`.

`http://localhost:1512/rest/V1.0/list/Article/bySearch?query=not characteristic('Ingredient') is empty`

We have these items with characteristic records:

ItemIdentifier	Characteristic	Characteristic record value
Item1	`Ingredient`	"Salt"
Item2	`Ingredient`	"Beans"
Item3	`Ingredient`	"Egg"
Item4	`BakingMaterials`	"Flour"

The query above will return **Item1,Item2,Item3**.

Searching for items which have specific characteristics and specific values

This call will return all items which have the value `15678` as a characteristic record for the characteristic "Numeric" and also have the value `"2017-12-31"` for the characteristic "TerminationDate"

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic('Numeric') = 15678 and characteristic('TerminationDate') =
"2017-12-31"
```

We have these items with characteristic records:

ItemIdentifier	Characteristic	Characteristic record value		Characteristic	Characteristic record value
Item1	`Numeric`	12345		`TerminationDate`	2017-12-31
Item2	`Numeric`	15678		`TerminationDate`	2017-12-31
Item3	`Numeric`	15678		`TerminationDate`	2017-11-20
Item4	`Numeric`	1335		`TerminationDate`	2018-12-31

The query above will only return **Item2**.

Searching for items in a specific language

This call will return all items which contain the value "soup recipe" for the characteristic "Description" in English

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic('Description',9) contains "soup recipe"
```

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic('Description',en) contains "soup recipe"
```

Both statements are equally valid.

We have these items with characteristic records:

ItemIdentifier	Characteristic	Language	Characteristic record value
Item1	Description	English	"soup without recipe"
Item2	Description	English	"this is a soup recipe"
Item3	Description	German	"Das ist ein Rezept für Suppe"
Item4	BakingMaterials	French	"soup recipe"

The query above will only return **Item2**.

Searching for items with lookup values

This call will return all items where the lookup value "Mushroom" of the lookup "Ingredients" is used regardless of the characteristic.

When trying to search by lookup values, it is required to use the entity proxy format: `'EXTERNAL_IDENTIFIER_OF_LOOKUP_VALUE'@'EXTERNAL_IDENTIFIER_OF_LOOKUP'`

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?query=characteristic() = 'Mushroom'@'Ingredients'
```

We have these items with characteristic records:

ItemIdentifier	Characteristic	Lookup of Characteristic	Characteristic record value
Item1	CookingIngredients	Ingredients	Salt
Item2	CookingIngredients	Ingredients	Mushroom
Item3	CookingIngredients	Ingredients	Egg
Item4	ToxicIngredients	Ingredients	Mushroom

The query above will return **Item2** and **Item 4**.

When specifying a characteristic, it is possible to use the lookup value identifier without the lookup identifier:

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic('CookingIngredients') = 'Mushroom'
```

The query above will only return **Item2.**

All other operators can be used as well just like working with the fieldpath:

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic('Ingredient') in
( 'Mushroom'@'Ingredients','Egg'@'Ingredients' ) and not
characteristic('TerminationDate') > 2017-12-31T15:00:00
```

We have these items with characteristic records:

ItemIdentifier	Characteristic	Lookup of Characteristic	Characteristic record value		Characteristic	Characteristic record value
Item1	Ingredient	Ingredients	Mushroom		TerminationDate	2017-12-28 17:30:00
Item2	Ingredient	Ingredients	Mushroom		TerminationDate	2020-12-12 15:00:00
Item3	Ingredient	Ingredients	Egg		TerminationDate	2017-12-05 15:00:00
Item4	BakingMaterials	Ingredients	Flour		TerminationDate	2017-12-31 13:37:00

The query above will only return **Item1** and **Item3**.

4.8.2 DESCENDANT Operator

Sometimes it is important to find items with certain values. For example to find all items containing sugar won in Brazil. You don't want items containing sugar from India or meat from Brazil in your result set. In order to maintain such item data, you probably modeled a characteristic "Ingredient" containing the values "Sugar" or "Meat" and a characteristic "Country of origin" which is a descendant of the characteristic "Ingredient" in the characteristic hierarchy. Using the characteristic operand from above you would find items containing the ingredient sugar and items with Brazil as country of origin. In order to tell the system that you only want items having the two values in a direct hierarchy, you can use the DESCENDANT operator.

4.8.2.1 Syntax

Operator	Allowed datatypes	Description
DESCENDANT	all datatypes except MIME	This operator searches for characteristics having a hierarchical relationship. Operands may be all binary expressions containing a characteristic except connectors like AND and OR. The left operand must be an expression containing a characteristic which is higher in the characteristic hierarchy than the characteristic contained by the right operand. Note: The left operand must not necessarily contain the root characteristic, but it must be higher in the characteristic hierarchy than the characteristic contained by the right operand.
>>		

So valid examples for the DESCENDANT operator are:

```
Operand1 DESCENDANT Operand2
```

```
Operand1 DESCENDANT Operand2 DESCENDANT Operand3
```

```
Operand1 >> Operand2
```

```
Operand1 >> Operand2 >> Operand3
```

```
Operand1 >>  NOT(Operand2) >> Operand3
```

```
Operand1 >> Operand2 DESCENDANT Operand 3
```

The following examples are **not** valid:

```
Operand1 >> Operand2 AND Operand3 >> Operand 4
```

```
Operand1 >> Operand2 OR Operand3 >> Operand 4
```

4.8.2.2 Limitations

It is **not** possible to use the same characteristic more than once in one DESCENDANT expression. So the following example ist **not** valid:

```
characteristic( 'ABC' ) equals otherValue DESCENDANT characteristic( 'ABC' ) equals otherValue DESCENDANT characteristic( 'ABC' ) equals otherValue
```

The descendant operator **cannot** be negated. So the following example ist **not** valid:

`NOT (characteristic( 'ABC' ) equals value DESCENDANT characteristic( 'ABCD' ) equals otherValue)`

4.8.2.3 Usage Examples

Rest call

```
GET http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic( 'Ingredient' ) equals 'SUGAR'@'Ingredients' DESCENDANT
characteristic( 'CountryOfOrigin' ) equals 'BRAZIL'@'OriginCountries'
```

Rest call - Alternative syntax with >>

```
GET http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=characteristic( 'Ingredient' ) equals 'SUGAR'@'Ingredients' >>
characteristic( 'CountryOfOrigin' ) equals 'BRAZIL'@'OriginCountries'
```

We have these items with characteristic records:

Item Identifier	Characteristic	Lookup of Characteristic	Characteristic record value
Item1	Ingredient	Ingredients	Sugar
Item1	CountryOfOrigin	OriginCountries	Thailand
Item2	Ingredient	Ingredients	Sugar
Item2	CountryOfOrigin	OriginCountries	India
Item2	Ingredient	Ingredients	Meat
Item2	CountryOfOrigin	OriginCountries	Brazil
Item3	Ingredient	Ingredients	Sugar

Item Identifier	Characteristic	Lookup of Characteristic	Characteristic record value
Item3	CountryOfOrigin	OriginCountries	Brazil

The query above will return **Item3**.

Rest call

```
GET http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=(characteristic( 'Ingredient' ) equals 'SUGAR'@'Ingredients' or
characteristic( 'Ingredient' ) equals 'SALT'@'Ingredients') >>
characteristic( 'CountryOfOrigin' ) equals 'BRAZIL'@'OriginCountries'
```

We have these items with characteristic records:

Item Identifier	Characteristic	Lookup of Characteristic	Characteristic record value
Item1	Ingredient	Ingredients	Sugar
Item1	CountryOfOrigin	OriginCountries	Thailand
Item2	Ingredient	Ingredients	Sugar
Item2	CountryOfOrigin	OriginCountries	India
Item2	Ingredient	Ingredients	Meat
Item2	CountryOfOrigin	OriginCountries	Brazil
Item3	Ingredient	Ingredients	Sugar
Item3	CountryOfOrigin	OriginCountries	Brazil

Item Identifier	Characteristic	Lookup of Characteristic	Characteristic record value
Item4	Ingredient	Ingredients	Salt
Item4	CountryOfOrigin	OriginCountries	Brazil

The query above will return **Item3** and **Item4**.

Rest call

```
GET http://localhost:1512/rest/V1.0/list/Article/bySearch?
query=(characteristic('Ingredient') = 'WOOL' >> characteristic('SerialNumber') = 55)
AND ( characteristic('Ingredient') = 'LEATHER' >> characteristic('CountryOfOrigin') =
'ES')
```

We have these items with characteristic records:

Item Identifier	Characteristic	Lookup of Characteristic	Characteristic record value
Item1	Ingredient	Ingredients	Wool
Item1	CountryOfOrigin	OriginCountries	Spain
Item1	SerialNumber	-	55
Item2	Ingredient	Ingredients	Wool
Item2	CountryOfOrigin	OriginCountries	India
Item2	SerialNumber	-	55
Item2	Ingredient	Ingredients	Leather

Item Identifier	Characteristic	Lookup of Characteristic	Characteristic record value
Item2	CountryOfOrigin	OriginCountries	Spain
Item3	Ingredient	Ingredients	Wool
Item3	CountryOfOrigin	OriginCountries	Brazil
Item4	Ingredient	Ingredients	Leather
Item4	CountryOfOrigin	OriginCountries	Spain

The query above will return **Item2.**

4.8.3 Limitation

Currently it is not possible to search for the datatype MIME in search expressions.

5 REST List API

The REST List API (see page 55) provides searching, reporting, navigation and finally result listing of nearly all objects in the system. Based on a query a homogeneous list of objects of a certain type is returned. The queries are parameterized. The resulting list is a two dimensional table with a given set of columns to control the informational character of the result.

- Overview (see page 55)
- List-Based Mass Data Access (see page 55)
- Root Entity vs. Sub Entity Access (see page 55)
- Report vs. Search (see page 55)
- Report Services (see page 56)
- Search Services (see page 56)
- Examples (see page 56)

5.1 Overview

The List API provides list-based read, write and delete functionality optimized for fast transmission of large amounts of data. It also contains a meta API in order to access meta information dynamically, e.g. for listing all available root entities in the Product Manager's repository.

5.2 List-Based Mass Data Access

The REST List API is based on the idea of ListModels, which is one central aspect of the Product Manager's data access layer as described in Data Models. List-based data access enables to transfer only the data that is needed for processing the current request, ommitting unnessesary chunks. Data can be accessed in fast manner, since virtual list models in turn support paging of the result table.

5.3 Root Entity vs. Sub Entity Access

As described in Domain Model (Repository), the PIM Core data model consists of entities, which in turn consist of sub entities. Since sub entity instances are distiguished by their dimensions, logical keys need to be provided when navigating on sub entity level - e.g. we need to fully qualify the path to a sub entity field (provide all logical keys) when accessing the English item short description.

5.4 Report vs. Search

A Report is a predefined query with a fixed set of parameters and a fixed semantic. Usually the report has a name which reflects it's main purpose, like "byCatalog" or "byStatus". The actual report implementation might be implemented by a Stored Procedure or by some other logic which is totally transparent for the REST API user. Take a look at the Entity Reporting page to learn more about Reports and how to contribute you own reports to the Heiler Product Manager.

A Search has no predefined query and only a limited number of fixed parameters. It's semantic is defined by the search query the user must provide for the execution of the search. Heiler Product Manager provides a generic search module which can be used to build simple and complex queries. Take a look at the Entity Search page to learn more about the possibilities of the generic search engine and also it's limitations.

5.5 Report Services

Such a query can represent a **predefined report**, provided to the Product Manager by an entity report contribution which can simply be executed by providing usually one or two parameters.

Because it is possible to extend the set of entity reports also through customizing, we cannot list the total number of entity reports which are available here. On different installations there might be a different set of entity reports. However, the rest API provides info resources which will list the available reports of a specific Product Manager instance. Using these info resources you are even able to create a generic UI for the end user in which he can pick the report he likes.

Samples:

- Show all items/products of the catalog(catalog, assortment)
- Show items/products classified by(structure system)
- Show items/products not classified by(structure system)
- Show items/products by status reached(status)
- Show items/products by status not reached(status)

5.6 Search Services

The search services are provided in the same way than the report services. The only specialty is the search query parameter which must follow the HSQ Syntax.

5.7 Examples

Additionally to the examples provided in this documentation, you can also use the ListAPI Postman collection: ListAPI.postman_collection.json

5.8 REST List API Info

The REST List API provides the possibility to get meta information about available reports and about the parameters of a specific report.

- List all entities with reports (see page 57)
  - Result (see page 57)
- List all available reports for an entity (see page 58)
  - Result (see page 58)
- List report specific parameters (see page 59)
  - Result (see page 60)
- Examples (see page 60)
  - List all entities with reports (see page 60)
  - Retrieving all reports for the entity Article (see page 61)
  - Retrieving all report specific parameters for the report byCatalog (see page 62)

5.8.1 List all entities with reports

Returns a list of all entities which have contributed reports

URL Pattern	`/list/info`
Method	`GET`
Parameters	No parameters are available
Media types	`text/html, application/json, application/xml`
Result	In case of media types *application/json* and *application/xml* a list of all entities which have contributed reports is returned. In case of media type text/html additionally some general information about the *REST List API* is returned.

5.8.1.1 Result

A list of entities is returned.

Properties of an `entities` element			
	Field	**Data type**	**Description**
	identifier	String	Entity identifier
	name	String	Language specific name
	description	String	Language specific description

5.8.2 List all available reports for an entity

Returns the list of all available reports for the specified entity.

URL Pattern	`/list/{entity-identifier}/info`
Method	`GET`
Parameters	No parameters are available
Media types	`text/html, application/json, application/xml`
Result	A list of all available reports for the specified entity is returned.

5.8.2.1 Result

A list of reports is returned. For each report the identifier, the name and the description is provided.

Properties of an `reports` element			
	Field	**Data type**	**Description**
	identifier	String	Report identifier
	name	String	Language specific name of the report
	description	String	Language specific description of the report

5.8.3 List report specific parameters

Returns a list of all report specific parameters.

URL Pattern	`/list/{entity-identifier}/{report-name}/info`
Method	`GET`
Parameters	No parameters are available
Media types	`text/html, application/json, application/xml`
Result	Returns a list of report specific parameters. If `text/html` is request, also the general, report independent parameters are returned.

5.8.3.1 Result

Properties of an `parameters` element

	Field	Data type	Description
	identifier	String	Parameter identifier
	name	String	Language specific name of the parameter
	description	String	Language specific description of the parameter
	dataType	String	
	entity	Objet	Proxy representing a repository entity. The proxy contains the properties `identifier`, `name` and `description.` Only set if data type is ENTITY_ITEM and an enumeration is not specified.
	enumeration	Object	Proxy representing an enumeration. The proxy contains the properties `identifier` and `name.`

5.8.4 Examples

5.8.4.1 List all entities with reports

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/list/info
```

The following JSON object is returned:

```
{"entities": [
    {
        "identifier": "Article",
        "name": "Item",
        "description": ""
    },
    {
        "identifier": "Structure",
        "name": "Structure",
        "description": ""
    },
    ...
]}
```

Rest Client Java Code

```
ListInfoRequest listInfoRequest = restClient.createListInfoRequest();
ReportInfos reportInfoList = listInfoRequest .getAllEntitiesWithReports();
```

5.8.4.2 Retrieving all reports for the entity Article

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/list/Article/info
```

The following JSON object is returned:

```
{"reports": [
    {
        "identifier": "byAssortment",
        "name": "Items in assortment",
        "description": "Determines all items in an assortment"
    },
    {
        "identifier": "byCatalog",
        "name": "Items from supplier catalog",
        "description": "Determines all items from a supplier catalog"
    },
    ...
]}
```

Rest Client Java Code

```
ListInfoRequest listInfoRequest = restClient.createListInfoRequest();
ReportInfos reportInfoList = listInfoRequest.getAllReports( "Article" );
```

5.8.4.3 Retrieving all report specific parameters for the report *byCatalog*

The report identifier has to be added (here "byCatalog" to request the items of a certain catalog):

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/list/Article/byCatalog/info
```

The following JSON object is returned:

```
{
    "parameters": [
        {
            "identifier": "catalog",
            "name": "Catalog",
            "description": "The catalog for which the items are to be determined",
            "mandatory": "false",
            "dataType": "ENTITY_ITEM",
            "entity": {},
            "enumeration": {
                "name": "Supplier catalogs and master catalog",
                "identifier": "Enum.SupplierCatalogsWithMaster"
            }
        }
    ]
}
```

Rest Client Java Code

```
ListInfoRequest listInfoRequest = restClient.createListInfoRequest();
ReportInfos reportInfoList = listInfoRequest..getReportParameters( "Article", "byCatalog" );
```

5.9 REST List API Read

The List API (see page 55) provides table based read access on objects via reports contributed to the system. List API read access has to be distinguished between reading root and sub entities in terms of the URL pattern and the query parameters being passed for the request as well as the resulting table being returned.

- Caching (see page 63)
  - Use Caching with Paging (see page 63)
  - Disable Caching (see page 63)

5.9.1 Caching

Table requests can be cached inside of the application server. This feature has been designed to improve the performance in situations in which you want to fetch not all entity items of a certain report/search at once, but in a paged manner. Without the caching the application server would re-execute the report/search every time you request the next page - which is a tremendous overhead.

5.9.1.1 Use Caching with Paging

When you intend to use the paging functionality with the `startIndex` and `pageSize` parameters, it's a good idea to also use the table caching functionality. Otherwise, the full table will be requested on server side every time you request the next page. This would lead to a big overhead and will slow down the client application, and, probably also the Product 360 server.

To use the caching you need to provide a cache id. This is an arbitrary alphanumeric string which you can define however you like. It's main purpose is to distinguish the request for a specific table from one client to the other. If you have multiple client connections which are able to share the same table, you can also define the same `cacheId`. It's up to the configuration of the caching in the application server how many tables and for how long they are stored in memory.

From a service API standpoint, it doesn't matter. The caching functionality is transparent to the caller. This means, in case the table has been purged from the cache in between two calls from the client, the table will be reloaded transparently.

5.9.1.2 Disable Caching

If you specify `cacheId=no-cache`, the caching functionality is disabled for this call, and the table will not be cached. Is is strongly recommended to provide `no-cache` every time you do not intend to reuse the table!

Please note that in version 1 of the Service API the default behavior of the `cacheId` parameter is not optimal. In fact, if you omit the `cacheId`, the table **will** be added to the cache and an arbitrary `cacheId` will be returned from the server.
In future versions of the Service API this behavior will be changed so that omitting the `cacheId` will not add the table to the cache.

Therefor we strongly recommend to always use `cacheId=no-cache` unless you really need the table cache. Otherwise a serious memory issue might be imposed on the application server, depending on how the list model cache is actually configured. (By default it's set to 1000 list models, eternal in memory lifetime!)

5.9.2 REST List API Read for Root Entities

In general, most common list-based read access will be utilized on root entity level, e.g. Products or StructureGroups, whereas fields can be requested on demand and need to be fully qualified (see REST Field Qualification (see page 31)) in case they belong to a sub entity of the requested root entity.

- Executing a specific report (see page 65)
  - Parameters (see page 66)
  - Result (see page 68)
  - Examples (see page 72)
  - Executing the report byItems with POST (see page 72)
  - Executing the report byCatalog for the catalog TOOLS (see page 73)
  - Executing the report byCatalog for the catalog TOOLS and specifying a list of fields (parameter formatData is false) (see page 74)
  - Executing the report byCatalog for the catalog TOOLS and specifying a list of fields (parameter formatData is true) (see page 75)
  - Searching for root entity objects (see page 77)
- Execute a specific report for root entities for MIME values (see page 78)
  - Examples for MIME values (see page 79)
  - Rest Java Client Example for MIME values (see page 79)

5.9.2.1 Executing a specific report

URL Pattern	`/list/{entity-identifier}/{report-name}`
Method	`GET or Post`
Parameters	Common list parameters are supported. Additional parameters of an entity report are report specific and can be obtained from the report service itself.
Accept Header	`application/json, application/xml`
Content-Type (**for POST only**)	`application/x-www-form-urlencoded` `All parameters besides the ones from the URL must be provided as form parameters`

Result	A REST List API Read for Root Entities (see page 65) which is optimized for fast transmitting and big item counts

Parameters

Beneath the query dependent parameters there are general parameters which can be passed to control the number of rows or the the specific columns which should be included in the query's result

General list query parameters				
Parameter	**Required**	**Default**	**Datatype**	**Parameter description**
`startIndex`	no	0	Integer	0-based index within the total list of results. The result list provided for that request will start from that index (paging the results). An integer value in the range between 0 and *totalSize*-1.
`pageSize`	no	100	Integer	Number of entries which should be provided with that request (paging the results). The list starts a the given *start index* (see above) and leads to the end index (*startIndex* + *pageSize)*. If the results *totalSize* is less than the requested *pageSize* or if *totalSize* - *startIndex* is less than *pageSize* the number of entries provided is less that the requested *pageSize*. You can use `-1` to obtain all items in one request - please note that this might cause serious performance problems on the client side, since the number of items can be quite large.
`fields` (see page 31)	no		String	Comma-separated list of fields to be provided as result. Please see the REST Field Qualification (see page 31) as well as REST Transition Fields (see page 36) for details on the syntax of this parameter.
`metaData`	no	false	Boolean	A Boolean parameter. If set to *true* the meta data of the requested fields (given in the parameter *fields* see above) are provided. As part of the meta data the language dependent name of the data fields is provided. The locale used by the HTTP request (see the *Accept-Language* http request-header field) is used to identify the language requested. If the locale can not be resolved or no language dependent name is available for the given locale the field's unique name is provided at this place.

General list query parameters				
`formatData`	no	false	Boolean	A Boolean parameter. If set to *true* the values having a locale/language dependent formatting (like date/time, numeric, etc.) returned are formatted corresponding the given locale used by the HTTP request (see the *Accept-Language* http request-header field). Also fields with an enumeration will be returned with the label of the enumeration's entry instead of it's key. In case the parameter is omitted or set to false, the data will be formatted in an easy to parse, locale neutral format. See the data type representation section for details.
`includeLabels (see page 28)`	no	true	Boolean	A boolean parameter. If set to true, columns of the data type ENTITY_ITEM also contain the human readable label of the entity item. For compatibility reasons this parameter is by default `true` for performance reasons clients should set it to `false`.
`orderBy`	no	0-ASC	String	The field(s) used to sort the result set can be specified by the this parameter. A 0-based index referring to the list of fields as given in the parameter *fields* has to be provided plus the ordering direction given by *ASC* (order ascending) or *DESC* (order descending). The syntax is as follows: `n-ASC/DESC` where *n* is the 0-based column index and either `ASC` or `DESC` has to be provided ("1-ASC" defines to sort ascending by the **second** field provided in the parameter fields). Multiple columns for ordering can be provided as a comma separated list ("1-ASC,0-DESC" sort by the second column ascending , then descending by the first column). **If not necessary for the client to process the rows in a sorted way, he should not provide this parameter. If not given the order of the rows is not defined - even if they look sorted. This might just be a coincidence because of internal processing algorithms. You can only rely on a sort order if the orderBy parameter is provided.**
`assortmentFilter`	no	none	EntityItem	An entity item representation of an assortment. The assortment's item entity must match with the entity of the items the query returns. Additionally to that, also the item parent must match. For example, it's no allowed to query all items of a Catalog "Henri" and specifying an assortment which is bound to the Mastercatalog. You will receive an appropriate error message in case the assortment does not fit.

General list query parameters				
`updateAssortment`	no	false	Boolean	This parameter works only in combination with the `assortment` parameter. If set to `true` the given assortment will be updated before it is used in the query. Please note that updating an assortment might be a performance intensive task, so take care when using this parameter and think about if you really need to have the assortment evaluated every time.
`cacheId`	no	-	String	See Caching (see page 63) for details
`softDeleteMode`	no	ALIVE_ONLY	String	The mode for loading soft deleted objects. Possible values: ALIVE_ONLY - Only include non-deleted objects in the result list. This is the default mode. DEAD_OR_ALIVE - Include both non-deleted and soft deleted objects into the result list. In terms of sub entity records, always the "newest" value is provided, meaning that if a non-deleted object for the current qualification exists, this one will be returned, and if only deleted objects exist, the one with the latest deletion time stamp will be returned, respectively.
`revision`	no	-	ENTITY_ITEM	The revision entity item for which the data should be retrieved

Result

The result of all List API service is always a list model. A list model is an object type independent data structure representing a list of results of a query. Basically it contains a list of list entries (**rows**). The list header provides general information on the number and type of list entries. Optionally the requested data fields' meta information can be provided (**columns**).

Each list entry consists of identifying fields which allow to identify the object behind the list entry uniquely - the navigable. Additionally each list entry might contain a list of object type specific data fields. These fields can be requested flexibly by passing a list of unique field identifiers. Optionally the requested fields' meta information can be provided.

```
{
  "cacheId": "20130403_164441_0",
  "entityIdentifier": "Article",
  "totalSize": 88640,
  "startIndex": 0,
```

```
  "pageSize": 100,
  "rowCount": 100,
  "columnCount": 2,
  "columns": [
  ...
  ],
  "rows": [
  ...
  ]
}
```

Header

The list header always contains the information about the **total number** of results. The result list is provided paged. The **start index** and the **number of entries** within the provided list model page are given in the list header information. The start index and the number of entries to be provided can be controlled by request parameters of the query (**startIndex** and **pageSize**). The actual number of returned list entries and colums within one request are stated with **rowCount** and **columnCount**. Paging of a result table in the web front-end or a virtual table functionality can be realized by this. The **entity type** of the entries of the result list is given as well generally in the list's header information.

List header	
Field	**Description**
cacheId	The id of the used cache.
entityIdentifier	The entity of the HPM repository the list model contains entries of.
totalSize	The total number of results of the query.
startIndex	The 0-based index of first entry of the currently provided list model page within the whole result set.
pageSize	The number of requested entries per list model page.
rowCount	Number of entries contained in the currently provided list model page (can be smaller than requested page size).

List header	
columnCount	Number of colums contained in the returned list model page (as requested by the **fields** query parameter).

Columns

The data fields' meta information for client side presentation and validation can be provided with the search result optionally. The meta data ("columns") is added to the resulting list model in case the query passes the parameter **metaData** with **true**.

The language dependent name as defined in the repository server side is provided. The language requested is identified by the locale provided with the HTTP request-header field *Accept-Language*. In case the requested language is not available a fallback the the repositories default language is implemented.

The columns are provided in the same sequence as requested by the parameter "fields" (see above) and identified by the unique field identifier the column stands for. Additionally the field's data type is provided. In future further information might be added for enhanced client side validations of the fields' content (typical information could be the minimum/maximum string length, minimum/maximum value of a numeric field, etc.).

```
"columns": [
{
  "identifier": "Article.SupplierAID",
  "dataType": "STRING",
  "name": "Item No."
},
...
]
```

Column	
Field	**Description**
identifier	The unique field identifiers including qualification.
datatype	The data type of the field.

Column	
name	The language dependent display name of the field. The language used is controlled by the HTTP request-header field *Accept-Language* when the parameter *formatData* is provided with *true*.

Rows

Each record in the resulting list is represented by a **list row**. Each row consists of a set of identifying fields, the so called navigable, which allow to identify the object behind the entry uniquely.

Additionally each entry contains a list of **values** provided for the data fields requested by the parameter "fields". The sequence of the values corresponds to the sequence the data fields in the parameter "fields" were requested.

The number of rows provided as a result for the certain query is controlled by the request parameter **pageSize**. The index within the whole result set of the first entry in the list provided is controlled by the request parameter **startIndex**.

```
"entries": [
{
  "object":
     {
       "id": "3264@1",
       "label": "IT-12345"
     },
  "values": [
    "IT-12345",
    "Hübscher brauner Schuh",
    "Nice brown shoe",
    "Atlas",
    "1.00"
  ]
},
...
]
```

<table>
<tr><th colspan="2">Entry</th></tr>
<tr><th>Field</th><th>Description</th></tr>
<tr><td>object</td><td>Entity Item Reference to the corresponding object within the PIM system.</td></tr>
<tr><td>values</td><td>A list of values provided per entry for the data fields requested by the parameter "fields" (a comma separated list of unique field identifiers including their qualifications). The sequence of the values corresponds to the sequence of the data fields in the parameter "fields" were requested. If no fields were specified the values element is an empty array.

The value is usually represented as a String except for fields with data type ENTITY_ITEM. In this case, the value is a JSON object with the properties <code>id</code> and <code>label</code> if the field has an upper-bound of 0 or 1. If the upper bound is greater than 1, the value is a JSON array of JSON objects with the properties <code>id</code> and <code>label.</code>
<table>
<tr><th>Data type</th><th>Upper bound</th><th>JSON type</th></tr>
<tr><td>ENTITY_ITEM</td><td>>1</td><td>Array of ProxyObjects (properties: <code>id</code> and <code>label</code>)</td></tr>
<tr><td>ENTITY_ITEM</td><td><=1</td><td>ProxyObject (properties: <code>id</code> and <code>label</code>)</td></tr>
<tr><td>not ENTITY_ITEM</td><td></td><td>String (delimiter for multiple values is semi-colon)</td></tr>
</table>
</td></tr>
</table>

Examples

Executing the report byItems with POST

The entity report "byItems" uses the ENTITY_ITEM Syntax of the service api. It's single mandatory parameter is "items". In case of this report, items means "entity items", not item as in product/item/variant. So this entity report is actually available for all entities.

```
curl --location --request POST 'http://localhost:1512/rest/V2.0/list/Article/byItems'
 \
--header 'Accept: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI=' \
```

```
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'items=11259@1,1257@1' \
--data-urlencode 'fields=Article.SupplierAID'
```

This example will return two rows, one for the Article entity item with the internal id 11259 of the master catalog (internal id=1), and one for the item with the internalId 1257, also from the master. The byItems entity report can handle thousands of entity items and is backed by a cache to resolve those items. You can also use the alternative entity item syntax with the identifiers. Like: 'MyItem'@'MASTER'.

Hint: Always use the byItems entity report instead of the bySearch in case you would just search for the identifier of the entity item. BySearch is NOT backed by an in memory cache, byItems is!

Executing the report *byCatalog* for the catalog *TOOLS*

The parameters of the report have to be added to the URL as query parameters:

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/list/Article/byCatalog?catalog=TOOLS
```

The following JSON object is returned:

```
{
    "cacheId": "20130403_164441_0",
    "entityIdentifier": "Article",
    "totalSize": 88640,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 100,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "86@1064",
                "label": "A1"
            },
            "values": []
        },
        ...
    ]
}
```

Rest Client Java Code

```
EntityItemReference catalog = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );

ReportQuery reportQuery = new ReportQuery( "byCatalog" );
reportQuery.addParameterValue( "catalog", catalog );

EntityItemTable resultList = restClient.createListReadRequest().getRootItems(
"Article", reportQuery );
```

Executing the report *byCatalog* for the catalog *TOOLS* and specifying a list of fields (parameter *formatData* is false)

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/list/Article/byCatalog?
catalog=TOOLS&fields=Article.SupplierAID,Article.ManufacturerName,Article.MainSupplie
r,Article.QuantityMin
&formatData=false&metaData=true&startIndex=0&pageSize=50&orderedBy=0-ASC
```

The following JSON object is returned:


```
{
    "cacheId": "20130403_164441_0",
    "entityIdentifier": "Article",
    "totalSize": 88640,
    "startIndex": 0,
    "pageSize": 50,
    "rowCount": 50,
    "columnCount": 4,
    "columns": [
        {
            "identifier": "Article.SupplierAID",
            "dataType": "STRING",
            "name": "Item no."
        },
        {
            "identifier": "Article.ManufacturerName",
            "dataType": "STRING",
            "name": "Manufacturer"
        },
        {
            "identifier": "Article.MainSupplier",
            "dataType": "ENTITY_ITEM",
            "name": "Main supplier"
        },
        {
            "identifier": "Article.QuantityMin",
            "dataType": "DECIMAL",
```

```
            "name": "Minimum order quantity"
        }
    ],
    "rows": [
        {
            "object": {
                "id": "1304@1",
                "label": "A1"
            },
            "values": [
                "A1",
                "Test Supplier 1",
                {
                    "id": "3",
                    "label": "Heiler Product Manager"
                },
                "1"
            ]
        },
        ...
    ]
}
```

Rest Client Java Code

```
EntityItemReference catalog = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );

ReportQuery reportQuery = new ReportQuery( "byCatalog" );
reportQuery.addParameterValue( "catalog", catalog );
EntityItemTable resultList = restClient.createListReadRequest()
                                                .setFields( "Article.SupplierAID",
"Article.ManufacturerName", "Article.MainSupplier", "Article.QuantityMin" )
                                                .setOrderBy( "0-DESC" )
                                                .setFormatData( false)
                                                .setMetaData( true )
                                                .setPageSize(50)
                                                .getRootItems( "Article", reportQuery );
```

Executing the report *byCatalog* for the catalog *TOOLS* and specifying a list of fields (parameter *formatData* is true)

Finally the additionally required parameters to control the result have to be added:

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/list/Article/byCatalog?
catalog=TOOLS&fields=Article.SupplierAID,Article.ManufacturerName,Article.MainSupplie
r,Article.QuantityMin
&formatData=true&metaData=true&startIndex=0&pageSize=50&orderedBy=0-ASC
```

The following JSON object is returned:


```
{
    "cacheId": "20130403_175512_0",
    "entityIdentifier": "Article",
    "totalSize": 88640,
    "startIndex": 0,
    "pageSize": 50,
    "rowCount": 50,
    "columnCount": 4,
    "columns": [
        {
            "identifier": "Article.SupplierAID",
            "dataType": "STRING",
            "name": "Item no."
        },
        {
            "identifier": "Article.ManufacturerName",
            "dataType": "STRING",
            "name": "Manufacturer"
        },
        {
            "identifier": "Article.MainSupplier",
            "dataType": "ENTITY_ITEM",
            "name": "Main supplier"
        },
        {
            "identifier": "Article.QuantityMin",
            "dataType": "DECIMAL",
            "name": "Minimum order quantity"
        }
    ],
    "entries": [
        {
            "object": {
                "id": "1304@1",
                "label": "A2"
            },
            "values": [
                "A2",
                "Test Supplier 1",
                {
                    "id": "3",
                    "label": "Heiler Product Manager"
                },
                "1.0000"
            ]
        }
    ]
}
```

Searching for root entity objects

In order to search for objects matching a specific criteria, a generic *bySearch* report is available for all root entities. The search criteria can be specified by providing a search expression in the *query* parameter. The search expression language to be used is documented in REST Search Query Language (see page 38). The *catalog* parameter can be used to specify in which catalog to look for (default is Master catalog).

Search for all items of the master catalog which contain "4711" in their GTIN number

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/list/Article/bySearch?catalog=MASTER&query=Article.EAN contains "4711"
```

Rest Client Java Code

```
EntityItemReference testCatalogRef = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );

ReportQuery reportQuery = new ReportQuery( "bySearch" );
reportQuery.addParameterValue( "query", "Article.EAN contains \"4711\"");

EntityItemTable resultList = restClient.createListReadRequest()
                                        .getRootItems( "Article", reportQuery );
```

Search for all items of the master catalog which are ordered in boxes, with German gross weight between 10 and 30 kg

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/list/Article/bySearch?query=Article.OrderUnit equals "Box" AND
ArticleLogistic.GrossWeight(Germany) >= 10.00 AND
ArticleLogistic.GrossWeight(Germany) <= 30.00
```

Search for tasks assigned to specific supplier

To request the list of tasks which are assigned to a supplier use following request:

Tasks bySupplier

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/list/Task/bySupplier?supplier=SUPPLIER_FOR_TASKS
```

NOTE: This report returns single (standard) tasks and workflow tasks. No task groups will be returned in the result. Only workflow tasks which have items (count > 0) will be returned

Parameters				
Parameter	**Required**	**Default**	**Datatype**	**Parameter description**
`supplier`	yes		Entity Item	Entity item representation of supplier (supplier proxy) for which the tasks should be provided. NOTE: supplier proxy should be represented using an internal ID or using supplier name (not identifier)
`includeCompleted`	no	false	Boolean	Indicates if completed tasks should be also provided in the result or not. Default value is false => completed tasks will be not provided in the result as default
`catalogs`	no		List of Entity Items	List with entity item presentations of catalogs (catalog proxies) which will restrict the list of provided in the result tasks. Optional patameter. If given, then only tasks which have content from requested supplier catalogs or empty tasks will be provided in the result. Useful e.g. for the Broker user.

5.9.2.2 Execute a specific report for root entities for MIME values

URL Pattern	`/list/{entity-identifier}/mimes/{report-name}`
Method	`GET and POST`
Parameters	The parameters `pageSize`, `startIndex`, `fields` (see page 31) and `qualificationFilter` are supported. Other List Read for sub entities parameters are ignored. Additional parameters of an entity report are report specific and can be obtained from the report service itself.
Media types	`application/octet-stream`

Result	A zip stream with MIME files including the folder structure from target system.

Examples for MIME values

Execute the report *byLookup* and retrieve the MIME files referenced by lookup "Apps" using GET

Rest Call

```
 curl -u rest:heiler -H "Accept: application/octet-stream" -X GET http://
localhost:1512/rest/V1.0/list/LookupValue/byLookup?
lookup=Apps&fields=LookupValue.MIMEValue
```

Execute the report *byLookup* and retrieve the MIME files referenced by LookupValue by specifying the field using POST

Rest Call

```
 curl -u rest:heiler -H "Accept: application/octet-stream" --data
"lookup=Apps&fields=LookupValue.MIMEValue" http://localhost:1512/rest/V1.0/list/
LookupValue/byLookup
```

Rest Java Client Example for MIME values

Following code retrieves first 100 MIME values including the empty ones from the lookup "Apps".

Usage example for the client API

```
//Create and login to the client...
RestClient restClient = new RestClient();


//Create and parameterize the report query you want to execute...
ReportQuery reportQuery = new ReportQuery( "byLookup" ).addParameterValue( "lookup",
"Apps" );


//Create and parameterize the list request
```

```
ListReadRequest mimeZipRequest = restClient.createListReadRequest();
InputStream mimeZipStream = mimeZipRequest.setPageSize( 100 )
                                          .setStartIndex( 0 )
                                          .setFields( "LookupValue.MIMEValue" )
                                          .getRootMIMEs( "LookupValue",
reportQuery );
```

5.9.3 REST List API Read for Sub Entities

Provides the ability to retrieve field content of a sub entity without providing a full qualification. An example is a list of all attribute names and values for a set of objects retrieved by a report.

- Execute a specific report for Read (see page 81)
  - Examples (see page 85)
- Execute a specific report of sub entities for MIME values (see page 95)
  - Parameters (see page 95)
  - Examples (see page 96)
  - Rest Java Client Example (see page 97)

5.9.3.1 Execute a specific report for Read

URL Pattern	`/list/{entity-identifier}/{sub-entity-identifier}/{report-name}`
Method	`GET or POST`
Parameters	Common list parameters are supported. Additional parameters of an entity report are report specific and can be obtained from the report service itself. At least one parameter has to be specified, otherwise the list of parameters for this report is returned.
Accept	`application/json, application/xml`
Content Type (only for POST)	x-www-form-urlencoded
Result	A list model which is optimized for fast transmitting and big item counts

Parameters

Beneath the query dependent parameters there are general parameters which have to be passed to control how the query's result has to be provided. These parameters are mostly the same as for root entities and therefore linked to the previous page.

General list query parameters					
	Parameter	**Required**	**Default**	**Datatype**	**Parameter description**
	`startIndex`	no	0	Integer	see REST List API Read for Root Entities (see page 66)
	`pageSize`	no	100	Integer	see REST List API Read for Root Entities (see page 66)
	`fields` (see page 31)	no		String	see REST List API Read for Root Entities (see page 66)
	`metaData`	no	false	Boolean	see REST List API Read for Root Entities (see page 66)
	`formatData`	no	false	Boolean	see REST List API Read for Root Entities (see page 66)

General list query parameters					
	`orderBy`	no	0-ASC	String	The field(s) used to sort the result set can be specified by the this parameter. A 0-based index referring to the list of fields as given in the parameter *fields* has to be provided plus the ordering direction given by *ASC* (order ascending) or *DESC* (order descending). The syntax is as follows: `n-ASC/DESC` where *n* is the 0-based column index and either `ASC` or `DESC` has to be provided ("1-ASC" defines to sort ascending by the **second** field provided in the parameter fields). Multiple columns for ordering can be provided as a comma separated list ("1-ASC,0-DESC" sort by the second column ascending , then descending by the first column). **In case of a sub-entities, rows which belong to the same root item are always returned in a group of rows. There will be all rows for ItemA, followed by all rows for ItemD, followed by all rows for ItemB. The order of those groups is not defined, but it is guaranteed that rows belonging to the same root item are together. So when you loop over this table, you can be sure that this item is not coming again as soon as the entity item reference of the row differs to the one of the previous row (aka if the group changes).** **If an orderBy parameter is given, this basic principle for sub-entities does not change! The sorting will only be applied WITHIN the groups of rows for the same entity item!** **If not necessary for the client to process the rows in those groups in a sorted way, he should not provide this parameter.**
	`qualificationFilter`	no		String	This parameter allows to restrict the output to certain qualifications. An example would a filter so that only Euro and Dollar prices for the customer Heiler are returned. The filter string is a comma-separated list of qualification settings. A qualification setting is a *qualification name* followed by a comma-separated list of values which is put into parentheses . The qualification name is defined in the repository and is included in the Meta-API. Example for prices in Euro and US-Dollar and customer Heiler: `qualificationFilter=currency(EUR,USD),customer(Heiler)`

General list query parameters					

	`fieldFilter` (since 10.1.0.02)	no		String	This parameter allows to restrict the output based on certain field filters. An example would be a filter so that only ArticleLang records are returned which have a certain keyword, or no keyword or any keyword.
					
					The filter string is a comma-separated list of filter settings. A filter setting is a *field identifier* followed by a comma-separated list of values which is put into parentheses.
					
					All REST Datatypes (see page 25) are supported for the parameters, except MIME_VALUE.
					
					Syntax: `fieldFilter= [not] <FieldIdentifier1>( [Parameter1], ..., [ParameterN]),[not] <FieldIdentifierN>([Parameter1], ..., [ParameterN])`
					
					Multiple values for a single filter are interpreted as OR, so the row is returned if any of the given values equals the field value of the row. In case multiple field filters are defined, all of them must be met for the row to be returned. Multiple field filters are treated as AND. The value of the fieldFilter must match to the field's datatype. Fields with enumerations can also use any of the synonym strings of the enumeration's values.
					
					Example for ArticleLang with "MyMarker" as keyword:
					`fieldFilter=ArticleLang.Keyword("MyMarker")`
					
					Example for ArticleLang which has **no** "MyMarker as keyword
					
					`fieldFilter=not ArticleLang.Keyword("MyMarker")`
					
					Example for ArticleLang which has **no** keyword
					
					`fieldFilter=ArticleLang.Keyword()`
					
					Example for ArticleLang which has **any** keyword
					
					`fieldFilter=not ArticleLang.Keyword()`
					
					Please also find more examples on this parameter in the ListAPI postman collection.
					See the Examples section of REST List API (see page 55).

General list query parameters					
	`revision`	no	-	ENTITY_ITEM	The revision entity item for which the data should be retrieved

Examples

Executing the report *byCatalog* and retrieve the values for short description and long description in all languages

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/list/Article/ArticleLang/byCatalog?fields=ArticleLang.DescriptionShort,ArticleLang.DescriptionLong
```

The following JSON object is returned:


```
{
    "cacheId": "20130404_115132_0",
    "entityIdentifier": "ArticleLang",
    "totalSize": 83,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 83,
    "columnCount": 2,
    "columns": [
        {
            "identifier": "ArticleLang.DescriptionShort",
            "dataType": "STRING",
            "name": "Short description (Language selectable)"
        },
        {
            "identifier": "ArticleLang.DescriptionLong",
            "dataType": "STRING",
            "name": "Long description (Language selectable)"
        }
    ],
    "rows": [
        {
            "object": {
                "id": "1304@1",
                "label": "A1"
            },
            "qualification": {
```

```
                "language": "German"
            },
            "values": [
                "Kurzbeschreibung A1",
                "Detailbeschreibung A1"
            ]
        },
        {
            "object": {
                "id": "1304@1",
                "label": "A1"
            },
            "qualification": {
                "language": "English"
            },
            "values": [
                "Short description A1",
                "Long description A1"
            ]
        },
        {
            "object": {
                "id": "1315@1",
                "label": "A2"
            },
            "qualification": {
                "language": "German"
            },
            "values": [
                "Kurzbeschreibung A2",
                "Detailbeschreibung A2"
            ]
        },
        {
            "object": {
                "id": "1315@1",
                "label": "A2"
            },
            "qualification": {
                "language": "English"
            },
            "values": [
                "Short description A2",
                "Long description A2"
            ]
        },
        ...
    ]
}
```

Rest Client Java Code

```
ReportQuery reportQuery = new ReportQuery( "byCatalog" );
EntityItemTable resultList = restClient.createListReadRequest()
                                                .setFields( "ArticleLang.DescriptionShort
", "ArticleLang.DescriptionLong" )
                                                .getSubItems( "Article", "ArticleLang",
reportQuery );
```

Executing the report *byCatalog* for the *MASTER* catalog and retrieve the values for attribute name, attribute datatype and attribute value

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/list/Article/ArticleAttribute/byCatalog?
fields=ArticleAttributeLang.Name,ArticleAttribute.Datatype,ArticleAttributeValue.Valu
e&metaData=true
```

The following JSON object is returned:


```
{
    "cacheId": "20130404_134526_0",
    "entityIdentifier": "ArticleAttribute",
    "totalSize": 88640,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 700,
    "columnCount": 3,
    "columns": [
        {
            "identifier": "ArticleAttributeLang.Name",
            "dataType": "STRING",
            "name": "Attribute name (Name selectable, Language selectable)"
        },
        {
            "identifier": "ArticleAttribute.Datatype",
            "dataType": "INTEGER",
            "name": "Attribute data type (Name selectable)"
        },
        {
            "identifier": "ArticleAttributeValue.Value",
            "dataType": "STRING",
            "name": "Attribute value (Name selectable, Language selectable,
Identifier selectable)"
```

```
        }
    ],
    "rows": [
        {
            "object": {
                "id": "1143@1",
                "label": "A1"
            },
            "qualification": {
                "name": "Farbe",
                "language": "German"
            },
            "values": [
                "Farbe",
                "String",
                "rot"
            ]
        },
        {
            "object": {
                "id": "1143@1",
                "label": "A1"
            },
            "qualification": {
                "name": "Größe",
                "language": "German"
            },
            "values": [
                "Größe",
                "Number",
                "43"
            ]
        },

        ...

    ]
}
```


Rest Client Java Code

```
ReportQuery reportQuery = new ReportQuery( "byCatalog" );

EntityItemTable resultList = restClient.createListReadRequest()
                                       .setFields( "ArticleAttributeLang.Name",
"ArticleAttribute.Datatype", "ArticleAttributeValue.Value" )
                                       .setMetaData( true )
                                       .getSubItems( "Article",
"ArticleAttribute", reportQuery );
```

Executing the report byItems with POST

The entity report "byItems" uses the ENTITY_ITEM Syntax of the service api. It's single mandatory parameter is "items". In case of this report, items means "entity items", not item as in product/item/variant. So this entity report is actually available for all entities. The byItems entity report can handle thousands of entity items and is backed by a cache to resolve those items. You can also use the alternative entity item syntax with the identifiers. Like: 'MyItem'@'MASTER'.

Hint: Always use the byItems entity report instead of the bySearch in case you would just search for the identifier of the entity item. BySearch is NOT backed by an in memory cache, byItems is!

```
curl --location --request POST 'http://localhost:1512/rest/V2.0/list/Article/
ArticleLang/byItems' \
--header 'Accept: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI=' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'items=11259@1,1257@1' \
--data-urlencode 'fields=ArticleLang.DescriptionShort'
```

This will return the short description field in all languages for the items with the internal id's 11259 and 1257 from the master catalog (= 1).


```
{
    "cacheId": "no-cache",
    "entityIdentifier": "ArticleLang",
    "totalSize": 2,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 2,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "1257@1",
                "entityId": 1000
            },
            "qualification": {
                "language": "9"
            },
            "values": [
                "humm"
            ]
        },
        {
            "object": {
                "id": "11259@1",
                "entityId": 1000
            },
            "qualification": {
```

```
                "language": "9"
            },
            "values": [
                "dummy"
            ]
        }
    ]
}
```

Executing the report *byItems* for an item in SampleCatalog and retrieve the values for all characteristics

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/Article/ArticleCharacteristicValue/byItems?items='AIW_6382437688'@'SampleCatalog'&fields=ArticleCharacteristicValue.Characteristic
```

The following JSON object is returned:


```
{
    "cacheId": "20181109_113416_0",
    "entityIdentifier": "ArticleCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 7,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "214@1200",
                "label": "AIW_6382437682",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "72f315fa2e73d20c:3964b945:16050a4147b:-7af4",
                "rootCharacteristic": {
                    "id": "78",
                    "label": "Sustainability Initiative [Sustainability]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "characteristic": {
```

```
                    "id": "78",
                    "label": "Sustainability Initiative [Sustainability]",
                    "entityId": 8000
                }
            },
            "values": [
                {
                    "id": "78",
                    "label": "Sustainability Initiative [Sustainability]",
                    "entityId": 8000
                }
            ]
        },
        {
            "object": {
                "id": "214@1200",
                "label": "AIW_6382437682",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "72f315fa2e73d20c:3964b945:16050a4147b:-7a03",
                "rootCharacteristic": {
                    "id": "78",
                    "label": "Sustainability Initiative [Sustainability]",
                    "entityId": 8000
                },
                "parentRecordKey": "72f315fa2e73d20c:3964b945:16050a4147b:-7af4",
                "characteristic": {
                    "id": "79",
                    "label": "Eco Footprint [Sustainability.EcoFootprint]",
                    "entityId": 8000
                }
            },
            "values": [
                {
                    "id": "79",
                    "label": "Eco Footprint [Sustainability.EcoFootprint]",
                    "entityId": 8000
                }
            ]
        }
        ...
    ]
}
```

Executing the report *byItems* for an item in SampleCatalog and retrieve the values for Order

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/Article/ArticleCharacteristicValue/byItems?items='AIW_6382437688'@'SampleCatalog'&fields=ArticleCharacteristicValue.Order
```

The following JSON object is returned:


```
{
    "cacheId": "20181109_113931_0",
    "entityIdentifier": "ArticleCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 7,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "214@1200",
                "label": "AIW_6382437682",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "72f315fa2e73d20c:3964b945:16050a4147b:-7af4",
                "rootCharacteristic": {
                    "id": "78",
                    "label": "Sustainability Initiative [Sustainability]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "characteristic": {
                    "id": "78",
                    "label": "Sustainability Initiative [Sustainability]",
                    "entityId": 8000
                }
            },
            "values": [
                "-32767"
            ]
        },
        {
            "object": {
                "id": "214@1200",
```

```
                "label": "AIW_6382437682",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "72f315fa2e73d20c:3964b945:16050a4147b:-7a03",
                "rootCharacteristic": {
                    "id": "78",
                    "label": "Sustainability Initiative [Sustainability]",
                    "entityId": 8000
                },
                "parentRecordKey": "72f315fa2e73d20c:3964b945:16050a4147b:-7af4",
                "characteristic": {
                    "id": "79",
                    "label": "Eco Footprint [Sustainability.EcoFootprint]",
                    "entityId": 8000
                }
            },
            "values": [
                "-32756"
            ]
        }
        ...
    ]
}
```


Executing the report *byItems* for an item in SampleCatalog and retrieve the values for characteristics language specific data

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/Article/ArticleCharacteristicValue/byItems?items='AIW_6382437688'@'SampleCatalog'&fields=ArticleCharacteristicValueLang.Language
```

The following JSON object is returned:


```
{
    "cacheId": "20181030_121849_0",
    "entityIdentifier": "ArticleCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 14,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
```

```
            "object": {
                "id": "211@1200",
                "label": "AIW_6382437688",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "6bfe197e8d5135c4:-1dd34fd6:1604f0fd364:-7ef1",
                "rootCharacteristic": {
                    "id": "88",
                    "label": "Return Policy [ReturnPolicy]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "Language independent",
                "characteristic": {
                    "id": "88",
                    "label": "Return Policy [ReturnPolicy]",
                    "entityId": 8000
                }
            },
            "values": [
                "Language independent"
            ]
        },

        ...

    ]
}
```


Example with Field Filter

This example requests all ArticleLang sub entity records of all items of the "MySpplierCatalog" catalog and returns the Short Description and the Keyword fields. Only records which have "communicativeness" as a keyword will be returned.

```
curl --location --request GET 'http://localhost:1512/rest/V1.0/list/Article/
ArticleLang/byCatalog?
fields=ArticleLang.DescriptionShort,ArticleLang.Keyword&fieldFilter=ArticleLang.Keywo
rd(%22communicativeness%22)&catalog=%27MySupplierCatalog%27' \
--header 'Accept: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI='
```

Please find additional examples for the field filters in the ListApi postman collection.

5.9.3.2 Execute a specific report of sub entities for MIME values

URL Pattern	`/list/{entity-identifier}/{sub-entity-identifier}/mimes/{report-name}`
Method	`GET and POST`
Parameters	The parameters `pageSize`, `startIndex`, `fields` (see page 31) and `qualificationFilter` are supported. Other List Read for sub entities parameters are ignored. Additional parameters of an entity report are report specific and can be obtained from the report service itself.
Media types	`application/octet-stream`
Result	A zip stream with MIME files including the folder structure from target system.

Parameters

The parameters used here are same as that of root entities.

Parameters valid for Article → ArticleCharacteristicValue entities only				
includeUnavailable	no	false	Boolean	Allows to return MIME files for available AND unavailable characteristics. Default value is false and means that only available characteristics will be considered. Read more about Characteristic values in KnowledgeBase → Characteristics - Dynamic data model → Characteristic values

Note: Parameter fields (see page 31) is not used for Article → ArticleCharacteristicValue.

Examples

Execute the report *byCatalog* and retrieve the MIME files referenced by Article characteristic values for items of MASTER catalog using GET

Rest Call

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X GET http://
localhost:1512/rest/V2.0/list/Article/ArticleCharacteristicValue/mimes/byCatalog
```

Execute the report *bySearch* and retrieve the MIME files referenced by Article characteristic values for specific item using GET

Rest Call

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X GET
http://localhost:1512/rest/V2.0/list/Article/ArticleCharacteristicValue/mimes/
bySearch?query=Article.SupplierAID = "ITEM_WITH_CHARACTERISTIC"
```

Execute the report *byStructureGroup* and retrieve the MIME files referenced by Article characteristic values qualified by characteristic record key using GET

Rest Call

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X GET
http://localhost:1512/rest/V2.0/list/Article/ArticleCharacteristicValue/mimes/
byStructureGroup?
structureGroup='GROUP_1_2'@'HeilerStandard'&qualificationFilter=recordKey("c003c0da0c
a388f7:-2c5e83f8:1614649c74c:-7e9d")
```

Execute the report *byLookup* and retrieve the MIME files referenced by LookupValueLang by specifying the field using POST

Rest Call

```
curl -u rest:heiler -H "Accept: application/octet-stream" --data
"lookup=Apps&fields=LookupValueLang.MIMEValue" http://localhost:1512/rest/V1.0/list/
LookupValue/LookupValueLang/byLookup
```

Rest Java Client Example

You can find a short example on how to use the client below. The full example classes can be viewed as part of the SDK package in the folder `examples\integration`.

Example 1

Following code block is used for retrieving first 100 MIME values from *masterCatalog* for the Characteristics with identifiers "CharacteristicIdentifier1" and "CharacteristicIdentifier2".

Usage example for the client API

```
//Create and login to the client...
RestClient restClient = new RestClient();


//Create and parameterize the report query you want to execute...
EntityItemReference masterCatalog = EntityItemReferenceFactory.createByIdentifier(
"MASTER" );
ReportQuery reportQuery = new ReportQuery( "byCatalog" ).addParameterValue(
"catalog", masterCatalog );


//Create and parameterize the list request
ListReadRequest mimeZipRequest = restClient.createListReadRequest();
InputStream mimeZipStream = mimeZipRequest.setPageSize( 100 )
                                          .setStartIndex( 0 )
                                          .setQualificationFilters( "characteristic('
CharacteristicIdentifier1')",

"characteristic('CharacteristicIdentifier2')" )
                                          .getSubMIMEs( "Article",
"ArticleCharacteristicValue", reportQuery );
```

Example 2

Following code block retrieves the first 100 MIME values from the subentity LookupValueLang.MIME.

Usage example for the client API

```
//Create and login to the client...
RestClient restClient = new RestClient();


//Create and parameterize the report query you want to execute...
ReportQuery reportQuery = new ReportQuery( "byLookup" ).addParameterValue( "lookup",
"Apps" );


//Create and parameterize the list request
ListReadRequest mimeZipRequest = restClient.createListReadRequest();
InputStream mimeZipStream = mimeZipRequest.setPageSize( 100 )
                                          .setStartIndex( 0 )
                                          .setFields( "LookupValueLang.MIMEValue" )
                                          .getSubMIMEs( "LookupValue",
"LookupValueLang", reportQuery ))
```

5.9.4 REST List API Read MIME files of Root and Sub Entities

The REST List API Read MIME files of Sub Entities (see page 99) provides the ability to determine MIME files referenced by a sub entity and provide them as a zip stream. An example is a zip stream containing MIME files for a set of objects retrieved by a report.

- Root Entity (see page 99)
  - Execute a specific report for root entities (see page 99)
  - Parameters (see page 100)
  - Examples (see page 100)
  - Rest Java Client Example (see page 101)
- Sub Entity (see page 101)
  - Execute a specific report for sub root entities (see page 101)
  - Parameters (see page 102)
  - Additional parameters (see page 102)
  - Examples (see page 102)
  - Rest Java Client Example (see page 104)

5.9.4.1 **Root Entity**

Execute a specific report for root entities

URL Pattern	`/list/{entity-identifier}/mimes/{report-name}`
Method	`GET and POST`
Parameters	The parameters `pageSize`, `startIndex`, `fields` (see page 31) and `qualificationFilter` are supported. Other List Read for sub entities parameters are ignored. Additional parameters of an entity report are report specific and can be obtained from the report service itself.
Media types	`application/octet-stream`
Result	A zip stream with MIME files including the folder structure from target system.

Parameters

Beneath the query dependent parameters below are the general parameters which can be passed to control the query's result:

General list query parameters				
Parameter	**Required**	**Default**	**Datatype**	**Parameter description**
startIndex	no	0	Integer	see REST List API Read for Root Entities (see page 66)
pageSize	no	100	Integer	see REST List API Read for Root Entities (see page 66)
qualificationFilter	no		String	This parameter allows to restrict the output to certain qualifications. An example would be a filter so that only Euro and Dollar prices for the customer Heiler are returned. The filter string is a comma-separated list of qualification settings. A qualification setting is a *qualification name* followed by a comma-separated list of values which is put into parentheses The qualification name is defined in the repository and is included in the Meta-API. Example for prices in Euro and US-Dollar and customer Heiler: `qualificationFilter=currency(EUR,USD),customer(Heiler)`
fields (see page 31)	no		String	Comma-separated list of fields to be provided as result. Please see the REST Field Qualification (see page 31) as well as REST Transition Fields (see page 36) for details on the syntax of this parameter. It is usually used for specifying which field contains the mime value.

Examples

Execute the report *byLookup* and retrieve the MIME files referenced by lookup "Apps" using GET

curl -u rest:heiler -H "Accept: application/octet-stream" -X GET http://localhost:1512/rest/V1.0/list/LookupValue/byLookup?lookup=Apps&fields=LookupValue.MIMEValue

Execute the report *byLookup* and retrieve the MIME files referenced by LookupValue by specifying the field using POST

```
curl -u rest:heiler -H "Accept: application/octet-stream" --data
"lookup=Apps&fields=LookupValue.MIMEValue" http://localhost:1512/rest/V1.0/list/
LookupValue/byLookup
```

Rest Java Client Example

Following code retrieves first 100 MIME values including the empty ones from the lookup "Apps".

Usage example for the client API

```
//Create and login to the client...
RestClient restClient = new RestClient();


//Create and parameterize the report query you want to execute...
ReportQuery reportQuery = new ReportQuery( "byLookup" ).addParameterValue( "lookup",
"Apps" );


//Create and parameterize the list request
ListReadRequest mimeZipRequest = restClient.createListReadRequest();
InputStream mimeZipStream = mimeZipRequest.setPageSize( 100 )
                                          .setStartIndex( 0 )
                                          .setFields( "LookupValue.MIMEValue" )
                                          .getRootMIMEs( "LookupValue",
reportQuery );
```

5.9.4.2 Sub Entity

Execute a specific report for sub root entities

URL Pattern	`/list/{entity-identifier}/{sub-entity-identifier}/mimes/{report-name}`
Method	`GET and POST`

Parameters	The parameters `pageSize`, `startIndex`, `fields` (see page 31) and `qualificationFilter` are supported. Other List Read for sub entities parameters are ignored. Additional parameters of an entity report are report specific and can be obtained from the report service itself.
Media types	`application/octet-stream`
Result	A zip stream with MIME files including the folder structure from target system.

Parameters

The parameters used here are same as that of root entities described above.

Additional parameters

There is an additional optional parameter to use in the calls for Characteristic MIME values.

Additional parameters				
includeUnavailable	no	false	Boolean	Allows to return MIME files for available AND unavailable characteristics. Default value is false and means that only available characteristics will be considered.

Parameter fields (see page 31) is ignored in the calls for Characteristic MIME values.

Read more about Characteristic values in KnowledgeBase → Characteristics - Dynamic data model → Characteristic values.

Examples

Execute the report *byCatalog* and retrieve the MIME files referenced by Article characteristic values for items of MASTER catalog using GET

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X GET http://localhost:1512/rest/V2.0/list/Article/ArticleCharacteristicValue/mimes/byCatalog
```

Execute the report *bySearch* and retrieve the MIME files referenced by Article characteristic values for specific item using GET

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X GET
http://localhost:1512/rest/V2.0/list/Article/ArticleCharacteristicValue/mimes/
bySearch?query=Article.SupplierAID = "ITEM_WITH_CHARACTERISTIC"
```

Execute the report *bySearch* and retrieve the MIME files referenced by Variant characteristic values for specific variant using GET

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X GET
http://localhost:1512/rest/V2.0/list/Variant/VariantCharacteristicValue/mimes/
bySearch?query=Variant.VariantNo equals "VARIANT_WITH_CHARACTERISTIC"
```

Execute the report *byStructureGroup* and retrieve the MIME files referenced by Article characteristic values qualified by characteristic record key using GET

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X  GET
http://localhost:1512/rest/V2.0/list/Article/ArticleCharacteristicValue/mimes/
byStructureGroup?
structureGroup='GROUP_1_2'@'HeilerStandard'&qualificationFilter=recordKey("c003c
0da0ca388f7:-2c5e83f8:1614649c74c:-7e9d")
```

Execute the report *byStructureGroup* and retrieve the MIME files referenced by Product characteristic values qualified by root characteristic using GET

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X  GET
http://localhost:1512/rest/V2.0/list/Product2G/Product2GCharacteristicValue/
mimes/byStructureGroup?
structureGroup='GROUP_2_1'@'MIME_Structure'&qualificationFilter=rootCharacterist
ic('Root1')
```

Execute the report *byLookup* and retrieve the MIME files referenced by LookupValueLang by specifying the field using POST

```
curl -u rest:heiler -H "Accept: application/octet-stream" --data
"lookup=Apps&fields=LookupValueLang.MIMEValue" http://localhost:1512/rest/V1.0/
list/LookupValue/LookupValueLang/byLookup
```

Rest Java Client Example

You can find a short example on how to use the client below. The full example classes can be viewed as part of the SDK package in the folder `examples\integration`.

Example 1

Following code block is used for retrieving first 100 MIME values from *masterCatalog* for the Characteristics with identifiers "CharacteristicIdentifier1" and "CharacteristicIdentifier2".

Usage example for the client API

```
//Create and login to the client...
RestClient restClient = new RestClient();


//Create and parameterize the report query you want to execute...
EntityItemReference masterCatalog = EntityItemReferenceFactory.createByIdentifier(
"MASTER" );
ReportQuery reportQuery = new ReportQuery( "byCatalog" ).addParameterValue(
"catalog", masterCatalog );


//Create and parameterize the list request
ListReadRequest mimeZipRequest = restClient.createListReadRequest();
InputStream mimeZipStream = mimeZipRequest.setPageSize( 100 )
                                          .setStartIndex( 0 )
                                          .setQualificationFilters( "characteristic('
CharacteristicIdentifier1')",

"characteristic('CharacteristicIdentifier2')" )
                                          .getSubMIMEs( "Article",
"ArticleCharacteristicValue", reportQuery );
```

Example 2

Following code block retrieves the first 100 MIME values from the subentity LookupValueLang.MIME.

Usage example for the client API

```
//Create and login to the client...
RestClient restClient = new RestClient();
```

```
//Create and parameterize the report query you want to execute...
ReportQuery reportQuery = new ReportQuery( "byLookup" ).addParameterValue( "lookup",
"Apps" );


//Create and parameterize the list request
ListReadRequest mimeZipRequest = restClient.createListReadRequest();
InputStream mimeZipStream = mimeZipRequest.setPageSize( 100 )
                                          .setStartIndex( 0 )
                                          .setFields( "LookupValueLang.MIMEValue" )
                                          .getSubMIMEs( "LookupValue",
"LookupValueLang", reportQuery ))
```

5.10 REST List API Write

- REST List API Write for Root Entities (see page 107)
- REST List API Write for Sub Entities (see page 120)

5.10.1 REST List API Write for Root Entities

The REST List API provides the possibility to set the values of several qualified fields for a list of given objects. The request returns a protocol containing the updated resp. created objects and a list of errors and warnings. It is possible to create new objects by providing external identifiers instead of internal IDs. Processing is distributed over multiple threads to speed up processing.

- Write values into qualified fields (see page 107)
  - Parameters (see page 108)
  - Content (see page 108)
  - Result (see page 109)
- Examples (see page 111)
  - Setting non-formatted values (see page 111)
  - Setting formatted values (see page 114)
  - Setting MIMEValues for entity (see page 115)

5.10.1.1 Write values into qualified fields

Writes the given values into qualified fields for one or more specified objects. All objects have to be of the same type.

URL Pattern	/list/{entity-identifier}
Method	POST
Parameters	formatData
Content types	application/json
Media type	application/json
Result	A protocol containing some statistical information like the number of errors and warnings, a list of all errors and warnings and the created resp. updated objects.

Parameters

Available parameters

	Parameter	Required	Default	Datatype	Parameter description
	formatData	no	false	Boolean	If set to true, language specific formatted values are expected.
	includeObjectsInProtocol	no	true	Boolean	If set to true, a list of created and updated objects are included in the protocol. To improve performance, this value should be set to false if the list of objects is not needed. This option is available since HPM 7.0.04.
	recreate	no	false	Boolean	If set to true, a sub entity item is always newly created (currently, only QualityStatusEntry objects are supported)

Content

The expected content has the same format as the result of a read request (see also REST List API Read for Root Entities (see page 68)) but not all properties have to be filled. Not required properties are ignored. It is possible to use the output of a read request, modify some values and use the modified read output as content for a write request.

Required properties are:

- `columns` containing
  - an array of objects. Each object must contain the property `identifier` which contains the qualified field identifier
- `rows` containing
  - the object id. If an external identifier is provided and a corresponding object does not exist, a new object is created.
  - a list of new values for each object.
- mimeValueArchives containing
  - File Reference id and originalFileName which is generated while uploading File using File upload API.
  - Multiple FileReference id and originalFileName can be passed.
  - This property is introduced to MIMEValue field only.

Formatting of values

- The values have to be provided as strings and formatted as described in REST Datatypes (see page 25).
- If an empty string is specified as value, the content of the field is cleared. If `null` is specified as value, the content of the field is left unchanged.
- In case a field has the type `ENTITY_ITEM` either a string or an JSON object can be provided. In case a string is provided, the string is treated as label if the field has an enumeration, otherwise as external identifier. In case a JSON object is provided, only the property `id` is evaluated.

Result

A protocol is returned as result. The protocol consists of a list of counters (number of errors, warnings etc.), a list of errors and warnings (members of the field `entries`) and a list of created and updated objects (members of the field `objects`). The list of created and updated objects is only included if the option *includeObjectsInProtocol* is set to true (default).

Properties of a `counters` element

	Field	Data type	Description
	errors	Integer	Total number of errors
	warnings	Integer	Total number of warnings
	createdObjects	Integer	Number of created objects
	updatedObjects	Integer	Number of updated objects
	objectsWithErrors	Integer	Number of objects with errors
	objectsWithWarnings	Integer	Number of objects with warnings

Properties of an `entries` element

	Field	Data type	Description

	row	Integer	The index of the row causing this error or warning. The first row has the index 0.
	objectType	String	The type of the object (language dependent label is used). Might be empty.
	object	EntityItem	The object the error or warning occurred in. Might be empty.
	severity	String	ERROR, WARNING or INFO
	category	String	Category of the error. Possible values are: SYSTEM, UNIQUENESS, DATATYPE, CARDINALITY, RANGE, CONSISTENCY, SECURITY, NOTE, SUMMARY
	propertyLabel	String	The field causing this error or warning (language dependent label is used). Might be empty.
	message	String	The message describing the error or warning.
	logDate	Date	Date when the error or warning occurred.
	logTime	Time	Time when the error or warning occurred.

Properties of an `objects` element

	Field	Data type	Description
	row	Integer	The index of the row which is responsible for creating or updating this object. The first row has the index 0.
	object	EntityItem	A reference to the updated or created object. Contains the id and the label of the object.

Properties of an `objects` element			
	status	List of Strings	The status of the object. Possible values are: CREATED, UPDATED, ERROR, WARNING

5.10.1.2 Examples

Setting non-formatted values

Sets values in the fields *Article.ManufacturerName*, *Article.OrderUnit* and *ArticlePriceValueSales.Amount*

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1501/rest/V1.0/list/Article
```

The following JSON object is provided as content:

```
{
   "columns": [
        {
            "identifier": "Article.ManufacturerName"
        },
        {
            "identifier": "Article.OrderUnit"
        },
        {
            "identifier": "ArticlePriceValueSales.Amount(1,nrp,USD,US,2010-08-06,1.5)"
        }
   ],
   "rows": [
        {
            "object":
            {
                   "id": "'A1'@'MASTER'"
        },
            "values": [
                "Heiler Software AG",
                {
                    "id": "39"
                },
                "12.34"
            ]
        },
        {
```

```
            "object":
            {
                   "id": "'A2'@'MASTER'"
        },
            "values": [
                "Heiler Software AG",
                {
                    "label": "fifteen kg drum"
                },
                "NotANumber"
            ]
        },
        {
            "object":
            {
                   "id": "'A3'@'MASTER'"
        },
            "values": [
                "",
                { },
                ""
            ]
        },
        {
            "object":
            {
                   "id": "'A4'@'MASTER'"
        },
            "values": [
                null,
                null,
                null
            ]
        }

    ]
}

```

Remarks:

- If the items A1, A2, A3 and A4 exists already, they are updated. Otherwise the items are newly created.
- In case of item A2, an error occurs while setting the value. An error description is provided in the protocol.
- In case of item A3, the content of the fields is cleared .
- In case of item A4, the content of the fields is left unchanged .

The following protocol is returned :

```
{
    "counters": {
        "errors": 1,
        "warnings": 0,
```

```
        "createdObjects": 4,
        "updatedObjects": 0,
        "objectsWithErrors": 1,
        "objectsWithWarnings": 0
    },
    "entries": [
        {
            "row": 1,
            "objectType": "Item",
            "object": {
                "id": "120015@1",
                "label": "A2"
            },
            "severity": "ERROR",
            "category": "DATATYPE",
            "propertyLabel": "Price (from 1.0000)",
            "message": "Value 'NotANumber' could not be converted into type java.math.BigDecimal.",
            "logDate": "2013-04-04",
            "logTime": "11:59:46"
        }
    ],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "120014@1",
                "label": "A1"
            },
            "status": [
                "CREATED"
            ]
        },
        {
            "row": 1,
            "object": {
                "id": "120015@1",
                "label": "A2"
            },
            "status": [
                "CREATED",
                "ERROR"
            ]
        },
        {
            "row": 2,
            "object": {
                "id": "120013@1",
                "label": "A3"
            },
            "status": [
                "CREATED"
            ]
```

```
        },
        {
            "row": 3,
            "object": {
                "id": "120012@1",
                "label": "A4"
            },
            "status": [
                "CREATED"
            ]
        }
    ]
}
```

Setting formatted values

Sets the values which are provided in a German format

```
curl -u rest:heiler -H "Accept: application/json" -H "Accept-Language: de-DE" -X POST
http://localhost:1501/rest/V1.0/list/Article?formatData=true
```

The following JSON object is provided as content:

```
{
   "columns": [
        {
            "identifier": "Article.ManufacturerName"
        },
        {
            "identifier": "Article.OrderUnit"
        },
        {
            "identifier":
"ArticlePriceValueSales.Amount(1,nrp,USD,US,2010-08-06,1.5)"
        }
   ],
   "rows": [
        {
            "object":
                {
                   "id": "'A1'@'MASTER'"
                },
            "values": [
                "Heiler Software AG",
                {
                    "label": "15 kg-Fass"
                },
                "12,34"
            ]
```

```
        }
    ]
}
```

The returned protocol looks like:

```
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 1,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "120033@1",
                "label": "A1"
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```

Setting MIMEValues for entity

To set MIMEValue to perticular column, first we have to upload Zip file containing MIMEValue we want to attach using File API. While uploading File using File upload API, user will get id and originalFileName object. After that object has to set to 'mimeValueArchives' as shown in example. User cann't reuse Filerefernce of uploaded file.

```
curl -u rest:heiler -H "Accept: application/json" -H 'Content-Type: application/json'
 -X POST http://localhost:1512/rest/V1.0/list/LookupValue
```

The following JSON object is provided as content:

```
{
   "columns": [
        {
```

```
            "identifier": "LookupValue.MIMEValue"
        }
    ],
    "rows": [
        {
             "object": {
                "id":"31@27",
                "label": "Amazon",
                "entityId": 7300
            },
            "values":
                [
                    {
                    "relativeFilePath": "17/8/32/fishstick.jpg"
                    }
            ]
        },
        {
             "object": {
                "id":"29@27",
                "label": "Netflix",
                "entityId": 7300
            },
                        "values":
                [
                    {
                    "label": "img1.jpg",
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        },
                {
             "object": {
                "id": "32@27",
                "label": "Skype",
                "entityId": 7300
            },
                        "values":
                [
                    {
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        }
    ],
    "mimeValueArchives":[
{
     "id": "b1f9e95f-f61e-4139-a429-efa16fbf4035",
     "originalFilename": "uploadMime.zip"
}
     ]
```

```
}
```

The returned protocol looks like:

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 3,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "31@27",
                "label": "Amazon",
                "entityId": 7300
            },
            "status": [
                "UPDATED"
            ]
        },
        {
            "row": 1,
            "object": {
                "id": "29@27",
                "label": "Netflix",
                "entityId": 7300
            },
            "status": [
                "UPDATED"
            ]
        },
        {
            "row": 2,
            "object": {
                "id": "32@27",
                "label": "Skype",
                "entityId": 7300
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
```

```
}
```

Remarks:

- If the object Netflix exists already, they are updated. Otherwise the items are newly created with MIMEValues if MIMEValue reference is set in 'mimeValueArchives'.

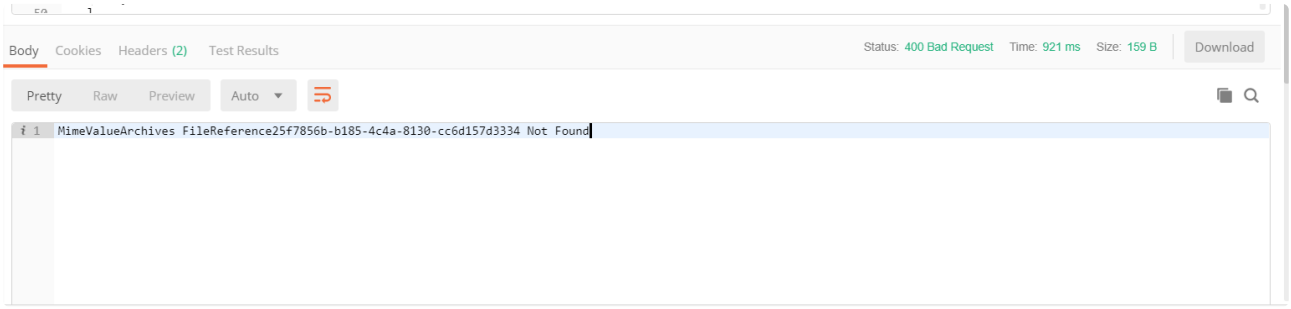
Example 2: If mimeValueArchives fileRefernces is invalid or used earlier then User will get "400: Bad Request" in Response. As shown in bellow:

```
{
   "columns": [
        {
            "identifier": "LookupValue.MIMEValue"
        }
   ],
   "rows": [
        {
             "object": {
                "id":"31@27",
                "label": "Amazon",
                "entityId": 7300
            },
            "values":
                [
                    {
                    "relativeFilePath": "17/8/32/fishstick.jpg"
                    }
            ]
        },
        {
             "object": {
                "id":"29@27",
                "label": "Netflix",
                "entityId": 7300
            },
                        "values":
                [
                    {
                    "label": "img1.jpg",
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        },
                {
             "object": {
                "id": "32@27",
                "label": "Skype",
```

```
                "entityId": 7300
            },
                    "values":
                [
                    {
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        }
    ],
    "mimeValueArchives":[
{
     "id": "25f7856b-b185-4c4a-8130-cc6d157d3334",
     "originalFilename": "uploadMime.zip"
}
     ]

}
```

The response will look like:



5.10.2 REST List API Write for Sub Entities

The REST List API provides the possibility to set the values of several fields of a specific sub entity for a list of given objects. The request returns a protocol containing the updated resp. created objects and a list of errors and warnings. It is possible to create new objects by providing external identifiers instead of internal IDs. Processing is distributed over multiple threads to speed up processing.

- Write values into fields of a certain sub entity (see page 120)
  - Parameters (see page 121)
  - Content (see page 121)
  - Result (see page 121)
- Examples (see page 122)
  - Setting values in sub entity ArticleLang (see page 122)
  - Setting values to subentity from root entity (see page 124)
  - Setting lookup value of an item (see page 125)
  - Setting MIMEValues for SubEntity (see page 127)

5.10.2.1 Write values into fields of a certain sub entity

Writes the given values into fields of a one specified sub entity for a list of provided objects. All objects have to be of the same type.

URL Pattern	/list/{entity-identifier}/{sub-entity-identifier}
Method	POST
Parameters	formatData
Content types	application/json
Media type	application/json
Result	A protocol containing some statistical information like the number of errors and warnings, a list of all errors and warnings and the created resp. updated objects.

Parameters

Available parameters					
	Parameter	**Required**	**Default**	**Datatype**	**Parameter description**
	formatData	no	false	Boolean	If set to true, language specific formatted values are expected.

Content

The expected content has the same format as the result of a read request (see also REST List API Read for Root Entities (see page 68)) but not all properties have to be filled. Not required properties are ignored. It is possible to use the output of a read request, modify some values and use the modified read output as content for a write request.

The expected content has the same format as the result of a report query, but not all properties have to be filled.

Required properties are

- `columns` containing
  - an array of objects. Each object must contain the property `identifier` which contains the qualified field identifier
- `rows` containing
  - the object `id.` If an external identifier is provided and a corresponding object does not exist, a new object is created.
  - the qualification of the sub entity
  - a list of new values for each object

Formatting of values

- The values have to be provided as strings and formatted as described in REST Datatypes (see page 25).
- If an empty string is specified as value, the content of the field is cleared. If `null` is specified as value, the content of the field is left unchanged.
- In case a field has the type `ENTITY_ITEM` either a string or an JSON object can be provided. In case a string is provided, the string is treated as label if the field has an enumeration, otherwise as external identifier. In case a JSON object is provided, only the property `id` is evaluated.

Result

A protocol is returned as result. The protocol consists of a list of counters (number of errors, warnings etc.), a list of single errors and warnings (in property `entries)` and a list of created and updated objects.

A detailed description can be found at REST List API Write for Root Entities (see page 109)

5.10.2.2 Examples

Setting values in sub entity ArticleLang

This example set values of the fields *ArticleLang.DescriptionShort* and *ArticleLang.Keyword*

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1501/rest/V1.0/list/Article/ArticleLang
```

The following JSON object is provided as content:

```
{
   "columns": [
        {
            "identifier": "ArticleLang.DescriptionShort"
        },
        {
            "identifier": "ArticleLang.Keyword"
        }
   ],
   "rows": [
        {
            "object":
            {
                   "id": "'A1'@'MASTER'"
            },
            "qualification" : { "language" : "en" },
            "values": [
                "An English short description",
                [ "EnglishKeyword1", "EnglishKeyword2" ]
            ]
        },
        {
            "object":
            {
                   "id": "'A1'@'MASTER'"
            },
            "qualification" : { "language" : "de" },
            "values": [
                "A German short description",
                ["GermanKeyword2", "GermanKeyword1" ]
            ]
        },
        {
            "object":
            {
                   "id": "'A2'@'MASTER'"
```

```
            },
            "qualification" : { "language" : "en" },
            "values": [
                "English short description for second item",
                null
            ]
        },
        {
            "object":
            {
                   "id": "'A3'@'MASTER'"
            },
            "qualification" : { "language" : "en" },
            "values": [
                "English short description for third item",
                []
            ]
        }
    ]
}
```

Remarks:

- In case of item A2, the content of the field *ArticleLang.Keyword* is cleared .
- In case of item A3, the content of the fields *ArticleLang.Keyword* is left unchanged .

The following protocol is returned :

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 3,
        "updatedObjects": 0,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "120044@1",
                "label": "A1"
            },
            "status": [
                "CREATED"
            ]
        },
        {
            "row": 2,
            "object": {
```

```
                "id": "120045@1",
                "label": "A2"
            },
            "status": [
                "CREATED"
            ]
        },
        {
            "row": 3,
            "object": {
                "id": "120043@1",
                "label": "A3"
            },
            "status": [
                "CREATED"
            ]
        }
    ]
}
```

Setting values to subentity from root entity

This example maps a unit to a specific unit system. In detail, a unit with ID 7001 will be taken and mapped to a unit system with ID 1. Therefore the values of the fields UnitSystemSpecific.Code and UnitSystemSpecificLang.Name in english and german will be set for the specified unit system.

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1512/rest/V1.0/list/Unit
```

The following JSON object is provided as content:

```
{
  "columns": [
    {
      "identifier": "UnitSystemSpecific.Code(1)"
    },
    {
      "identifier": "UnitSystemSpecificLang.Name(1,9)"
    },
    {
      "identifier": "UnitSystemSpecificLang.Name(1,7)"
    }

  ],
  "rows": [
    {
      "object": {
        "id": "7001"
      },
      "values": [
```

```
        "ZzZ",
        "My unit name for this unit system",
        "Mein Name für die Einheit in diesem Einheitemsystem"
      ]
    }
  ]
}
```

The following protocol is returned :

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 1,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "7001",
                "label": "Dose",
                "entityId": 3100
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```

Setting lookup value of an item

This example set values of the fields *ArticleCharacteristicValue.LookupValue*

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1512/rest/V1.0/list/Article/ArticleCharacteristicValue
```

The following JSON object is provided as content:

```
{
  "columns": [
    {
```

```
      "identifier": "ArticleCharacteristicValue.LookupValue"
    }
  ],
  "rows": [
    {
      "object": {
                "id": "3@1100"
      },
      "qualification": {
                "recordKey":
"72f315fa2e73d20c:3964b945:16050a4147b:-79d4",
                "rootCharacteristic": {
                    "id": "49"
                },
                "parentRecordKey": "root",
                "characteristic": {
                    "id": "49",
                    "label": "Ingredient [Ingredient]",
                    "entityId": 8000
                }
      },
      "values": [
        [
                    {
                        "id": "163@111",
                        "label": "Egg",
                        "entityId": 7300
                    }
                ]
      ]
    }
  ]
}
```

The following protocol is returned :

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 1,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "3@1100",
                "label": "AIW_6382437684",
```

```
                "entityId": 1000
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```

Setting MIMEValues for SubEntity

To set MIMEValue to perticular column, first we have to upload Zip file containing MIMEValue we want to attach using File API. While uploading File using File upload API, user will get id and originalFileName object. After that object has to set to 'mimeValueArchives' as shown in example. User cann't reuse Filerefernce of uploaded file.

```
curl -u rest:heiler -H "Accept: application/json" -H 'Content-Type: application/json'
 -X POST http://localhost:1512/rest/V1.0/list/LookupValue/LookupValueLang
```

The following JSON object is provided as content:

```
{
  "columns": [
        {
            "identifier": "LookupValueLang.MIMEValue"
        }
  ],
  "rows": [
        {
             "object": {
                "id": "31@27"
            },
            "qualification": {
                "language": "German"
            },
            "values":
                [
                    {
                        "relativeFilePath": "17/8/32/fishstick.jpg"
                    }

            ]
        },
        {
             "object": {
                "id":"29@27",
                "label": "Netflix",
```

```
                "entityId": 7300
            },
            "qualification": {
                "language": "German"
            },
              "values":
                [
                    {
                    "label": "img1.jpg",
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        },
                {
             "object": {
                "id": "32@27",
                "label": "Skype",
                "entityId": 7300
            },
            "qualification": {
                "language": "German"
            },
                        "values":
                [
                    {
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        }
   ],
   "mimeValueArchives":[
{
    "id": "1f66a149-937f-4401-a3e5-737a05c35a25",
    "originalFilename": "uploadMime.zip"
}
]
}
```

The returned protocol looks like:

```
 {
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 3,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
```

```
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "31@27",
                "label": "Amazon",
                "entityId": 7300
            },
            "status": [
                "UPDATED"
            ]
        },
        {
            "row": 1,
            "object": {
                "id": "29@27",
                "label": "Netflix",
                "entityId": 7300
            },
            "status": [
                "UPDATED"
            ]
        },
        {
            "row": 2,
            "object": {
                "id": "32@27",
                "label": "Skype",
                "entityId": 7300
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```


Remarks:

- If the object Netflix exists already, they are updated.

Example 2: If mimeValueArchives fileRefernces is invalid or used earlier then User will get "400: Bad Request" in Response. As shown in bellow:

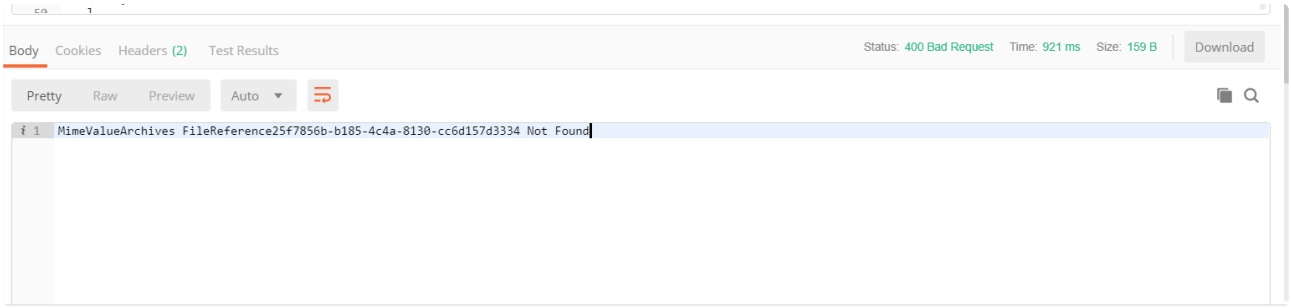

```
{
   "columns": [
        {
            "identifier": "LookupValueLang.MIMEValue"
```

```
        }
    ],
"rows": [
        {
             "object": {
                "id": "31@27"
            },
            "qualification": {
                "language": "German"
            },
            "values":
                [
                    {
                        "relativeFilePath": "17/8/32/fishstick.jpg"
                    }

            ]
        },
        {
             "object": {
                "id":"29@27",
                "label": "Netflix",
                "entityId": 7300
            },
            "qualification": {
                "language": "German"
            },
              "values":
                [
                    {
                    "label": "img1.jpg",
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        },
                {
             "object": {
                "id": "32@27",
                "label": "Skype",
                "entityId": 7300
            },
            "qualification": {
                "language": "German"
            },
                        "values":
                [
                    {
                    "mimeType": "image/jpeg",
                    "relativeFilePath": "18/1/img1.jpg"
                    }
            ]
        }
```

```
    ],
    "mimeValueArchives":[
{
     "id": "25f7856b-b185-4c4a-8130-cc6d157d3334",
     "originalFilename": "uploadMime.zip"
}
]
}
```

The response will look like:



5.11 REST List API Delete

The REST List API provides the possibility to delete objects using a report. All objects returned by the report are deleted.

- Delete all objects returned by a report (see page 132)
  - Result (see page 132)
- Delete sub entity records of objects returned by a report (see page 133)
  - Result (see page 134)
- Examples (see page 135)
  - Delete all item in the supplier catalog TOOLS (see page 135)
  - Delete all sales prices with currency USD in the supplier catalog TOOLS (see page 136)
  - Delete a Characteristic QUALITY in a revision DAILY_VERSION (see page 137)
  - Delete all Characteristic Values of an Item in a revision DAILY_VERSION (see page 137)

5.11.1 Delete all objects returned by a report

URL Pattern	`/list/{entity-identifier}/{report-name}`
Method	`DELETE`
Parameters	Only the parameters of the specified entity report are available.
Content-Type	`application/x-www-form-urlencoded`
Media types	`application/json, application/xml`
Result	A protocol containing information about the number of retrieved and deleted items and a list of errors and warnings.

5.11.1.1 Result

A protocol is returned as result. The protocol consists of a list of counters (number of errors, warnings and retreived and deleted objects), a list of errors and warnings (members of the field `entries)` and a list of created and updated objects (members of the field `objects`).

Properties of a `counters` element

	Field	Data type	Description
	errors	Integer	Total number of errors
	warnings	Integer	Total number of warnings
	retrievedObjects	Integer	Number of objects retrieved by the report
	deletedObjects	Integer	Number of deleted objects

The properties of an entries element are the same as when writing values and is described at REST List API Write for Root Entities (see page 109).

Warning

Be aware that when deleting objects via the List API, all objects belonging to the specified report will be deleted at once. So once the deletion request is fired, there will NOT appear any "Are you really sure..?" confirmation dialog 🙂!

Limit delete rights of Rest users as much as possible

As a best practise, it is recommended to restrict the delete rights of users accessing the Product Manager via Rest Service API as much as possible.

5.11.2 Delete sub entity records of objects returned by a report

For example "delete all prices for a specific customer from all items of the master catalog"

Since (PIM 8.0)

URL Pattern	`/list/{entity-identifier}/{sub-entity-identifier}/{report-name}`
Method	`DELETE`

Content-Type	`application/x-www-form-urlencoded`
Media types	`application/json, application/xml`
Result	A protocol containing information about the number of retrieved and deleted items and a list of errors and warnings.

Parameters

Parameter	Required	Default	Datatype	Parameter description
report parameters	no		String	Parameters of the specified entity report.
qualificationFilter	no		String	This parameter allows to restrict the deletion to certain qualifications. An example would a filter so that only Euro and Dollar prices for the customer Heiler are deleted. The filter string is a comma-separated list of qualification settings. A qualification setting is a *qualification name* followed by a comma-separated list of values which is put into parentheses . The qualification name is defined in the repository and is included in the Meta-API. Example for prices in Euro and US-Dollar and customer Heiler: `qualificationFilter=currency(EUR,USD),customer(Heiler)`

5.11.2.1 Result

A protocol is returned as result. The protocol consists of a list of counters (number of errors, warnings and retreived and deleted objects), a list of errors and warnings (members of the field `entries)` and a list of created and updated objects (members of the field `objects`).

Properties of a `counters` element

Field	Data type	Description
errors	Integer	Total number of errors
warnings	Integer	Total number of warnings
retrievedObjects	Integer	Number of objects retrieved by the report
deletedObjects	Integer	Number of deleted objects

The properties of an entries element are the same as when writing values and is described at REST List API Write for Root Entities (see page 109).

Warning

Be aware that when deleting objects via the List API, all objects belonging to the specified report will be deleted at once. So once the deletion request is fired, there will NOT appear any "Are you really sure..?" confirmation dialog!

5.11.3 Examples

5.11.3.1 Delete all item in the supplier catalog TOOLS

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type: application/x-www-form-urlencoded"  -X DELETE http://localhost:1501/rest/V1.0/list/Article/byCatalog?catalog=TOOLS
```

The returned protocol looks like

```
{
    "counters": {
        "retrievedObjects": 3,
        "warnings": 0,
```

```
        "errors": 0,
        "deletedObjects": 3
    },
    "entries": []
}
```


Rest Client Java Code

```
EntityItemReference toolsCatalogRef = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );

ReportQuery reportQuery = new ReportQuery( "byCatalog" );
reportQuery.addParameterValue( "catalog", toolsCatalogRef );

ListDeleteRequest deleteRequest = getRestClient().createListDeleteRequest();

DeleteProtocol protocol = deleteRequest.deleteRootItems( "Article", reportQuery );
```

5.11.3.2 Delete all sales prices with currency USD in the supplier catalog TOOLS

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type: application/x-
www-form-urlencoded"  -X DELETE http://localhost:1501/rest/V1.0/list/Article/
ArticlePriceSales/byCatalog?catalog=TOOLS&qualificationFilter=currency(USD)
```

Rest Client Java Code

```
EntityItemReference toolsCatalogRef = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );

ReportQuery reportQuery = new ReportQuery( "byCatalog" );
reportQuery.addParameterValue( "catalog", toolsCatalogRef );

ListDeleteRequest deleteRequest = getRestClient().createListDeleteRequest();

DeleteProtocol protocol = deleteRequest.deleteSubItems( "Article",
"ArticlePriceSales", reportQuery, "currency(USD)" );
```

5.11.3.3 Delete a Characteristic QUALITY in a revision DAILY_VERSION

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type: application/x-www-form-urlencoded"  -X DELETE http://localhost:1512/rest/V1.0/list/Characteristic/bySearch?query=Characteristic.Identifier equals "QUALITY"&revision='DAILY_VERSION'
```

Rest Client Java Code

```
ReportQuery reportQuery = new ReportQuery( "bySearch" );
reportQuery.addParameterValue( "query", "Characteristic.Identifier equals 'QUALITY'");
reportQuery.addParameterValue( "revision", "'DAILY_VERSION'");

ListDeleteRequest deleteRequest = getRestClient().createListDeleteRequest();

DeleteProtocol protocol = deleteRequest.deleteRootItems( "Characteristic", reportQuery );
```

5.11.3.4 Delete all Characteristic Values of an Item in a revision DAILY_VERSION

Deletion of Subentity records would be a change of an object in that revision and is not supported.

Warning

Any revision other than the default revision will result in an error message stating that the given revision is not supported!

5.12 REST List API Errors

5.12.1 General List API Errors

For any list query, the following errors may occur depending on the executed query.

Occasion	Return code	Message
Misspelled / unqualified field	400 (Bad request)	The field list "<Field List>" could not be interpreted: Error 1 (e.g. ArticleLang.DescriptionShort / ArticleLang.DescriptionShort() -> unqualified) Error 2 (e.g. ArticleLang.DescriptionShort(de, en, 123) -> Invalid qualification) Error 3 (e.g. ...)
Invalid entity	404 (Not found)	Entity '<entity identifier>' does not exist
Entity is no root entity	404 (Not found)	Entity '<entity identifier>' is not a root entity.
Report/Search not found	404 (Not found)	Report '<report name>' does not exist
Any other unexpected error	500 (internal server error)	• *The response body will contain the error message of the original exception as plain text without stack trace in this case* • *The stack trace of the exception is logged on the server*

5.12.2 Report Specific Errors

If the result of a predefined report is requested, the following errors may occur additionally to the General Errors of the List API.

Occasion	Return code	Message
Missing report parameters	400 (Bad request)	The following mandatory report parameters have not been provided: A, B, C

Occasion	Return code	Message
Invalid values for report parameters	400 (Bad request)	If paramter value could not be converted: "Value of given parameter X has a wrong format" If proxy does not exist: "Given item for parameter X does not exist"
Missing report	404 (Not found)	Report '<ReportName>' does not exist.

5.12.3 Search Specific Errors

If a search query is performed using the search query language, the following errors may happen in addition to the General Errors of the List API.

Occasion	Return code	Message
Invalid query syntax	400 (Bad request)	Search Query syntax is incorrect Example: Unexpected character found at position 11 a = 1 AND b / 2 ^
No search available	404 (Resource not found)	

5.13 REST List API Tutorial

5.13.1 Loading Data For Examples

1. Create catalog *RestTutorial*
2. Import the attached zip file RestTutorial.zip (Perspective *Import*, menu *File / Load import project* / *from zip file*) into the newly created folder *RestTutorial.*

5.13.2 Tools

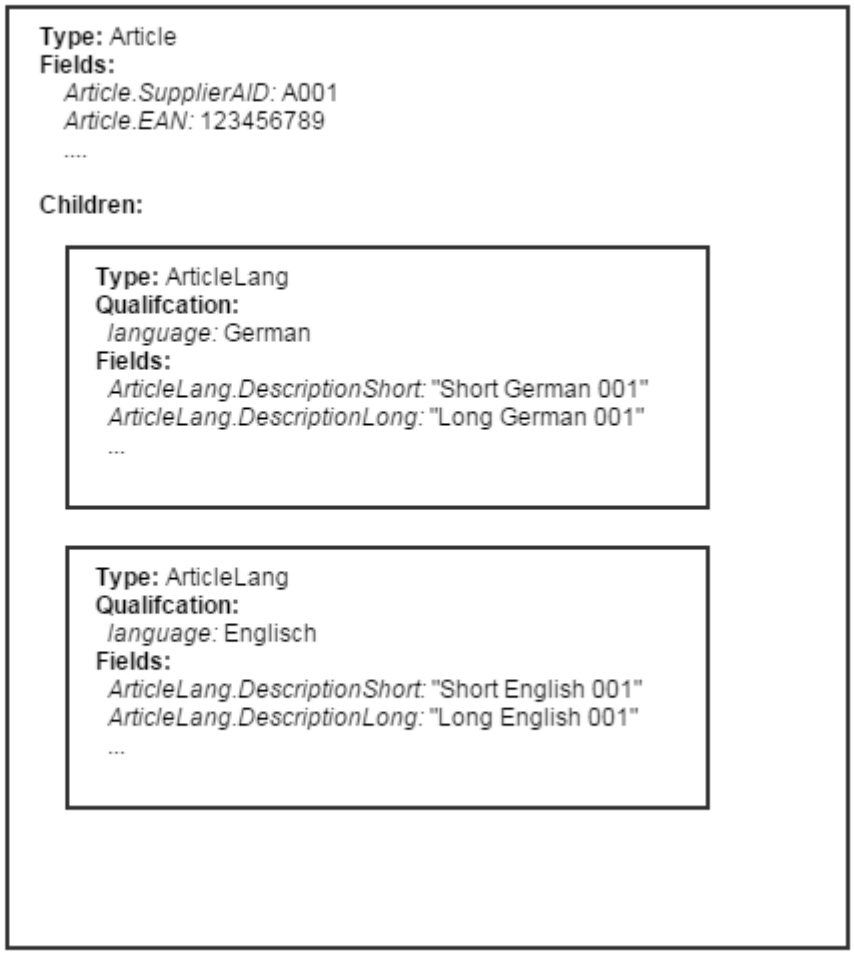
Postman for Google Chrome as REST client (http://www.getpostman.com/)

5.13.3 REST API Tutorial - Data Model

5.13.3.1 Data Model

Objects that can contain embedded/other objects. Embedded objects are uniquely identified by qualifiers (also called logical keys).

Links to other objects are possible.



Explore the data model via HTML Browser

http://localhost:1501/rest/V1.0/meta/

Qualifier and fields have types

See REST Datatypes (see page 25)

Note that references to other objects use a special syntax.

Qualifier and fields can contain enumerations and proposal enumerations

Example for qualifier with enumeration

http://localhost:1501/rest/V1.0/meta/Article/ArticleLang/ArticleLangType.LK.Language

Example for values of an enumeration (link is included in qualifier)

http://localhost:1501/rest/V1.0/enum/Enum.Language

5.13.4 REST API Tutorial - List Read for Root Entities

The list read for root entities is similar to a root entity table in the Rich Client.

The following information has to be specified for a list read:

5.13.4.1 1. Which objects should be included (rows) -> Report

List of available reports:

http://localhost:1501/rest/V1.0/list/info

Please note that there is a general report called *bySearch* which allows powerful search queries (see also REST Search Query Language (see page 38)).

General information about reports: Entity Reporting

5.13.4.2 2. Which fields are required (columns)

The fields are specified in the URL parameter *fields*
Fields of sub entities have to be qualified, see also REST Field Qualification. (see page 31)

5.13.4.3 3. Options

Various options are available, like sort order, paging and formatting information. They are described in section *Parameters* in REST List API Read for Root Entities (see page 65)
Please note that the options includeLabels and cacheId have an impact on performance (unfortunately, the default settings are not the fastest settings). If not otherwise required, set *includeLabels* to *false* and *cacheId* to *no-cache*.
Note that the locale, the request is executed in, has to be set directly in the http header (property *Accept-Language*, the format is *<language>-<country>*, e.g., *en-US* or *de-DE*).

5.13.4.4 Examples

Root entity fields only

Report:

byCatalog whereby parameter *catalog* is set to *RestTutorial*

Fields:

Article.SupplierAID

Article.ManufacturerName
Article.MainSupplier
Article.QuantityMin

Options:
metaData is set to *true* (shows meta information for the specified fields)

pageSize is set to -1 (return all items)
chacheId is set to *no-cache* (result of report execution is not cached, paged access is not possible)

```
GET http://localhost:1501/rest/V1.0/list/Article/byCatalog?
catalog=RestTutorial&fields=Article.SupplierAID,Article.ManufacturerName,Article.MainSupp
lier,Article.QuantityMin&metaData=true&pageSize=-1&cacheId=no-cache
```

Root fields and qualified sub entity fields

Fields:

Article.SupplierAID
ArticleLang.DescriptionShort(German)
ArticleLang.DescriptionShort(English)
ArticleAttributeValue.Value(Length,"Language independent",DEFAULT)
ArticleAttributeValue.Value("Front Color",English,DEFAULT)

```
GET http://localhost:1501/rest/V1.0/list/Article/byCatalog?
catalog=RestTutorial&fields=Article.SupplierAID, ArticleLang.DescriptionSho
rt(German),ArticleLang.DescriptionShort(English),ArticleAttributeValue.Value(Len
gth,"Language independent",DEFAULT),ArticleAttributeValue.Value("Front
Color",English,DEFAULT)&metaData=true
```

Remark:

If a qualification is an enumeration, the label, one of the synonyms or the key can be used.

So instead of ArticleLang.DescriptionShort(German) also the following expressions are possible:

ArticleLang.DescriptionShort(de) (synoym)
ArticleLang.DescriptionShort(7) (key)

5.13.4.5 Transition field access

Since 7.1.03 it is possible to use transition fields that allow to access values of linked objects directly (currently read access only).
For example to get the acronym of the main supplier from the example above, *Article.MainSupplier->Party.Acronym* can be used.
See also REST Transition Fields (see page 36).

Fields:

Article.SupplierAID
Article.MainSupplier->Party.Name
Article.MainSupplier->Party.Acronym

```
GET http://localhost:1501/rest/V1.0/list/Article/byCatalog?
catalog=RestTutorial&fields=Article.SupplierAID,Article.MainSupplier-
>Party.Name,Article.MainSupplier->Party.Acronym&metaData=true&pageSize=-1
```

5.13.4.6 Paged access

Use the parameters *startIndex*, *pageSize* and *cacheId* to perform a page based access.

First call (without parameter *cacheId*):

```
GET http://localhost:1501/rest/V1.0/list/Article/byCatalog?catalog= RestTutorial &fields=Article.SupplierAID,ArticleLang.DescriptionShort(German)&startIndex=0&pageSize=1
```

Second call (with parameter *cacheId*):

```
GET http://localhost:1501/rest/V1.0/list/Article/byCatalog?catalog= RestTutorial &fields=Article.SupplierAID,ArticleLang.DescriptionShort(German)&startIndex=1&pageSize=1&cacheId=<Returned cacheId>
```

5.13.5 REST API Tutorial - List Write for Root Entities

The input of list write and the output of list read uses the same structure. You can perform the list read, modify some values and write them back.

5.13.5.1 Update

The HTTP method POST instead of GET is used, the body contains the data to be written. The data in the body has the same structure as a read result, but only the values of *columns*, *formatData, includeObjectsInProtocol* and *rows* are relevant.

The fields have to be specified in the *columns* section.

Example

The short description and attribute value is updated, new values are *Update short English A001* resp. *light blue* for *A001* and *red* as attribute value for *A002.* The value of the short description of *A002* remains unchanged by setting the value to *null.*

The example uses the external identifier instead of the internal id as the internal ID is different on each system and cannot be used for copy and paste example.

External identifier are written in single quotes: `'A001'@'RestTutorial'` instead of somethink like `39861@11639` (see also entry *ENTITY_ITEM* in REST Datatypes (see page 25)).

`POST http://localhost:1501/rest/V1.0/list/Article`

Message body

```
{
  "columns": [
    {
      "identifier": "ArticleLang.DescriptionShort(English)"
    },
    {
      "identifier": "ArticleAttributeValue.Value(\"Front
Color\",English,DEFAULT)"
    }
  ],
  "rows": [
    {
      "object": {
        "id": "'A001'@'RestTutorial'"
      },
      "values": [
        "Updated Short English A001",
```

```
        "light blue"
      ]
    },
    {
      "object": {
        "id": "'A002'@'RestTutorial'"
      },
      "values": [
        null,
        "red"
      ]
    }
  ]
}
```

A protocol is returned containing created resp. updated items and errors resp. warnings.

If the object part of the protocol is not required, set *includeObjectsInProtocol* to *false*. This improves performance significantly.

Example with error

Errors are reported in the returned protocol. This example sets the attribute *Length* to *unknown* for item *A001* which is not a valid decimal value.

Message body

```
{
  "columns": [
    {
      "identifier": "ArticleAttributeValue.Value(Length,\"Language
independent\",DEFAULT)"
    }
  ],
  "rows": [
    {
      "object": {
        "id": "'A001'@'RestTutorial'"
      },
      "values": [
        "unknown"
      ]
    },
    {
      "object": {
        "id": "'A002'@'RestTutorial'"
      },
```

```
      "values": [
        "23"
      ]
    }
  ]
}
```

5.13.5.2 Create

The request is the same POST request as for update, but the *object* entry has to use the external identifier instead of the internal id.

So if you want to create an item with the external identifier (Supplier AID) *A001* in the master catalog, you have to write it as 'A001'@1 or 'A001'@'MASTER'.

See also entry *ENTITY_ITEM* in REST Datatypes (see page 25).

Example

A new item is created with four fields.

```
POST http://localhost:1501/rest/V1.0/list/Article
```

Message Body

```
{
  "columns": [
    {
      "identifier": "ArticleLang.DescriptionShort(English)"
    },
    {
      "identifier": "ArticleAttributeValue.Value(\"Front
color\",English,DEFAULT)"
    }
  ],
  "rows": [
    {
      "object": {
        "id": "'A003'@'RestTutorial'"
      },
      "values": [
        "Created Short English A003",
        "green"
      ]
    }
  ]
}
```

5.13.6 REST API Tutorial - List Read for Sub Entities

5.13.6.1 Motivation

Reads on sub entity level are necessary when

Qualifications are unknown

- Attribute names
- References to other items (the referenced item is part of the qualification)

Too many fields

- Getting values for 100 attributes would require 100 qualified fields

5.13.6.2 Query

Reading on sub entity level is like reading on entity level, but

- URL contains the identifier of the sub entity additionally to the root entity
- Fields are not qualified
- Results contains the qualification for each row

Remark

Only direct sub entities can be specified, but the queries can contains fields from sub sub entities.

Examples

Language data: Query the values of the field *ArticleLang.DescriptionShort* from the sub entity *ArticleLang* for all languages

```
GET http://localhost:1501/rest/V1.0/list/Article/ArticleLang/byCatalog?
catalog=RestTutorial&fields=ArticleLang.DescriptionShort&metaData=true&pageSize=-1
```

Attributes: Query all values of the fields *ArticleAttribute.Datatype* and *ArticleAttributeValue.Value* (which is a field of a sub sub entity) from the sub entitry *ArticleAttribute*

```
GET http://localhost:1501/rest/V1.0/list/Article/ArticleAttribute/byCatalog?
catalog=RestTutorial&fields=  ArticleAttribute.Datatype ,ArticleAttributeValue.Value&m
etaData=true&pageSize=-1
```

References: Query all values of the fields *ArticleReference.Type*, *ArticleReference.ReferencedArticle* and *ArticleReference.Quantity* from the sub entity *ArticleReference*

```
GET http://localhost:1501/rest/V1.0/list/Article/ArticleReference/byCatalog?
catalog=RestTutorial&fields=ArticleReference.Type ,ArticleReference.ReferencedArticle,
ArticleReference.Quantity&metaData=true&pageSize=-1
```

5.13.6.3 Qualification filter

A qualification filter allows to filter the returned values. The filtername can be looked up in qualifier description in the entry *Qualification filter identifier*

Example

In this example only the English short descriptions returned. Therefore the parameter *qualificationFilter* is set to *language(English)*.

The filter name can be found in the entry *Qualification filter identifier* of http://localhost:1501/rest/V1.0/meta/Article/ArticleLang/ArticleLangType.LK.Language

```
GET http://localhost:1501/rest/V1.0/list/Article/ArticleLang/byCatalog?catalog=RestTutorial&fields=ArticleLang.DescriptionShort&qualificationFilter=language(English)&metaData=true&pageSize=-1
```

5.13.7 REST API Tutorial - List Write for Sub Entities

Writing on sub entity level is similar to writing on root entity level.

Differences:

- URL contains the identifier of the sub entity additionally to the root entity
- Fields are not qualified
- Qualification entry is required for each row

5.13.7.1 Example

The attributes *Length* and *Front Color* are added to item *A003*

```
POST http://localhost:1501/rest/V1.0/list/Article/ArticleAttribute
```

Message body

```
{
  "columns": [
    {
      "identifier": "ArticleAttribute.Datatype"
    },
    {
      "identifier": "ArticleAttributeValue.Value"
    }
  ],
  "rows": [
    {
      "object": {
        "id": "'A003'@'RestTutorial'"
      },
      "qualification": {
        "name": "Length",
        "language": "Language independent",
        "identifier": "DEFAULT"
      },
      "values": [
        "Decimal",
        "3"
      ]
    },
    {
      "object": {
        "id": "'A003'@'RestTutorial'"
      },
      "qualification": {
        "name": "Front Color",
        "language": "German",
```

```
        "identifier": "DEFAULT"
      },
      "values": [
        "Character string",
        "rot"
      ]
    },
    {
      "object": {
        "id": "'A003'@'RestTutorial'"
      },
      "qualification": {
        "name": "Front Color",
        "language": "English",
        "identifier": "DEFAULT"
      },
      "values": [
        "Character string",
        "red"
      ]
    }
  ]
}
```

5.14 REST List API Troubleshooting and How To's

A collection of small how to's and hints as how to work with the list api. This page also serves as a troubleshooting page which summarizes solutions we encountered during practical usage of the List API.

5.14.1 User and User Groups

5.14.1.1 Assigning User Groups to a User

Use the following REST API call to assign user groups to a user:

POST /rest/V1.0/list/User

```
{
    "columns": [{"identifier": "User.UserGroups"}],
    "rows": [{
            "object": { "id": "'restuser'" },
            "values": [ [ { "id" : "'InfaBPM'" }, { "id" :
"'Categorization'" } ] ]
        }]
}
```

In this example the user *restuser* gets the user groups *InfaBPM* and *Categorization* assigned.

To add just one user group no list element is needed. For example
`{ "id" : "'InfaBPM'" }`

would be sufficient to assign group *InfaBPM* to a user.

5.14.2 Characteristics

- When searching for a string value which is in the entity proxy format like: `characteristic('StringCharacteristic') = "'LookupValue'@'Lookup'"`, the value will still be interpreted as a lookup value and not as a string.
- Please make sure when using the search that there is **no leading whitespace** in the URL: `list/Article/bySearch?query= characteristic('23124123') is empty`

5.14.3 Structure Groups

5.14.3.1 Query the list of items assigned to multiple structure groups of a structure

Use the following REST request to get a list of all items assigned to "SG_1" and "SG_2" of "Structure" structure.

Note: in this request, the "ArticleStructureGroupMap" (read-only) entity is used since there is one entry for each item - structure group mapping.

```
http://localhost:1512/rest/V1.0/list/Article/bySearch?query=(not
(ArticleStructureGroupMap.StructureProxy("Structure",
"'SG_1'@'Structure'") is empty) ) and (not
(ArticleStructureGroupMap.StructureProxy("Structure",
"'SG_2'@'Structure'") is empty))&fields=Article.SupplierAID
```

5.14.3.2 Updating Product's Structure Group with Rest API

Use the following REST API call to change the structure group of a product:

POST /rest/V1.0/list/Product2G

```
{
  "entityIdentifier": "Product2G",
  "rowCount": 1,
  "columnCount": 1,
  "columns": [
    {
        "identifier": "Product2GStructureMap.ManualMap(\"Brands\")"
    }
  ],
  "rows": [{
      "object": {
        "id": "87534@1",
        "label": "MGRTN0000003381699"
      },
      "values": [ "NEW BRAND" ]
  }]
}
```

In this example the object with the id 87534@1 is assigned to the "NEW BRAND" structure group within the Brands structure system.



You have to specify the Manual Map fields as a list of Strings, not entity items. You need to set the actual string identifier of the structure groups, not using the entity item syntax with the internal ID's.

You can also use this approach to change structure groups of articles and variants.

5.14.3.3 Access to structure group fails

The following call fails:

```
GET http://<hostname>:1501/rest/V1.0/list/StructureGroup/byParentGroup?
structureGroup=20153@17969&fields=StructureGroup.Level
```

with error:

```
ERROR [qtp2129920-124] [FragmentManager] execute: Error while initializing resp.
executing fragments
java.lang.IllegalStateException: Couldn't find a column index for column identifier:
com.heiler.ppm.structure.db.model.StructureGroupRevision.structureId
    at
com.heiler.ppm.fragment.server.hql.FragmentHqlContext.getColumnIndex(FragmentHqlConte
xt.java:238)
```

while the following succeeds:

```
GET http://<hostname>:1501/rest/V1.0/list/StructureGroup/byParentGroup?
structureGroup=20153@17969&fields=StructureGroup.Level,StructureGroup.StructureProxy
```

5.14.3.4 StructureGroup levels

Query *bySearch* on an attribute which is based on an enumeration:

```
GET http://<hostname>:1501/rest/V1.0/list/Article/bySearch?
query=Article.Channelkennzeichen contains "technisat"
```

- A multi-selection should also be possible, e.g. “technisat.de, ekshop.de”
- In general there is also the problem, that the data also has to be written. How could this be achieved? The DB contains comma separated values.

5.14.3.5 Filtering on structure group attribute

The following call for example returns the structure group attributes:

```
http://<hostname>:1501/rest/V1.0/list/StructureGroup/StructureGroupAttribute/
bySearch?
 query=StructureGroup.Identifier%20=%20%221365663226877%22
 &structure=TechniSat
 &fields=StructureGroupAttributeValue.Value
 &qualificationFilter=language(de)
```

It would be helpful a filter on sub attributes could be added to the query, e.g.

```
StructureGroupAttribute.IsMandatory = false
```

however adding this filter to the query leads to an internal error:

```
java.lang.NullPointerException
 at
com.heiler.ppm.search.server.internal.hql.exp.logicalkey.DefaultLogicalKeyGenerator.g
enerateHQLCondition(DefaultLogicalKeyGenerator.java:97)
```

5.14.3.6 Not possible to search by Structure Group Attribute Value

The following call:

```
GET http://localhost:1501/rest/V1.0/list/StructureGroup/bySearch?metaData=true
  &structure=10024
  &query=StructureGroupAttributeValue.Value("Brand ID",en,DEFAULT) equals "TEST"
  &pageSize=1000&orderBy=0-ASC
  &fields=StructureGroup.Identifier
```

leads to the following internal error:

```
ERROR [qtp461258525-162] [CoreExceptionMapper] Error while interpreting object
"StructureGroupAttributeValue.Value("BrandID","en",DEFAULT) equals "TEST"":
Error while parsing condition 'StructureGroupAttributeValue.Value("Brand
ID","en",DEFAULT) equals "TEST"':
Unknown enum entry provided for enum-based field (StructureGroupAttributeValue.Value)
- Entry for synonym 'TEST' not found!
com.heiler.ppm.std.core.dsl.DslParseException: Error while interpreting object
"StructureGroupAttributeValue.Value("Brand ID","en",DEFAULT) equals "TEST"":
      at
com.heiler.ppm.std.core.internal.dsl.ParseUtils.performParse(ParseUtils.java:53)
      at com.heiler.ppm.search.core.hsq.HSQParser.parseSearchQuery(HSQParser.java:62)
```

See also https://mysupport.informatica.com/message/89002?et=watches.email.thread

5.14.4 Media Assets

5.14.4.1 Get media asset document with attributes fails

The following call:

```
GET http://<hostname>:1501/rest/V1.0/list/MediaAsset/MediaAssetDocument/byItems?
items=14&fields=MediaAssetDocument.Identifier,MediaAssetDocumentAttributes.Resolution
```

leads to an internal error:

```
java.lang.IllegalArgumentException: Sort settings at index = 0 specify a field path
for which the list model doesn't have appropriate column
    at
com.heiler.ppm.std.core.list.util.ListEntryComparator.<init>(ListEntryComparator.java
:269)
```

The following call fails with: Entity 'MediaAssetDocumentAttributes' is not a direct child entity of entity 'MediaAsset'.

```
GET http://localhost:1501/rest/V1.0/list/MediaAsset/MediaAssetDocumentAttributes/
byItems?items=14
```

5.14.5 How to read and write values for characteristics

This document describes how you can use the service api to read and write item, product or variant values for characteristics.

Characteristics are the new alternative to attributes. They allow to store additional data on products, variants and items without the need to adjust the repository and without the restriction to have the information on each and every product, variant and item. Characteristics are similar to attributes in that you can configure them to have a specific data type and define if they are mandatory or multi value. They are also dynamic in nature and can be changed during runtime. In contrast to attributes, they allow a hierarchy of values and dependencies between them. This allows for a value to only be maintained dependent on the selected value for the parent characteristic.

Please see the Knowledge Base article "Characteristics - Dynamic data model" for more details on the possibilities and the general handling within Product 360.

The Service API for characteristics follows the general List API contract and has the same parameters. Please make yourself familiar with the REST List API (see page 55) before you read this how-to.

5.14.5.1 Definitions

Name	
`Characteristic`	The model definition. Name, Datatype, Lookup Values, etc.
`CharacteristicRecord`	A characteristic record contains the actual value for the characteristic for a specific entity-item like item, product or variant. Additionally to the value itself it also contains the reference to the characteristic as well as a unique key of the record and the reference to the parent record.
`ArticleCharacteristicValue`, `Product2GCharacteristicValue`, `VariantCharacteristicValue`	The repository entity for a characteristic record of the corresponding root entity

Name	
`SimpleArticleCharacteristicValue`, `SimpleProduct2GCharacteristicValue`, `SimpleVariantCharacteristicValue`	The repository entity for a simple characteristic record of the corresponding root entity. A simple characteristic is a root characteristic without children. Values for simple characteristics can be assigned to items, products or variants by specifying only the simple characteristic and the language.

5.14.5.2 Qualifications

Qualification	Datatype	
`characteristic`	`ENTITY_ITEM`	The characteristic to which this value belongs as ENTITY_ITEM (see page 25)
`rootCharacteristic`	`ENTITY_ITEM`	The root characteristic (not necessarily the parent) of the characteristic as ENTITY_ITEM (see page 25). *This qualification is only used for read access, it can be omitted for write requests.*
`recordKey`	`String`	The key of the characteristic record, is unique within the characteristic. The default value for this qualification is `0000.0000.RK` The record key is mandatory in case there are multiple records for the same characteristic. We recommend to use the default record key as long as you don't have multiple records for the same characteristic, this will allow you to obtain values for this characteristic also by means of fully qualifying a field.

Qualification	Datatype	
`language`	`String`	The key or code or name of the language in which the value should be interpreted. This defaults to `-1` for characteristics which are not defined as language specific. Only `Text` or `MIME` characteristics can be language dependent.

5.14.5.3 Datatypes

Characteristic Datatype	Corresponding Rest Datatype
`TEXT`	STRING (see page 25)
`INTEGER`	INTEGER (see page 25)
`DECIMAL`	DECIMAL (see page 26)
`BOOLEAN`	BOOLEAN (see page 27)
`DATE`	DATE (see page 26)
`DATETIME`	DATETIME (see page 27)
`MIME`	MIME_VALUE (see page 29)
`LOOKUP`	ENTITY_ITEM (see page 28)

Characteristic Datatype	Corresponding Rest Datatype
`NONE`	This datatype is typically used to build logical groups of characteristics with a common parent. For example: Dimensions with the children height, width and depth. Records of characteristics with this datatype do not hold an own value.

5.14.5.4 Fields

The field which contains the actual value of a characteristic record is `ArticleCharacteristicValueLang.Value`

This field is always a multi-value field which means it will always be returned as array. Some characteristics are multi-value and others are not, in this case the client can treat them all the same. The datatype with regards to the Service API is ANY, which means any of the datatypes which are supported by the characteristics will be returned - depending on the definition of the record's characteristic.

Special Case: LookupValues

In order to provide the generic Field transition functionality for characteristic values of datatype lookup a specialized field is provided: `ArticleCharacteristicValue.LookupValue`

This field only contains a value if the datatype of the characteristic is lookup value and it can actually be used to build a transition field like:
`ArticleCharacteristicValue.LookupValue→LookupValueLang.Description(de)` which would return the German description of this lookup value.

5.14.5.5 Read Access

CharacteristicsValues can be read in the same way as any other sub-entity can be read. Either by fully qualifying a field parameter with *all* qualifications or by requesting all records for this sub-entity and specifying *some* of the qualifications as optional filters.

The nature of the characteristics makes it not easy to use in a fully qualified manner, as the `recordKey` can be different for each item/characteristic combination.
So one fully qualified column might not return values for all items in the table as the record key must not match.

Examples based on the root entity

As mentioned above, reading characteristic values with fully qualified fields is difficult in general. But it is quite easy for simple characteristics as the following examples show.

GET Simple characteristic values for Item

```
http://<server>:<port>/rest/V2.0/list/Article/bySearch?query=Article.SupplierAID
startsWith
"ItemRestTest"&fields=SimpleArticleCharacteristicValueLang.Value(COLOR,-1),SimpleArti
cleCharacteristicValueLang.Value(STATEMENT,en),SimpleArticleCharacteristicValueLang.V
alue(STATEMENT,de)&includeLabels=true
```

As a result the following structure of characteristic values will be provided

Simple characteristic values for Item


```
{
    "cacheId": "no-cache",
    "entityIdentifier": "Article",
    "totalSize": 2,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 2,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "158684@1",
                "label": "ItemRestTest",
                "entityId": 1000
            },
            "values": [
                [
                    {
                        "id": "1455@945",
                        "label": "green",
                        "entityId": 7300
                    }
                ],
                [
                    "Statement"
                ],
                [
                    "Erklärung"
                ]
            ]
        },
        {
            "object": {
                "id": "158685@1",
                "label": "ItemRestTest2",
                "entityId": 1000
```

```
            },
            "values": [
                [
                    {
                        "id": "1456@945",
                        "label": "blue",
                        "entityId": 7300
                    }
                ],
                [
                    "No statement necessary"
                ],
                [
                    "Keine Erklärung nötig"
                ]
            ]
        }
    ]
}
```

GET MIME value meta data and MIME values for Product

GET MIME value meta data for Product

```
http://<server>:<port>/rest/V2.0/list/Product2G/bySearch?query=Product2G.ProductNo
startsWith
"RestTestProduct"&fields=SimpleProduct2GCharacteristicValueLang.Value(LOGO,-1)&includ
eLabels=true
```

As a result the following structure of characteristic values will be provided

MIME values meta data for Product

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "Product2G",
    "totalSize": 2,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 2,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "158686@1",
                "label": "RestTestProduct",
                "entityId": 1100
```

```
            },
            "values": [
                [
                    {
                        "label": "logo.png",
                        "mimeType": "image/png",
                        "relativeFilePath": "29\\27\\19\
\logo.908f8b16002fbfb2_4955b4f5_170d4d6aaae_-7e4a.png"
                    }
                ]
            ]
        },
        {
            "object": {
                "id": "158687@1",
                "label": "RestTestProduct2",
                "entityId": 1100
            },
            "values": [
                [
                    {
                        "label": "logo_global.png",
                        "mimeType": "image/png",
                        "relativeFilePath": "10\\46\\16\
\logo_global.908f8b16002fbfb2_4955b4f5_170d4d6aaae_-7e3b.png"
                    }
                ]
            ]
        }
    ]
}
```

GET MIME values for Product

```
http://<server>:<port>/rest/V2.0/list/Product2G/mimes/bySearch?
query=Product2G.ProductNo startsWith
"RestTestProduct"&fields=SimpleProduct2GCharacteristicValueLang.Value(LOGO,-1)&includ
eLabels=true
```

As result a zip file containing the MIME files will be provided

Examples based on the sub-entity

GET Characteristic values for Item

```
http://<server>:<port>/rest/V2.0/list/Article/ArticleCharacteristicValue/bySearch?
query=Article.SupplierAID equals
```

```
"ArticleRestTest"&fields=ArticleCharacteristicValue.RecordKey,ArticleCharacteristicVa
lueLang.Value&includeLabels=true
```

As a result the following structure of characteristic values will be provided

Characteristic values for Items


```
{
    "cacheId": "no-cache",
    "entityIdentifier": "ArticleCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 1,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "10126@1",
                "label": "ArticleRestTest",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "10769",
                    "label": "LookupRestArticleTestCharacteristic
[LookupRestArticleTestCharacteristic]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "",
                "characteristic": {
                    "id": "10769",
                    "label": "LookupRestArticleTestCharacteristic
[LookupRestArticleTestCharacteristic]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    {
                        "id": "10615@3",
                        "label": "ActualValue",
                        "entityId": 7300
                    }
                ]
            ]
        }
```

```
    ]
}
```

GET Characteristic values for Product

```
http://<server>:<port>/rest/V2.0/list/Product2G/Product2GCharacteristicValue/
bySearch?query=Product2G.ProductNo equals
"RestTestProductNo"&fields=Product2GCharacteristicValue.RecordKey,Product2GCharacteri
sticValueLang.Value&includeLabels=true
```

As a result the following structure of characteristic values will be provided

Characteristic values for Product

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "Product2GCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 10,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "10123@1",
                "label": "RestTestProductNo",
                "entityId": 1100
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "10759",
                    "label": "TextRestTestCharacteristic
[TextRestTestCharacteristic]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "-1",
                "characteristic": {
                    "id": "10759",
                    "label": "TextRestTestCharacteristic
[TextRestTestCharacteristic]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
```

```
                [
                    "TextRestTestCharacteristicValue"
                ]
            ]
        },
    ]
}
```

GET Characteristic values for Variant

```
http://<server>:<port>/rest/V2.0/list/Variant/VariantCharacteristicValue/bySearch?
query=Variant.VariantNo equals
"VariantRestTest"&fields=VariantCharacteristicValue.RecordKey,VariantCharacteristicVa
lueLang.Value&includeLabels=true
```

As a result the following structure of characteristic values will be provided

Characteristic values for Variant

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "VariantCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 10,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "10123@1",
                "label": "RestTestVariantNo",
                "entityId": 1100
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "10759",
                    "label": "TextRestTestCharacteristic
[TextRestTestCharacteristic]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "-1",
                "characteristic": {
                    "id": "10759",
                    "label": "TextRestTestCharacteristic
[TextRestTestCharacteristic]",
```

```
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "TextRestTestCharacteristicValue"
                ]
            ]
        },
    ]
}
```

GET Characteristic values and lookup value codes for Item

```
http://<server>:<port>/rest/V2.0/list/Article/ArticleCharacteristicValue/bySearch?
query=Article.SupplierAID equals
"ItemRestTest"&fields=ArticleCharacteristicValue.RecordKey,ArticleCharacteristicValue
Lang.Value,ArticleCharacteristicValue.LookupValue ->
LookupValue.Code&includeLabels=true
```

As a result the following structure of characteristic values and lookup value codes will be provided

Characteristic values and lookup value codes for Item

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "ArticleCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 1,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "158684@1",
                "label": "ItemRestTest",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "3422",
                    "label": "Color [COLOR]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
```

```
                "language": "-1",
                "characteristic": {
                    "id": "3422",
                    "label": "Color [COLOR]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    {
                        "id": "1455@945",
                        "label": "green",
                        "entityId": 7300
                    }
                ],
                "GREEN"
            ]
        }
    ]
}
```

GET Characteristic values for Item for specific languages

```
http://<server>:<port>/rest/V2.0/list/Article/ArticleCharacteristicValue/bySearch?
query=Article.SupplierAID equals
"ItemRestTest"&fields=ArticleCharacteristicValueLang.Value&includeLabels=true&qualifi
cationFilter=language(en,de,fr)
```

As a result the following structure of characteristic values will be provided

Characteristic values for Item for speficic languages

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "ArticleCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 3,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "158684@1",
                "label": "ItemRestTest",
                "entityId": 1000
            },
```

```
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "4300",
                    "label": "Statement [STATEMENT]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "9",
                "characteristic": {
                    "id": "4300",
                    "label": "Statement [STATEMENT]",
                    "entityId": 8000
                }
            },
            "values": [
                [
                    "Statement"
                ]
            ]
        },
        {
            "object": {
                "id": "158684@1",
                "label": "ItemRestTest",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "4300",
                    "label": "Statement [STATEMENT]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "12",
                "characteristic": {
                    "id": "4300",
                    "label": "Statement [STATEMENT]",
                    "entityId": 8000
                }
            },
            "values": [
                [
                    "Déclaration"
                ]
            ]
        },
        {
            "object": {
                "id": "158684@1",
                "label": "ItemRestTest",
                "entityId": 1000
```

```
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "4300",
                    "label": "Statement [STATEMENT]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "7",
                "characteristic": {
                    "id": "4300",
                    "label": "Statement [STATEMENT]",
                    "entityId": 8000
                }
            },
            "values": [
                [
                    "Erklärung"
                ]
            ]
        }
    ]
}
```

GET versioned Characteristic values for Product

```
http://<server>:<port>/rest/V2.0/list/Product2G/Product2GCharacteristicValue/
bySearch?query=Product2G.ProductNo equals
"P_3"&revision='ProductVersion2'&fields=Product2GCharacteristicValue.RecordKey,Produc
t2GCharacteristicValueLang.Value&includeLabels=true
```

As a result the following structure of characteristic values will be provided

Versioned Characteristic values for Product

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "Product2GCharacteristicValue",
    "revision": {
        "id": "10",
        "label": "ProductVersion2",
        "entityId": 5600
    },
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 5,
    "columnCount": 0,
```

```
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "17@1",
                "label": "P_3",
                "entityId": 1100
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "24",
                    "label": "Marketing information [marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "0000.0000.RK",
                "language": "-1",
                "characteristic": {
                    "id": "26",
                    "label": "Coupon family code [couponFamilyCode]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "54321"
                ]
            ]
        },
        {
            "object": {
                "id": "17@1",
                "label": "P_3",
                "entityId": 1100
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "24",
                    "label": "Marketing information [marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "",
                "characteristic": {
                    "id": "24",
                    "label": "Marketing information [marketingInformation]",
                    "entityId": 8000
                }
```

```
            },
            "values": [
                "0000.0000.RK",
                [
                    ""
                ]
            ]
        },
        {
            "object": {
                "id": "17@1",
                "label": "P_3",
                "entityId": 1100
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCcharacteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "0000.0000.RK",
                "language": "7",
                "characteristic": {
                    "id": "31",
                    "label": "Included accessories
[tradeItemIncludedAccessories]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "Version ProductVersion2"
                ]
            ]
        },
        {
            "object": {
                "id": "17@1",
                "label": "P_3",
                "entityId": 1100
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "0000.0000.RK",
```

```
                "language": "9",
                "characteristic": {
                    "id": "31",
                    "label": "Included accessories
[tradeItemIncludedAccessories]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "Version ProductVersion2"
                ]
            ]
        },
        {
            "object": {
                "id": "17@1",
                "label": "P_3",
                "entityId": 1100
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "0000.0000.RK",
                "language": "-1",
                "characteristic": {
                    "id": "25",
                    "label": "Build-In product type [buildInProductType]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "Version ProductVersion2"
                ]
            ]
        }
    ]
}
```


GET versioned Characteristic values for Variant

```
http://<server>:<port>/rest/V2.0/list/Variant/VariantCharacteristicValue/bySearch?
query=Variant.VariantNo equals
```

```
"V_3"&revision='ProductVersion2'&fields=VariantCharacteristicValue.RecordKey,VariantC
haracteristicValueLang.Value&includeLabels=true
```

As a result the following structure of characteristic values will be provided

Versioned Characteristic values for Variant


```
{
    "cacheId": "no-cache",
    "entityIdentifier": "VariantCharacteristicValue",
    "revision": {
        "id": "10",
        "label": "ProductVersion2",
        "entityId": 5600
    },
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 2,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "94@1",
                "label": "V_3",
                "entityId": 1200
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "",
                "characteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    ""
                ]
            ]
```

```
        },
        {
            "object": {
                "id": "94@1",
                "label": "V_3",
                "entityId": 1200
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "0000.0000.RK",
                "language": "9",
                "characteristic": {
                    "id": "39",
                    "label": "Key words [tradeItemKeyWords]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "Variant versioned with product in ProductVersion2"
                ]
            ]
        }
    ]
}
```

GET versioned Characteristic values for Item

```
http://<server>:<port>/rest/V2.0/list/Article/ArticleCharacteristicValue/bySearch?
query=Article.SupplierAID equals
"I_3"&revision='ProductVersion2'&fields=ArticleCharacteristicValue.RecordKey,ArticleC
haracteristicValueLang.Value&includeLabels=true
```

As a result the following structure of characteristic values will be provided

Versioned Characteristic values for Item

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "ArticleCharacteristicValue",
    "revision": {
        "id": "10",
```

```
        "label": "ProductVersion2",
        "entityId": 5600
    },
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 2,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "95@1",
                "label": "I_3",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCcharacteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                },
                "parentRecordKey": "0000.0000.RK",
                "language": "9",
                "characteristic": {
                    "id": "39",
                    "label": "Key words [tradeItemKeyWords]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "Item versioned with variant and product in
ProductVersion2"
                ]
            ]
        },
        {
            "object": {
                "id": "95@1",
                "label": "I_3",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
```

```
                },
                "parentRecordKey": "root",
                "language": "",
                "characteristic": {
                    "id": "24",
                    "label": "Marketing information
[marketingInformation]",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    ""
                ]
            ]
        }
    ]
}
```

GET Characteristic values including unavailable records for Item

```
http://<server>:<port>/rest/V2.0/list/Article/ArticleCharacteristicValue/bySearch?
query=Article.SupplierAID equals
"UnavailableCharacteristicTest"&fields=ArticleCharacteristicValue.RecordKey,ArticleCh
aracteristicValueLang.Value
&includeUnavailable=true
```

As a result the following structure of characteristic values will be provided

Characteristic values for Item including unavailable records

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "ArticleCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 1,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "10127@1",
                "entityId": 1000
            },
            "qualification": {
                "recordKey": "0000.0000.RK",
```

```
                "rootCharacteristic": {
                    "id": "10770",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "-1",
                "characteristic": {
                    "id": "10770",
                    "entityId": 8000
                }
            },
            "values": [
                "0000.0000.RK",
                [
                    "Some Unavailable Value"
                ]
            ]
        }
    ]
}
```

GET MIME value metadata and MIME values for Product

GET MIME value metadata for Product

```
http://<server>:<port>/rest/V2.0/list/Product2G/Product2GCharacteristicValue/
bySearch?query=Product2G.ProductNo equals
"RestTestProductNo"&fields=Product2GCharacteristicValueLang.Value&includeLabels=true
```

MIME value metadata for Product

```
{
    "cacheId": "no-cache",
    "entityIdentifier": "Product2GCharacteristicValue",
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 10,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "10123@1",
                "label": "RestTestProductNo",
                "entityId": 1100
            },
```

```
            "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "10765",
                    "label": "MimeRestTestCharacteristic
[MimeRestTestCharacteristic]",
                    "entityId": 8000
                },
                "parentRecordKey": "root",
                "language": "-1",
                "characteristic": {
                    "id": "10765",
                    "label": "MimeRestTestCharacteristic
[MimeRestTestCharacteristic]",
                    "entityId": 8000
                }
            },
            "values": [
                [
                    {
                        "label":
"880x495_cmsv2_298e3b01-877d-57e3-9ce0-0542084c5af4-3217366.jpg",
                        "mimeType": "image/jpeg",
                        "relativeFilePath": "35\\18\\14\
\880x495_cmsv2_298e3b.b245cc1ddda0ddd7_1023f19d_16bad3f75bb_-7bf7.jpg"
                    }
                ]
            ]
        }
    ]
}
```

GET MIME values for Product

```
http://<server>:<port>/rest/V2.0/list/Product2G/Product2GCharacteristicValue/mimes/
bySearch?query=Product2G.ProductNo equals
"RestTestProductNo"&fields=Product2GCharacteristicValueLang.Value
```

As result a zip file containing the MIME files will be provided

5.14.5.6 Write Access

Examples based on the root entity

As mentioned above, writing characteristic values with fully qualified fields is difficult in general. But it is quite easy for simple characteristics as the following example show.

POST Simple characteristic values for Item

```
http://<server>:<port>/rest/V2.0/list/Article
```

This POST request requires the following request body

Simple characteristic values for Item

```
{
  "columns": [
    { "identifier":
"SimpleArticleCharacteristicValueLang.Value(STATEMENT,en)" },
    { "identifier":
"SimpleArticleCharacteristicValueLang.Value(STATEMENT,de)" },
    { "identifier":
"SimpleArticleCharacteristicValueLang.Value(STATEMENT,fr)" }
  ],
  "rows": [
    {
      "object": { "id": "'ItemRestTest'@'MASTER'" }
      ,
      "values": [
        "Another EN statement",
        "Another DE statement",
        "Another FR statement"
      ]
    }
  ]
}
```

As a result the following answer will be provided

Simple characteristic values for Item Result

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 1,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
```

```
            "object": {
                "id": "2@1",
                "label": "ItemRestTest",
                "entityId": 1000
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```

Examples based on the sub-entity

POST Characteristic values for Item: lookup value

```
http://<server>:<port>/rest/V2.0/list/Article/ArticleCharacteristicValue
```

This POST request requires the following request body

Characteristic values for Item

```
{
  "columns": [
    {
      "identifier": "ArticleCharacteristicValueLang.Value"
    }
  ],
  "rows": [
    {
      "object": {
                "id": "10126@1"
      },
      "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "10769"
                },
                "parentRecordKey": "root",
                "language": "-1",
                "characteristic": {
                    "id": "10769",
                    "label": "LookupRestArticleTestCharacteristic
[LookupRestArticleTestCharacteristic]",
                    "entityId": 8000
                }
```

```
      },
      "values": [
        [
                    {
                        "id": "10617@3",
                        "label": "ActualLookupValueLabel",
                        "entityId": 7300
                    }
                ]
      ]
    }
  ]
}
```

As a result the following answer will be provided

Characteristic values for Item Result

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 1,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "10126@1",
                "label": "ArticleRestTest",
                "entityId": 1100
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```

POST Characteristic values for Product:: language-dependent string values

```
http://<server>:<port>/rest/V2.0/list/Product2G/Product2GCharacteristicValue
```

This POST request requires the following request body

Characteristic values for Product


```
{
  "columns": [
    {
      "identifier": "Product2GCharacteristicValueLang.Value"
    }
  ],
  "rows": [
    {
      "object": {
                "id": "'RestTestProduct'@'MASTER'"
      },
      "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "4300"
                },
                "parentRecordKey": "root",
                "language": "de",
                "characteristic": {
                    "id": "4300"
                }
      },
      "values": [
        [
           "CHANGED German statement"
         ]
      ]
    },
    {
      "object": {
                "id": "'RestTestProduct'@'MASTER'"
      },
      "qualification": {
                "recordKey": "0000.0000.RK",
                "rootCharacteristic": {
                    "id": "4300"
                },
                "parentRecordKey": "root",
                "language": "en",
                "characteristic": {
                    "id": "4300"
                }
      },
      "values": [
        [
           "CHANGED English statement"
         ]
      ]
    }
```

```
  ]
}
```

As a result the following answer will be provided

Characteristic values for Product Result


```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 1,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "158686@1",
                "label": "RestTestProduct",
                "entityId": 1100
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
```


POST Characteristic values for Variant: MIME values

Step 1: Upload MIME zip file

In a first step you have to upload a zip file containing all files that are to be used as MIME values. Details can be found in the chapter REST File API (see page 339).

The result is needed for the next step.

Result of MIME file upload


```
{
    "id": "6942cf90-a4c4-449d-896a-51bdc8e45906",
    "originalFilename": "logos.zip"
}
```

Step 2: Use MIME data to set characteristic values

Important: The uploaded MIME archive can be used for one POST request only, it cannot be re-used for additional requests. So, if that archive contains multiple files, they should all be used within one request.

```
http://<server>:<port>/rest/V2.0/list/Variant/VariantCharacteristicValue
```

This POST request requires the following request body, the result of the file upload is used as `mimeValueArchives` value:

Characteristic values for Variant

```
{
  "columns": [
    {
      "identifier": "VariantCharacteristicValueLang.Value"
    }
  ],
  "rows": [
    {
      "object": {
        "id": "'RestTestVariant'@'MASTER'"
      },
      "qualification": {
        "recordKey": "0000.0000.RK",
        "rootCharacteristic": {
          "id": "4298"
        },
        "parentRecordKey": "root",
        "language": "-1",
        "characteristic": {
          "id": "4298"
        }
      },
      "values": [
        [
          {
            "relativeFilePath": "logo.png"
          }
         ]
      ]
    },
    {
      "object": {
        "id": "'RestTestVariant2'@'MASTER'"
      },
      "qualification": {
        "recordKey": "0000.0000.RK",
```

```
        "rootCharacteristic": {
          "id": "4298"
        },
        "parentRecordKey": "root",
        "language": "-1",
        "characteristic": {
          "id": "4298"
        }
      },
      "values": [
        [
          {
            "relativeFilePath": "logo_global.png"
          }
         ]
      ]
    }
  ],
  "mimeValueArchives": [
    {
        "id": "6942cf90-a4c4-449d-896a-51bdc8e45906",
        "originalFilename": "logos.zip"
    }
  ]
}
```

As a result the following answer will be provided

Characteristic values for Variant Result

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 2,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "158688@1",
                "label": "RestTestVariant",
                "entityId": 1200
            },
            "status": [
                "UPDATED"
            ]
```

```
        },
        {
            "row": 1,
            "object": {
                "id": "158689@1",
                "label": "RestTestVariant2",
                "entityId": 1200
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```

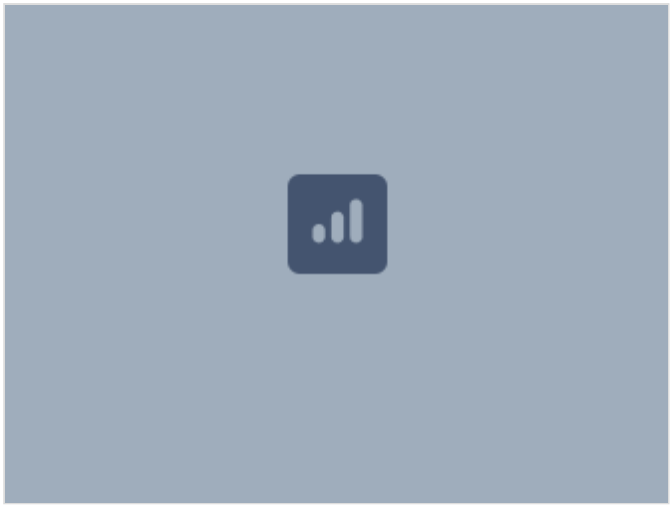
6 REST Object API

With Version 10.5 we introduce an optimized version of the Read Object API (see page 230) and a new Write Object API (see page 272). The usage of Version 1 of the Read Object Api (see page 192) is supported, but discouraged.

With this all typical CRUD (Create, Read, Update and Delete) operations can be performed for all repository based entities which support the object API.

6.1 Overview Presentation

This powerpoint presentation gives a rough overview of the Version 2 of the Object API. Please also see the partner portals for the demo videos which accompany this ppt.



6.2 Examples

Each Object API chapter has some examples directly on the page. For your convenience we also provide a Postman Collection (2.1 Version) with examples ready to use and try out the API. The collection also contains example calls to publish the api calls to the Apache MQ. For details on this see also ObjectAPI Queue.

Download the Postman Collection (V2.1) here: ObjectAPI.postman_collection.json

6.3 OpenAPI Support

The Object API supports the OpenAPI standard 3.0.3 for all its operations. As the repository is very individual, a static api file would not be sufficient. Therefor a new Service API endpoint has been created with Product 360 10.5.0.01 to obtain the open api spec individually.

URL Pattern	`/object/{entity-identifier}/info`
Method	`GET`
Accept	`application/json` If multiple accept headers are defined, they will be evaluated by their order. The wildcard media type (*/*) will result in XML.
Example	`/rest/V2.0/object/Article/info`
Return Codes	**Code** / **Reason** `200 (OK)` — Everything went fine `404 (Not Found)` — Entity couldn't be found `403 (Forbidden)` — User has no permissions for this entity `500` — Internal server error, please check the server logs

The response object corresponds to the OpenAPI standard 3.0.3 and can directly be used in 3rd party applications

6.3.1 Permissions

6.3.1.1 Entity Permissions

The OpenAPI document will not contain methods for which the user who requests the api document has no permission. E.g., the delete method is missing is case the user which is authenticated for the Service Api has no delete entity permission for the requested entity.

6.3.1.2 Field Permissions

In case the authenticated user has no write permission for a specific field, the field is returned as readOnly field. In case no read permission is available, the field will not be returned.

6.4 REST Object API Read V1

Discouraged

With introduction of REST Object API Read V2 (see page 230) we discourage the usage of this version. Version 2 has improvements which optimize the performance of the api because the response is significantly smaller. Additionally to that, the structure is compatible with REST Object API Write (see page 272)

The object api provides a canonical rest api for entity items. Contrary to the ListAPI, the object api is optimized for single (multiple) item use and can return all data of the items in the full hierarchical form of them. It is not the correct api for mass data retrival of a small set of columns. Please use the ListApi for such use cases.

- Read Requests (see page 192)
  - Single Item (see page 192)
  - Multiple Items (since 10.1.0.02) (see page 193)
  - Query Parameters (see page 195)
  - Permissions (see page 197)
    - Object Permissions (see page 197)
    - Field Permissions (see page 198)
    - Qualification Permissions (aka Qualified Field Permissions) (see page 198)
  - Result (see page 198)
    - Repository Entities and Fields (see page 198)
      - Names (see page 198)
      - Datatypes (see page 198)
    - Meta attributes (see page 198)
    - Data Element (see page 199)
    - Example Result (see page 199)
    - Characteristic Values (see page 223)
      - Additional Meta Attributes (see page 223)
      - Characteristic Records Example (see page 223)
  - Examples (see page 229)

6.4.1 Read Requests

6.4.1.1 Single Item

URL Pattern	`/object/{entity-identifier}/{entity_item}`
Method	`GET`

Accept	`application/json` `application/xml (default)`
Example	`/rest/V1.0/object/Article/4711@1` `/rest/V1.0/object/Article/'myItem'@'myCatalog'`
Return Codes	**Code** / **Reason** `200 (OK)` — Everything went fine `404 (Not Found)` — Object couldn't be found `403 (Forbidden)` — User has no read permission for this object `500` — Internal server error, please check the server logs

6.4.1.2 Multiple Items (since 10.1.0.02)

This endpoint provides a multi-item interface in which the client can provide multiple items which should be returned in one call. As query parameters do have a size limitation, this api is designed as POST. This api has restrictions on the number of items which can be called. In case the limit is exceeded a corresponding return code is provided.
Contrary to the single item GET request, a missing object permission or an unknown item id does not lead to an error return code; it will just not be returned.

URL Pattern	`/object/{entity-identifier}/byItems`
Method	`POST`

Headers

Header	Supported Values	Default
Accept	`application/json` `application/xml`	application/xml
Content-Type	`application/x-www-form-urlencoded`	

Form Parameters

Parameter	Datatype	Description	Example
items	List of ENTITY_ITEM	String based entity item syntax. The form parameter can be provided multiple times and also as a comma separated list or even a mix of both	`items=4711@1` `items='myItem'@'myCatalog'` `items=4711@1,'myItem'@'myCatalog'`

Example

`/rest/V1.0/object/Article/byItems`

Return Codes

Code	Reason
`200 (OK)`	Everything went fine
`413 (Payload too large)`	The amount of entity item ids exceeds the configured limit (default: 100). The response contains the "p360-ObjectAPI-MaxItems" header which returns the limit.
`500`	Internal server error, please check the server logs

6.4.1.3 Query Parameters

Please note that the qualification filter applies to all requested sub-entities which have this qualification. So, for example, if you filter for language=English, the ArticleLang and ArticleAttributeValueLang entities will both only return the English values!

Parameter	Required	Default	Datatype	Parameter description	Example
`entityFilter`	no	none	String	Comma separated list of entity identifiers which should be part of the result. If omitted, the full object with all data is returned. We recommend to provide a list of entities which are required in order to gain performance. Note: In case an entityFilter is provided, also the root entity is a filter value. So by only providing the root entity as filter value, only the fields of the root level are returned in the data.	Only return the root fields: `entityFilter=Article` Only return the root fields, and the sales prices `entityFilter=Article,ArticleSalesPrice` Only return the language specific data like Short and Long Description `entityFilter=ArticleLang`
`qualificationFilter`	no	none	String	This parameter allows to restrict the output to certain qualifications. One example would be a filter so that only Euro and US-Dollar prices for the customer Informatica are returned. The filter string is a comma-separated list of qualification settings. A qualification setting is a *qualification name* followed by a comma-separated list of values which is put into parentheses . The qualification name is defined in the repository and is included in the Meta-API.	Example for prices in Euro and US-Dollar and customer Informatica: `qualificationFilter=currency(EUR,USD),customer(Informatica)`

Parameter	Required	Default	Datatype	Parameter description	Example

`revision`	no	root	ENTITY_ITEM	The revision for which the data should be retrieved	`revision='root'`
					`revision=1`
					`revision='myRevisionIdentifier'`
					`revision=4711` (where 4711 is the internal id of the revision).

`includeLabels` (available with 10.1.0.02)	no	`false`	boolean	If set to true, the returned document will contain _label elements for all fields or qualifications which have an enumeration. The label will be returned in the locale of the request like this: `"orderUnit" : { "_current" : { "_key" : { "_entityId" : 3100, "_internalId" : "932", "_externalId" : "'C62'" }, "_code" : "C62", "_label" : "piece" } }`	`includeLabels=true` `includeLabels=false`

6.4.1.4 Permissions

Object Permissions

The user needs to have the READ object permission for this API. In case he doesn't, the api returns HTTP 403 (forbidden) in case of the single item call with GET, and skip this item in case of the multiple items call.

Field Permissions

The user needs to have the READ field permissions for a field. If he doesn't, the field will not be part of the data element, but no error is returned.

Qualification Permissions (aka Qualified Field Permissions)

The user needs to have the READ permission for a qualification, e.g. for Language = English. If he doesn't, sub-entities which are qualified for this will not be part of the data element, but no error is returned.

6.4.1.5 Result

Repository Entities and Fields

Names

A new repository property "shortIdentifier" has been added in the custom section which is used to define the entity and field names for the json structure. The short identifier is unique within its parent entity only!

The standard repository already contains a short identifier for all fields and enabled qualifications.

Datatypes

- ENTITY_ITEM objects always contain the `_entityId, _internalId` and `_externalId` attributes
- MIME_VALUE objects always contain the relative `_filePath`, the `_label` and the `_mimeType`
- timestamp: ISO 8601, e.g.: `2012-04-23T18:25:43.511Z`
- date: ISO 8601, e.g.: `2019-07-04`
- numbers: standard JSON
- "no value" is either provided as null, or the attribute is not in the document.
- Empty string is returned as null, which is omitted when possible!
- Enumeration fields
  - `_key` : always holds the unique key of the enumeration entry
    In case the enumeration field is of datatype ENTITY_ITEM, then the key will be provided with the ENTITY_ITEM syntax (see above)
  - `_code` : the external code of the enumeration entry, if the enum entry has one, and it differs from the key!
  - `_label` : the locale dependent label of the enumeration entry. Only if `includeLabels` is set to true. The locale of the http request is used for this.

Meta attributes

All meta attributes are prefixed with an underscore.

- `_entity` = the root entity identifier
- `_entityItem` = the entity item in the ENTITY_ITEM syntax

- `_revision` = the revision of the data as ENTITY_ITEM
- `_data` = The actual entity item's data
- `_current` = the current value of the field
  As the structure of the document is similar to the EntityItemChange Document which is used for audit trail the `_current` attribute for the current value is required.
- `_qualification` = the qualification of the record. **The combination of the qualification values in the qualification object define this record as unique within its parent**.

Data Element

In order to be part of the data element of the object api, a sub-entity or field must fulfill multiple criteria:

- The entity or field must be `active` in the Repository
- The entity or field must have `supportsServiceApi = true` in the Repository
- The entity or field must have `supportsAuditTrail = true` in the Repository
- The entity of field `must have a shortIdentifier` in the Repository
- The user must have `READ permission` for the `Entity` or `Field`
- The user must have `READ permission` for the `qualification` of the sub-entity
- The field must must `not be a password Field` in the Repository
- `Qualifications` must be `editable` in the Repository

Entities, Fields or Qualifications which do not comply to any of these points will be omitted in the data element. No exception is thrown.

Example Result


```
{
  "_entity": "Article",
  "_entityItem": {
    "_internalId": "13989@1120",
    "_entityId": 1000,
    "_externalId": "\u0027supplierAID-840\u0027@\u0027NEW_CATALOG\u0027"
  },
  "_revision": {
    "_internalId": "1",
    "_entityId": 5600,
    "_externalId": "\u0027root\u0027"
  },
  "_container": {
    "_internalId": "1120",
    "_entityId": 7000,
    "_externalId": "\u0027NEW_CATALOG\u0027"
  },
  "_data": {
    "article": {
      "identifier": {
        "_current": "supplierAID-840"
```

```
},
"manufacturerAID": {
  "_current": "L000000000839"
},
"manufacturerName": {
  "_current": "Rolex"
},
"deliveryTime": {
  "_current": 3.8900
},
"orderUnit": {
  "_current": {
    "_key": {
      "_entityId": 3100,
      "_internalId": "145",
      "_externalId": "\u0027AMH\u0027"
    },
    "_code": "AMH"
  }
},
"contentUnit": {
  "_current": {
    "_key": {
      "_entityId": 3100,
      "_internalId": "310",
      "_externalId": "\u0027GLL\u0027"
    },
    "_code": "GLL"
  }
},
"noCUperOU": {
  "_current": 842.3100
},
"priceQuantity": {
  "_current": 219.5200
},
"quantityMin": {
  "_current": 6.0000
},
"quantityInterval": {
  "_current": 1.0000
},
"mainSupplier": {
  "_current": {
    "_key": {
      "_entityId": 2800,
      "_internalId": "3",
      "_externalId": "\u0027Heiler Product Manager\u0027"
    },
    "_code": "Heiler Product Manager"
  }
},
```

```
      "currentStatus": {
        "_current": {
          "_key": 100,
          "_code": "NEW"
        }
      },
      "kitParent": {
        "_current": false
      },
      "kitComponent": {
        "_current": false
      },
      "soldOnlyInKits": {
        "_current": false
      },
      "lang": [
        {
          "_qualification": {
            "language": {
              "_key": 9,
              "_code": "eng"
            }
          },
          "descriptionShort": {
            "_current": "the little value he put on his own good qualities.
Elizabeth was pleased to find that he had not betrayed the interference of his"
          },
          "descriptionLong": {
            "_current": "practice. I have told Miss Bennet several times, that
she will never play really well unless she practises more; and though Mrs.
Collins has no instrument, she is very welcome, as I have often told her, to
come to Rosings every day, and play on the pianoforte in Mrs. Jenkinson’s room.
She would be in nobody’s way, you know, in that part of the house.” Mr. Darcy
looked a little ashamed of his aunt’s ill-breeding, and made no answer. When
coffee was over, Colonel Fitzwilliam reminded Elizabeth of having promised to
play to him; and she sat down directly to the i"
          },
          "keywords": {
            "_current": [
              "MERCHANTIBILITY"
            ]
          }
        },
        {
          "_qualification": {
            "language": {
              "_key": 1046,
              "_code": "por"
            }
          },
          "descriptionShort": {
            "_current": "daughters. Mr. Collins was punctual to his time, and
```

```
was receive"
          },
          "descriptionLong": {
            "_current": "against herself; and his disappointed feelings became
the object of compassion. His attachment excited gratitude, his general
character respect; but she could not approve him; nor could she for a moment
repent her refusal, or feel the slightest inclination ever to see him again. In
her own past behaviour, there was a constant source of vexation and regret; and
in the unhappy defects of her family, a subject of yet heavier chagrin. They
were hopeless of remedy. Her father, contented with laughing at them, would
never exert himself to restrain the wild giddiness of his youngest daughters;
and her mother, with manners so far fr"
          },
          "keywords": {
            "_current": [
              "busy",
              "_particularly_",
              "nowhere"
            ]
          }
        },
        {
          "_qualification": {
            "language": {
              "_key": 7,
              "_code": "deu"
            }
          },
          "descriptionShort": {
            "_current": "has now living, better than any other person.”
Elizabeth was at no loss to understand from whence this defe"
          },
          "descriptionLong": {
            "_current": "or expense to the user, provide a copy, a means of
exporting a copy, or a means of obtaining a copy upon request, of the work in
its original “Plain Vanilla ASCII” or other form.  Any alternate format must
include the full Project Gutenberg-tm License as specified in paragraph 1.E.1.
1.E.7.  Do not charge a fee for access to, viewing, displaying, performing,
copying or distributing any Project Gutenberg-tm works unless you comply with p"
          }
        }
      ],
      "pricePurchase": [
        {
          "_qualification": {
            "supplier": {
              "_key": {
                "_entityId": 2800,
                "_internalId": "3",
                "_externalId": "\u0027Heiler Product Manager\u0027"
              },
              "_code": "Heiler Product Manager"
            },
```

```
    "type": {
      "_key": 1,
      "_code": "net_list"
    },
    "currency": {
      "_key": "USD"
    },
    "territory": {
      "_key": "DE"
    }
  },
  "validFrom": {
    "_current": "1899-12-30"
  },
  "validTo": {
    "_current": "9999-12-31"
  },
  "value": [
    {
      "_qualification": {
        "lowerBound": 1.0000
      },
      "amount": {
        "_current": 872327.120000
      },
      "factor": {
        "_current": 1.0000
      }
    }
  ]
},
{
  "_qualification": {
    "supplier": {
      "_key": {
        "_entityId": 2800,
        "_internalId": "3",
        "_externalId": "\u0027Heiler Product Manager\u0027"
      },
      "_code": "Heiler Product Manager"
    },
    "type": {
      "_key": 2,
      "_code": "gross_list"
    },
    "currency": {
      "_key": "CHF"
    },
    "territory": {
      "_key": "DE"
    }
  },
```

```
    "validFrom": {
      "_current": "1899-12-30"
    },
    "validTo": {
      "_current": "9999-12-31"
    },
    "value": [
      {
        "_qualification": {
          "lowerBound": 3.0000
        },
        "amount": {
          "_current": 239011.140000
        },
        "factor": {
          "_current": 1.0000
        }
      }
    ]
  }
],
"priceSales": [
  {
    "_qualification": {
      "customer": {
        "_key": {
          "_entityId": 2800,
          "_internalId": "1",
          "_externalId": "\u0027Public\u0027"
        }
      },
      "type": {
        "_key": 3,
        "_code": "net_customer"
      },
      "currency": {
        "_key": "CHF"
      },
      "territory": {
        "_key": "DE"
      }
    },
    "validFrom": {
      "_current": "1899-12-30"
    },
    "validTo": {
      "_current": "9999-12-31"
    },
    "value": [
      {
        "_qualification": {
          "lowerBound": 2.0000
```

```
        },
        "amount": {
          "_current": 60216.410000
        },
        "factor": {
          "_current": 1.0000
        }
      }
    ]
  }
],
"logistic": [
  {
    "_qualification": {
      "territory": {
        "_key": "CH"
      }
    },
    "packageUnit": {
      "_current": {
        "_key": {
          "_entityId": 3100,
          "_internalId": "747",
          "_externalId": "\u0027NEW\u0027"
        },
        "_code": "NEW"
      }
    },
    "packageWeight": {
      "_current": 36.5000
    },
    "originCountry": {
      "_current": {
        "_key": "BB"
      }
    },
    "grossWeight": {
      "_current": 43.9700
    },
    "grossWeightUnit": {
      "_current": {
        "_key": {
          "_entityId": 3100,
          "_internalId": "723",
          "_externalId": "\u0027MON\u0027"
        },
        "_code": "MON"
      }
    }
  }
],
"logisticExtension": [
```

```
{
  "_qualification": {
    "party": {
      "_key": {
        "_entityId": 2800,
        "_internalId": "3",
        "_externalId": "\u0027Heiler Product Manager\u0027"
      },
      "_code": "Heiler Product Manager"
    },
    "packagingUnit": {
      "_key": {
        "_entityId": 3100,
        "_internalId": "380",
        "_externalId": "\u00275\u0027"
      },
      "_code": "5"
    },
    "language": {
      "_key": 10,
      "_code": "esl"
    }
  },
  "code128": {
    "_current": "62243677464153991935"
  },
  "code39": {
    "_current": "25028067122421819836"
  },
  "length": {
    "_current": 52873820.0761
  },
  "lengthUnit": {
    "_current": {
      "_key": {
        "_entityId": 3100,
        "_internalId": "936",
        "_externalId": "\u0027AR\u0027"
      },
      "_code": "AR"
    }
  },
  "height": {
    "_current": 6794560.1841
  },
  "heightUnit": {
    "_current": {
      "_key": {
        "_entityId": 3100,
        "_internalId": "7053",
        "_externalId": "\u0027X_PPC\u0027"
      },
```

```
              "_code": "X_PPC"
            }
          }
        }
      ],
      "referencedItem": [
        {
          "_qualification": {
            "type": {
              "_key": 8,
              "_code": "diff_orderunit"
            },
            "referencedIdentifier": "supplierAID-648",
            "referencedCatalogIdentifier": "NEW_CATALOG"
          },
          "referencedItem": {
            "_current": {
              "_entityId": 1000,
              "_internalId": "14452@1120",
              "_externalId":
"\u0027supplierAID-648\u0027@\u0027NEW_CATALOG\u0027"
            }
          },
          "quantity": {
            "_current": 1
          }
        }
      ],
      "structureMap": [
        {
          "_qualification": {
            "structure": {
              "_key": 15,
              "_code": "ECLASS-6.1"
            }
          },
          "structureGroups": {
            "_current": [
              {
                "_entityId": 3000,
                "_internalId": "23576@15",
                "_externalId": "\u0027AKL316004\u0027@\u0027ECLASS-6.1\u0027"
              }
            ]
          }
        }
      ],
      "structureGroupMap": [
        {
          "_qualification": {
            "structure": {
              "_key": {
```

```
          "_entityId": 2300,
          "_internalId": "15",
          "_externalId": "\u0027ECLASS-6.1\u0027"
        },
        "_code": "ECLASS-6.1"
      },
      "structureGroup": {
        "_entityId": 3000,
        "_internalId": "23576@15",
        "_externalId": "\u0027AKL316004\u0027@\u0027ECLASS-6.1\u0027"
      }
    },
    "sequence": {
      "_current": 2147483647
    }
  }
],
"mediaAsset": [
  {
    "_qualification": {
      "type": {
        "_key": "logo"
      }
    },
    "name": {
      "_current": "be the meaning of it? It"
    },
    "category": {
      "_current": "nonproprietary"
    },
    "mediaAsset": {
      "_current": {
        "_entityId": 2400,
        "_internalId": "250",
        "_externalId": "\u0027MediaAsset_1596466573432152\u0027"
      }
    },
    "document": [
      {
        "_qualification": {
          "quality": {
            "_key": "highres"
          },
          "language": {
            "_key": 7,
            "_code": "deu"
          }
        },
        "identifier": {
          "_current": "hlr-system/nhvjsixjcjucktt.jpg"
        },
        "imageIdentifier": {
```

```
          "_current": "hlr-system/nhvjsixjcjucktt.jpg"
        },
        "order": {
          "_current": 1
        }
      }
    ]
  },
  {
    "_qualification": {
      "type": {
        "_key": "thumbnail"
      }
    },
    "name": {
      "_current": "from Jane. “I do not know whe"
    },
    "category": {
      "_current": "counterbalance"
    },
    "mediaAsset": {
      "_current": {
        "_entityId": 2400,
        "_internalId": "255",
        "_externalId": "\u0027MediaAsset_1596466573432157\u0027"
      }
    },
    "document": [
      {
        "_qualification": {
          "quality": {
            "_key": "highres"
          },
          "language": {
            "_key": 56,
            "_code": "kor"
          }
        },
        "identifier": {
          "_current": "hlr-system/ajomgjomrxyjews.jpg"
        },
        "imageIdentifier": {
          "_current": "hlr-system/ajomgjomrxyjews.jpg"
        },
        "order": {
          "_current": 1
        }
      }
    ]
  }
],
"log": [
```

```
{
  "_qualification": {
    "channel": {
      "_key": "100_classifier.id"
    }
  },
  "creationDate": {
    "_current": "2020-10-07T13:56:25.990Z"
  },
  "creationUser": {
    "_current": {
      "_key": {
        "_entityId": 2600,
        "_internalId": "1",
        "_externalId": "\u0027Administrator\u0027"
      }
    }
  },
  "deletionDate": {
    "_current": "9999-12-31T00:00:00.000Z"
  }
},
{
  "_qualification": {
    "channel": {
      "_key": "HPM"
    }
  },
  "creationDate": {
    "_current": "2020-10-07T13:56:25.990Z"
  },
  "creationUser": {
    "_current": {
      "_key": {
        "_entityId": 2600,
        "_internalId": "1",
        "_externalId": "\u0027Administrator\u0027"
      }
    }
  },
  "deletionDate": {
    "_current": "9999-12-31T00:00:00.000Z"
  }
},
{
  "_qualification": {
    "channel": {
      "_key": "101_classifier.id"
    }
  },
  "creationDate": {
    "_current": "2020-10-07T13:56:25.990Z"
```

```
        },
        "creationUser": {
          "_current": {
            "_key": {
              "_entityId": 2600,
              "_internalId": "1",
              "_externalId": "\u0027Administrator\u0027"
            }
          }
        },
        "deletionDate": {
          "_current": "9999-12-31T00:00:00.000Z"
        }
      }
    ],
    "ownLog": [
      {
        "modificationDate": {
          "_current": "2020-10-07T13:56:26.010Z"
        }
      }
    ],
    "nutrient": [
      {
        "_qualification": {
          "targetMarket": {
            "_key": "CA"
          },
          "preparationState": {
            "_key": "READY_TO_EAT"
          }
        },
        "lang": [
          {
            "_qualification": {
              "language": {
                "_key": 12,
                "_code": "fra"
              }
            },
            "servingSizeDescription": {
              "_current": "of general complaisance, and in all that he said
she heard an accent so removed from _hauteur_ or disdain"
            },
            "dailyValueIntakeReference": {
              "_current": "said Elizabeth, struck with other ideas. “She looks
sickly and cross. Yes, she will do for"
            }
          },
          {
            "_qualification": {
              "language": {
```

```
                  "_key": 16,
                  "_code": "ita"
                }
              },
              "servingSizeDescription": {
                "_current": "her mother. “And then when you go away, you may
leave one or two of my sisters behind you; and I dare say I shall get husbands
for them before the winter is over.” “I thank you for my share of the favour,”
said Elizabeth; “but I do not particularly like your way of getting husbands.”
Their visitors were not to remain above ten days with them. Mr. Wickham had
received his commission before he"
              },
              "dailyValueIntakeReference": {
                "_current": "duty of a young man who has been so fortunate as I
have been in early preferment; and I trust I am resigned. Perhaps not the less
so from feeling a doubt of my positive happiness had my fair cousin honoured me
with her hand; for I have often observed that resignation is n"
              }
            }
          ]
        }
      ],
      "_characteristicRecords": [
        {
          "_qualification": {
            "characteristic": {
              "_key": {
                "_entityId": 8000,
                "_internalId": "12424",
                "_externalId": "\u0027isHomogenised\u0027"
              },
              "_code": "isHomogenised"
            },
            "recordKey": "0000.0000.RK",
            "parentRecordKey": "root"
          },
          "_datatype": "LOOKUP",
          "order": {
            "_current": 1
          },
          "_recordLang": [
            {
              "_qualification": {
                "language": {
                  "_key": -1,
                  "_code": "zxx"
                }
              },
              "values": {
                "_current": [
                  {
                    "_entityId": 7300,
```

```
                    "_internalId": "1209@121",
                    "_externalId":
"\u0027FALSE\u0027@\u0027NonBinaryLogicCodes\u0027"
                  }
                ]
              }
            }
          ]
        },
        {
          "_qualification": {
            "characteristic": {
              "_key": {
                "_entityId": 8000,
                "_internalId": "12441",
                "_externalId": "\u0027nutrientFormatTypeCodeReference\u0027"
              },
              "_code": "nutrientFormatTypeCodeReference"
            },
            "recordKey": "0000.0000.RK",
            "parentRecordKey": "root"
          },
          "_datatype": "LOOKUP",
          "order": {
            "_current": 1
          },
          "_recordLang": [
            {
              "_qualification": {
                "language": {
                  "_key": -1,
                  "_code": "zxx"
                }
              },
              "values": {
                "_current": [
                  {
                    "_entityId": 7300,
                    "_internalId": "3494@151",
                    "_externalId":
"\u0027AB\u0027@\u0027nutrientFormatTypeCodeReference\u0027"
                  }
                ]
              }
            }
          ]
        },
        {
          "_qualification": {
            "characteristic": {
              "_key": {
                "_entityId": 8000,
```

```
        "_internalId": "12377",
        "_externalId": "\u0027juiceContentPercentage\u0027"
      },
      "_code": "juiceContentPercentage"
    },
    "recordKey": "0000.0000.RK",
    "parentRecordKey": "root"
  },
  "_datatype": "DECIMAL",
  "_formatPattern": "###.##",
  "order": {
    "_current": 1
  },
  "_recordLang": [
    {
      "_qualification": {
        "language": {
          "_key": -1,
          "_code": "zxx"
        }
      },
      "values": {
        "_current": [
          3169.03
        ]
      }
    }
  ]
},
{
  "_qualification": {
    "characteristic": {
      "_key": {
        "_entityId": 8000,
        "_internalId": "12339",
        "_externalId": "\u0027fatPercentageInDryMatter\u0027"
      },
      "_code": "fatPercentageInDryMatter"
    },
    "recordKey": "0000.0000.RK",
    "parentRecordKey": "root"
  },
  "_datatype": "DECIMAL",
  "_formatPattern": "##.###",
  "order": {
    "_current": 1
  },
  "_recordLang": [
    {
      "_qualification": {
        "language": {
          "_key": -1,
```

```
                "_code": "zxx"
              }
            },
            "values": {
              "_current": [
                4176.55
              ]
            }
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "12340",
              "_externalId": "\u0027isRindEdible\u0027"
            },
            "_code": "isRindEdible"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": {
          "_current": 1
        },
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": {
              "_current": [
                {
                  "_entityId": 7300,
                  "_internalId": "1296@121",
                  "_externalId":
"\u0027NOT_APPLICABLE\u0027@\u0027NonBinaryLogicCodes\u0027"
                }
              ]
            }
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
```

```
            "_key": {
              "_entityId": 8000,
              "_internalId": "12337",
              "_externalId": "\u0027cheeseMaturationPeriodDescription\u0027"
            },
            "_code": "cheeseMaturationPeriodDescription"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "TEXT",
        "order": {
          "_current": 1
        },
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": 17,
                "_code": "jpn"
              }
            },
            "values": {
              "_current": [
                "of his confidence in his friend. How grievous then was the"
              ]
            }
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "12338",
              "_externalId":
"\u0027cheeseMaturationProcessContainerTypeCode\u0027"
            },
            "_code": "cheeseMaturationProcessContainerTypeCode"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": {
          "_current": 1
        },
        "_recordLang": [
          {
            "_qualification": {
              "language": {
```

```
                  "_key": -1,
                  "_code": "zxx"
                }
              },
              "values": {
                "_current": [
                  {
                    "_entityId": 7300,
                    "_internalId": "2062@123",
                    "_externalId":
"\u0027BRINE_SOLUTION\u0027@\u0027cheeseMaturationProcessContainerTypeCode\u0027
"
                  }
                ]
              }
            }
          ]
        },
        {
          "_qualification": {
            "characteristic": {
              "_key": {
                "_entityId": 8000,
                "_internalId": "12375",
                "_externalId": "\u0027ingredientOfConcernCode\u0027"
              },
              "_code": "ingredientOfConcernCode"
            },
            "recordKey": "0000.0000.RK",
            "parentRecordKey": "root"
          },
          "_datatype": "LOOKUP",
          "order": {
            "_current": 1
          },
          "_recordLang": [
            {
              "_qualification": {
                "language": {
                  "_key": -1,
                  "_code": "zxx"
                }
              },
              "values": {
                "_current": [
                  {
                    "_entityId": 7300,
                    "_internalId": "2720@136",
                    "_externalId":
"\u0027RAW_MILK\u0027@\u0027ingredientOfConcernCode\u0027"
                  }
                ]
```

```
      }
    }
  ]
},
{
  "_qualification": {
    "characteristic": {
      "_key": {
        "_entityId": 8000,
        "_internalId": "12376",
        "_externalId": "\u0027ingredientStatement\u0027"
      },
      "_code": "ingredientStatement"
    },
    "recordKey": "0000.0000.RK",
    "parentRecordKey": "root"
  },
  "_datatype": "TEXT",
  "order": {
    "_current": 1
  },
  "_recordLang": [
    {
      "_qualification": {
        "language": {
          "_key": 11,
          "_code": "fin"
        }
      },
      "values": {
        "_current": [
          "anything about it, they found at last, on examining th"
        ]
      }
    }
  ]
},
{
  "_qualification": {
    "characteristic": {
      "_key": {
        "_entityId": 8000,
        "_internalId": "12341",
        "_externalId": "\u0027surfaceOfCheeseAtEndOfRipeningCode\u0027"
      },
      "_code": "surfaceOfCheeseAtEndOfRipeningCode"
    },
    "recordKey": "0000.0000.RK",
    "parentRecordKey": "root"
  },
  "_datatype": "LOOKUP",
  "order": {
```

```
            "_current": 1
          },
          "_recordLang": [
            {
              "_qualification": {
                "language": {
                  "_key": -1,
                  "_code": "zxx"
                }
              },
              "values": {
                "_current": [
                  {
                    "_entityId": 7300,
                    "_internalId": "1913@122",
                    "_externalId":
"\u0027NO_RIND\u0027@\u0027surfaceOfCheeseAtEndOfRipeningCode\u0027"
                  }
                ]
              }
            }
          ]
        },
        {
          "_qualification": {
            "characteristic": {
              "_key": {
                "_entityId": 8000,
                "_internalId": "12342",
                "_externalId": "\u0027dietTypeDescription\u0027"
              },
              "_code": "dietTypeDescription"
            },
            "recordKey": "0000.0000.RK",
            "parentRecordKey": "root"
          },
          "_datatype": "TEXT",
          "order": {
            "_current": 1
          },
          "_recordLang": [
            {
              "_qualification": {
                "language": {
                  "_key": 4,
                  "_code": "chi"
                }
              },
              "values": {
                "_current": [
                  "and, for a few moments, she flattered herself tha"
                ]
```

```
            }
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "12555",
              "_externalId": "\u0027doPackagingMaterialContainLatex\u0027"
            },
            "_code": "doPackagingMaterialContainLatex"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": {
          "_current": 1
        },
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": {
              "_current": [
                {
                  "_entityId": 7300,
                  "_internalId": "1294@121",
                  "_externalId":
"\u0027TRUE\u0027@\u0027NonBinaryLogicCodes\u0027"
                }
              ]
            }
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "12556",
              "_externalId": "\u0027numberOfPackagesForSerPiecesGTIN\u0027"
            },
            "_code": "numberOfPackagesForSerPiecesGTIN"
          },
```

```
      "recordKey": "0000.0000.RK",
      "parentRecordKey": "root"
    },
    "_datatype": "INTEGER",
    "order": {
      "_current": 1
    },
    "_recordLang": [
      {
        "_qualification": {
          "language": {
            "_key": -1,
            "_code": "zxx"
          }
        },
        "values": {
          "_current": [
            4410
          ]
        }
      }
    ]
  },
  {
    "_qualification": {
      "characteristic": {
        "_key": {
          "_entityId": 8000,
          "_internalId": "12332",
          "_externalId": "\u0027fatInMilkContent\u0027"
        },
        "_code": "fatInMilkContent"
      },
      "recordKey": "0000.0000.RK",
      "parentRecordKey": "root"
    },
    "_datatype": "DECIMAL",
    "_formatPattern": "##.###",
    "order": {
      "_current": 1
    },
    "_recordLang": [
      {
        "_qualification": {
          "language": {
            "_key": -1,
            "_code": "zxx"
          }
        },
        "values": {
          "_current": [
            58.92
```

```
              ]
            }
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "12333",
              "_externalId": "\u0027rennetTypeCode\u0027"
            },
            "_code": "rennetTypeCode"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": {
          "_current": 1
        },
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": {
              "_current": [
                {
                  "_entityId": 7300,
                  "_internalId": "1931@126",
                  "_externalId":
"\u0027VEGETABLE_RENNET\u0027@\u0027rennetTypeCode\u0027"
                }
              ]
            }
          }
        ]
      }
    ]
  }
}
```

Characteristic Values

Characteristic values are rendered in a specialized structure in order to honor their hierarchical nature. This structure is not identical to the repository as it is fully hierarchical. This is the reason for extra meta attributes which help to not get in conflict with repository based data.

Additional Meta Attributes

- `_characteristicRecords`
  Top level element for all characteristic records
- `_recordLang`
  Language specific record values. In case the characteristic is not language specific, the qualification for "not language specific" is returned (-1 or xxz)
  In case the value of the record is a lookup value and the includeLabels parameter has been set, an additional _label element is provided.
- `_children`
  children of the current characteristic record
- `_datatype`
  The characteristic data type - to be able to interpret the values even in case the characteristic is no longer in the system
- `_formatPattern`
  The format pattern of the characteristic - to be able to format the values even in case the characteristic is no longer in the system

Characteristic Records Example


```
{
  "_entity": "Article",
  "_entityItem": {
    "_internalId": "13989@1120",
    "_entityId": 1000,
    "_externalId": "\u0027supplierAID-840\u0027@\u0027NEW_CATALOG\u0027"
  },
  "_revision": {
    "_internalId": "1",
    "_entityId": 5600,
    "_externalId": "\u0027root\u0027"
  },
  "_container": {
    "_internalId": "1120",
    "_entityId": 7000,
    "_externalId": "\u0027NEW_CATALOG\u0027"
  },
  "_data": {
    "article": {
      "identifier": {
        "_current": "supplierAID-841"
      },
      "_characteristicRecords": [
        {
```

```
          "_qualification": {
            "characteristic": {
              "_key": {
                "_entityId": 8000,
                "_internalId": "7",
                "_externalId": "\u0027AnimalIngredient\u0027"
              },
              "_code": "AnimalIngredient"
            },
            "recordKey": "0000.0000.RK",
            "parentRecordKey": "root"
          },
          "_dataType": "LOOKUP",
          "order": {
            "_current": -32767
          },
          "_recordLang": [
            {
              "_qualification": {
                "language": {
                  "_key": -1,
                  "_code": "zxx"
                }
              },
              "values": {
                "_current": [
                  {
                    "_entityId": 7300,
                    "_internalId": "23@5",
                    "_externalId":
"\u0027Down\u0027@\u0027AnimalIngredient\u0027"
                  }
                ]
              }
            }
          ],
          "_children": [
            {
              "_qualification": {
                "characteristic": {
                  "_key": {
                    "_entityId": 8000,
                    "_internalId": "8",
                    "_externalId": "\u0027CertDown\u0027"
                  },
                  "_code": "CertDown"
                },
                "recordKey": "0000.0000.RK",
                "parentRecordKey": "0000.0000.RK"
              },
              "_dataType": "LOOKUP",
              "order": {
```

```
  "_current": -32766
},
"_recordLang": [
  {
    "_qualification": {
      "language": {
        "_key": -1,
        "_code": "zxx"
      }
    },
    "values": {
      "_current": [
        {
          "_entityId": 7300,
          "_internalId": "25@6",
          "_externalId": "\u0027Other\u0027@\u0027CertDown\u0027"
        }
      ]
    }
  }
],
"_children": [
  {
    "_qualification": {
      "characteristic": {
        "_key": {
          "_entityId": 8000,
          "_internalId": "11",
          "_externalId": "\u0027CertDownExpDate\u0027"
        },
        "_code": "CertDownExpDate"
      },
      "recordKey": "0000.0000.RK",
      "parentRecordKey": "0000.0000.RK"
    },
    "_dataType": "DATE",
    "order": {
      "_current": -32765
    },
    "_recordLang": [
      {
        "_qualification": {
          "language": {
            "_key": -1,
            "_code": "zxx"
          }
        },
        "values": {
          "_current": [
            "1977-05-28"
          ]
        }
```

```
        }
      ]
    }
  ]
},
{
  "_qualification": {
    "characteristic": {
      "_key": {
        "_entityId": 8000,
        "_internalId": "8",
        "_externalId": "\u0027CertDown\u0027"
      },
      "_code": "CertDown"
    },
    "recordKey": "0000.0000.RK",
    "parentRecordKey": "0000.0000.RK"
  },
  "_dataType": "LOOKUP",
  "order": {
    "_current": -32766
  },
  "_recordLang": [
    {
      "_qualification": {
        "language": {
          "_key": -1,
          "_code": "zxx"
        }
      },
      "values": {
        "_current": [
          {
            "_entityId": 7300,
            "_internalId": "25@6",
            "_externalId": "\u0027Other\u0027@\u0027CertDown\u0027"
          }
        ]
      }
    }
  ],
  "_children": [
    {
      "_qualification": {
        "characteristic": {
          "_key": {
            "_entityId": 8000,
            "_internalId": "11",
            "_externalId": "\u0027CertDownExpDate\u0027"
          },
          "_code": "CertDownExpDate"
        },
```

```
              "recordKey": "0000.0000.RK",
              "parentRecordKey": "0000.0000.RK"
            },
            "_dataType": "DATE",
            "order": {
              "_current": -32765
            },
            "_recordLang": [
              {
                "_qualification": {
                  "language": {
                    "_key": -1,
                    "_code": "zxx"
                  }
                },
                "values": {
                  "_current": [
                    "1977-05-28"
                  ]
                }
              }
            ]
          }
        ]
      }
    ]
  }
],
"_children": [
  {
    "_qualification": {
      "characteristic": {
        "_key": {
          "_entityId": 8000,
          "_internalId": "8",
          "_externalId": "\u0027CertDown\u0027"
        },
        "_code": "CertDown"
      },
      "recordKey": "0000.0000.RK",
      "parentRecordKey": "0000.0000.RK"
    },
    "_dataType": "LOOKUP",
    "order": {
      "_current": -32766
    },
    "_recordLang": [
      {
        "_qualification": {
          "language": {
            "_key": -1,
            "_code": "zxx"
```

```
            }
          },
          "values": {
            "_current": [
              {
                "_entityId": 7300,
                "_internalId": "25@6",
                "_externalId": "\u0027Other\u0027@\u0027CertDown\u0027"
              }
            ]
          }
        }
      ],
      "_children": [
        {
          "_qualification": {
            "characteristic": {
              "_key": {
                "_entityId": 8000,
                "_internalId": "11",
                "_externalId": "\u0027CertDownExpDate\u0027"
              },
              "_code": "CertDownExpDate"
            },
            "recordKey": "0000.0000.RK",
            "parentRecordKey": "0000.0000.RK"
          },
          "_dataType": "DATE",
          "order": {
            "_current": -32765
          },
          "_recordLang": [
            {
              "_qualification": {
                "language": {
                  "_key": -1,
                  "_code": "zxx"
                }
              },
              "values": {
                "_current": [
                  "1977-05-28"
                ]
              }
            }
          ]
        }
      ]
    }
  ]
}
```

```
  }
}
```

6.4.1.6 Examples

Please see the attached postman example collection for you convenience: ObjectAPI.postman_collection.json

Get Item by ID

```
curl --location --request GET 'http://localhost:1512/rest/V1.0/object/Article/
14260@1120' \
--header 'Content-Type: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI='
```

Get Item by Identifier As XML

```
curl --location --request GET 'http://localhost:1512/rest/V1.0/object/
Article/'\''supplierAID-42'\''@'\''MySupplierCatalog'\''' \
--header 'Content-Type: application/xml' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI='
```

Get Filtered Item (entity and qualification filter)

```
curl --location --request GET 'http://localhost:1512/rest/V1.0/object/
Article/'\''supplierAID-42'\''@'\''MySupplierCatalog'\''?
qualificationFilter=language(eng)&entityFilter=ArticleLang' \
--header 'Content-Type: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI='
```

6.5 REST Object API Read V2



Version 2 of the Object Read API introduces a new parameter "`includeIds`" and a streamlined response payload which no longer contains the extra "_current" element. Besides that, the response payload's _data element can directly be used for the Write API (see page 272).
If a field or entity is supported by the Object API V2 has now been bound to the newly introduced "supportsObjectApi" flag in the repository. A better control over the fields and entities which are supported is possible with this.

The object api provides a canonical rest api for entity items. Contrary to the ListAPI, the object api is optimized for single (multiple) item use and can return all data of the items in the full hierarchical form of them. It is not the correct api for mass data retrival of a small set of columns. Please use the ListApi for such use cases.

- Read Requests (see page 230)
  - Single Item (see page 230)
  - Multiple Items (see page 231)
  - Query Parameters (see page 233)
  - Permissions (see page 237)
    - Object Permissions (see page 237)
    - Field Permissions (see page 237)
    - Qualification Permissions (aka Qualified Field Permissions) (see page 237)
  - Result (see page 237)
    - Repository Entities and Fields (see page 237)
      - Names (see page 237)
      - Datatypes (see page 238)
    - Meta attributes (see page 238)
    - Data Element (see page 238)
    - Example Result (see page 239)
    - Characteristic Values (see page 264)
      - Additional Meta Attributes (see page 264)
      - Response Examples (see page 265)
  - Examples (see page 270)

6.5.1 Read Requests

6.5.1.1 Single Item

URL Pattern	`/object/{entity-identifier}/{entity_item}`
Method	`GET`

Accept	`application/json`
	`application/xml`
	If multiple accept headers are defined, they will be evaluated by their order.
	The wildcard media type (*/*) will result in XML.

Example	`/rest/V2.0/object/Article/4711@1` `/rest/V2.0/object/Article/'myItem'@'myCatalog'`
Return Codes	**Code** / **Reason** `200 (OK)` — Everything went fine `404 (Not Found)` — Object couldn't be found `403 (Forbidden)` — User has no read permission for this object `500` — Internal server error, please check the server logs

6.5.1.2 Multiple Items

This endpoint provides a multi-item interface in which the client can provide multiple items which should be returned in one call. As query parameters do have a size limitation, this api is designed as POST. This api has restrictions on the number of items which can be called. In case the limit is exceeded a corresponding return code is provided.
Contrary to the single item GET request, a missing object permission or an unknown item id does not lead to an error return code; it will just not be returned.

URL Pattern	`/object/{entity-identifier}/byItems`
Method	`POST`

Headers

Header	Supported Values

Accept	`application/json`
	`application/xml`
	If multiple accept headers are defined, they will be evaluated by their order.
	The wildcard media type (*/*) will result in XML.

Content-Type	`application/x-www-form-urlencoded`

Form Parameters

Parameter	Datatype	Description	Example

items	List of ENTITY_ITEM	String based entity item syntax.
		The form parameter can be provided multiple times and also
		as a comma separated list or even a mix of both
			`items=4711@1`
			`items='myItem'@'myCatalog'`
			`items=4711@1,'myItem'@'myCatalog'`

Example

`/rest/V2.0/object/Article/byItems`

Return Codes

Code	Reason
`200 (OK)`	Everything went fine
`413 (Payload too large)`	The amount of entity item ids exceeds the configured limit (default: 100). The response contains the "p360-ObjectAPI-MaxItems" header which returns the limit.
`500`	Internal server error, please check the server logs

6.5.1.3 Query Parameters

Please note that the qualification filter applies to all requested sub-entities which have this qualification. So, for example, if you filter for language=English, the ArticleLang and ArticleAttributeValueLang entities will both only return the English values!

Parameter	Required	Default	Datatype	Parameter description	Example
`entityFilter`	no	none	`String`	Comma separated list of entity identifiers which should be part of the result. If omitted, the full object with all data is returned. We recommend to provide a list of entities which are required in order to gain performance. Note: In case an `entityFilter` is provided, also the root entity is a filter value. So by only providing the root entity as filter value, only the fields of the root level are returned in the data.	Only return the root fields: `entityFilter=Article` Only return the root fields, and the sales prices `entityFilter=Article,ArticleSalesPrice` Only return the language specific data like Short and Long Description `entityFilter=ArticleLang`
`qualificationFilter`	no	none	`String`	This parameter allows to restrict the output to certain qualifications. One example would be a filter so that only Euro and US-Dollar prices for the customer Informatica are returned. The filter string is a comma-separated list of qualification settings. A qualification setting is a *qualification name* followed by a comma-separated list of values which is put into parentheses . The qualification name is defined in the repository and is included in the Meta-API.	Example for prices in Euro and US-Dollar and customer Informatica: `qualificationFilter=currency(EUR,USD),customer(Informatica)`

Parameter	Required	Default	Datatype	Parameter description	Example

`revision`	no	root	`ENTITY_ITEM`	The revision for which the data should be retrieved	`revision='root'`
					`revision=1`
					`revision='myRevisionIdentifier'`
					`revision=4711` (where 4711 is the internal id of the revision).

`includeLabels`	no	`false`	`boolean`	If set to true, the returned document will contain _label elements for all fields or qualifications which have an enumeration. The label will be returned in the locale of the request like this: `"orderUnit" : { "_current" : { "_key" : { "_entityId" : 3100, "_internalId" : "932", "_externalId" : "'C62'" }, "_code" : "C62", "_label" : "piece" } }`	`includeLabels=true` `includeLabels=false`

Parameter	Required	Default	Datatype	Parameter description	Example

`includeIds`	no	`false`	`boolean`	If set to false, the payload avoids to contain any internal ids which might be system specific. This specifically applies to the ENTITY_ITEM datatype which will then no longer contain the "`_internalId`" element, only the "`_externalId`". Also the "`_entityId`" element will be replaced by an "`entity`" element which then holds the alphanumeric identifier of the entity and not it's numeric id.
				
				While reading and writing to the same Product 360 environment, we always recommend to include the ids since write performance is increased as no internal lookup needs to be done. However, when reading from one system and applying to another P360 system (different database schemas!), `includeIds` should be set to `false`, otherwise the write operation might fail since the internal ids do not match to the target system.
				
				In case the object is read for a 3rd party system, you might very well disable the `internalIds` to further reduce the response size!
					`includeIds=true`
					
					`includeIds=false`

Parameter	Required	Default	Datatype	Parameter description	Example
`includeMimeValueArchive`	no	`false`	`boolean`	If set to true a ZIP Archive will be provided on the server which can be downloaded using the File API (see page 339). This archive will contain all MIME_VALUE binaries of all objects which have been returned by the call to the object API. Is the single item read request is used, only the mime values for this single item are part of the archive. In case the multi-item read request is used the archive will contain the values for all items.	

6.5.1.4 Permissions

Object Permissions

The user needs to have the READ object permission for this API. In case he doesn't, the api returns HTTP 403 (forbidden) in case of the single item call with GET, and skip this item in case of the multiple items call.

Field Permissions

The user needs to have the READ field permissions for a field. If he doesn't, the field will not be part of the data element, but no error is returned.

Qualification Permissions (aka Qualified Field Permissions)

The user needs to have the READ permission for a qualification, e.g. for Language = English. If he doesn't, sub-entities which are qualified for this will not be part of the data element, but no error is returned.

6.5.1.5 Result

Repository Entities and Fields

Names

A new repository property "shortIdentifier" has been added in the custom section which is used to define the entity and field names for the json structure. The short identifier is unique within its parent entity only!

The standard repository already contains a short identifier for all fields and enabled qualifications.

Datatypes

- ENTITY_ITEM objects. Depending on the includeIds parameter they contain a different subset:
  - `includeIds = true` : `_entityId, _internalId` and `_externalId`
  - `includeIds = false` : `_entity` and `_externalId`
- MIME_VALUE objects always contain the relative `_filePath` , the `_label` and the `_mimeType`
- timestamp: ISO 8601, e.g.: `2012-04-23T18:25:43.511Z`
- date: ISO 8601, e.g.: `2019-07-04`
- numbers: standard JSON
- "no value" is either provided as null, or the attribute is not in the document.
- Empty string is returned as null, which is omitted when possible!
- Enumeration fields
  - `_key` : always holds the unique key of the enumeration entry
    In case the enumeration field is of datatype ENTITY_ITEM, then the key will be provided with the ENTITY_ITEM syntax (see above)
    The key is omitted in case `includeIds` parameter is set to `false` , and the enumeration entry does have a code.
  - `_code` : the external code of the enumeration entry, if the enum entry has one, and it differs from the key!
  - `_label` : the locale dependent label of the enumeration entry. Only if `includeLabels` is set to true. The locale of the http request is used for this.

Meta attributes

All meta attributes are prefixed with an underscore.

- `_entity` = the root entity identifier
- `_entityItem` = the entity item in the ENTITY_ITEM syntax
- `_revision` = the revision of the data as ENTITY_ITEM
- `_data` = The actual entity item's data
- `_qualification` = the qualification of the record. **The combination of the qualification values in the qualification object define this record as unique within its parent**.

Data Element

In order to be part of the data element of the object api, a sub-entity or field must fulfill multiple criteria:

- The entity or field must be `active` in the repository
- The entity or field must have `supportsObjectApi = true` in the repository.
  In case the corresponding FieldType or EntityType does not support object api, enabling it in the custom area has no effect.
- The user must have `READ permission` for the `Entity` or `Field`
- The user must have `READ permission` for the `qualification` of the sub-entity

- The field must must `not be a password Field` in the repository
- `Qualifications` must be `editable` in the repository

Entities, Fields or Qualifications which do not comply to any of these points will be omitted in the data element. No exception is thrown.

Example Result

`includeIds = true`	`includeIds = false`

{
"_entity": "Article",
"_entityItem": {
"_internalId": "18@1",
"_entityId": 1000,
"_externalId":
"\u0027supplierAID-1\u0027@\u0027MASTER\u0027"
},
"_revision": {
"_internalId": "1",
"_entityId": 5600,
"_externalId": "\u0027root\u0027"
},
"_container": {
"_internalId": "1",
"_entityId": 2900,
"_externalId": "\u0027MASTER\u0027"
},
"_data": {
"identifier": "supplierAID-1",
"manufacturerAID": "A000000000000",
"noCUperOU": 907.3600,
"deliveryTime": 1.6000,
"currentStatus": {
"_key": 100,
"_code": "NEW"
},
"manufacturerName": "UQ Communications",
"catalog": {
"_key": {
"_entityId": 2900,
"_internalId": "1",
"_externalId": "\u0027MASTER\u0027"
},
"_code": "MASTER"
},
"quantityMin": 2.0000,
"mainSupplier": {
"_key": {
	{
	"_entity": "Article",
	"_entityItem": {
	"_entity": "Article",
	"_externalId":
	"\u0027supplierAID-1\u0027
	@\u0027MASTER\u0027"
	},
	"_revision": {
	"_entity": "Revision",
	"_externalId":
	"\u0027root\u0027"
	},
	"_container": {
	"_entity":
	"MasterCatalog",
	"_externalId":
	"\u0027MASTER\u0027"
	},
	"_data": {
	"identifier":
	"supplierAID-1",
	"manufacturerAID":
	"A000000000000",
	"noCUperOU": 907.3600,
	"deliveryTime":
	1.6000,
	"currentStatus": {
	"_code": "NEW"
	},
	"manufacturerName":
	"UQ Communications",
	"catalog": {
	"_code": "MASTER"
	},
	"quantityMin": 2.0000,
	"mainSupplier": {
	"_code": "Heiler
	Product Manager"

```
        "_entityId": 2800,
        "_internalId": "3",
        "_externalId": "\u0027Heiler Product
Manager\u0027"
      },
      "_code": "Heiler Product Manager"
    },
    "priceQuantity": 501.0400,
    "kitComponent": false,
    "contentUnit": {
      "_key": {
        "_entityId": 3100,
        "_internalId": "536",
        "_externalId": "\u0027MQH\u0027"
      },
      "_code": "MQH"
    },
    "quantityInterval": 1.0000,
    "kitParent": false,
    "orderUnit": {
      "_key": {
        "_entityId": 3100,
        "_internalId": "539",
        "_externalId": "\u0027CMQ\u0027"
      },
      "_code": "CMQ"
    },
    "soldOnlyInKits": false,
    "lang": [
      {
        "_qualification": {
          "language": {
            "_key": 7,
            "_code": "deu"
          }
        },
        "keywords": [
          "disinterestedness",
          "incomprehensible"
        ],
        "descriptionLong": "one side. Elizabeth
smiled at the recollection of all that she had
heard of its inhabitants. At length the
Parsonage was discernible. The garden sloping to
the road, the house standing in it, the green
pales, and the laurel hedge, everything declared
they were arriving. Mr. Collins and Charlotte
appeared at the door, and the carriage stopped
at the small gate which led by a short gravel
walk to the house, amidst the nods and smiles of
the whole party. In a moment they were all out
of the chaise, rejoicing at the sight of each
    },
    "priceQuantity":
501.0400,
    "kitComponent": false,
    "contentUnit": {
      "_code": "MQH"
    },
    "quantityInterval":
1.0000,
    "kitParent": false,
    "orderUnit": {
      "_code": "CMQ"
    },
    "soldOnlyInKits":
false,
    "lang": [
      {
        "_qualification":
{
          "language": {
            "_code": "eng"
          }
        },
        "keywords": [
          "curious",

"disinterestedness"
        ],

"descriptionShort":
"thanks and assurances of
happiness. She had spent
six weeks with great
enjoyment; and the
pleasure of being with
Charlotte, and the"
      },
      {
        "_qualification":
{
          "language": {
            "_code": "fra"
          }
        },
        "keywords": [

"communicativeness"
        ],
        "descriptionLong":
"yet it would seem, by her
manner of talking, as if
she wanted to persuade
```

```
other. Mrs. Collins welcomed her friend with the liveliest pleasure, and Elizabeth was more and more satisfied with coming when she found herself so affectionately received. She saw instantly that her cousin’s manners w",
        "descriptionShort": "a few minutes’ conversation with Charlotte, but was scarcely ever prevailed upon to get out. Very few days passed in which Mr."
      },
      {
        "_qualification": {
          "language": {
            "_key": 12,
            "_code": "fra"
          }
        },
        "keywords": [
          "communicativeness"
        ],
        "descriptionLong": "yet it would seem, by her manner of talking, as if she wanted to persuade herself that he is really partial to Miss Darcy. I cannot understand it. If I were not afraid of judging harshly, I should be almost tempted to say that there is a strong appearance of duplicity in all this. But I will endeavour to banish every painful thought, and think only of what will make me happy—your affection, and the invariable kindness of my dear uncle and aunt. Let me hear from you very soon. Miss Bingley said something of his never returning to Netherfield again, of giving up the house, but not with any certainty. We had better not mention it. I am extremely glad that you have such pleasant accounts from our friends at Hunsford. Pray go to see them, with Sir William and Maria. I am sure you",
        "descriptionShort": "it is not true. A great many changes have happened in the neighbourhood, since you went away. Miss Lucas is married and settled. And"
      },
      {
        "_qualification": {
          "language": {
            "_key": 9,
            "_code": "eng"
          }
        },
        "keywords": [
          "curious",
          "disinterestedness"
herself that he is really partial to Miss Darcy. I cannot understand it. If I were not afraid of judging harshly, I should be almost tempted to say that there is a strong appearance of duplicity in all this. But I will endeavour to banish every painful thought, and think only of what will make me happy—your affection, and the invariable kindness of my dear uncle and aunt. Let me hear from you very soon. Miss Bingley said something of his never returning to Netherfield again, of giving up the house, but not with any certainty. We had better not mention it. I am extremely glad that you have such pleasant accounts from our friends at Hunsford. Pray go to see them, with Sir William and Maria. I am sure you",

"descriptionShort": "it is not true. A great many changes have happened in the neighbourhood, since you went away. Miss Lucas is married and settled. And"
      },
      {
        "_qualification": {
          "language": {
            "_code": "deu"
          }
        },
        "keywords": [

"disinterestedness",

"incomprehensible"
        ],
        "descriptionLong": "one side. Elizabeth
```

```
        ],
        "descriptionShort": "thanks and
assurances of happiness. She had spent six weeks
with great enjoyment; and the pleasure of being
with Charlotte, and the"
      },
      {
        "_qualification": {
          "language": {
            "_key": 10,
            "_code": "esl"
          }
        },
        "keywords": [
          "intermarriage",
          "disinterestedness",
          "stretched"
        ],
        "descriptionLong": "invitation was
accepted of course, and at a proper hour they
joined the party in Lady Catherine’s drawing-
room. Her ladyship received them civilly, but it
was plain that their company was by no means so
acceptable as when she could get nobody else;
and she was, in fact, almost engrossed by her
nephews, speaking to them, especially to Darcy,
mu",
        "descriptionShort": "in the best manner
that his profession might allow—and if he took
orders, desired that a valuable family living
might be his as soon as it becam"
      }
    ],
    "pricePurchase": [
      {
        "_qualification": {
          "supplier": {
            "_key": {
              "_entityId": 2800,
              "_internalId": "3",
              "_externalId": "\u0027Heiler
Product Manager\u0027"
            },
            "_code": "Heiler Product Manager"
          },
          "currency": {
            "_key": "GBP"
          },
          "type": {
            "_key": 1,
            "_code": "net_list"
          },
```


smiled at the recollection of all that she had heard of its inhabitants. At length the Parsonage was discernible. The garden sloping to the road, the house standing in it, the green pales, and the laurel hedge, everything declared they were arriving. Mr. Collins and Charlotte appeared at the door, and the carriage stopped at the small gate which led by a short gravel walk to the house, amidst the nods and smiles of the whole party. In a moment they were all out of the chaise, rejoicing at the sight of each other. Mrs. Collins welcomed her friend with the liveliest pleasure, and Elizabeth was more and more satisfied with coming when she found herself so affectionately received. She saw instantly that her cousin’s manners w",


```
"descriptionShort": "a few
minutes’ conversation with
Charlotte, but was
scarcely ever prevailed
upon to get out. Very few
days passed in which Mr."
      },
      {
        "_qualification":
{
          "language": {
            "_code": "esl"
          }
        },
        "keywords": [
          "intermarriage",
          "stretched",

"disinterestedness"
        ],
        "descriptionLong":
"invitation was accepted
```

```
          "territory": {
            "_key": "DE"
          },
          "validFrom": "1899-12-30",
          "validTo": "9999-12-31"
        },
        "value": [
          {
            "_qualification": {
              "lowerBound": 3.0000
            },
            "amount": 224648.610000,
            "factor": 1.0000
          }
        ]
      }
    ],
    "priceSales": [
      {
        "_qualification": {
          "currency": {
            "_key": "CHF"
          },
          "type": {
            "_key": 5,
            "_code": "nrp"
          },
          "customer": {
            "_key": {
              "_entityId": 2800,
              "_internalId": "1",
              "_externalId":
"\u0027Public\u0027"
            }
          },
          "territory": {
            "_key": "DE"
          },
          "validFrom": "1899-12-30",
          "validTo": "9999-12-31"
        },
        "value": [
          {
            "_qualification": {
              "lowerBound": 1.0000
            },
            "amount": 45558.800000,
            "factor": 1.0000
          }
        ]
      },
      {
```



```
of course, and at a proper
hour they joined the party
in Lady Catherine’s
drawing-room. Her ladyship
received them civilly, but
it was plain that their
company was by no means so
acceptable as when she
could get nobody else; and
she was, in fact, almost
engrossed by her nephews,
speaking to them,
especially to Darcy, mu",

"descriptionShort": "in
the best manner that his
profession might allow—and
if he took orders, desired
that a valuable family
living might be his as
soon as it becam"
      }
    ],
    "priceSales": [
      {
        "_qualification":
{
          "currency": {
            "_code": "CHF"
          },
          "type": {
            "_code": "nrp"
          },
          "customer": {
            "_key": {
              "_entity":
"Party",

"_externalId":
"\u0027Public\u0027"
            }
          },
          "territory": {
            "_code": "DE"
          },
          "validFrom":
"1899-12-30",
          "validTo":
"9999-12-31"
        },
        "value": [
          {
```

```
        "_qualification": {
          "currency": {
            "_key": "USD"
          },
          "type": {
            "_key": 3,
            "_code": "net_customer"
          },
          "customer": {
            "_key": {
              "_entityId": 2800,
              "_internalId": "1",
              "_externalId":
"\u0027Public\u0027"
            }
          },
          "territory": {
            "_key": "DE"
          },
          "validFrom": "1899-12-30",
          "validTo": "9999-12-31"
        },
        "value": [
          {
            "_qualification": {
              "lowerBound": 3.0000
            },
            "amount": 18416.580000,
            "factor": 1.0000
          }
        ]
      }
    ],
    "logistic": [
      {
        "_qualification": {
          "territory": {
            "_key": "MX"
          }
        },
        "grossWeight": 32.3000,
        "packageWeight": 23.0700,
        "packageUnit": {
          "_key": {
            "_entityId": 3100,
            "_internalId": "7042",
            "_externalId": "\u0027HEP\u0027"
          },
          "_code": "HEP"
        },
        "originCountry": {
          "_key": "AT"
        },
        "_qualification": {

"lowerBound": 1.0000
            },
            "amount":
45558.800000,
            "factor":
1.0000
          }
        ]
      },
      {
        "_qualification":
{
          "currency": {
            "_code": "USD"
          },
          "type": {
            "_code":
"net_customer"
          },
          "customer": {
            "_key": {
              "_entity":
"Party",

"_externalId":
"\u0027Public\u0027"
            }
          },
          "territory": {
            "_code": "DE"
          },
          "validFrom":
"1899-12-30",
          "validTo":
"9999-12-31"
        },
        "value": [
          {

"_qualification": {

"lowerBound": 3.0000
            },
            "amount":
18416.580000,
            "factor":
1.0000
          }
        ]
      }
    ],
```

```
        },
        "grossWeightUnit": {
          "_key": {
            "_entityId": 3100,
            "_internalId": "7119",
            "_externalId": "\u0027mL/(72.h)
\u0027"
          },
          "_code": "mL/(72.h)"
        }
      }
    ],
    "logisticExtension": [
      {
        "_qualification": {
          "packagingUnit": {
            "_key": {
              "_entityId": 3100,
              "_internalId": "1027",
              "_externalId": "\u0027DI\u0027"
            },
            "_code": "DI"
          },
          "language": {
            "_key": 7,
            "_code": "deu"
          },
          "party": {
            "_key": {
              "_entityId": 2800,
              "_internalId": "3",
              "_externalId": "\u0027Heiler
Product Manager\u0027"
            },
            "_code": "Heiler Product Manager"
          }
        },
        "length": 99122788.3034,
        "code128": "090409902676672680 90",
        "code39": "27655316938891158712",
        "lengthUnit": {
          "_key": {
            "_entityId": 3100,
            "_internalId": "3013",
            "_externalId":
"\u0027mig_WA_DATANORM40\u0027"
          },
          "_code": "mig_WA_DATANORM40"
        },
        "heightUnit": {
          "_key": {
            "_entityId": 3100,
    "pricePurchase": [
      {
        "_qualification":
{
          "supplier": {
            "_code":
"Heiler Product Manager"
          },
          "currency": {
            "_code": "GBP"
          },
          "type": {
            "_code":
"net_list"
          },
          "territory": {
            "_code": "DE"
          },
          "validFrom":
"1899-12-30",
          "validTo":
"9999-12-31"
        },
        "value": [
          {
"_qualification": {
"lowerBound": 3.0000
            },
            "amount":
224648.610000,
            "factor":
1.0000
          }
        ]
      }
    ],
    "logistic": [
      {
        "_qualification":
{
          "territory": {
            "_code": "MX"
          }
        },
        "grossWeight":
32.3000,
        "packageWeight":
23.0700,
        "packageUnit": {
          "_code": "HEP"
```

```
            "_internalId": "804",
            "_externalId": "\u0027BH\u0027"
          },
          "_code": "BH"
        },
        "height": 36999719.5867
      },
      {
        "_qualification": {
          "packagingUnit": {
            "_key": {
              "_entityId": 3100,
              "_internalId": "456",
              "_externalId": "\u0027KL\u0027"
            },
            "_code": "KL"
          },
          "language": {
            "_key": 9,
            "_code": "eng"
          },
          "party": {
            "_key": {
              "_entityId": 2800,
              "_internalId": "3",
              "_externalId": "\u0027Heiler
Product Manager\u0027"
            },
            "_code": "Heiler Product Manager"
          }
        },
        "length": 28554703.0225,
        "code128": "82915251799710733746",
        "code39": "21863593155379329992",
        "lengthUnit": {
          "_key": {
            "_entityId": 3100,
            "_internalId": "131",
            "_externalId": "\u0027RM\u0027"
          },
          "_code": "RM"
        },
        "heightUnit": {
          "_key": {
            "_entityId": 3100,
            "_internalId": "404",
            "_externalId": "\u0027JOU\u0027"
          },
          "_code": "JOU"
        },
        "height": 27955841.1420
      },
```

```
        },
        "originCountry": {
          "_code": "AT"
        },
        "grossWeightUnit":
{
          "_code": "mL/
(72.h)"
        }
      }
    ],
    "logisticExtension": [
      {
        "_qualification":
{
          "packagingUnit":
{
            "_code": "DI"
          },
          "language": {
            "_code": "deu"
          },
          "party": {
            "_code":
"Heiler Product Manager"
          }
        },
        "length":
99122788.3034,
        "code128":
"09040990267667268090",
        "code39":
"27655316938891158712",
        "lengthUnit": {
          "_code":
"mig_WA_DATANORM40"
        },
        "heightUnit": {
          "_code": "BH"
        },
        "height":
36999719.5867
      },
      {
        "_qualification":
{
          "packagingUnit":
{
            "_code": "KL"
          },
          "language": {
            "_code": "eng"
```

```
      {
        "_qualification": {
          "packagingUnit": {
            "_key": {
              "_entityId": 3100,
              "_internalId": "1013",
              "_externalId": "\u0027WR\u0027"
            },
            "_code": "WR"
          },
          "language": {
            "_key": 29,
            "_code": "sve"
          },
          "party": {
            "_key": {
              "_entityId": 2800,
              "_internalId": "3",
              "_externalId": "\u0027Heiler
Product Manager\u0027"
            },
            "_code": "Heiler Product Manager"
          }
        },
        "length": 53266349.3016,
        "code128": "55240848258305284982",
        "code39": "44402274049146575392",
        "lengthUnit": {
          "_key": {
            "_entityId": 3100,
            "_internalId": "378",
            "_externalId": "\u0027HTZ\u0027"
          },
          "_code": "HTZ"
        },
        "heightUnit": {
          "_key": {
            "_entityId": 3100,
            "_internalId": "928",
            "_externalId": "\u0027D79\u0027"
          },
          "_code": "D79"
        },
        "height": 37828907.4796
      }
    ],
    "referencedItem": [
      {
        "_qualification": {
          "referencedIdentifier":
"supplierAID-889",
          "referencedCatalogIdentifier":
          },
          "party": {
            "_code":
"Heiler Product Manager"
          }
        },
        "length":
28554703.0225,
        "code128":
"82915251799710733746",
        "code39":
"21863593155379329992",
        "lengthUnit": {
          "_code": "RM"
        },
        "heightUnit": {
          "_code": "JOU"
        },
        "height":
27955841.1420
      },
      {
        "_qualification":
{
          "packagingUnit":
{
            "_code": "WR"
          },
          "language": {
            "_code": "sve"
          },
          "party": {
            "_code":
"Heiler Product Manager"
          }
        },
        "length":
53266349.3016,
        "code128":
"55240848258305284982",
        "code39":
"44402274049146575392",
        "lengthUnit": {
          "_code": "HTZ"
        },
        "heightUnit": {
          "_code": "D79"
        },
        "height":
37828907.4796
      }
    ],
```

```
"MASTER",
          "type": {
            "_key": 9,
            "_code": "consists_of"
          }
        },
        "quantity": 1,
        "referencedItem": {
          "_entityId": 1000,
          "_internalId": "326@1",
          "_externalId":
"\u0027supplierAID-889\u0027@\u0027MASTER\u0027"
        }
      }
    ],
    "structureMap": [
      {
        "_qualification": {
          "structure": {
            "_key": 15,
            "_code": "ECLASS-6.1"
          }
        },
        "structureGroups": [
          {
            "_entityId": 3000,
            "_internalId": "22884@15",
            "_externalId":
"\u0027AKL536004\u0027@\u0027ECLASS-6.1\u0027"
          }
        ],
        "manualMap": [
          "36-13-90-90"
        ]
      }
    ],
    "structureGroupMap": [
      {
        "_qualification": {
          "structure": {
            "_key": {
              "_entityId": 2300,
              "_internalId": "15",
              "_externalId":
"\u0027ECLASS-6.1\u0027"
            },
            "_code": "ECLASS-6.1"
          },
          "structureGroup": {
            "_entityId": 3000,
            "_internalId": "22884@15",
            "_externalId":
"\u0027AKL536004\u0027@\u0027ECLASS-6.1\u0027"
```

```
    "referencedItem": [
      {
        "_qualification":
{

"referencedIdentifier":
"supplierAID-889",

"referencedCatalogIdentifi
er": "MASTER",
          "type": {
            "_code":
"consists_of"
          }
        },
        "quantity": 1,
        "referencedItem":
{
          "_entity":
"Article",
          "_externalId":
"\u0027supplierAID-889\u00
27@\u0027MASTER\u0027"
        }
      }
    ],
    "structureMap": [
      {
        "_qualification":
{
          "structure": {
            "_code":
"ECLASS-6.1"
          }
        },
        "structureGroups":
[
          {
            "_entity":
"StructureGroup",
            "_externalId":
"\u0027AKL536004\u0027@\u0
027ECLASS-6.1\u0027"
          }
        ],
        "manualMap": [
          "36-13-90-90"
        ]
      }
    ],
    "structureGroupMap": [
      {
```

```
          }
        }
      }
    ],
    "mediaAsset": [
      {
        "_qualification": {
          "type": {
            "_key": "BACK_VIEW"
          }
        },
        "mediaAsset": {
          "_entityId": 2400,
          "_internalId": "22",
          "_externalId":
"\u0027MediaAsset_1643730037754020\u0027"
        },
        "name": "not have made the offer of your
hand in any possible way that would have te",
        "category": "preservative",
        "document": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              },
              "quality": {
                "_key": "highres"
              }
            },
            "identifier": "hlr-system/
ugngddhkkynrymx.jpg",
            "imageIdentifier": "hlr-system/
ugngddhkkynrymx.jpg",
            "order": 1
          }
        ]
      },
      {
        "_qualification": {
          "type": {
            "_key": "thumbnail"
          }
        },
        "mediaAsset": {
          "_entityId": 2400,
          "_internalId": "8",
          "_externalId":
"\u0027MediaAsset_1643730037754004\u0027"
        },
        "name": "settled, that had given the",
        "_qualification":
{
          "structure": {
            "_code":
"ECLASS-6.1"
          },

"structureGroup": {
            "_entity":
"StructureGroup",
            "_externalId":
"\u0027AKL536004\u0027@\u0
027ECLASS-6.1\u0027"
          }
        }
      }
    ],
    "mediaAsset": [
      {
        "_qualification":
{
          "type": {
            "_code":
"thumbnail"
          }
        },
        "mediaAsset": {
          "_entity":
"MediaAsset",
          "_externalId":
"\u0027MediaAsset_16437300
37754004\u0027"
        },
        "name": "settled,
that had given the",
        "category":
"lessened",
        "document": [
          {

"_qualification": {
              "language":
{
                "_code":
"dut"
              },
              "quality": {
                "_key":
"highres"
              }
            },
            "identifier":
```

```
        "category": "lessened",
        "document": [
          {
            "_qualification": {
              "language": {
                "_key": 19,
                "_code": "dut"
              },
              "quality": {
                "_key": "highres"
              }
            },
            "identifier": "hlr-system/
uwhrxrrlxhohqnn.jpg",
            "imageIdentifier": "hlr-system/
uwhrxrrlxhohqnn.jpg",
            "order": 1
          }
        ]
      }
    ],
    "log": [
      {
        "_qualification": {
          "channel": {
            "_key": "HPM"
          }
        },
        "creationUser": {
          "_key": {
            "_entityId": 2600,
            "_internalId": "1",
            "_externalId":
"\u0027Administrator\u0027"
          },
          "_code": "Administrator"
        },
        "creationDate":
"2022-03-11T14:52:36.300Z"
      }
    ],
    "ownLog": [
      {
        "modificationDate":
"2022-03-11T14:52:36.583Z"
      }
    ],
    "marketing": [
      {
        "_qualification": {
          "targetMarket": {
            "_key": "EN"
"hlr-system/
uwhrxrrlxhohqnn.jpg",

"imageIdentifier": "hlr-
system/
uwhrxrrlxhohqnn.jpg",
            "order": 1
          }
        ]
      },
      {
        "_qualification":
{
          "type": {
            "_code":
"BACK_VIEW"
          }
        },
        "mediaAsset": {
          "_entity":
"MediaAsset",
          "_externalId":
"\u0027MediaAsset_16437300
37754020\u0027"
        },
        "name": "not have
made the offer of your
hand in any possible way
that would have te",
        "category":
"preservative",
        "document": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              },
              "quality": {
                "_key":
"highres"
              }
            },
            "identifier":
"hlr-system/
ugngddhkkynrymx.jpg",

"imageIdentifier": "hlr-
system/
ugngddhkkynrymx.jpg",
            "order": 1
```

```
          }
        },
        "specialItemCode": [
          {
            "_key": "GIFT_WITH_PURCHASE"
          }
        ],
        "couponFamilyCode": [
          "23736760493674713570",
          "59114057533905607311"
        ],
        "lang": [
          {
            "_qualification": {
              "sequence": 1,
              "language": {
                "_key": 12,
                "_code": "fra"
              }
            },
            "marketingMessage": "not so good an
income as yours. Do you draw?” “No, not at all.”
“What, none of you?” “Not one.” “That is very
strange. But I suppose you had no opportunity.
Your mother should have taken you to town every
spring for the benefit of masters.” “My mother
would have had no objection, but my father hates
London.” “Has your governess left you?” “We
never had any governess.” “No governess! How was
that possible? Five daughters brought up at home
without a governess! I never"
          },
          {
            "_qualification": {
              "sequence": 0,
              "language": {
                "_key": 10,
                "_code": "esl"
              }
            },
            "marketingMessage": "brother-in-law,
Mr. Hurst, merely looked the gentleman; but his
frie"
          }
        ]
      },
      {
        "_qualification": {
          "targetMarket": {
            "_key": "DE"
          }
        },
          }
        ]
      }
    ],
    "log": [
      {
        "_qualification":
{
          "channel": {
            "_code": "HPM"
          }
        },
        "creationUser": {
          "_code":
"Administrator"
        },
        "creationDate":
"2022-03-11T14:52:36.300Z"
      }
    ],
    "ownLog": [
      {

"modificationDate":
"2022-03-11T14:52:36.583Z"
      }
    ],
    "marketing": [
      {
        "_qualification":
{
          "targetMarket":
{
            "_code": "DE"
          }
        },
        "specialItemCode":
[
          {
            "_code":
"LIMITED_PRODUCTION"
          }
        ],

"couponFamilyCode": [

"83337210683270677496",

"68772233910188927019"
        ],
        "lang": [
          {
```

```
        "specialItemCode": [
          {
            "_key": "LIMITED_PRODUCTION"
          }
        ],
        "couponFamilyCode": [
          "83337210683270677496",
          "68772233910188927019"
        ],
        "lang": [
          {
            "_qualification": {
              "sequence": 1,
              "language": {
                "_key": 9,
                "_code": "eng"
              }
            },
            "marketingMessage": "It was a
journey of only twenty-four miles, and they
began it so early as to be in Gracechurch Street
by noon. As they drove to Mr. Gardiner’s door,
Jane was at a drawing-room window watching
their"
          },
          {
            "_qualification": {
              "sequence": 0,
              "language": {
                "_key": 7,
                "_code": "deu"
              }
            },
            "marketingMessage": "a mixture of
pride and obsequiousness, self-importance and
humility. Having now a good house and a very
sufficient income, he intended to marry; and in
seeking a reconciliation with"
          }
        ]
      }
    ],
    "_characteristicRecords": [
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "829",
              "_externalId":
"\u0027nutrientFormatTypeCodeReference\u0027"
            },
"_qualification": {
              "sequence":
1,
              "language":
{
                "_code":
"eng"
              }
            },

"marketingMessage": "It
was a journey of only
twenty-four miles, and
they began it so early as
to be in Gracechurch
Street by noon. As they
drove to Mr. Gardiner’s
door, Jane was at a
drawing-room window
watching their"
          },
          {

"_qualification": {
              "sequence":
0,
              "language":
{
                "_code":
"deu"
              }
            },

"marketingMessage": "a
mixture of pride and
obsequiousness, self-
importance and humility.
Having now a good house
and a very sufficient
income, he intended to
marry; and in seeking a
reconciliation with"
          }
        ]
      },
      {
        "_qualification":
{
          "targetMarket":
{
            "_code": "EN"
```

```
            "_code":
"nutrientFormatTypeCodeReference"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "4129@305",
                  "_externalId":
"\u0027US_FDA_SFP_MULTI_VITAMINS_IN_PACKAGES\u00
27@\u0027nutrientFormatTypeCodeReference\u0027"
                },
                "_code":
"US_FDA_SFP_MULTI_VITAMINS_IN_PACKAGES"
              },
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "4056@305",
                  "_externalId":
"\u0027US_FDA_NFP_2020_STANDARD_INFANTS_TO_12_MO
NTHS\u0027@\u0027nutrientFormatTypeCodeReference
\u0027"
                },
                "_code":
"US_FDA_NFP_2020_STANDARD_INFANTS_TO_12_MONTHS"
              },
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "2997@305",
                  "_externalId":
"\u0027NUTRIENT_DATA_EXEMPT\u0027@\u0027nutrient
FormatTypeCodeReference\u0027"
                },
                "_code": "NUTRIENT_DATA_EXEMPT"
              }
            ]
          }
          }
        },
        "specialItemCode":
[
          {
            "_code":
"GIFT_WITH_PURCHASE"
          }
        ],

"couponFamilyCode": [

"23736760493674713570",

"59114057533905607311"
        ],
        "lang": [
          {

"_qualification": {
              "sequence":
1,
              "language":
{
                "_code":
"fra"
              }
            },

"marketingMessage": "not
so good an income as
yours. Do you draw?” “No,
not at all.” “What, none
of you?” “Not one.” “That
is very strange. But I
suppose you had no
opportunity. Your mother
should have taken you to
town every spring for the
benefit of masters.” “My
mother would have had no
objection, but my father
hates London.” “Has your
governess left you?” “We
never had any governess.”
“No governess! How was
that possible? Five
daughters brought up at
home without a governess!
I never"
          },
          {
```

```
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "765",
              "_externalId":
"\u0027juiceContentPercentage\u0027"
            },
            "_code": "juiceContentPercentage"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "DECIMAL",
        "_formatPattern": "###.##",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              1635.15
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "764",
              "_externalId":
"\u0027ingredientStatement\u0027"
            },
            "_code": "ingredientStatement"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "TEXT",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "sequence":
0,
              "language":
{
                "_code":
"esl"
              }
            },

"marketingMessage":
"brother-in-law, Mr.
Hurst, merely looked the
gentleman; but his frie"
          }
        ]
      }
    ],

"_characteristicRecords":
[
      {
        "_qualification":
{

"characteristic": {
            "_code":
"nutrientFormatTypeCodeRef
erence"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"LOOKUP",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              {
                "_code":
```

```
              "language": {
                "_key": 11,
                "_code": "fin"
              }
            },
            "values": [
              "did she think it for Bingley and
her sister that s"
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "763",
              "_externalId":
"\u0027ingredientOfConcernCode\u0027"
            },
            "_code": "ingredientOfConcernCode"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "2518@283",
                  "_externalId":
"\u0027IODISED_SALT\u0027@\u0027ingredientOfConc
ernCode\u0027"
                },
                "_code": "IODISED_SALT"
              }
            ]
          }
        ]
      },
      {
"US_FDA_SFP_MULTI_VITAMINS
_IN_PACKAGES"
              },
              {
                "_code":
"NUTRIENT_DATA_EXEMPT"
              },
              {
                "_code":
"US_FDA_NFP_2020_STANDARD_
INFANTS_TO_12_MONTHS"
              }
            ]
          }
        ]
      },
      {
        "_qualification":
{

"characteristic": {
            "_code":
"juiceContentPercentage"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"DECIMAL",
        "_formatPattern":
"###.##",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              1635.15
            ]
          }
        ]
      },
      {
        "_qualification":
{
```

```
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "730",
              "_externalId":
"\u0027dietTypeDescription\u0027"
            },
            "_code": "dietTypeDescription"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "TEXT",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": 29,
                "_code": "sve"
              }
            },
            "values": [
              "admiration of Captain Carter, and
her hope of seeing him in the course of the day"
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "729",
              "_externalId":
"\u0027surfaceOfCheeseAtEndOfRipeningCode\u0027"
            },
            "_code":
"surfaceOfCheeseAtEndOfRipeningCode"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,

"characteristic": {
            "_code":
"ingredientStatement"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"TEXT",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"fin"
              }
            },
            "values": [
              "did she
think it for Bingley and
her sister that s"
            ]
          }
        ]
      },
      {
        "_qualification":
{

"characteristic": {
            "_code":
"ingredientOfConcernCode"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"LOOKUP",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
```

```
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "1580@275",
                  "_externalId":
"\u0027SOFT_RIPENED_MOULD_RIND\u0027@\u0027surfa
ceOfCheeseAtEndOfRipeningCode\u0027"
                },
                "_code":
"SOFT_RIPENED_MOULD_RIND"
              }
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "728",
              "_externalId":
"\u0027isRindEdible\u0027"
            },
            "_code": "isRindEdible"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "704@271",
                  "_externalId":
"\u0027FALSE\u0027@\u0027NonBinaryLogicCodes\u00
27"
                },
{
                  "_code":
"zxx"
                }
              },
              "values": [
                {
                  "_code":
"IODISED_SALT"
                }
              ]
            }
          ]
        },
        {
          "_qualification":
{

"characteristic": {
              "_code":
"dietTypeDescription"
            },
            "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
          },
          "_datatype":
"TEXT",
          "order": 1,
          "_recordLang": [
            {

"_qualification": {
                "language":
{
                  "_code":
"sve"
                }
              },
              "values": [
                "admiration
of Captain Carter, and her
hope of seeing him in the
course of the day"
              ]
            }
          ]
        },
        {
          "_qualification":
{
```

```
              "_code": "FALSE"
            }
          ]
        }
      ]
    },
    {
      "_qualification": {
        "characteristic": {
          "_key": {
            "_entityId": 8000,
            "_internalId": "727",
            "_externalId":
"\u0027fatPercentageInDryMatter\u0027"
          },
          "_code": "fatPercentageInDryMatter"
        },
        "recordKey": "0000.0000.RK",
        "parentRecordKey": "root"
      },
      "_datatype": "DECIMAL",
      "_formatPattern": "##.###",
      "order": 1,
      "_recordLang": [
        {
          "_qualification": {
            "language": {
              "_key": -1,
              "_code": "zxx"
            }
          },
          "values": [
            641.07
          ]
        }
      ]
    },
    {
      "_qualification": {
        "characteristic": {
          "_key": {
            "_entityId": 8000,
            "_internalId": "726",
            "_externalId":
"\u0027cheeseMaturationProcessContainerTypeCode\
u0027"
          },
          "_code":
"cheeseMaturationProcessContainerTypeCode"
        },
        "recordKey": "0000.0000.RK",
        "parentRecordKey": "root"
      },
      "_datatype": "LOOKUP",
      "order": 1,
      "_recordLang": [
        {
          "_qualification": {
            "language": {
              "_code": "zxx"
            }
          },
          "values": [
            {
              "_code":
"SOFT_RIPENED_MOULD_RIND"
            }
          ]
        }
      ]
    },
    {
      "_qualification": {
        "characteristic": {
          "_code":
"surfaceOfCheeseAtEndOfRip
eningCode"
        },
        "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
      },
      "_datatype":
"LOOKUP",
      "order": 1,
      "_recordLang": [
        {

"_qualification": {
            "language":
{
              "_code":
"zxx"
            }
          },
          "values": [
            {
              "_code":
"SOFT_RIPENED_MOULD_RIND"
            }
          ]
        }
      ]
    },
    {
      "_qualification":
{

"characteristic": {
          "_code":
"isRindEdible"
        },
        "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
      },
      "_datatype":
"LOOKUP",
      "order": 1,
      "_recordLang": [
        {

"_qualification": {
            "language":
```

```
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "1545@273",
                  "_externalId":
"\u0027MOULD\u0027@\u0027cheeseMaturationProcess
ContainerTypeCode\u0027"
                },
                "_code": "MOULD"
              }
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "725",
              "_externalId":
"\u0027cheeseMaturationPeriodDescription\u0027"
            },
            "_code":
"cheeseMaturationPeriodDescription"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "TEXT",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": 4,
                "_code": "chi"
              }
            },
            "values": [
{
                "_code":
"zxx"
              }
            },
            "values": [
              {
                "_code":
"FALSE"
              }
            ]
          }
        ]
      },
      {
        "_qualification":
{

"characteristic": {
            "_code":
"fatPercentageInDryMatter"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"DECIMAL",
        "_formatPattern":
"##.###",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              641.07
            ]
          }
        ]
      },
      {
        "_qualification":
{

"characteristic": {
```

```
              "were well, I hope, when you left
Londo"
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "721",
              "_externalId":
"\u0027rennetTypeCode\u0027"
            },
            "_code": "rennetTypeCode"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "1335@276",
                  "_externalId":
"\u0027NO_RENNET\u0027@\u0027rennetTypeCode\u002
7"
                },
                "_code": "NO_RENNET"
              }
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "720",
              "_externalId":
            "_code":
"cheeseMaturationProcessCo
ntainerTypeCode"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"LOOKUP",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              {
                "_code":
"MOULD"
              }
            ]
          }
        ]
      },
      {
        "_qualification":
{

"characteristic": {
            "_code":
"cheeseMaturationPeriodDes
cription"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"TEXT",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
```

```
"\u0027fatInMilkContent\u0027"
            },
            "_code": "fatInMilkContent"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "DECIMAL",
        "_formatPattern": "##.###",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              1724.34
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "944",
              "_externalId":
"\u0027numberOfPackagesForSerPiecesGTIN\u0027"
            },
            "_code":
"numberOfPackagesForSerPiecesGTIN"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "INTEGER",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              711
            ]
          }
```

```
              "language":
{
                "_code":
"chi"
              }
            },
            "values": [
              "were well,
I hope, when you left
Londo"
            ]
          }
        ]
      },
      {
        "_qualification":
{

"characteristic": {
            "_code":
"rennetTypeCode"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"LOOKUP",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              {
                "_code":
"NO_RENNET"
              }
            ]
          }
        ]
      },
      {
        "_qualification":
{
```

```
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "943",
              "_externalId":
"\u0027doPackagingMaterialContainLatex\u0027"
            },
            "_code":
"doPackagingMaterialContainLatex"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "704@271",
                  "_externalId":
"\u0027FALSE\u0027@\u0027NonBinaryLogicCodes\u00
27"
                },
                "_code": "FALSE"
              }
            ]
          }
        ]
      },
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "812",
              "_externalId":
"\u0027isHomogenised\u0027"
            },
            "_code": "isHomogenised"
          },
          "recordKey":
"0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype":
"DECIMAL",
        "_formatPattern":
"##.###",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              1724.34
            ]
          }
        ]
      },
      {
        "_qualification":
{
          "characteristic": {
            "_code":
"numberOfPackagesForSerPie
cesGTIN"
          },
          "recordKey":
"0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype":
"INTEGER",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "characteristic": {
                "_code":
"fatInMilkContent"
              },
              "recordKey":
"0000.0000.RK",
              "parentRecordKey": "root"
            },
            "_datatype":
"DECIMAL",
            "_formatPattern":
"##.###",
            "order": 1,
            "_recordLang": [
              {
                "_qualification": {
                  "language":
{
                    "_code":
"zxx"
                  }
                },
                "values": [
                  1724.34
                ]
              }
            ]
          },
          {
            "_qualification":
{
              "characteristic": {
                "_code":
"numberOfPackagesForSerPie
cesGTIN"
              },
              "recordKey":
"0000.0000.RK",
              "parentRecordKey": "root"
            },
            "_datatype":
"INTEGER",
            "order": 1,
            "_recordLang": [
              {
                "_qualification": {
```

```
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": 1,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId": "795@271",
                  "_externalId":
"\u0027NOT_APPLICABLE\u0027@\u0027NonBinaryLogic
Codes\u0027"
                },
                "_code": "NOT_APPLICABLE"
              }
            ]
          }
        ]
      }
    ]
  }
}
```

```
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              711
            ]
          }
        ]
      },
      {
        "_qualification":
{

"characteristic": {
            "_code":
"doPackagingMaterialContai
nLatex"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"LOOKUP",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              {
                "_code":
"FALSE"
              }
            ]
          }
        ]
      },
      {
        "_qualification":
{
```

```
"characteristic": {
            "_code":
"isHomogenised"
          },
          "recordKey":
"0000.0000.RK",

"parentRecordKey": "root"
        },
        "_datatype":
"LOOKUP",
        "order": 1,
        "_recordLang": [
          {

"_qualification": {
              "language":
{
                "_code":
"zxx"
              }
            },
            "values": [
              {
                "_code":
"NOT_APPLICABLE"
              }
            ]
          }
        ]
      }
    ]
  }
}
```


Characteristic Values

Characteristic values are rendered in a specialized structure in order to honor their hierarchical nature. This structure is not identical to the repository as it is fully hierarchical. This is the reason for extra meta attributes which help to not get in conflict with repository based data.

Additional Meta Attributes

- `_characteristicRecords`
  Top level element for all characteristic records
- `_recordLang`
  Language specific record values. In case the characteristic is not language specific, the qualification for "not

language specific" is returned (-1 or xxz)
In case the value of the record is a lookup value and the `includeLabels` parameter has been set, an additional `_label` element is provided.

- `_children`
  children of the current characteristic record
- `_datatype`
  The characteristic data type - to be able to interpret the values even in case the characteristic is no longer in the system
- `_formatPattern`
  The format pattern of the characteristic - to be able to format the values even in case the characteristic is no longer in the system

Response Examples

`includeIds = true`	`includeIds = false`

<pre>{
  "_entity": "Article",
  "_entityItem": {
  "_internalId": "18@1",
  "_entityId": 1000,
  "_externalId":
"\u0027supplierAID-1\u0027@\u0027MAST
ER\u0027"
  },
  "_revision": {
  "_internalId": "1",
  "_entityId": 5600,
  "_externalId": "\u0027root\u0027"
  },
  "_container": {
  "_internalId": "1",
  "_entityId": 2900,
  "_externalId":
"\u0027MASTER\u0027"
  },
  "_data": {
  "identifier": "supplierAID-1",
  "noCUperOU": 1,
  "currentStatus": {
  "_key": 100,
  "_code": "NEW"
  },
  "catalog": {
  "_key": {
  "_entityId": 2900,
  "_internalId": "1",
  "_externalId":</pre>
	<pre>{
	  "_entity": "Article",
	  "_entityItem": {
	  "_entity": "Article",
	  "_externalId":
	"\u0027supplierAID-1\u0027@\u0027MASTE
	R\u0027"
	  },
	  "_revision": {
	  "_entity": "Revision",
	  "_externalId": "\u0027root\u0027"
	  },
	  "_container": {
	  "_entity": "MasterCatalog",
	  "_externalId":
	"\u0027MASTER\u0027"
	  },
	  "_data": {
	  "identifier": "supplierAID-1",
	  "noCUperOU": 1,
	  "currentStatus": {
	  "_code": "NEW"
	  },
	  "catalog": {
	  "_code": "MASTER"
	  },
	  "quantityMin": 1,
	  "mainSupplier": {
	  "_code": "Heiler Product
	Manager"
	  },
	  "priceQuantity": 1,</pre>

```
"\u0027MASTER\u0027"
      },
      "_code": "MASTER"
    },
    "quantityMin": 1,
    "mainSupplier": {
      "_key": {
        "_entityId": 2800,
        "_internalId": "3",
        "_externalId": "\u0027Heiler
Product Manager\u0027"
      },
      "_code": "Heiler Product
Manager"
    },
    "priceQuantity": 1,
    "quantityInterval": 1,
    "kitParent": false,
    "soldOnlyInKits": false,
    "_characteristicRecords": [
      {
        "_qualification": {
          "characteristic": {
            "_key": {
              "_entityId": 8000,
              "_internalId": "2176",
              "_externalId":
"\u0027AnimalIngredient\u0027"
            },
            "_code":
"AnimalIngredient"
          },
          "recordKey":
"0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": -32767,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_key": -1,
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_key": {
                  "_entityId": 7300,
                  "_internalId":
"6075@799",
                  "_externalId":
    "quantityInterval": 1,
    "kitParent": false,
    "soldOnlyInKits": false,
    "_characteristicRecords": [
      {
        "_qualification": {
          "characteristic": {
            "_code":
"AnimalIngredient"
          },
          "recordKey": "0000.0000.RK",
          "parentRecordKey": "root"
        },
        "_datatype": "LOOKUP",
        "order": -32767,
        "_recordLang": [
          {
            "_qualification": {
              "language": {
                "_code": "zxx"
              }
            },
            "values": [
              {
                "_code": "Down"
              }
            ]
          }
        ],
        "_children": [
          {
            "_qualification": {
              "characteristic": {
                "_code": "CertDown"
              },
              "recordKey":
"0000.0000.RK",
              "parentRecordKey":
"0000.0000.RK"
            },
            "_datatype": "LOOKUP",
            "order": -32766,
            "_recordLang": [
              {
                "_qualification": {
                  "language": {
                    "_code": "zxx"
                  }
                },
                "values": [
                  {
                    "_code": "Other"
```

```
"\u0027Down\u0027@\u0027AnimalIngredient\u0027"
                },
                "_code": "Down"
              }
            ]
          }
        ],
        "_children": [
          {
            "_qualification": {
              "characteristic": {
                "_key": {
                  "_entityId": 8000,
                  "_internalId": "2177",
                  "_externalId": "\u0027CertDown\u0027"
                },
                "_code": "CertDown"
              },
              "recordKey": "0000.0000.RK",
              "parentRecordKey": "0000.0000.RK"
            },
            "_datatype": "LOOKUP",
            "order": -32766,
            "_recordLang": [
              {
                "_qualification": {
                  "language": {
                    "_key": -1,
                    "_code": "zxx"
                  }
                },
                "values": [
                  {
                    "_key": {
                      "_entityId": 7300,
                      "_internalId": "6077@800",
                      "_externalId": "\u0027Other\u0027@\u0027CertDown\u0027"
                    },
                    "_code": "Other"
                  }
                ]
              }
            ],
            "_children": [
                  }
                ]
              }
            ],
            "_children": [
              {
                "_qualification": {
                  "characteristic": {
                    "_code": "CertDownExpDate"
                  },
                  "recordKey": "0000.0000.RK",
                  "parentRecordKey": "0000.0000.RK"
                },
                "_datatype": "DATE",
                "order": -32765,
                "_recordLang": [
                  {
                    "_qualification": {
                      "language": {
                        "_code": "zxx"
                      }
                    },
                    "values": [
                      "1977-05-28"
                    ]
                  }
                ]
              },
              {
                "_qualification": {
                  "characteristic": {
                    "_code": "CertDownDesc"
                  },
                  "recordKey": "0000.0000.RK",
                  "parentRecordKey": "0000.0000.RK"
                },
                "_datatype": "TEXT",
                "order": -32763,
                "_recordLang": [
                  {
                    "_qualification": {
                      "language": {
                        "_code": "zxx"
                      }
```

```
              {
                "_qualification": {
                  "characteristic": {
                    "_key": {
                      "_entityId":
8000,
                      "_internalId":
"2180",
                      "_externalId":
"\u0027CertDownExpDate\u0027"
                    },
                    "_code":
"CertDownExpDate"
                  },
                  "recordKey":
"0000.0000.RK",
                  "parentRecordKey":
"0000.0000.RK"
                },
                "_datatype": "DATE",
                "order": -32765,
                "_recordLang": [
                  {
                    "_qualification":
{
                      "language": {
                        "_key": -1,
                        "_code":
"zxx"
                      }
                    },
                    "values": [
                      "1977-05-28"
                    ]
                  }
                ]
              },
              {
                "_qualification": {
                  "characteristic": {
                    "_key": {
                      "_entityId":
8000,
                      "_internalId":
"2179",
                      "_externalId":
"\u0027CertDownDesc\u0027"
                    },
                    "_code":
"CertDownDesc"
                  },
                  "recordKey":
                    },
                    "values": [
                      "Language
Independent Description"
                    ]
                  }
                ]
              },
              {
                "_qualification": {
                  "characteristic": {
                    "_code":
"CertDownZert"
                  },
                  "recordKey":
"0000.0000.RK",
                  "parentRecordKey":
"0000.0000.RK"
                },
                "_datatype": "MIME",
                "order": -32764,
                "_recordLang": [
                  {
                    "_qualification":
{
                      "language": {
                        "_code": "zxx"
                      }
                    },
                    "values": [
                      {
                        "_path": "34\
\48\\47\
\testMimeValue.c4995547687a9880_-2df8f
d04_17f9d0348a3_-7970.jpg",
                        "_label":
"testMimeValue.jpg",
                        "_type":
"image/jpeg"
                      }
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
  }
}
```

```
"0000.0000.RK",
                  "parentRecordKey":
"0000.0000.RK"
                },
                "_datatype": "TEXT",
                "order": -32763,
                "_recordLang": [
                  {
                    "_qualification":
{
                      "language": {
                        "_key": -1,
                        "_code":
"zxx"
                      }
                    },
                    "values": [
                      "Language
Independent Description"
                    ]
                  }
                ]
              },
              {
                "_qualification": {
                  "characteristic": {
                    "_key": {
                      "_entityId":
8000,
                      "_internalId":
"2178",
                      "_externalId":
"\u0027CertDownZert\u0027"
                    },
                    "_code":
"CertDownZert"
                  },
                  "recordKey":
"0000.0000.RK",
                  "parentRecordKey":
"0000.0000.RK"
                },
                "_datatype": "MIME",
                "order": -32764,
                "_recordLang": [
                  {
                    "_qualification":
{
                      "language": {
                        "_key": -1,
                        "_code":
"zxx"
                      }
```

```
                    },
                    "values": [
                      {
                        "_path": "8\
\2\\18\
\testMimeValue.931b5ed859092878_29b7f
765_17f9d0ab375_-796c.jpg",
                        "_label":
"testMimeValue.jpg",
                        "_type":
"image/jpeg"
                      }
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
  }
}
```


6.5.1.6 Examples

Please see the attached postman example collection for you convenience: ObjectAPI.postman_collection.json

Get Item by ID

```
curl --location --request GET 'http://localhost:1512/rest/V2.0/object/Article/
14260@1120' \
--header 'Content-Type: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI='
```

Get Item by Identifier As XML

```
curl --location --request GET 'http://localhost:1512/rest/V2.0/object/
Article/'\''supplierAID-42'\''@'\''MySupplierCatalog'\''' \
--header 'Content-Type: application/xml' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI='
```

Get Filtered Item (entity and qualification filter)

```
curl --location --request GET 'http://localhost:1512/rest/V2.0/object/
Article/''\''supplierAID-42'\''@'\''MySupplierCatalog'\''?
qualificationFilter=language(eng)&entityFilter=ArticleLang' \
--header 'Content-Type: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI='
```

6.6 REST Object API Write

Provides the Create, Update and Delete calls for objects. Sub-entity deletion is considered an update of the object. Update calls will check if the given item already exists and will create it when not.

- Create/Update Object (see page 272)
  - Permissions (see page 272)
  - Body (see page 272)
  - Response (see page 278)
  - Request Syntax (see page 282)
- Examples (see page 289)
  - Create an Item in the Master catalog (see page 289)

6.6.1 Create/Update Object

6.6.1.1 Permissions

Entity Permissions

The user needs to have the corresponding entity permission. They need to have `CREATE` in case a new item should be created and `EDIT` for any update of an existing item. In case sub-entities have separate entity permissions they need to apply too.

Object Permissions

The user needs to have the WRITE object permission in case of an update of an existing entity item. In case he doesn't, the api returns HTTP 403 (forbidden)

Field Permissions

The user needs to have the WRITE field permissions for all fields he wants to write.

Qualification Permissions (aka Qualified Field Permissions)

The user needs to have the WRITE permission for a qualification, e.g. for Language = English. If he doesn't, sub-entities which are qualified for this will not be persisted, and an error is returned in the protocol

6.6.1.2 Body

The structure of the body for the object api is similar, no matter if a post or put is executed. With one exception:

The attribute which represents the external, alphanumeric identifier can be omitted in case of the create api, as the framework will generate a new identifier automatically.

Fields and Entities

The repository property "shortIdentifier" in the custom section is used to define the entity and field names for the json/xml structure. The short identifier is unique within its parent entity only!

The standard repository already contains a short identifier for all fields and enabled qualifications.

Meta attributes

All meta attributes are prefixed with an underscore. Here is a list of the meta attributes used in the body besides the ones already described in the datatypes (see page 274)section.

`_qualification`	The qualification of the record. **The combination of the qualification values in the qualification object define this record as unique within its parent**.
`_changeType`	Optional attribute which defines how a sub-entity record should be handled. `put`: The sub-entity record is **updated** in case it already exists. It will be **created** if it doesn't exist. This is the default mode in case this attribute is missing `remove`: The sub-entity record is **removed** in case it exists. If not, nothing happens. To uniquely identify the record to be removed, the qualification element is needed. Other attributes can be omitted in this case.

Datatypes

`ENTITY_ITEM`	Objects can be specified by either the `_internalId` or the `_externalId` attribute. In case both is given, the `_internalId` takes precedence due to performance reasons, there is no fallback logic to the `_externalId`.
	In case the entity item can not be resolved an error is returned.
	
	Please note the Syntax (don't overlook the single quotes for external id!):
	
	Entity Items with container:
	
	`_externalId: 'Identifier'@'ContainerIdentifier'`
	
	`_internalId: InternalID@ContainerInternalID`
	
	Entity Items without container:
	
	`_externalId: 'Identifier'`
	
	`_internalId: InternalID`
	
	Single quotes as part of the identifiers need to be escaped with a backslash. Like
	
	'Iden\'tifier'@'ContainerIdentifir'
	
	Please note that quotes, backslashes or the @ symbol should be escaped in URLs. Please take a look at the REST Datatypes (see page 25) page for more information.
		Entity item with the external id:
		
		`"catalog": { "_externalId": "'MY_SUPPLIER_CATALOG'" }`
		
		Entity item with the internal id:
		
		`"catalog": { "_internalId": "1120" }`

`MIME_VALUE`

A reference to a mime file which must be provided in a separate zip archive
A mime value has a label. The label is shown in the UI in case a preview picture is not used there. The file path defines the location of the file inside of the zip archive when you download or upload the mime value.

The label and type are optional in case you want to write a mime value. They will be extracted from the `filePath` if omitted.
Please note that the relative file path of a mime value will change once it's uploaded and saved as Product 360 makes sure that every mime value is unique in the system. If you retrieve the mime value again after the upload, the `path` will be different. It should always be treated as a black box.

MIME_VALUE object, which at least must contain the relative `_filePath`.
The file path must correspond to the file location within the uploaded archive file.

`_label`	Optional: Human readable label for the mime value. Defaults to the filename in the path attribute (without extension)
`_type`	Optional: The official mime type of the file. If omitted, the mime type will be determined based on the file extension using the mapping provided in the mime.types file in the Product 360 configuration folder.
`_path`	**Mandatory:** File patch to the mime file. The path is relative to the ZIP Archive which has been uploaded prior to the REST call with the REST File API (see page 339).

```
{
  "_label": "Example Picture",
  "_type": "image/jpg",
  "_path":
"mySubFolderInTheZip\examplePictur
e.jpg"
}
```

`timestamp`	ISO 8601 `(YYYY-MM-DDTHH:mm:ss.sss)`	`2012-04-23T18:25:43.511`
`date`	ISO 8601 `(YYYY-MM-DD)`	`2019-07-04`
`time`	ISO 8601 `(HH:mm:ss.sss)`	`18:25:43.511`
`numbers`	Standard JSON. A dot is the decimal separator, no thousand separators **Rounding logic for decimal fields:** The number of fraction digits is defined in the repository with the scale attribute. In case the given value has a higher precision (a higher scale) the API will apply a rounding algorithm to the value to bring it to the maximum scale which is defined in the repository. *The rounding algorithm used it HALF_UP:* Rounding mode to round towards "nearest neighbor" unless both neighbors are equidistant, in which case round up. If the discarded fraction is ≥ 0.5 it's rounded up; otherwise, down. Note that this is the rounding mode commonly taught at school. For more details on this rounding mode you can have a look at the Java documentation for it.	`123.45`
`text`	Standard JSON	`"My Text"`
`boolean`	Standard JSON, true/false	`true` | `false`

Enumeration Fields

Enumeration fields can have `_key` or `_code` or `_label`. Always only one of the three possibilities is considered, even is multiples are provided. Key is used before code, and code is used before label.

In case the label must be used, the HTTP requests locale is used to interpret it. For performance reasons we strongly recommend to use `_key` or `_code`.

In case the enumeration field is of datatype ENTITY_ITEM, then the `_key` must be provided with the ENTITY_ITEM syntax (see above)

No Value / Empty / Null

If an attribute is given with `null`, `null` will be applied to the entity item's field. An empty string will be treated as null for text fields.

If an attribute is **not part of the document, it will be ignored**. In case the attribute is defined as mandatory, a corresponding error will be returned in the protocol

Multi-Value fields

Single fields which are defined as multi-value in the repository (upperBound>1) must be provided as array. It is not possible to add or remove a single value. The existing values will be replaced with the given ones.

Mime-Value fields

To be able to set mime values you need to first upload the binaries in form of a ZIP archive which contains all files which are referenced from the JSON payload. If the file is missing, a corresponding error is returned.

The file can contain folders and sub folders, however, the JSON payload needs to include the full path to the picture within the archive.

See the REST File API (see page 339) for details on how to upload the archive.

A single archive file can be used for multiple items. It will automatically be deleted 24 hours after it has been uploaded.

Examples

Example which uses only external ids for ENTITY_ITEM fields or the code for enumeration fields

This is the typical scenario in which the client of this api has no deeper knowledge on the internal id's or keys of entities and enumeration values.

```
{
  "identifier": "identifier-1",
  "manufacturerAID": "A000000000000",
  "manufacturerName": "BMW",
  "catalog": {
```

```
    "_code": "MySupplierCatalog"
  },
  "orderUnit": {
    "_code": "C62"
  },
  "lang": [
    {
      "_qualification": {
        "language": {
          "_code": "deu"
        }
      },
      "keywords": [
        "shift"
      ],
      "segment": "be treating his father’s favourite in such a manner, one
whom",
      "manufacturerTypeDescription": "of him. Proud tha",
      "descriptionShort": "feel he had been wrong; he had liberality, and he had
the mean",
      "remarks": "be at present; that you actually declared your resolution of
never taking orders"
    }
  ]
}
```


6.6.1.3 Response

The response contains useful meta attributes to easily identify the newly created entity item. The entity as well as the new identifier and the container it has been created in as well as the ready to use ENTITY_ITEM syntax.

Meta attributes

`_status`	The http status code of the response
`_entity`	The root entity identifier
`_entityItem`	The entity item in the ENTITY_ITEM syntax

`_identifier`	The external, alphanumeric identifier of the entity item. Although this identifier is already part of the `_entityItem` attribute, it's added separately for your convenience. The identifier is unique within the container
`_container`	The container of the entity item in the ENTITY_ITEM syntax. For items, products and variants it's the catalog, for structure groups it is the structure system, etc. Not all entities do have a containers.
`_protocol`	An array of protocol entries which provide details in case the validation returned errors or warnings. See below for details


```
{
  "_entity": "Article",
  "_entityItem": {
    "_internalId": "13989@1120",
    "_entityId": 1000,
    "_externalId": "\u0027NewItem\u0027@\u0027MY_SUPPLIER_CATALOG\u0027"
  },
  "_identifier": "NewItem",
  "_container": {
    "_internalId": "1120",
    "_entityId": 7000,
    "_externalId": "\u0027MY_SUPPLIER_CATALOG\u0027"
  },
  "_protocol": {
    "infoCounter": 0,
    "warningCounter": 0,
    "errorCounter": 1,
    "entries": [
      {
        "severity": "ERROR",
        "category": "DATATYPE",
        "property": "Article.OrderUnit",
        "message": "\u0027orderUnit\u0027 (Article.OrderUnit): Code value
\u0027NoIdea\u0027 not found in enumeration \u0027Enum.OrderUnits\u0027. Either
the code is not part of the enumeration or the user has no read permission"
      }
    ]
  }
}
```


Protocol

`infoCounter`	Number of entries with INFO severity

`warningCounter`	Number of entries with WARNING severity
`errorCounter`	Number of entries with ERROR severity
`entries`	Array of protocol entries

Protocol Entry

`severity`	INFO, WARNING or ERROR

<table>
<tr><td><code>category</code></td><td>The category of the protocol entry.
Possible categories are:<table>
<tr><td><code>SYSTEM</code></td><td>Category for general system errors</td></tr>
<tr><td><code>UNIQUENESS</code></td><td>Category for identity checks, unique constraints etc.</td></tr>
<tr><td><code>DATATYPE</code></td><td>Category for data type conversion problems as well as parsing issues</td></tr>
<tr><td><code>CARDINALITY</code></td><td>Category for problems with mandatory fields/elements as well as min/max number of elements (e.g. keywords)</td></tr>
<tr><td><code>RANGE</code></td><td>Category for min-/max length or value range violations</td></tr>
<tr><td><code>CONSISTENCY</code></td><td>Category for general consistency checks</td></tr>
<tr><td><code>SECURITY</code></td><td>Category for any kind of permission/security problems</td></tr>
<tr><td><code>NOTE</code></td><td>Category for info notes</td></tr>
<tr><td><code>SUMMARY</code></td><td>Category for summary entries</td></tr>
</table></td></tr>
<tr><td><code>property</code></td><td>Some form of unique identifier of the field or entity. Might be a combination of field type identifier and entity identifier or, like in this example, the field identifier. In any case it uniquely identifies the repository element</td></tr>
</table>

`message`	A human readable message in the locale of the request


```
{
  "_protocol": {
    "infoCounter": 0,
    "warningCounter": 0,
    "errorCounter": 1,
    "entries": [
      {
        "severity": "ERROR",
        "category": "DATATYPE",
        "property": "Article.OrderUnit",
        "message": "\u0027orderUnit\u0027 (Article.OrderUnit): Code value
\u0027NoIdea\u0027 not found in enumeration \u0027Enum.OrderUnits\u0027. Either
the code is not part of the enumeration or the user has no read permission"
      }
    ]
  }
}
```


6.6.1.4 Request Syntax

Create Object

Creates a new object of the given entity. In case the payload contains the corresponding external identifier field, this identifier will be used for the object.
In case no identifier is given, the system will automatically create one for the new object.

If the entity requires a container (e.g. for Items the container is the catalog), the corresponding container field must also be part of the payload or provided as query parameter.

URL Pattern	`/object/{entity-identifier}`
Method	`POST`
Content Type (for the body)	`application/json (recommended!)` `application/xml`

<table>
<tr><td>Accept (for the response)</td><td><code>application/json (recommended)</code>
<code>application/xml</code></td></tr>
<tr><td>URL Parameters</td><td><table><tr><td><code>{entity-identifier}</code></td><td>Unique identifier of the repository entity</td></tr></table></td></tr>
<tr><td>Query Parameter</td><td><table>
<tr><th>Name</th><th>Datatype</th><th>Default</th><th>Description</th></tr>
<tr><td><code>updateIfExists</code></td><td><code>boolean</code></td><td><code>false</code></td><td>Based on this parameter an already existing entity item will be updated.
In order for this to work, the identifier and container (if the entity requires one) must be part of the payload.</td></tr>
<tr><td><code>mimeValueArchive</code></td><td><code>String</code></td><td></td><td>The unique ID of the uploaded mime archive file as returned from the REST File API (see page 339)</td></tr>
<tr><td><code>container</code></td><td><code>String</code></td><td></td><td>Optional parameter which will set or override the container as it is defined in the JSON payload.
In case of an Item, this would be a ENTITY_ITEM Syntax of a catalog, as the Catalog is the container entity of Item.
This parameter is especially useful in testing scenarios in which the same payload needs to be processed repeatedly for different containers.</td></tr>
</table></td></tr>
<tr><td>Example</td><td><code>/rest/V2.0/object/Article</code></td></tr>
</table>

Return Codes

Code	Reason
`200 (Ok)`	Entity item has been updated successfully. (only in case the `updateIfExists` parameter is true and the unique identifier of the object has been provided in the payload)
`201 (Created)`	Entity Item has been created successfully
`400 (Bad Request)`	Entity item has not been created/updated successfully due to at least one validation error. The validation protocol is part of the response
`403 (Forbidden)`	User has no create permission for this entity
`404 (Not Found)`	Entity has not been found
`429 (Too many Requests)`	The number of concurrent write calls is too high. Clients should slow down their requests or use the message queue interface instead (recommended). Details on thread pool configuration for the Object API can be found here: REST Object API (see page 189)
`500`	Entity item has not been created/updated due to an unexpected server error. The server log should be checked.

	Code	Reason
	`503 (Service unavailable)`	In case the execution has been interrupted or cancelled unexpectedly any currently waiting executions will return server unavailable. This is typically when the server is shutdown forcefully while there are still executions waiting.

Update Object

An existing entity item can be updated with this endpoint.

URL Pattern	`/object/{entity-identifier}/{entityItem}`
Method	`PUT`
Content Type (for the body!)	`application/json (recommended)` `application/xml`
Accept (for the response!)	`application/json (recommended)` `application/xml`
URL Parameters	`{entity-identifier}` — Unique identifier of the repository entity `{entityItem}` — The ENTITY_ITEM either in the `internalId` or `externalId` syntax. E.g. `'myItem'@'MASTER'` or `42@1`

Query Parameters

Name	Datatype	Default	Description
`createIfMissing`	`boolean`	`true`	Based on this parameter a missing entity item will be created with the given identifier. In order for this to work, the `externalId` must be used for the `entityItem` parameter. In case the `internalId` is used and the object is not in the system a 404 code will be returned.
`mimeValueArchive`	`String`		The unique ID of the uploaded mime archive file as returned from the REST File API (see page 339)

Example

```
/rest/V2.0/object/Article/'myItem'@'MASTER'
```

```
/rest/V2.0/object/Article/42@1
```

Return Codes

Code	Reason
`200 (Ok)`	Entity item has been updated successfully
`201 (Created)`	Entity item has been created successfully
`400 (Bad Request)`	Entity item has not been created/updated successfully due to at least one validation error
`403 (Forbidden)`	User has no create or edit permission for this entity or object. Object permission is only checked in case of update
`404 (Not Found)`	▪ Entity has not been found ▪ Entity item has not been found. Only in case the `internalId` is used for the `entityItem` parameter, or the `createIfMissing` parameter is set to `false`.
`429 (Too many Requests)`	The number of concurrent write calls is too high. Clients should slow down their requests or use the message queue interface instead (recommended). Details on thread pool configuration for the Object API can be found here: REST Object API (see page 189)
`500`	Entity item has not been created/updated due to an unexpected server error. The server log should be checked.

Code	Reason
`503 (Service unavailable)`	In case the execution has been interrupted or cancelled unexpectedly any currently waiting executions will return server unavailable. This is typically when the server is shutdown forcefully while there are still executions waiting.

Delete Object

This API is essentially performing the same deletion as the REST List API Delete (see page 132). In case many items should be deleted, always use the List API. Executing the Object API's delete method for many items is causing more load on the system than executing the REST List API Delete (see page 132) just once because the application can not optimize the database requests.

URL Pattern	`/object/{entity-identifier}/{entityItem}`
Method	`DELETE`
Content Type	none
Accept Type	none
URL Parameters	`{entity-identifier}` — Unique identifier of the repository entity `{entityItem}` — The ENTITY_ITEM either in the `internalId` or `externalId` syntax. E.g. `'myItem'@'MASTER'` or `42@1`

Example	`/rest/V2.0/object/Article/'myItem'@'MASTER'` `/rest/V2.0/object/Article/42@1`
Return Codes	

Code	Reason
`200 (Ok)`	Entity item has been deleted successfully
`404 (Not Found)`	Entity item has not been found
`403 (Forbidden)`	User has no delete permission for this entity or object
`500`	Entity item has not been deleted due to an unexpected server error. The server log should be checked.

6.6.2 Examples

6.6.2.1 Create an Item in the Master catalog

This call will create a new item in the master catalog with an auto generated identifier. This is the most minimal call to create a new item. It's of course always better to directly provide all data the item should have.

Request

```
curl --location --request POST 'http://localhost:1512/rest/V2.0/object/Article' \
--header 'Accept: application/json' \
--header 'Content-Type: application/json' \
--header 'Authorization: Basic cmVzdDpoZWlsZXI=' \
--data-raw '{
    "catalog": {
        "_key": {
            "_externalId": "'\''MASTER'\''"
        }
    }
```

```
}'
```

Response


```
{
    "_entity": "Article",
    "_entityItem": {
        "_internalId": "5@1",
        "_entityId": 1000,
        "_externalId": "'Article_1643730037713001'@'MASTER'"
    },
    "_identifier": "Article_1643730037713001",
    "_container": {
        "_internalId": "1",
        "_entityId": 2900,
        "_externalId": "'MASTER'"
    },
    "_protocol": {
        "infoCounter": 0,
        "warningCounter": 0,
        "errorCounter": 0,
        "entries": []
    }
}
```

6.6.3 Payload Examples

Examples are usually better than most other documentations. This page will provide many hands-on examples. In order to have it easier to understand, all examples are written in JSON - but work likewise as XML.

The xml syntax is identical to the JSON one. Please have also a look at the attached Article_00.xml file which contains a full example of a write payload in the XML format. See also section Full Example.

6.6.3.1 Create or Update

The payload for create and update is essentially the same. The only difference is how an already existing item is handled. In case of a create call, existing items will return a corresponding error, in case of an update they will be updated.
There are query parameters to change this behavior for update and create. Please see the request definitions on REST Object API Write (see page 272).

6.6.3.2 General Examples

This section shows some general examples which basically work for all entities which are defined in the repository. For ease of understanding the examples are also made with the "Item" entity, but as mentioned, a generic by nature.

Simple fields on the root level of the entity

This example has no identifier, so the identifier will be generated in case it's used for create. In case it's used for update, the identifier needs to be provided in the request URL anyway.

```
{
  "catalog": {
    "_key": {
      "_externalId": "'ObjectApiWriterTest'"
    }
  },
  "gtin": "123456789",
  "orderUnit": {
    "_code": "C62"
  },
  "deliveryTime": 42.42
}
```

Remove a sub-entity record

This example will remove the english language sub-entity record

```
{
  "lang": [
    {
      "_changeType": "remove",
      "_qualification": {
        "language": {
          "_code": "en"
        }
      }
    }
  ]
}
```

Item, Product, Variants

Create prices for an item

The prices sub-entity has a small specialty. Contrary to the repository/meta definition, the object api does not have the "`validAt`" qualification. The object api used the `validFrom` and `validTo` qualifications instead.

This example creates an item with three purchase price records

```
{
  "pricePurchase": [
    {
      "_qualification": {
        "supplier": {
          "_code": "MAIN_SUPPLIER"
        },
        "type": {
          "_code": "net_list"
        },
        "currency": {
          "_code": "EUR"
        },
        "territory": {
          "_code": "DE"
        },
        "validFrom": "2021-01-01",
        "validTo": "2021-05-28"
      },
      "value": [
        {
          "_qualification": {
            "lowerBound": 1.0
          },
          "amount": 11.11
        }
      ]
```

```
    },
    {
      "_qualification": {
        "supplier": {
          "_code": "MAIN_SUPPLIER"
        },
        "type": {
          "_code": "net_list"
        },
        "currency": {
          "_code": "EUR"
        },
        "territory": {
          "_code": "DE"
        },
        "validFrom": "2021-05-29",
        "validTo": "2021-09-30"
      },
      "value": [
        {
          "_qualification": {
            "lowerBound": 1.0
          },
          "amount": 22.22
        }
      ]
    },
    {
      "_qualification": {
        "supplier": {
          "_code": "MAIN_SUPPLIER"
        },
        "type": {
          "_code": "net_list"
        },
        "currency": {
          "_code": "EUR"
        },
        "territory": {
          "_code": "DE"
        },
        "validFrom": "2021-10-01",
        "validTo": "2021-12-31"
      },
      "value": [
        {
          "_qualification": {
            "lowerBound": 1.0
          },
          "amount": 33.33
        }
      ]
    }
```

```
  ]
}
```

Update a single price

To update the amount of a single price, you need to specify the full list of qualifications in order to uniquely identify the record to update.

```
{
  "pricePurchase": [
    {
      "_qualification": {
        "supplier": {
          "_code": "MAIN_SUPPLIER"
        },
        "type": {
          "_code": "net_list"
        },
        "currency": {
          "_code": "EUR"
        },
        "territory": {
          "_code": "DE"
        },
        "validFrom": "2021-05-29",
        "validTo": "2021-09-30"
      },
      "value": [
        {
          "_qualification": {
            "lowerBound": 1.0
          },
          "amount": 44.44
        }
      ]
    }
  ]
}
```

Add Attribute with multiple values

Adds a new attribute "Material" with String as datatype and adds three values for it.

```
{
    "attribute": [
        {
            "_qualification": {
                "nameInKeyLang": "Material"
            },
```

```
            "datatype": {
                "_code": "String"
            },
            "multiValue": true,
            "mandatory": false,
            "value": [
                {
                    "_qualification": {
                        "language": {
                            "_code": "eng"
                        },
                        "identifier": "SECOND"
                    },
                    "value": "Steel"
                },
                {
                    "_qualification": {
                        "language": {
                            "_code": "eng"
                        },
                        "identifier": "THIRD"
                    },
                    "value": "Copper"
                },
                {
                    "_qualification": {
                        "language": {
                            "_code": "eng"
                        },
                        "identifier": "DEFAULT"
                    },
                    "value": "Wood"
                }
            ]
        }
    ]
}
```

Remove item attribute value

Removes a single value from a multi-value item attribute

```
{
    "attribute": [
        {
            "_qualification": {
                "nameInKeyLang": "Material"
            },
            "value": [
                {
                    "_qualification": {
```

```
                    "language": {
                        "_code": "eng"
                    },
                    "identifier": "SECOND"
                },
                "_changeType": "remove"
            }
        ]
    }
]
}
```

Remove item attribute (with all values)

```
{
    "attribute": [
        {
            "_qualification": {
                "nameInKeyLang": "Material"
            },
            "_changeType": "remove"
        }
    ]
}
```

Assign an item to a structure group

The ArticleStructureMap sub-entity is not available in the object api. It's successor, the ArticleStructureGroupMap is. To make it easier for everyone, we omit the Structure as a qualification field. It would have redundant since the StructureGroup is already a qualification field.

This element assigns the item, product or variant to "MyNode" within the "MyStructure" structure system. Please note that the StructureGroup field has no enumeration, therefor we can not use the code syntax for it but must use the entity_items

```
{
  "structureGroupMap": [
    {
      "_qualification": {
        "structureGroup": {
          "_externalId": "'MyNode'@'MyStructure'"
        }
      }
    }
  ]
```

```
}
```

You can also combine this with `"_changeType"="remove"` which would remove the assignment.

Set Attribute Values (based on Structure Attributes)

In case the structure is a maintenance structure, the structure can define attributes which then can have actual values on the item level. Here is an example how to set the Attribute value with a mapping to the structure feature.

```
{
   "attribute": [
    {
        "_qualification": {
            "nameInKeyLang": "Product name"
        },
        "identifier": "BAA316003",
        "datatype": {
            "_code": "String"
        },
        "value": [
            {
                "_qualification": {
                    "language": {
                        "_code": "eng"
                    },
                    "identifier": "DEFAULT"
                },
                "value": "spirits. Mrs. Bennet invited him to dine with
them; but, with many expres"
            }
        ],
        "structureGroupAttributeMap": [
            {
                "_qualification": {
                    "structureGroup": {
                        "_entity": "StructureGroup",
                        "_externalId": "'AKL837004'@'ECLASS-6.1'"
                    }
                },
                "structureAttribute": {
                    "_key": {
                        "_entity": "StructureFeature",
                        "_externalId": "'BAA316003'@'ECLASS-6.1'"
                    }
                }
            }
        ]
    }
}
```

Add characteristic records with child records

This example creates/updates an item's characteristic values.

The characteristic model which has been used looks roughly like this:

```
AnimalIngredients (Characteristic, Root)
|
|-- CertificateDown (1:1 for AnimalIngredient: DOWN)
|   |-- CertDownZert (1:1 for CertificateDown: OTHER)
|   |-- CertDownDesc (1:1 for CertificateDown: OTHER)
|   |-- CertDownExpDate (1:1 for CertificateDown: OTHER)
```

Characteristic values are hierarchical by nature, but the repository model for them is different. The normal generic logic can not be used here as it would be way too complicated.

- The `recordKey` qualification can be omitted in case there is only one record for this characteristic. In this case the default (0000.0000.RK) is used.
- The `parentRecordKey` can always be omitted as the parent is defined by the hierarchical structure itself
- The `language` qualification can be omitted in case the value is not language dependent. (In case you want to provide it anyway, please use `zxx` as language code (this means "language independent")
- All validations which are defined in the validation expressions are executed and will be reported in the result object!

```
{
  "_characteristicRecords": [
    {
      "_qualification": {
        "characteristic": {
          "_code": "AnimalIngredient"
        }
      },
      "_recordLang": [
        {
          "values": [
            {
              "_code": "Down"
            }
          ]
        }
      ],
      "_children": [
        {
          "_qualification": {
            "characteristic": {
              "_code": "CertDown"
```

```
        }
      },
      "_recordLang": [
        {
          "values": [
            {
              "_code": "Other"
            }
          ]
        }
      ],
      "_children": [
        {
          "_qualification": {
            "characteristic": {
              "_code": "CertDownExpDate"
            }
          },
          "_recordLang": [
            {
              "values": [
                "1977-05-28"
              ]
            }
          ]
        },
        {
          "_qualification": {
            "characteristic": {
              "_code": "CertDownZert"
            }
          },
          "_recordLang": [
            {
              "values": [
                {
                  "_path": "archiveSubFolder/testMimeValue.jpg"
                }
              ]
            }
          ]
        },
        {
          "_qualification": {
            "characteristic": {
              "_code": "CertDownDesc"
            }
          },
          "_recordLang": [
            {
              "values": [
                "Language Independent Description"
              ]
```

```
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

Remove a root characteristic record

To remove a characteristic record of an item, use the `_changeType` attribute. It will remove also all children of this record.

```
{
  "catalog": {
    "_code": "ObjectApiWriterTest"
  },
  "_characteristicRecords": [
    {
      "_changeType" : "remove",
      "_qualification": {
        "characteristic": {
          "_code": "AnimalIngredient"
        },
        "recordKey": "0000.0000.RK"
      }
    }
  ]
}
```

Structures

Create a new child node below the "MyParent" node.

The structure for groups is the same as the catalog for items. Their container object. It can be omitted depending on how the object api is called. We left it in this example for educational purposes.

```
{
  "structure": {
    "_entity": "Structure",
    "_externalId": "'MyStructure'"
  },
  "parent": {
    "_entity": "StructureGroup",
```

```
    "_externalId": "'MyParent'@'MyStructure'"
  }
}
```

6.6.3.3 Full Example

This example contains a value for pretty much all fields which we have defined in the repository, this includes GDSN fields. However, as the values are artificially generated, they do not make too much sense, but at least persisting this object will not lead to an error (assuming all dependencies are met).

It's merely here to give a big example as how the object api looks like. We have the same data created once as JSON document, and once as XML document. The xml document has a root element named <_data>, json obviously has no name of the root object.

Full Example in XML: Article_00.xml

Full Example in JSON: Article_00.json

7 REST Management API

The REST Management API (see page 302) provides access to control the server side management processes like Import, Merge, Export, and others. Typically these processes will be able to be started or scheduled, the process status can be evaluated and the process results can be requested and accessed.

- REST Assortment API (see page 303) — The Assortment API combines all methods around assortments. Currently the retrieval of the content of an assortment is possible as well as updating it's content by reevaluating dynamic rules. A modification of assortments rules is not yet possible using the Service API.
- REST Data Quality API (see page 304) — The Rest Data Quality API allows to schedule data quality jobs.
- REST Environment API (see page 321)
- REST Export API (see page 330) — The Rest Export API allows to schedule new export jobs and to cancel running export jobs.
- REST File API (see page 339) — The Rest File API can be used for uploading and downloading files to the Product Manager Server, e.g. for providing the source data of an import or for downloading the mime values of other api's.
- REST Import API (see page 342) — The Rest Import API allows to schedule new import jobs and to cancel running import jobs.
- REST KPI API (see page 350) — The Rest KPI API provides some management aspect features, e.g. to schedule jobs to calculate and persist key performance indicator values or delete certain persisted KPI results.
- REST Merge API (see page 358) — The Rest Merge API allows to schedule merge jobs.
- REST Revision API (see page 370) — The Rest Revision API allows to create revisions and add entity items to them (this process is called "release to a revision" or a revision release job. The API is available with Version 8.1.0.01 of the product
- REST System API (see page 378)
- REST Task API (see page 385) — The Rest Task API supports the creation, updating and content setting of PIM tasks.
- REST Workflow API (see page 399) — The Rest Workflow API allows to manage workflows, workflow status and process status. This API is used for the Informatica BPM workflow engine integration.

7.1 REST Assortment API

The Assortment API combines all methods around assortments. Currently the retrieval of the content of an assortment is possible as well as updating it's content by reevaluating dynamic rules. A modification of assortments rules is not yet possible using the Service API.

- Get assortment content (see page 303)

7.1.1 Get assortment content

URL Pattern	/manage/assortment/{assortment-id}/content
Method	GET
Parameters	`updateAssortment:` update the assortment if set to true, by default - false.
Content types	-
Media types	application/json
Result	report result object

Returns report result object, which contains all information from internal report result + entity identifier.

If `updateAssortment` is set, corresponding assortment will be updated (which can be time consuming) before returning the result.

7.2 REST Data Quality API

The Rest Data Quality API allows to schedule data quality jobs.

- Rule Execution (since 8.0.03) (see page 304)
  - Content (see page 305)
  - Result (see page 305)
  - Examples (see page 307)
- Schedule Rule Execution (see page 310)
  - Query Parameters (see page 311)
  - Content (see page 312)
  - Result (see page 313)
  - Workflow Callback (since 8.0.03) (see page 313)
  - Workflow Callback (since 10.0) (see page 313)
- Examples (see page 315)
  - Executing a DQ checks for the catalog TOOLS (see page 315)
- Deprecated (see page 317)
  - Execute a data quality rule configuration group immediately (see page 317)
  - Execute a set of data quality rule configurations (see page 318)
  - Execute channel's set of data quality rule configurations (see page 319)
  - Examples (see page 320)

7.2.1 Rule Execution (since 8.0.03)

Executes all specified rules for a given set of items. The rules to be executed can be defined either by a list of single rule configurations, rule configuration groups, channels or a combination of all options. Please note that this is a synchronous call. No job will be triggered and the call will return as soon as the execution is finished. It's not recommended to use this method for large data sets as there might be HTTP based connection issues during the call in case it takes too long. On the contrary, it definitly is recommended to use this method for small data sets as it does not impose the overhead which comes with the job framework and it's easier to handle (especially in workflow situations)

URL Pattern	`/manage/dataquality/executions`
Method	POST
Content types	`application/json, application/xml`
Media types	`application/json, application/xml`

Result	A rule execution result object containing the protocol of the execution as well as the detail status information for each item and rule

7.2.1.1 Content

The content has to be a DataQualityProfile JSON object which is described in the following table

Field	Required	Default	Datatype	Parameter description
rules	no		`StringArray`	Optional parameter which specifies individual rule names in form of a string array. Either `rules`, `ruleGroups`, `channels` or a combination of them must be specified!
ruleGroups	no		`StringArray`	Optional parameter which specifies rule group names in form of a string array. Either `rules`, `ruleGroups`, `channels` or a combination of them must be specified!
channels	no		`ENTITY_ITEM`	Optional parameter which specifies channels in form of an entity item array. Either `rules`, `ruleGroups`, `channels` or a combination of them must be specified!
`reportQuery`	`yes`		`ReportQuery`	the report query which defines the input data set for the data quality check.
`entityIdentifier`	`yes`		`String`	the entity identifier which describes the data type for which the data quality check will be executed. It must correspond to the entity identifier of the rule configuration group.

7.2.1.2 Result

A data quality result object which contains all relevant information about the execution for each item.

Properties of the returned object		
Field	**Data type**	**Description**
`ruleIds`	`Map`	A map of rule names to rule ID's. The ID's are only valid in this result object and are needed to reference the results of each item
`numberOfSuccessfulItems`	Integer	The number of items which completed all rules successfully
`numberOfFailedItems`	Integer	The number of items which failed for at least one rule
items		**An array of item objects which provide the results for all executed rules per item**
`entityItem`	ENTITY_ITEM	The reference to the entity item for which the rules have been executed
`status`	String	The overall status for all rules. Might be SUCCESSFUL, FAILED or UNKNOWN. SUCCESSFUL means that **all** rules have completed successfully FAILED means that **at least** a single rule failed UNKNOWN means that for **at least** one rule no results are available. Usually this can only happen in case some unexpected error happens - please check the protocol in this case.
`failedRuleIds`	Array of Integer	The id's of all rules which have failed for this item (see ruleIds above)
`successfulRuleIds`	Array of Integer	The id's of all rules which have succeeded for this item (see ruleIds above)

`protocol`		**The protocol (also known as problem log) of the execution.**
`infoCounter`	Integer	number of protocol entries with the INFO severity
`warningCounter`	Integer	number of protocol entries with the WARNING severity
`errorCounter`	Integer	number of protocol entries with the ERROR severity
`entries`	***Array of protocol entries***	
`severity`	String	The severity of the protocol entry. Might be INFO, WARNING or ERROR
`category`	String	The category of the protocol entry
`message`	String	The message of the protocol entry
`logDate`	Date	The date when the protocol entry has been created
`logTime`	Time	The time when the protocol entry has been created

7.2.1.3 Examples

In the following examples we assume that there are several rule configurations configured in the PIM system.

Java Rest Client Example

Rest Client Java Code

```
EntityItemReference toolsCatalog = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );
EntityItemReference webShopChannel = EntityItemReferenceFactory.createByIdentifier(
"WebShop" );
EntityItemReference erpChannel = EntityItemReferenceFactory.createByIdentifier(
"ERP" );

ReportQuery reportQuery = new ReportQuery( "byCatalog" ); //$NON-NLS-1$
reportQuery.addParameterValue( "catalog", toolsCatalog );


DataQualityProfile profile = new DataQualityProfile();
profile.setReportQuery( reportQuery );
profile.setEntityIdentifier( "Article" );
profile.addRuleConfigurations( "item_description_rule1", "item_description_rule3" );
profile.addRuleConfigurationGroups( "Item Texts", "Item Attributes" );
profile.addChannels( webShopChannel, erpChannel );

DataQualityRequest request = getRestClient().createDataQualityRequest();
DataQualityResult result = request.execute( profile );
```

JSON Examples

Execute individual rules "item_description_rule1" and "item_description_rule3" on the TOOLS catalog

```
//POST to http://localhost:1501/rest/V1.0/manage/dataquality/executions
{
  "rules":["item_description_rule1","item_description_rule3"],
  "ruleGroups":["Item Texts","Item Attributes"],
  "channels":["'WebShop'","'ERP'"],
  "entityIdentifier":"Article",
  "reportQuery":{
    "identifier":"byCatalog",
    "parameterList":[
     {
       "key":"Catalog",
       "value":"'TOOLS'"
     }
    ]
  }
}
```

DataQuality Result object in JSON format


```
{
  "ruleIds": {
    "CheckGtinMED": 12
  },
  "numberOfSuccessfulItems": 5,
  "numberOfFailedItems": 0,
  "items": [
    {
      "entityItem": {
        "id": "15@1"
      },
      "status": "SUCCESSFUL",
      "failedRuleIds": [],
      "successfulRuleIds": [
        12
      ]
    },
    {
      "entityItem": {
        "id": "137@1"
      },
      "status": "SUCCESSFUL",
      "failedRuleIds": [],
      "successfulRuleIds": [
        12
      ]
    },
    {
      "entityItem": {
        "id": "121@1"
      },
      "status": "SUCCESSFUL",
      "failedRuleIds": [],
      "successfulRuleIds": [
        12
      ]
    },
    {
      "entityItem": {
        "id": "19@1"
      },
      "status": "SUCCESSFUL",
      "failedRuleIds": [],
      "successfulRuleIds": [
        12
      ]
    },
    {
      "entityItem": {
```

```
        "id": "149@1"
      },
      "status": "SUCCESSFUL",
      "failedRuleIds": [],
      "successfulRuleIds": [
        12
      ]
    }
  ],
  "protocol": {
    "infoCounter": 3,
    "warningCounter": 0,
    "errorCounter": 0,
    "entries": [
      {
        "severity": "INFO",
        "category": "SUMMARY",
        "message": "1 Regel wird auf 5 Objekte des Typs 'Artikel' angewendet",
        "logDate": "2016-01-04",
        "logTime": "14:52:00"
      },
      {
        "severity": "INFO",
        "category": "SUMMARY",
        "message": "Ausgeführte Regeln: CheckGtinMED",
        "logDate": "2016-01-04",
        "logTime": "14:52:00"
      },
      {
        "severity": "INFO",
        "category": "SUMMARY",
        "message": "Verarbeitung der Regeln beendet.",
        "logDate": "2016-01-04",
        "logTime": "14:52:00"
      }
    ]
  }
}
```


7.2.2 Schedule Rule Execution

Executes all rules for a given amount of items. The rules to be executed can be defined either by a list of single rule configurations, rule configuration groups, channels or a combination of all options. Please note that this method replaces all other methods which are now deprecated (see below).

URL Pattern	`/manage/dataquality/jobs`

Method	POST
Content types	`application/json, application/xml`
Media types	`application/json, application/xml`
Result	The job object of the scheduled data quality job

7.2.2.1 Query Parameters

Parameter	Required	Default	Datatype	Parameter description
`workflowServiceEndpoint`	`no`		`String`	Informatica BPM callback parameter. Defines the name of the service endpoint which must be available in an attached Informatica BPM instance.
`workflowCorrelationId`	`no`		`String`	Informatica BPM callback parameter. An arbitrary id which is used by the Informatica BPM workflow to identify the correct workflow process.
`workflowCommunicationMode`	`no`	`REST`	`REST / QUEUE`	Informatica BPM callback parameter. Defines the communication mode which can be using JMS message queue and REST communication.

Parameter	Required	Default	Datatype	Parameter description
`workflowQueueId`	`no`	`First trigger queue id in server.properties`	`String`	Informatica BPM callback parameter. An queue id defined in the server properties in the message queue section which is used as response queue

7.2.2.2 Content

The content has to be a DataQualityProfile JSON object which is described in the following table

Field	Required	Default	Datatype	Parameter description
rules	no		`String Array`	Optional parameter which specifies individual rule names in form of a string array. Either `rules`, `ruleGroups`, `channels` or a combination of them must be specified!
ruleGroups	no		`String Array`	Optional parameter which specifies rule group names in form of a string array. Either `rules`, `ruleGroups`, `channels` or a combination of them must be specified!
channels	no		`ENTITY_ITEM`	Optional parameter which specifies channels in form of an entity item array. Either `rules`, `ruleGroups`, `channels` or a combination of them must be specified!
`reportQuery`	`yes`		`Report Query`	the report query which defines the input data set for the data quality check.

Field	Required	Default	Datatype	Parameter description
`entityIdentifier`	`yes`		`String`	the entity identifier which describes the data type for which the data quality check will be executed. It must correspond to the entity identifier of the rule configuration group.

7.2.2.3 Result

An object reference to the data quality job.

Properties of the returned object		
Field	**Data type**	**Description**
id	`Integer`	The job ID

7.2.2.4 Workflow Callback (since 8.0.03)

If a dataquality run is executed with the workflowServiceEnpoint parameter given, the job will create a callback request to the Informatica BPM server with additional information by the time of finish.

Additional information include job information, as well as report information about the filtered entity objects. Based on the report ids it is even possible using the list api to retrieve the items regarding the different status groups.

7.2.2.5 Workflow Callback (since 10.0)

It is possible to add the query parameters workflowServiceEndpoint and workflowQueueId and workflowCommunicationMode which allows to specify that the response is send via the message queue. The workflowServiceEndpoint is send back as JMS property P360TargetService which allows BPM to call workflow endpoints. The workflowQueueId parameter specifies a queue configured in the server.properties with syntax "queue.[queueId].name".

Workflow Callback Example

In the following example two items have been processed with two rules (GTINRule,ManufacturerRule).

The report results lists reports for following entry keys:

- SUCCESSFUL
- FAILED
- ManufacturerRule_FAILED
- GTINRule_SUCCESSFUL
- GTINRule_FAILED

The first two categories (SUCCESSFUL, FAILED) shows how many items did fail with any rule (FAILED) or were sucessful for all rules (SUCCESFUL).

The further categories are rule specific (ManufacturerRule_FAILED,GTINRule_SUCCESSFUL,GTINRule_FAILED). They show e.g. for the GTINRule_FAILED entry, how many items did fail specific for the 'GTINRule' rule.

With the report result id it is possible the retrieve the affected items via the byReport query of the List API.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<jobFinished>
    <jobId>24</jobId>
    <stateIdentifier>finished.info</stateIdentifier>
    <stateLabel>Completed</stateLabel>
    <reportResults>
        <entry key="SUCCESSFUL"/>
        <entry key="FAILED">
            <reportResult>
                <id>30</id>
                <dataSource>PCM_MASTER</dataSource>
                <type>1</type>
                <purpose>1</purpose>
                <resultTableName>ReportStoreTempB7</resultTableName>
                <count>2</count>
                <entityIdentifier>Article</entityIdentifier>
            </reportResult>
        </entry>
        <entry key="UNKNOWN"/>
        <entry key="GTINRule_FAILED">
            <reportResult>
                <id>21</id>
                <dataSource>PCM_MASTER</dataSource>
                <type>1</type>
                <purpose>1</purpose>
                <resultTableName>ReportStoreTempB8</resultTableName>
                <count>1</count>
                <entityIdentifier>Article</entityIdentifier>
            </reportResult>
        </entry>
        <entry key="ManufacturerRule_FAILED">
            <reportResult>
                <id>30</id>
                <dataSource>PCM_MASTER</dataSource>
                <type>1</type>
```

```
                <purpose>1</purpose>
                <resultTableName>ReportStoreTempB7</resultTableName>
                <count>2</count>
                <entityIdentifier>Article</entityIdentifier>
            </reportResult>
        </entry>
        <entry key="GTINRule_SUCCESSFUL">
            <reportResult>
                <id>32</id>
                <dataSource>PCM_MASTER</dataSource>
                <type>1</type>
                <purpose>1</purpose>
                <resultTableName>ReportStoreTempB9</resultTableName>
                <count>1</count>
                <entityIdentifier>Article</entityIdentifier>
            </reportResult>
        </entry>
    </reportResults>
</jobFinished>
```

7.2.3 Examples

7.2.3.1 Executing a DQ checks for the catalog *TOOLS*

In the following examples we assume that there are several rule configurations configured in the PIM system.

Java Rest Client Example

Rest Client Java Code

```
EntityItemReference toolsCatalog = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );
EntityItemReference webShopChannel = EntityItemReferenceFactory.createByIdentifier(
"WebShop" );
EntityItemReference erpChannel = EntityItemReferenceFactory.createByIdentifier(
"ERP" );

ReportQuery reportQuery = new ReportQuery( "byCatalog" ); //$NON-NLS-1$
reportQuery.addParameterValue( "catalog", toolsCatalog );


DataQualityProfile profile = new DataQualityProfile();
profile.setReportQuery( reportQuery );
profile.setEntityIdentifier( "Article" );
```

```
profile.addRuleConfigurations( "item_description_rule1", "item_description_rule3" );
profile.addRuleConfigurationGroups( "Item Texts", "Item Attributes" );
profile.addChannels( webShopChannel, erpChannel );


DataQualityRequest request = getRestClient().createDataQualityRequest();
EntityItemReference job = request.scheduleExecution( profile );
```

JSON Examples

Execute individual rules "item_description_rule1" and "item_description_rule3" on the TOOLS catalog

```
//POST to http://localhost:1501/rest/V1.0/manage/dataquality/jobs
{
  "rules":["item_description_rule1","item_description_rule3"],
  "ruleGroups":["Item Texts","Item Attributes"],
  "channels":["'WebShop'","'ERP'"],
  "entityIdentifier":"Article",
  "reportQuery":{
    "identifier":"byCatalog",
    "parameterList":[
     {
       "key":"Catalog",
       "value":"'TOOLS'"
     }
    ]
  }
}
```

Execute all rules for the rule groups "Item Texts" and "Item Attributes"

```
//POST to http://localhost:1501/rest/V1.0/manage/dataquality/jobs
{
  "ruleGroups":["Item Texts","Item Attributes"],
  "entityIdentifier":"Article",
  "reportQuery":{
    "identifier":"byCatalog",
    "parameterList":[
     {
       "key":"Catalog",
       "value":"'TOOLS'"
     }
    ]
  }
}
```

Execute all rules which are mapped to the "WebShop" and "ERP" channels

```
//POST to http://localhost:1501/rest/V1.0/manage/dataquality/jobs
{
  "channels":["'WebShop'","'ERP'"],
  "entityIdentifier":"Article",
  "reportQuery":{
    "identifier":"byCatalog",
    "parameterList":[
     {
       "key":"Catalog",
       "value":"'TOOLS'"
     }
    ]
  }
}
```

7.2.4 Deprecated

Please use `/manage/dataquality/jobs` instead of the following old URL's.

7.2.4.1 Execute a data quality rule configuration group immediately

General Info

URL Pattern	/manage/dataquality/job/{rule-configuration-group-name}
Method	POST
Parameters	-
Content types	application/json
Media types	application/json
Result	The job object of the scheduled data quality job

Content

The content has to be a JSON object which includes the properties listed below. It's called DataQualityProfile.

DataQualityProfile Properties					
	Field	**Required**	**Default**	**Datatype**	**Parameter description**
	reportQuery	yes		ReportQuery	the report query which defines the input data set for the data quality check.
	entityIdentifier	yes		String	the entity identifier which describes the data typ for which the data quality check will be executed. It must correspond to the entity identifier of the rule configuration group.

Result

An object reference to the data quality job.

Properties of the returned object			
	Field	**Data type**	**Description**
	id	Integer	The job ID

7.2.4.2 Execute a set of data quality rule configurations

General Info

URL Pattern	/manage/dataquality/execution
Method	POST

Parameters	`rules=<rule conf name1>,<conf name 2>,...`
Content types	application/json
Media types	application/json
Result	The job object of the scheduled data quality job

Content and result are the same as for the configuration group.

7.2.4.3 Execute channel's set of data quality rule configurations

General Info

URL Pattern	/manage/dataquality/execution
Method	POST
Parameters	`channel=<channelId>`
Content types	application/json
Media types	application/json
Result	The job object of the scheduled data quality job

Content and result are the same as for the configuration group. *ChannelId* is an internal id as it appears in

```
http://localhost:1501/rest/V1.0/list/Channel/bySearch?
query=%20not%20(Channel.Identifier%20is%20empty)&fields=ChannelLang.Name(en)
```

or an external id in single quotes.

7.2.4.4 Examples

Executing a dq check on the item descriptions of the catalog *TOOLS*

In the following example we assume that there is a rule configuration group which is called "item_descriptions" configured within the PIM system. To run this rule configuration immediately against the catalog "TOOLS" you need to execute the following code snippet.

Rest Client Java Code

```
EntityItemReference catalog = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );

ReportQuery reportQuery = new ReportQuery( "byCatalog" );
reportQuery.addParameterValue( "catalog", catalog );


DataQualityProfile dataQualityProfile= new DataQualityProfile();
dataQualityProfile.setReportQuery( reportQuery );
dataQualityProfile.setEntityIdentifier( "Article" );

EntityItemReference result =
getRestClient().createDataQualityRequest().scheduleRuleConfigurationGroup( "item_desc
riptions",

dataQualityProfile );
```

To run individual rules item_description_rule1 and item_description_rule3 on the same catalog with JSON

JSON example

```
//POST to http://localhost:1501/rest/V1.0/manage/dataquality/execution?
rules=item_description_rule1,item_description_rule3
{
  "reportQuery":{
    "identifier":"byCatalog",
    "parameterList":[{"key":"Catalog","value":"'TOOLS'"}]
  },
  "entityIdentifier":"Article"
}
```

7.3 REST Environment API

7.3.1 REST Environment APIs used for Extraction, Download and Integration of Environment transfer

7.3.2 **Extraction**

URL Pattern	`/manage/environment/transfer/extraction`
Method	POST
Content types	`application/json, application/xml`
Parameters	`transferHandlers` (eg: characteristics,lookups)
Returns	jobId, currentState and progress of extraction job If the job is finished, also the link to download the extraction archive as well as the protocol of the job will be returned

7.3.2.1 Query Parameters

Parameter	**Required**	**Default**	**Datatype**	**Parameter description**
`transferHandlers`	no	all the available handlers	`String`	A comma separated string of transfer handlers that specify the required transfer parameters for extraction of the environment.

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" "Content-Type: application/json" -X
POST http://localhost:1512/rest/V1.0/manage/environment/transfer/extraction?
transferHandlers=lookups,characteristics
```

7.3.2.2 Result

An object including required details as follows:

JSON Result:

```
{
    "job": {
        "id": "615"
    },
    "currentState": "scheduled",
    "progress": 0
}
```

XML Result:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<jobProgress>
    <job>
        <id>615</id>
    </job>
    <currentState>scheduled</currentState>
    <progress>0</progress>
</jobProgress>
```

7.3.3 **Download**

URL Pattern	/manage/environment/transfer/{jobId}/archive
Method	GET
Content types	`application/json, application/xml`

Parameters	`jobId` received by triggering the extraction job
Returns	stream of the object to be downloaded.

7.3.3.1 URL Parameters

Parameter	Required	Default	Datatype	Parameter description
`{jobId}`	`yes`		`Integer`	Unique ID of the job responsible for extraction

Rest Call

```
curl -u rest:heiler -H "Accept: application/octet-stream" "Content-Type: application/
json" -X GET http://localhost:1512/rest/V1.0/manage/environment/transfer/615/archive
```

7.3.3.2 Result

A stream of the object to be downloaded is returned.

7.3.4 **Integration**

URL Pattern	`/manage/environment/transfer/integrate`
Method	POST
Content types	multipart/form-data
Parameters	`transferHandlers` (eg: characteristics,lookups)

Returns	jobId and currentState and progress of integrate job

7.3.4.1 Query Parameters

Parameter	Required	Default	Datatype	Parameter description
transferHandlers	no	all the available handlers	String	A comma separated string of transfer handlers that specify the required transfer parameters for integration of the environment.

7.3.4.2 Form Data Parameters

Parameter	Required	Default	Datatype	Parameter description
file	`yes`		`Stream`	An handler for the file stream that is required for environment integration

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" "Content-Type: multipart/form-data"
 --data-binary "@C:/users/shared/pim_2018-08-22_05_23_24.428.zip" -X POST http://
localhost:1512/rest/V1.0/manage/environment/transfer/integrate?
transferHanlers=lookups,characteristics
```

7.3.4.3 Result

An object including required details as follows:

JSON Result:

```
{
```

```
    "job": {
        "id": "616"
    },
    "currentState": "scheduled",
    "progress": 0
}
```

XML Result:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<jobProgress>
    <job>
        <id>616</id>
    </job>
    <currentState>scheduled</currentState>
    <progress>0</progress>
</jobProgress>
```

7.3.5 Checking the Progress of the Extraction/Integration Job

URL Pattern	`/manage/environment/transfer/{jobId}`
Method	GET
Content types	`application/json, application/xml`
Parameters	`jobId` received by triggering the extraction job
Returns	jobId and currentState and progress of extraction job If the job is finished, also the link to download the extraction archive as well as the protocol of the job will be returned only for extraction job

7.3.5.1 URL Parameters

Parameter	Required	Default	Datatype	Parameter description
`{jobId}`	`yes`		`Integer`	Unique ID of the job to check progress

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" "Content-Type: application/json" -X
GET http://localhost:1512/rest/V1.0/manage/environment/transfer/615
```

7.3.5.2 Result

An object including required details as follows:

JSON Result:

```
{
    "job": {
        "id": "615"
    },
    "currentState": "finished.info",
    "progress": 100,
    "archiveUrl": "http://localhost:1512/rest/V1.0/manage/environment/transfer/615/
archive",
    "protocol": {
        "infoCounter": 0,
        "warningCounter": 0,
        "errorCounter": 0,
        "entries": []
    }
}
```

XML Result:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<jobProgress>
    <job>
        <id>615</id>
    </job>
    <currentState>finished.info</currentState>
```

```
    <progress>100</progress>
    <archiveUrl>http://localhost:1512/rest/V1.0/manage/environment/transfer/615/
archive</archiveUrl>
    <protocol>
        <infoCounter>0</infoCounter>
        <warningCounter>0</warningCounter>
        <errorCounter>0</errorCounter>
        <entries/>
    </protocol>
</jobProgress>
```

7.3.5.3 Possible Values of "currentState"

State	Value	Discription
SCHEDULED	scheduled	Server Job state constant which indicated that the job is currently scheduled
RUNNING_OK	running.ok	Server Job state constant which indicated that the job is currently running and no protocol info is available, till now
RUNNING_INFO	running.info	Server Job state constant which indicated that the job is currently running and info protocol entries are available
RUNNING_WARNING	running.warning	Server Job state constant which indicated that the job is currently running and warning protocol entries are available
RUNNING_ERROR	running.error	Server Job state constant which indicated that the job is currently running and error protocol entries are available
FINISHED_OK	finished.ok	Server Job state constant which indicated that the job is currently currently finished, having only info protocol entries
FINISHED_INFO	finished.info	Server Job state constant which indicated that the job is currently finished, having at least one warning protocol entry

State	Value	Discription
FINISHED_WARNING	finished.warning	Server Job state constant which indicated that the job is currently finished, having at least one error protocol entry
FINISHED_ERROR	finished.error	Server Job state constant which indicated that the job has been canceled by the user
CANCELED_OK	canceled.ok	Server Job state constant which indicated that the job has been canceled by the user, having only info protocol entries
CANCELED_INFO	canceled.info	Server Job state constant which indicated that the job has been canceled by the user, having at least one warning protocol entry
CANCELED_WARNING	canceled.warning	Server Job state constant which indicated that the job has been canceled by the user, having at least one error protocol entry
CANCELED_ERROR	canceled.error	Server Job state constant which indicated that the job has been canceled by a kill of the server (and not a proper shutdown)
CANCELED_SYSTEM	canceled.system	Server Job state constant which indicated that the job has been canceled due to a serious exception
CANCELED_CRITICAL	canceled.critical	Server Job state constant which indicated that the job has been canceled during a proper server shutdown
CANCELED_SHUTDOWN	canceled.shutdown	Server Job state constant which indicated that the job is still running, but a cancel has been requested, either by the user or by a server shutdown
CANCELED_PENDING	canceled.pending	Server Job state constant which indicates that the job is queued and waiting for the next free execution slot

State	Value	Discription
QUEUED_WORKLOAD	queued.workload	All Job States, which indicate, that the job is running.

7.4 REST Export API

The Rest Export API allows to schedule new export jobs and to cancel running export jobs.

- Schedule Export Job (see page 330)
  - Query Parameters (see page 331)
  - Content (see page 331)
    - Property variables (available with 8.0.02) (see page 332)
    - Property dataSources (available with 8.0.02) (see page 333)
  - Result (see page 335)
  - Complete example (see page 335)
- Information about an export profile (see page 336)
  - Result (see page 337)
  - Example (see page 337)
- Cancel Export Job (see page 338)
  - URL Parameters (see page 338)
  - Result (see page 338)

7.4.1 Schedule Export Job

Schedules an export.

URL Pattern	`/manage/export`
Method	POST
Content types	`application/json, application/xml`
Media types	`application/json, application/xml`
Result	The job object of the scheduled export job

7.4.1.1 Query Parameters

Parameter	Required	Default	Datatype	Parameter description
`workflowServiceEndpoint`	`no`		`String`	Informatica BPM callback parameter. Defines the name of the service endpoint which must be available in an attached Informatica BPM instance.
`workflowCorrelationId`	`no`		`String`	Informatica BPM callback parameter. An arbitrary id which is used by the Informatica BPM workflow to identify the correct workflow process.
`workflowCommunicationMode`	`no`	`REST`	`REST / QUEUE`	Informatica BPM callback parameter. Defines the communication mode which can be using JMS message queue and REST communication.
`workflowQueueId`	`no`	`First trigger queue id in server.properties`	`String`	Informatica BPM callback parameter. An queue id defined in the server properties in the message queue section which is used as response queue

7.4.1.2 Content

The content has to be a JSON object or an exportSchedulingProfile entity which includes the properties listed below.

Properties				
Field	**Required**	**Default**	**Datatype**	**Parameter description**
`profileName`	`yes`		`String`	Export profile name
`comment`	`no`		`String`	Comment for the process overview
`executionDate`	`no`	now	`DATETIME`	Date and time when the export should be started (Note: Before 8.0.02 the following format has to be used: `"yyyy-MM-dd HH:mm:ss"` , e.g., `"2015-11-16 16:06:34"`)
`variables` (available with 8.0.02)	no		`Map<String, Object>`	Sets the variables specified in the export template
`dataSources` (available with 8.0.02)	no		`Map<String, DataSource>`	Allows to set a report query for each data source
`parameters` (*deprecated* with 8.0.02)	`no`		`Map<String, String>`	Map of export parameters (key/value pairs)

Property `variables (available with 8.0.02)`

The property `variables` is a set of key / value pairs, whereby the key is the name of the variable. The value has to be valid according to the underlying datatype and the enumeration of the variable.

The datatype and the enumeration (if available) can be obtained by requesting information about an export profile (`/manage/export/{ExportProfileName}` , see below) . The valid formatting for the data types is described in REST Datatypes (see page 25).

Example:

```
{
    "profileName":"MyExportProfile",
    "variables": {
        "BooleanVar": "true",
        "CurrencyVar": "USD",
        "DateAndTimeVar": "2015-11-16T16:06:34",
        "DateTimeVar": "2015-11-16",
        "LongVar": "87",
        "StringVar": "Hello Export"
    }
}
```

Property dataSources (available with 8.0.02)

The property `dataSources` is a key value map which allows to set a report query for each data source. The key is the name of the data source, the value is an object containing the report query.

The available data sources can be obtained by requesting information about an export profile (`/manage/export/{ExportProfileName}` , see below).

Internally, the property `assortment` of the data source in the export profile is used for the report query. **This property has to be set to editable in the data source otherwise it is not possible to specify a report query for that data source.** Note that the report query allows to specify an assortment, so it is still possible to specify an assortment.

The report query has the following properties:

Properties				
Field	**Required**	**Default**	**Datatype**	**Parameter description**
`identifier`	`yes`		`String`	Identifier of the report
`parameters`	`no`		`Map<String,String>`	Parameters

Properties				
`assortmentFilter`	`no`		ENTITY_ITEM	Reference to an assortment (must be a reference object)
`updateAssortment`	no	false	`Boolean`	Specifies if the assortment should be updated before the

Examples:

```
{
    "profileName":"MyExportProfile",
    "dataSources": {
        "Item list" : {
            "reportQuery": {
                "identifier": "bySearch",
                "parameters": {
                    "query": "ArticleLang.DescriptionShort(en) startsWith \"Long live the Export\""
                },
                "assortmentFilter" : { "id": "4" },
                "updateAssortment" : true
            }
        }
    }
}
```

```
{"profileName":"MyExportProfile",
    "dataSources": {
        "Item list" : {
            "reportQuery": {
                "identifier": "byCatalog",
                "parameters": {
                    "catalog": "'mySupplierCatalog'"
                }
            }
        }
    }
}
```

parameters (deprecated since 8.0.02)

Variables can be passed as special parameters. The name of the variable has to end with `##variable`, e.g., `myVar##variable`.

```
{
  "profileName":"REST",
  "parameters":{"myVar##variable":"sampleValue"}
}
```

or

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<exportSchedulingProfile>
  <profileName>REST</profileName>
  <parameters>
        <entry>
            <key>myVar##variable</key>
            <value>sampleValue</value>
        </entry>
  </parameters>
</exportSchedulingProfile>
```

Note that "##variable" **has** to be appended to the variable name to create a corresponding parameter.

7.4.1.3 Result

Field	Data type	Description
`id`	`Integer`	The ID of the export job
`label`	`String`	The label of the export job

7.4.1.4 Complete example

A complete example with two variables and two data sources

```
{
   "profileName":"MyExportProfile",
   "comment":"MyComment",
   "executionDate":"2015-11-06T12:00:00",
   "variables":{
       "FirstVar":"Value for first varibale",
```

```
        "SecondVar":"Value for second varibale",
    },
    "dataSources": {
        "Item list 1" :{
            "reportQuery": {
                "identifier": "bySearch",
                "parameters": {
                    "query": "ArticleLang.DescriptionShort(en) startsWith \"Long live the Export\""
                },
                "assortmentFilter" : { "id": "4"},
                "updateAssortment" : true
            }
        },
        "Item list 2" :{
            "reportQuery": {
                "identifier": "bySearch",
                "parameters": {
                    "query": "Article.SupplierAID startsWith \"A1\""
                }
            }
        }
    }
}
```

7.4.2 Information about an export profile

Returns the available variables and data sources for the specified export profile.

URL Pattern	`/manage/export/{ExportProfileName}`
Method	GET
Media types	`application/json, application/xml`
Result	Available variables and data sources of the specified export profile.

7.4.2.1 Result

Field	Data type	Description
`profileName`	`String`	The name of the export profile
`variables`	`Map<String,VarInfo>`	Contains variable names together with information about the data type, the enumeration (if available), if the variable is mandatory and the default value
`dataSources`	`Map<String, DataSourceInfo>`	Contains the available data sources and for each data source the entity

7.4.2.2 Example

```
{
  "profileName": "MyExportProfile",
  "variables": {
    "BooleanVar": {
      "dataType": "BOOLEAN",
      "mandatory": false,
      "defaultValue": "0"
    },
    "CurrencyVar": {
      "dataType": "STRING",
      "enumeration": "Enum.Currency",
      "mandatory": false,
      "defaultValue": "Euro"
    },
  },
  "dataSources": {
    "Item list 1": {
      "entity": "Article"
    },
    "Item list 2": {
      "entity": "Article"
    }
  }
}
```

7.4.3 Cancel Export Job

Cancels a scheduled or running exportjob. If the job has been finished already, this request has no effect.

URL Pattern	`/manage/export/{jobId}/cancel`
Method	POST
Content types	`application/json, application/xml`
Media types	`application/json, application/xml`
Result	The job-id, the old job state and the new job state.

7.4.3.1 URL Parameters

Parameter	Required	Default	Datatype	Parameter description
`{jobId}`	`yes`		`Integer`	Unique ID of the job to cancel

7.4.3.2 Result

Field	Data type	Description
`id`	`Integer`	The job ID
`oldState`	`String`	The old state of the job.
`newState`	`String`	The new state of the job.

7.5 REST File API

The Rest File API can be used for uploading and downloading files to the Product Manager Server, e.g. for providing the source data of an import or for downloading the mime values of other api's.

- Downloading a File (see page 339)
- Uploading a File (see page 339)
  - Result (see page 340)
  - Examples (see page 340)
- Retention of uploaded files (see page 341)

7.5.1 Downloading a File

Downloads the binary content of the file identified by the file id parameter.

URL Pattern	`/manage/file/{fileId}`
Method	`GET`
URL Parameter	`fileId` — the unique id of the file. It's typically in the form of a UUID, e.g. `4033ac68-eb3b-4cdb-bfac-953c06a819ea` This is the same id which is returned from the upload api.
Accept types	`application/octet-stream`
Returns	The binary file stream of the file for the given file id

7.5.2 Uploading a File

Uploads the binary stream into a new folder on the server which uniquely identifies this upload. The file will be created in this folder with the given original filename. Every upload will create a separate folder as not to interfere with each other.

URL Pattern	`/manage/file`
Method	`POST`
Query Parameter	`originalFilename`
Content-Type	`application/octet-stream`
Accept	`application/json` `application/xml`
Returns	a file object containing the ID of the upload folder and the file name

7.5.2.1 Result

An object including an ID and a file name is returned, e.g.:

```
{
   "originalFilename": "Data1.xls",
   "id": "5b588807-5ad5-47db-a08b-3b3ca6573b41"
}
```

The file name is the same as provided in the parameter, the id represents the folder name the file has been stored in (under the upload directory of the Product Manager).

7.5.2.2 Examples

Uploading a file with file name Data1.xls

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type:application/octet-stream"  --data-binary "@Data.xlsx" -X POST http://localhost:1501/rest/V1.0/manage/file?originalFilename=Data.xls
```

An object including an ID and an URL is returned:

```
{
  "id":"fc645958-b169-44eb-91df-6ce30b3c4b11",
  "originalFilename":"Data.xls"
}
```

Rest Client Java Code

```
FileUploadRequest fileUploadRequest = restClient.createFileUploadRequest();

FileReference result = fileUploadRequest.uploadFile( "Data.xlsx", inputStream );
```

Uploading big files

When uploading big files with curl the `-T` command is recommended because the `--data-binary` command loads the complete file into memory and can lead to out of memory issues.

Using `-T` requires to send the header `-H "Transfer-Encoding: chunked"` in addition.

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type:application/octet-stream" -H "Transfer-Encoding: chunked" -T "BigData.xlsx" -X POST "http://localhost:1501/rest/V1.0/manage/file?originalFilename=BigData.xlsx"
```

7.5.3 Retention of uploaded files

Uploaded files will be deleted either by the process which uses them or by an automated cleanup process.

If you upload a file to use it for an import process, the import will delete the file once it has copied it to his internal structure. If you request a `mimeValueArchive` for download, the file will be deleted latest after 24 hours.

The retention time of files which are automatically deleted can be configured on the server. By default the job executes every hour and will delete all files which are older than 24 hours.

7.6 REST Import API

The Rest Import API allows to schedule new import jobs and to cancel running import jobs.

- Schedule Import Job (see page 342)
  - Query Parameters (see page 343)
  - Content (see page 343)
  - Result (see page 346)
- Cancel Import Job (see page 347)
  - Result (see page 347)
- Examples (see page 348)
  - Scheduling an import for a certain date and time (see page 348)
  - Cancel a running import job (see page 349)

7.6.1 Schedule Import Job

Schedules an import. The import files have to be uploaded first using the REST File API (see page 339).

URL Pattern	`/manage/import`
Method	POST
Content types	`application/json`
Media types	`application/json, application/xml`
Result	The job object of the scheduled import job

7.6.1.1 Query Parameters

Parameter	Required	Default	Datatype	Parameter description
`workflowServiceEndpoint`	`no`		`String`	Informatica BPM callback parameter. Defines the name of the service endpoint which must be available in an attached Informatica BPM instance.
`workflowCorrelationId`	`no`		`String`	Informatica BPM callback parameter. An arbitrary id which is used by the Informatica BPM workflow to identify the correct workflow process.
`workflowCommunicationMode`	`no`	`REST`	`REST / QUEUE`	Informatica BPM callback parameter. Defines the communication mode which can be using JMS message queue and REST communication.
`workflowQueueId`	`no`	`First trigger queue id in server.properties`	`String`	Informatica BPM callback parameter. An queue id defined in the server properties in the message queue section which is used as response queue

7.6.1.2 Content

The content has to be a JSON object which includes the properties listed below.

Properties				
Field	**Required**	**Default**	**Datatype**	**Parameter description**
`executionDate`	no	now	`DATETIME` (see page 27)	Date and time when the import should be started
`files`	yes		`A list of file objects`	A list of objects describing an uploaded file. The object has to include the property `id`. The number and order of the files has to match the files specified in the import mapping.
`mapping`	yes		`ENTITY_ITEM`	A reference to the import mapping stored in the database. Import mappings have the entity *ImportProfile*. The property `id` has to be specified.
`entitySpecificData`	no		`A list of JSON objects`	Each object contains information that is specific to the imported entity, for example, the catalog in case of *Articles*.
`options`	no		`JSON object`	A JSON object containing the import options

Properties of the `entitySpecificData` element				
Field	**Required**	**Default**	**Datatype**	**Parameter description**
`entityIdentifier`	yes		`String`	The identifier of the entity that should be imported, for example, Article

Properties of the `entitySpecificData` element

Field	Required	Default	Datatype	Parameter description
`properties`	no		`JSON object`	Properties for the specified entity

Properties of the `properties` element for entity *Article* (standard without customizing)

Field	Required	Default	Datatype	Parameter description
`parent`	yes		`String`	The id of the catalog.
`groupAssignmentMode`	no	0	`Integer`	If set to • 0: References to structure groups are added to existing structure groups • 1: References to structure groups replace existing structure groups

Properties of the `options` element

Field	Required	Default	Datatype	Parameter description
`createNewObjects`	no	`true`	`Boolean`	If set to `true`, new objects are created during the import. If set to `false`, objects that do not already exist are not imported.

Properties of the `options` element				
Field	**Required**	**Default**	**Datatype**	**Parameter description**
`updateExistingObjects`	no	`true`	`Boolean`	If set to `true`, objects that exist already are updated. If set to `false`, objects that do not already exist are not imported.
`dryRunBeforeImport`	no	`false`	`Boolean`	A dry run is executed before the actual import. Only if the dry run does not contain any errors, the actual import is executed.
`dryRunOnly`	no	`false`	`Boolean`	Only a dry run is executed, no objects are imported.
`errorMode`	no	`TOLERANT`	`String`	Either TOLERANT (objects containing errors are imported as far as possible) or RESTRICTIVE (objects
`numberOfThreads`	no	`Depends on server settings`	`Integer`	Number of threads which are used for the import. The number of threads can be reduced to lower the server load. *Currently not supported, will be available in HPM 7.0.01*.
useMappingDefault	no	false	boolean	If set to true, the import settings of the import mapping will be used, instead of the options. (since version 8.0.6.05)

7.6.1.3 Result

An object reference to the import job.

Properties of the returned object		
Field	**Data type**	**Description**
id	Integer	The job ID
label	String	The label of the job

7.6.2 Cancel Import Job

Cancels a scheduled or running import job. If the job has been finished already, this request has no effect.

URL Pattern	`/manage/import/{jobId}/cancel`
Method	POST
Content types	`application/json`
Result	The job-id, the old job state and the new job state.

7.6.2.1 Result

Properties of the returned object		
Field	**Data type**	**Description**
id	Integer	The job ID
oldState	String	The old state of the job.

Properties of the returned object		
Field	**Data type**	**Description**
newState	String	The new state of the job.

7.6.3 Examples

7.6.3.1 Scheduling an import for a certain date and time

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type:application/octet-stream"  --data-binary "@Data.xlsx" -X POST http://localhost:1501/rest/V1.0/manage/file?originalFilename=Data.xlsx
```

```
{   "executionDate": "2013-04-11T11:00:00",
    "files": [
        { "id": "fc645958-b169-44eb-91df-6ce30b3c4b11" }
    ],
    "mapping": { "id": "73" },
    "entitySpecificData": [ {
        "entityIdentifier" : "Article",
        "properties" :  {
            "parent": "1",
            "groupAssignmentMode": "1"
        }
    } ],
    "options": {
        "createNewObjects": true,
        "updateExistingObjects": true,
        "dryRunBeforeImport": false,
        "dryRunOnly": false,
        "errorMode": "TOLERANT",
        "numberOfThreads": 10
    }
}
```

A reference to the import job is returned:

```
{
    "id": "291",
```

```
    "label": "Import_291"
}
```

7.6.3.2 Cancel a running import job

Request to cancel the job with the id 291:

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1501/rest/V1.0/manage/import/291/cancel
```

Response containing the old and the new state of the job.

```
{
  "oldState": "scheduled",
  "newState": "canceled.ok",
  "id": "291"
}
```

7.7 REST KPI API

The Rest KPI API provides some management aspect features, e.g. to schedule jobs to calculate and persist key performance indicator values or delete certain persisted KPI results.

7.7.1 Schedule calculation of key performance indicator (KPI) values

Schedules a KPI calculation job that calculates and persists KPI values for a list of specified KPIs and a time interval.

Returns the scheduled job as EntityItemReference.

The user executing this request must have the permission 'Manage KPI values'. Otherwise, no calculation job will be scheduled and a response with HTTP status forbidden (403) will be returned.

URL Pattern	`/manage/kpi/calculationJob`
Method	POST
Content types	`application/json, application/xml`
Media types	`application/json, application/xml`
Result	The job object of the scheduled KPI calculation job

7.7.1.1 Content

The content has to be a KPICalculationRequestParameter JSON object which is described in the following table.

Properties of the POST request				
Field	**Required**	**Default**	**Data type**	**Parameter description**
`kpisToCalculate`	no	-	String Array	Specifies a list of KPIs for which the calculation of values should be done. If a KPI with a specific identifier is not registered, then there will be no calculation for it (will be logged as warning), but the remaining KPIs in the list are still considered. In case this parameter is not given, then all KPIs will be used that are contributed as KPI extension point.
`referenceTime`	no	Defaults to the current date when the job is scheduled.	DATETIME	Reference time that specifies the end time of all day intervals that will be considered.
`daysToBeBacktracked`	no	Defaults individually for each KPI to the daysToBeBacktracked from the server preferences. If then is no value provided for a specific KPI, defaults individually for each KPI to the `daysToBeBacktracked` from the KPI extension point.	Integer	Specifies a global value for the days to be backtracked ending at the date from the `referenceTime` parameter. The global value is used for all KPIs in the `kpisToCalculate` list. Must be convertible to an integer that is >= 0.

7.7.1.2 Result

An object reference to the KPI calculation job.

Properties of the returned object in the POST response		
Field	**Data type**	**Description**
`id`	Integer	The job ID
`label`	String	The label of the job

7.7.1.3 Examples

In the examples it is assumed that there are KPI extension points contributed that have "AverageTimeSpentInWorkflow" and "ProcessTimes" as KPI identifier.

Schedule KPI values calculation job for KPIs "AverageTimeSpentInWorkfow" and "ProcessTimes", a reference time of "2013-04-11T11:00:00" and 2 days to be backtracked

With a referenceTime set to "2013-04-11T11:00:00" and "daysToBeBacktracked" set to "2" the following day intervals will be passed to the calculators of each KPI:

```
[2013-04-09T00:00:00 - 2013-04-10T00:00:00), [2013-04-10T00:00:00 -
2013-04-11T00:00:00), [2013-04-11T00:00:00 - 2013-04-11T11:00:00)
```

Java Rest Client

Rest Client Java Code

```
KPICalculationRequestParameter requestParameter = new
 KPICalculationRequestParameter();
requestParameter.setKpisToCalculate( Arrays.asList( "AverageTimeSpentInWorkflow",
"ProcessTimes" ) );
requestParameter.setReferenceTime( "2013-04-11T11:00:00" );
requestParameter.setDaysToBeBacktracked( "2" );

KPICalculationRequest calculationRequest =
getRestClient().createKPICalculationRequest();

calculationRequest.scheduleCalculateKPIValuesJob( requestParameter );
```

JSON

```
//POST to http://localhost:1501/rest/V1.0/manage/kpi/calculationJob as body
{
    "kpisToCalculate": ["AverageTimeSpentInWorkflow", "ProcessTimes"],
    "referenceTime": "2013-04-11T11:00:00",
    "daysToBeBacktracked": "2"
}
```

The response contains a reference to the scheduled calculation job.

Returned response for scheduling a KPI values calculation

```
//Returned response contains job reference
{
    "id": "291",
    "label": "CalculateKPIValues_SingleJob_291"
}
```

7.7.2 Remove persisted results of key performance indicator (KPI) calculations

Removes all persisted values and execution data for certain KPIs and time interval. Thus, by doing this, it is possible to force a recalculation of KPI of the desired time interval which are also persisted again.

Reasons to use this call may be that

- the calculation logic of the contributed KPI calculator was wrong, so the persisted KPI values are wrong as well.
- certain data for a specific time period was not available at the time the calculation execution was done. By removing the execution results it is then possible to have the additional data added as well.

Returns Deletion Result objects containing a Webservice Protocol. This contains log informations (information, warning, error entries) that have been written to the (transient) problem log during the deletion operation.

The user executing this request must have the permission 'Manage KPI values'. Otherwise, no calculation job will be scheduled and a response with HTTP status forbidden (403) will be returned.

URL Pattern	`/manage/kpi/removeCalculationResults`
Method	POST
Content types	`application/json, application/xml`

Media types	`application/json, application/xml`
Result	A Deletion Result object containing a Webservice Protocol of the execution.

7.7.2.1 Content

The content has to be a KPICalculationRequestParameter JSON object which is described in the following table

Properties of the POST request				
Field	**Required**	**Default**	**Data type**	**Parameter description**
`kpisToDelete`	yes	-	String Array	Specifies a list of KPIs for which persisted results (values and execution information) are to be deleted. If a KPI with a specific identifier is not registered, then there will be no deletion for it (will be logged as warning), but the remaining KPIs in the list are still considered. In case this parameter is not given, then all KPIs will be used that are contributed as KPI extension point.
`referenceTime`	yes	-	DATETIME	Reference time that specifies the end time of the time inteval that will be considered.
`daysToBeBacktracked`	yes	-	Integer	Specifies the days to be backtracked ending at the Date from the `referenceTime` parameter. Must be convertible to an integer that is >= 0.

7.7.2.2 Result

A Deletion result object containing the protocol of the deletion execution.

Properties of the returned object in the POST response		
Field	**Data type**	**Description**
`protocol`		**The protocol (also known as problem log) of the execution.**
`infoCounter`	Integer	number of protocol entries with the INFO severity
`warningCounter`	Integer	number of protocol entries with the WARNING severity
`errorCounter`	Integer	number of protocol entries with the ERROR severity
`entries`	***Array of protocol entries***	
`severity`	String	The severity of the protocol entry. Might be INFO, WARNING or ERROR
`category`	String	The category of the protocol entry
`message`	String	The message of the protocol entry
`logDate`	Date	The date when the protocol entry has been created
`logTime`	Time	The time when the protocol entry has been created

7.7.2.3 Examples

In the examples it is assumed that there is a KPI extension point contributed that has "AverageTimeSpentInWorkflow" as KPI identifier.

However there is no KPI extension contributed with the identifier "ProcessAllTimes".

Remove KPI calculation results for contributed KPI "AverageTimeSpentInWorkfow" and not contributed "ProcessAllTimes", a reference time of "2013-04-11T11:00:00" and 2 days to be backtracked

With a referenceTime set to "2013-04-11T11:00:00" and "daysToBeBacktracked" set to "2" the following time interval will be passed to the deletion execution for each KPI:

```
[2013-04-09T00:00:00 - 2013-04-11T11:00:00)
```

and there will be a warning entry in the protocol object of the result that there is no KPI registered with the identifier "ProcessAllTimes":

```
The KPI with identifier 'ProcessAllTimes' (which is defined in the rest parameters)
is not registered.
```

Java Rest Client

Rest Client Java Code

```
KPIResultsDeletionRequestParameter requestParameter = new
 KPIResultsDeletionRequestParameter();
requestParameter.setKpisToDelete( Arrays.asList( "AverageTimeSpentInWorkflow",
"ProcessAllTimes" ) );
requestParameter.setReferenceTime( "2013-04-11T11:00:00" );
requestParameter.setDaysToBeBacktracked( "2" );

KPIResultsDeletionRequest deletionRequest =
getRestClient().createKPIResultsDeletionRequest();

deletionRequest.deleteKPIExecutions( requestParameter );
```

JSON

```
//POST to http://localhost:1501/rest/V1.0/manage/kpi/removeCalculationResults as body
{
    "kpisToDelete": ["AverageTimeSpentInWorkflow", "ProcessAllTimes"],
    "referenceTime": "2013-04-11T11:00:00",
    "daysToBeBacktracked": "2"
}
```

The response contains a KPIDeletionResult object.

KPIDeletionResult object in JSON format


```
{
    "protocol": {
        "infoCounter": 1,
        "warningCounter": 1,
        "errorCounter": 0,
        "entries": [
          {
            "severity": "WARNING",
            "category": "CONSISTENCY",
            "message": "The KPI with identifier 'ProcessAllTimes' (which is defined
in the rest parameters) is not registered.",
            "logDate": "2018-01-04",
            "logTime": "14:52:00"
          },
          {
            "severity": "INFO",
            "category": "SUMMARY",
            "message": "Removal of KPI values finished successfully.",
            "logDate": "2017-09-20",
            "logTime": "08:21:22"
          }
        ]
    }
 }
```

7.8 REST Merge API

The Rest Merge API allows to schedule merge jobs.

- Schedule Merge Job (see page 358)
  - Query Parameters (see page 359)
  - Content (see page 360)
  - Result (see page 366)
- Workflow Callback (see page 367)
  - Workflow Callback (since 10.0) (see page 367)
  - Workflow Callback Example (see page 367)
- Examples (see page 368)
  - Schedule merge of the full "Apparel" catalog (see page 368)

7.8.1 Schedule Merge Job

Schedules a merge.

URL Pattern	`/manage/merge`
Method	`POST`
Content types	`application/json`
Media types	`application/json`
Result	The job object of the scheduled merge job

7.8.1.1 Query Parameters

Parameter	Required	Default	Datatype	Parameter description
`updateAssortment`	`false`	`false`	`Boolean`	If `updateAssortment` is set, corresponding assortment will be updated (which can be time consuming) **before** performing merge.
`workflowServiceEndpoint`	`no`		`String`	Informatica BPM callback parameter. Defines the name of the service endpoint which must be available in an attached Informatica BPM instance.
`workflowCorrelationId`	`no`		`String`	Informatica BPM callback parameter. An arbitrary id which is used by the Informatica BPM workflow to identify the correct workflow process.
`workflowCommunicationMode`	`no`	REST	`REST / QUEUE`	Informatica BPM callback parameter. Defines the communication mode which can be using JMS message queue and REST communication.
`workflowQueueId`	`no`	`First trigger queue id in server.properties`	`String`	Informatica BPM callback parameter. An queue id defined in the server properties in the message queue section which is used as response queue

7.8.1.2 Content

The content has to be a `MergeProfile` JSON object which includes the properties listed below.

Field	Required	Default	Datatype	Parameter description
`rootEntity`	no	"Article"	`String`	The identifier of the root entity this merge profile refers to. At the moment only the "Article" entity is supported - so this parameter is optional.
`catalog`	either catalog or entityReportQuery is mandatory!		`ENTITY_ITEM`	The supplier catalog to be merged from. If no assortment is given, all objects of the specified rootEntity will be merged from this catalog
`assortment`	`no`		`ENTITY_ITEM`	Optional assortment to be merged
entityReportQuery	no		`EntityReportQuery`	An entity report query which allows to use the available reports like 'byCatalog', 'byItems'... (Since 8.0.03)
entityReportQueryExecution	no	DEFERRED	`IMMEDIATE` or `DEFERRED`	Defines when the content of the entity report query is resolved: at the moment of calling the rest method or when the merge job runs. If set to DEFERRED, the report is executed when the merge job begins to run. (Since 8.0.03)

Field	Required	Default	Datatype	Parameter description
executionDate	no	now	DATETIME (see page 27)	Date and time when the merge should be started.
defaultAction	no	"SUPPLEMENT"	String	Default merge mode for all fields in case the corresponding supplier catalog has no persisted merge profile. "SUPPLEMENT", "OVERWRITE", "CLEAN_COPY", "NO_MERGE" This parameter doubles as the default action for the entityQualification parameter object. ⚠ There is a priority order for the merge actions! Highest priority has the `entityQualification` parameter. In case this one is given, then this is how the merge is going to be performed. If this parameter is **not** given, the merge will be done as defined in the merge settings of the supplier catalog! The merge settings of the supplier catalog can be adjusted in the desktop client only. Last priority has the `defaultAction` parameter. In case the `entityQualification` is omitted **and** there are no supplier catalog merge settings, then all fields and entities will be merged according to this `defaultAction` parameter. Finally, in case this one is also not there, a SUPPLEMENT merge is performed.
entityQualification	no		EntityQualification	Optional parameter to fine grain the merge actions down to the field level. Provides also the ability to filter based on qualifications (like: merge only the short description in English). See also special chapter below.

Entity Report Query

The items to be merged can be defined in two different ways. You can either specify the supplier catalog and optionally an assortment, or a more sophisticated entityReportQuery. The entity report query option provides

a fine grained way to define even single items. For this you can use all available entity reports of the system. What reports are available can differ from Product 360 to Product 360 installation and therefore are documented by the Service API itself. The documentation can be obtained using the List Info API.

Using byCatalog report query

POST /rest/V1.0/manage/merge

```
{
 "entityReportQuery": {
  "identifier": "byCatalog",
  "entityIdentifier": "Article",
  "parameterList": [{
   "key": "catalog",
   "value": "'Apparel'"
  }]
 }
}
```

Using byItems report query

Note: the items must have the same container, it is not possible to specify items from multiple catalogs

POST /rest/V1.0/manage/merge

```
{
 "entityReportQuery": {
  "identifier": "byItems",
  "entityIdentifier": "Article",
  "parameterList": [{
   "key": "items",
   "value": "'Article1'@'CatalogIdentifier',123@1000"
  }]
 }
}
```

Using bySearch report query

Note: the items must have the same container, it is not possible to specify items from multiple catalogs

POST /rest/V1.0/manage/merge

```
{
  "entityReportQuery":{
    "identifier":"bySearch",
```

```
    "entityIdentifier":"Article",
    "parameterList":[
        {
          "key":"query",
          "value":"Article.EAN in ( \"1234\" , \"1235\" )"
        } ,
        {
          "key":"catalog",
          "value":"'TestCatalog'"
        }
     ]
}
```

Merge Actions

Mode	Description	Field	Supplier Item	Master Item (Before)	Master Item (After)
NO_MERGE	This is probably the easiest one. If this action is defined, no merge will be performed for the entity or field	`Description (en)`	"My **Changed** English Value"	"My English Value"	"My English Value"
SUPPLEMENT	A field or entity is only merged in case the master item has not already an object/value for this entity or field	`Description (en)`	"My **Changed** English Value"	"My English Value"	"My English Value"
		`Description (en)`	"My **Changed** English Value"		"My **Changed** English Value"
OVERWRITE	A field or entity is always merged, in case the master catalog already has a value for this field, the value is overwritten	`Description (en)`	"My **Changed** English Value"	"My English Value"	"My **Changed** English Value"

Mode	Description	Field	Supplier Item	Master Item (Before)	Master Item (After)
CLEAN_COPY	Similar to OVERWRITE for fields, but in case of entities is also deletes sub-entities which do not exist in the supplier item The example shows that the master item has a value for German and English, but after the clean copy merge, only the value from the supplier item survives.	`Description (en)`	"My **Changed** English Value"	"My English Value"	"My **Changed** English Value"
		`Description (de)`		"Mein Deutscher Wert"	

EntityQualification

The `EntityQualification` parameter is supported with version 8.0.6.01 and following only.

The `EntityQualification` is a complex data structure which defines how exactly each field of each sub-entity should be merged. The EntityQualification structure is **recursive**, thus, it has children which have the same structure.

It is required to qualify the whole path to the desired subentity. The qualification has to include the root entity.

On each level you can define a defaultAction which sets the action for this entity and it's fields and all sub-entities which are not explicitly part of the EntityQualification. So, you only need to specify EntityQualification objects and their fieldActions in case they differ from the corresponding defaultAction.

Each EntityQualification defines the identifier of the `entity`, the **`defaultAction`**, optional `qualificationFilters` (like, only for a certain language), the `fieldActions` (in case they are different than the defaultAction) and the **`children`** (in case they have a different actions as the defaultAction).

Attribute	Datatype	Description
`entity`	`String`	the unique identifier of the entity / sub-entity. See the Meta API (see page 477) for details on the possible entities.
`defaultAction`	`String`	one of the Merge Actions (`NO_MERGE, SUPPLEMENT, OVERWRITE, CLEAN_COPY`)
`qualificationFilters`	`String[]`	With the `qualificationFilters` array you can limit this EntityQualification so it only applies for a subset (just a specific language for example) The `qualificationFilters` is an array of qualification settings. A qualification setting is a *qualification name* followed by a comma-separated list of values which is put into parentheses . The qualification name is defined in the repository and is included in the Meta-API (see page 477). Example for German language: "qualificationFilters" : ["language(DE)"] The syntax is quite similar to the List API (see page 81) for sub-entities.
`fieldActions`	`FieldAction[]`	Array of `FieldAction` elements. Each `FieldAction` holds the `fieldIdentifier` of a field and an `fieldAction` . In case a field is not part of this array, it's treated like defined in the `defaultAction` .
`children`	`EntityQualification[]`	optional array of `EntityQualification` elements to define the sub-entities. If omitted, the sub-entities will be merged like this entity

Example

The following example defines as default " `NO_MERGE` " which means, that by default, nothing should be merged. It really depends on the use case what to pick as default. Usually the default should be whatever the most fields/children have, so you don't need to specify too much.

So in this example we only want to merge the German (= `language(DE)`) Name (= `ArticleLang.Name`) of an item and the English (= `language(EN)`) Description (= `ArticleLang.Description`). In order to

be able to have different settings based on the qualification or the entity, you can specify an EntityQualification multiple times. However, you must make sure that the qualification values do not overlap (otherwise the result of the merge for this will not be predictable).

All Fields of the ArticleLang entity in SUPPLEMENT mode, but the Name in OVERWRITE

```
"entityQualification": {
   "entity": "Article",
   "defaultAction": "NO_MERGE",
   "children": [
       {
           "entity": "ArticleLang",
           "defaultAction": "NO_MERGE",
           "qualificationFilters": [
               "language(DE)"
           ],
           "fieldActions": [
               {
                   "fieldIdentifier": "ArticleLang.DescriptionShort",
                   "fieldAction": "OVERWRITE"
               }
           ]
       },
       {
           "entity": "ArticleLang",
           "defaultAction": "NO_MERGE",
           "qualificationFilters": [
               "language(EN)"
           ],
           "fieldActions": [
               {
                   "fieldIdentifier": "ArticleLang.DescriptionLong",
                   "fieldAction": "OVERWRITE"
               }
           ]
       }
   ]
}
```

7.8.1.3 Result

An object reference to the merge job.

Field	Data type	Description
id	Integer	The job ID

Field	Data type	Description
label	String	The label of the job

7.8.2 Workflow Callback

If a merge job is scheduled with a `workflowServiceEndpoint` parameter, a callback is given to the Informatica BPM server.

This call will include data about the job itself as well as reports about the merge result. With help of the id of the report information it is even possible to retrieve the merged items (using the List API (see page 63)) .

7.8.2.1 Workflow Callback (since 10.0)

It is possible to add the query parameters `workflowQueueServiceEndpoint` and `workflowQueueId` and `workflowCommunicationMode` which allows to specify that the response is send via the message queue. The workflowServiceEndpoint is send back as JMS property `P360TargetService` which allows BPM to call workflow endpoints. The `workflowQueueId` parameter specifies a queue configured in the server.properties with syntax "queue.[queueId].name".

7.8.2.2 Workflow Callback Example

An example callback body to the service endpoint of Informatica BPM if a workflow service endpoint parameter has been specified.

Workflow Callback Example XML

```
<jobFinished>
 <jobId>35</jobId>
 <stateIdentifier>finished.info</stateIdentifier>
 <stateLabel>Beendet</stateLabel>
 <reportResults>
  <entry key="TOTAL">
   <reportResult>
    <id>2557</id><!-- With help of this reportid and the byReportId report of the
list API you can retrieve all items related to this report result -->
    <dataSource>PCM_MASTER</dataSource>
    <type>1</type>
    <purpose>3</purpose>
    <resultTableName>ReportStore0</resultTableName>
    <count>2</count>
    <entityIdentifier>Article</entityIdentifier>
   </reportResult>
```

```
  </entry>
  <entry key="UNMODIFIED"/>
  <entry key="NEW">
   <reportResult>
    <id>2558</id>
    <dataSource>PCM_MASTER</dataSource>
    <type>1</type>
    <purpose>3</purpose>
    <resultTableName>ReportStore1</resultTableName>
    <count>2</count>
    <entityIdentifier>Article</entityIdentifier>
   </reportResult>
  </entry>
  <entry key="UPDATED"/>
  <entry key="NEW_AND_UPDATED">
   <reportResult>
    <id>2559</id>
    <dataSource>PCM_MASTER</dataSource>
    <type>1</type>
    <purpose>3</purpose>
    <resultTableName>ReportStore2</resultTableName>
    <count>2</count>
    <entityIdentifier>Article</entityIdentifier>
   </reportResult>
  </entry>
  <entry key="FILTERED_BY_DQ"/>
  <entry key="FILTERED_BY_PRODUCT_FINDER"/>
 </reportResults>
</jobFinished>
```

7.8.3 Examples

7.8.3.1 Schedule merge of the full "Apparel" catalog

Java Client API: Merge full "Apparel" catalog with OVERWRITE

```
    MergeProfile mergeProfile = new MergeProfile();
    mergeProfile.setCatalog( EntityItemReferenceFactory.createByIdentifier(
"Apparel" ) );
    mergeProfile.setDefaultAction( "OVERWRITE" );

    MergeRequest mergeRequest = getRestClient().createMergeRequest();
    EntityItemReference mergeJobReference =
mergeRequest.scheduleMergeJob( mergeProfile );
```

Merge Profile: Full Apparel catalog with OVERWRITE action for everything

```
  "catalog" : {
    "id" : "Apparel"
  },
  "rootEntity" : "Article",
  "defaultAction" : "OVERWRITE"
```

Merge Profile: Full Apparell Catalog, but only the Long Description in English

```
{
  "rootEntity" : "Article",
  "catalog" : {
    "id" : "Apparel"
  },
  "defaultAction" : "NO_MERGE",
  "entityQualification" : {
    "entity" : "Article",
    "defaultAction" : "NO_MERGE",
    "children" : [ {
      "entity" : "ArticleLang",
      "defaultAction" : "NO_MERGE",
      "qualificationFilters" : [ "language(EN)" ],
      "fieldActions" : [ {
        "fieldIdentifier" : "ArticleLang.DescriptionLong",
        "fieldAction" : "OVERWRITE"
      } ]
    }
  }
}
```

7.9 REST Revision API

The Rest Revision API allows to create revisions and add entity items to them (this process is called "release to a revision" or a revision release job. The API is available with Version 8.1.0.01 of the product

Adding objects to a revision is a performance and database storage intensive process. Essentially the object will be copied including all the objects it references. Clients and Projects must always consider if the use case really needs the revision logic. Most times the audit trail feature is sufficient!

- Schedule Revision Release Job (see page 370)
  - Query Parameters (see page 371)
  - Content (see page 372)
  - Result (see page 375)
- Get Revision Job Information (see page 375)
  - Result (see page 376)

7.9.1 Schedule Revision Release Job

Schedules a revision release job to be executed as soon as possible and returns the job id for the scheduled job

URL Pattern	`/manage/revision`
Example	http://localhost:1512/rest/V2.0/manage/revision
Method	`POST`
Content types	`application/json, application/xml`
Media types	`application/json, application/xml`
Result	HTTP-202 (accepted) and the job object of the scheduled revision release job

Please note that the Revision Process will also add all entity items which are referenced from the entity items specified by the report! For example: If you add a single Item to a revision, also it's variant will be added and also the structure group the item belongs to. So, the full dependency tree will be added to the revision. It can be compared with a snapshot. In case an object has already

been added to the revision and it's content didn't change meanwhile, it will not be added a second time. In case the content did change, it will be overwritten in that revision. So there is always only one copy of any object within a revision.

7.9.1.1 Query Parameters

Parameter	Required	Default	Datatype	Parameter description
`workflowServiceEndpoint`	`no`		`String`	Informatica BPM callback parameter. Defines the name of the service endpoint which must be available in an attached Informatica BPM instance.
`workflowCorrelationId`	`no`		`String`	Informatica BPM callback parameter. An arbitrary id which is used by the Informatica BPM workflow to identify the correct workflow process.
`workflowCommunicationMode`	`no`	`REST`	`REST / QUEUE`	Informatica BPM callback parameter. Defines the communication mode which can be using JMS message queue and REST communication.
`workflowQueueId`	`no`	`First trigger queue id in server.properties`	`String`	Informatica BPM callback parameter. An queue id defined in the server properties in the message queue section which is used as response queue

Workflow Callback

If a revision job is scheduled with a `workflowServiceEndpoint` parameter, a callback is given to the Informatica BPM server.

This call will include data about the job itself

An example callback body to the service endpoint of Informatica BPM if a workflow service endpoint parameter has been specified.

Workflow Callback Example XML

```
<jobFinished>
 <jobId>35</jobId>
 <stateIdentifier>finished.info</stateIdentifier>
 <stateLabel>Finished</stateLabel>
</jobFinished>
```

Workflow Callback (since 10.0)

It is possible to add the query parameters `workflowQueueServiceEndpoint` and `workflowQueueId` and `workflowCommunicationMode` which allows to specify that the response is send via the message queue. The workflowServiceEndpoint is send back as JMS property `P360TargetService` which allows BPM to call workflow endpoints. The `workflowQueueId` parameter specifies a queue configured in the server.properties with syntax "queue.[queueId].name".

7.9.1.2 Content

The content has to be a JSON/XML object which includes the properties listed below.

Field	Required	Default	Datatype	Parameter description
`revision`	yes		`EntityItem`	The revision to add objects to. The revision will be created in case it does not already exist. Otherwise it will be used and the objects will be just added to it
`entityReportQuery`	yes		`EntityReportQuery`	An entity report query which allows to use the available reports like 'byCatalog', 'byItems'... (Since 8.0.03)

Entity Report Query

The items to be released into the revision must be defined by an entity report query. For this you can use all available entity reports of the system. What reports are available can differ from Product 360 to Product 360 installation and therefore are documented by the Service API itself. The documentation can be obtained using the List Info API (see page 57).

The EntityReportQuery also allows to define an optional assortment.

Parameter	Required	Default	Datatype	Parameter description
`identifier`	yes		`String`	Identifier of the entity report
`entityIdentifier`	yes		`String`	The unique identifier of the root entity
`parameterList`	no		`Parameter`	A list of parameters, each having a key and the value. Which parameters are supported or required for which entity report can be obtained with the List Info API (see page 57).
`assortmentFilter`	no		`EntityItem`	An entity item representation of an assortment. The assortment's item entity must match with the entity of the items the report returns. Additionally to that, also the item parent must match. For example, it's no allowed to query all items of a Catalog "Henri" and specifying an assortment which is bound to the master catalog. You will receive an appropriate error message in case the assortment does not fit.
`updateAssortment`	no	false	`Boolean`	This parameter works only in combination with the `assortment` parameter. If set to `true` the given assortment will be updated before it is used. Please note that updating an assortment might be a performance intensive task, so take care when using this parameter and think about it if you really need to have the assortment evaluated every time.

JSON Examples

Using byCatalog report query with an assortment

POST /rest/V2.0/manage/revision

```
{
 "revision": "'MyRevisionIdentifier'",
 "entityReportQuery": {
  "identifier": "byCatalog",
  "entityIdentifier": "Article",
```

```
  "parameterList": [{
   "key": "catalog",
   "value": "'Apparel'"
  }],
  "assortmentFilter": "4711",
  "updateAssortment": false
 }
}
```

Using byItems report query

Note: the items must have the same container, it is not possible to specify items from multiple catalogs

POST /rest/V2.0/manage/revision

```
{
 "revision": "'MyRevisionIdentifier'",
 "entityReportQuery": {
  "identifier": "byItems",
  "entityIdentifier": "Article",
  "parameterList": [{
   "key": "items",
   "value": "'Article1'@'CatalogIdentifier',123@1000"
  }]
 }
}
```

Using bySearch report query

Note: the items must have the same container, it is not possible to specify items from multiple catalogs

POST /rest/V2.0/manage/revision

```
{
 "revision": "'MyRevisionIdentifier'",
 "entityReportQuery":{
  "identifier":"bySearch",
  "entityIdentifier":"Article",
  "parameterList":
   [{
   "key":"query",
   "value":"Article.EAN in ( \"1234\" , \"1235\" )"
   },{
   "key":"catalog",
   "value":"'TestCatalog'"
   }]
}
```

Example with the Rest Client API

Schedule revision of the full "TOOLS" catalog

Java Client API: Merge full "Apparel" catalog with OVERWRITE

```
EntityItemReference catalog = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );
ReportQuery reportQuery = new ReportQuery( "byCatalog" );
reportQuery.addParameterValue( "catalog", catalog );

RevisionRequest request = getRestClient().createRevisionRequest();
RevisionResult result = request.scheduleRevisionJob( "Article", reportQuery );
```

7.9.1.3 Result

A result object containing the entity item of the job

Field	Data type	Description
job	EntityItem	The job entity item

Example of the result in JSON

```
{
  "id": "20045",
  "label": "com.heiler.ppm.revision.jobType_20045"
}
```

7.9.2 Get Revision Job Information

Returns current job information about a revision job including the protocol of the job

URL Pattern	`/manage/revision/{jobId}`

Example	http://localhost:1512/rest/V2.0/manage/revision/20045
Method	GET
Content types	`application/json, application/xml`
Media types	`application/json, application/xml`
Result	HTTP-200 (ok) and the job information object about the current state of the release job
	HTTP-404 (not found) in case the job id does not exist or is not a revision release job

7.9.2.1 Result

Example of the result in JSON


```
{
    "id": 20045,
    "jobType": "com.heiler.ppm.revision.jobType",
    "jobGroup": "DataMaintenanceGroup",
    "user": "rest",
    "creationTime": "2018-01-05T15:07:35:920+0100",
    "modificationTime": "2018-01-05T15:07:36:470+0100",
    "scheduledAt": "2018-01-05T15:07:35:920+0100",
    "currentState": "finished.info",
    "progress": 100,
    "serverIdentifier": "pim-server1",
    "problemLogIdentifier": "com.heiler.ppm.revision.jobType_20046_20046",
    "protocol": {
        "infoCounter": 1,
        "warningCounter": 0,
        "errorCounter": 0,
        "entries": [
            {
                "objectType": "Item",
                "severity": "INFO",
                "category": "SUMMARY",
                "message": "1 objects released in 200 ms",
                "logDate": "2018-01-05",
```

```
                "logTime": "15:07:36"
            }
        ]
    }
}
```

Example of the result in XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<revisionJobInfo>
    <id>20045</id>
    <jobType>com.heiler.ppm.revision.jobType</jobType>
    <jobGroup>DataMaintenanceGroup</jobGroup>
    <user>rest</user>
    <creationTime>2018-01-05T15:07:35:920+0100</creationTime>
    <modificationTime>2018-01-05T15:07:36:470+0100</modificationTime>
    <scheduledAt>2018-01-05T15:07:35:920+0100</scheduledAt>
    <currentState>finished.info</currentState>
    <progress>100</progress>
    <serverIdentifier>pim-server1 (DEW1PC05KDRB IP: 10.43.104.101)</serverIdentifier>
    <problemLogIdentifier>com.heiler.ppm.revision.jobType_20046_20046</
problemLogIdentifier>
    <protocol>
        <infoCounter>1</infoCounter>
        <warningCounter>0</warningCounter>
        <errorCounter>0</errorCounter>
        <entries>
            <entry>
                <objectType>Item</objectType>
                <severity>INFO</severity>
                <category>SUMMARY</category>
                <message>1 objects released in 200 ms</message>
                <logDate>2018-01-05T00:00:00+01:00</logDate>
                <logTime/>
            </entry>
        </entries>
    </protocol>
</revisionJobInfo>
```

7.10 REST System API

7.10.1 Create a thread dump

URL Pattern	/manage/system/info/threaddump
Method	GET
Parameters	
Content types	
Returns	a thread dump from the current server

Request:

```
curl -u rest:heiler -X GET http://localhost:1501/rest/V1.0/manage/system/info/
threaddump
```

Response:

```
a thread dump in a common format which can be read by automated systems like
fastthread.io
```

7.10.2 Create thread dumps for all servers

URL Pattern	/manage/system/info/threaddump/allServers
Method	GET
Parameters	

Content types	
Returns	a zip containing a thread dump for every server

Request:

```
curl -u rest:heiler -X GET http://localhost:1501/rest/V1.0/manage/system/info/
threaddump/allservers
```

Response:

```
a zip with thread dumps in a common format which can be read by automated systems
like fastthread.io
```

7.10.3 Echo Text

Simple function to test server availability, authentication and correct charset/encoding use.

URL Pattern	/manage/system/echo/{echoText}
Method	GET
Parameters	`echoText:` the text string which get echoes back.
Content types	application/json
Returns	the given echoText with the server internal charset/encoding.

7.10.3.1 Example:

Request:

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/manage/system/echo/helloWorld
```

Response:

```
helloWorld
```

7.10.4 Create token for Token Basic Authentication

Creates a token, which can be used to authenticate, instead of supplying username/password (for further details see chapter REST Service Authentication (see page 23)). The token is generated for the authenticated user of this REST call and is valid for one hour.

URL Pattern	/manage/system/security/token
Method	POST
Parameters	`grant_type` : the type of token. " `authorization_code` " for an access token
Content types	application/json
Returns	A JSON Object, containing: `access-token` : the token `expires-in` : period of validity of the token (in seconds)

7.10.4.1 Example:

Request:

```
POST /rest/V1.0/manage/system/security/token
grant_type: authorization_code
```

Response:

```
{
  "access-token":
"4ABE78F118D7068322B9A1AEB5B76C385C32777D52FB971FF70593C28CE94C8DB43BD5E7EF4EAB84DA8A
E435F231B5F274F3643A2B2527350861E38029A31D5CD826A28D68D70625E8D1F47...",
  "expires-in": 3600
```

```
}
```

7.10.5 Create token for Token Basic Authentication, based on a SAML Response

When using SAML for authentication (see chapter SAML Configuration), a token can by retrieved for the user contained in a SAML Response.

URL Pattern	/manage/system/security/tokenForSaml
Method	POST
Parameters	`grant_type` : the type of token. " `authorization_code` " for an access token `saml_response` : Base64-Encoded SAML 2.0 Response (containing a SAML Assertion)
Content types	application/json
Returns	A JSON Object, containing: `access-token` : the token `expires-in` : period of validity of the token (in seconds)

The token validity time can be configured via the preference `com.heiler.ppm.webservice.server/accessTokenExpirationTime.SAML` (in the server's `plugin_customization.ini`), and is one day by default.

7.10.5.1 Example:

Request:

```
POST /rest/V1.0/manage/system/security/tokenForSaml
grant_type: authorization_code

saml_response:
28CE94C8DB43BD5E7EF4EAB84DA8AE435F231B5F274F3643A2B2527350861E38029A31D5CD826A28D68D7
0625E8D1F47FFC79712AEC227AD53818203C0496EB6D38E6BFE4B68E...
```

Response:

```
{
  "access-token":
"4ABE78F118D7068322B9A1AEB5B76C385C32777D52FB971FF70593C28CE94C8DB43BD5E7EF4EAB84DA8A
E435F231B5F274F3643A2B2527350861E38029A31D5CD826A28D68D70625E8D1F47...",
  "expires-in": 86400
}
```

7.10.6 REST System API for License

Available since 7.0.04

7.10.6.1 Retrieve License Information

Retrieve license information containing maximum users and available

URL Pattern	/manage/system/license
Method	GET
Parameters	-
Content types	application/json
Media types	application/json
Result	The license object

Result

An object containing license information.

Properties of the returned object			
	Field	**Data type**	**Description**
	maxPimUsers	Integer	The maximum number of PIM users.
	maxCollaborative Users	Integer	The maximum number of collaborative users.

Properties of the returned object			
	languages	List of String	The names of all language locales that have been licensed, sorted ascending. The locales are formatted in the two conventional Java pattern - Language in ISO 639 + "_" + country code in ISO 3166-1 alpha-2, f.e. en_US.

7.11 REST Task API

This page does **not** describe the handling of special "Workflow Tasks". Workflow tasks are visualized in the same UI components than tasks, but have a fundamentally different behavior. For details on how to use Workflow tasks please see REST Workflow Task (see page 416)

The Rest Task API supports the creation, updating and content setting of PIM tasks.

- Create Task (see page 385)
  - Content (see page 386)
  - Result (see page 393)
- Update Task (see page 393)
  - Content (see page 393)
- Get Task (see page 393)
- Delete Task (see page 394)
- Update Task Content (see page 394)
  - Content (see page 395)
- Get Task Content (see page 395)
- Workflow callback (see page 396)
- Information Page (see page 396)
- Examples (see page 396)
  - Create a task for all items in the catalog TOOLS (see page 396)
  - Create a task for supplier (see page 397)
  - Update a task: Change name and description (see page 398)
  - Update a task's content: Add another item (see page 398)

7.11.1 Create Task

URL Pattern	`/manage/task`
Method	`POST`
Content types	`application/json`
Media types	`application/json`

Result	ENTITY_ITEM object of the new task.

7.11.1.1 Content

The content has to be a JSON object `TaskCreationProfile` which includes the properties listed below.

Field	Required	Datatype	Parameter description
task	yes	Task	The task's initial properties. Following Task fields are mandatory: name
content	no	ReportQuery	The report query which defines the content of the task.

Properties of the `Task` object

Field	Datatype	ReadOnly	Parameter description	Remarks
`taskType`	`String`		Type of the task, either "SingleTask" or "TaskGroup"	Mandatory. Error if invalid type is provided. Can only be changed for tasks without content.
`dynamic`	`Boolean`		Whether task is based on a dynamic query or has static content.	Default is static (false). Can only be changed using the appropriate Update Task Content `contentMode`.
`entity`	`String`		Entity identifier of the task's content type.	If task *content* is provided, *entity* must be set. Error if invalid entity. Can only be changed for tasks without content.

Properties of the `Task` object				
Field	**Datatype**	**ReadOnly**	**Parameter description**	**Remarks**
`name`	`String`		The name of the task. Ideally a meaningful short description of what is to do.	**Only if feature 'Language specific Task data' is disabled**. Mandatory.
`description`	`String`		Optional long description of the task.	**Only if feature 'Language specific Task data' is disabled**.
`parent`	`ENTITY_ITEM`		Parent task group, in case task shall belong to a group.	Error if parent task group does not exist. Error if set for SingleTask.
`user`	`ENTITY_ITEM`		The user the task will be assigned to.	At least one of the parameters *user* or *userGroup* or *supplier* have to be given. If none of them is given the creating user (see below "creationUser") will be used as assigned user by default. If *user* and *userGroup* are given the assignee *user* has priority. Error if *supplier* and *user*/*userGroup* are given. Error if *user* specified does not exist.

Properties of the `Task` object				
Field	**Datatype**	**ReadOnly**	**Parameter description**	**Remarks**
`userGroup`	`ENTITY_ITEM`		The user group the task will be assigned to.	One of the users of the given user group has to accept the task and will be the assigned user then. At least one of the parameters *user* or *userGroup* or *supplier* have to be given. If none of them is given the creating user (see below "creationUser") will be used as assigned user by default. If *user* and *userGroup* are given the assignee *user* has priority. Error if *supplier* and *user*/*userGroup* are given. Error if *userGroup* specified does not exist.
`supplier`	`ENTITY_ITEM`		The supplier the task will be assigned to.	At least one of the parameters *user* or *userGroup* or *supplier* have to be given. If none of them is given the creating user (see below "creationUser") will be used as assigned user by default. Error if *supplier* and *user*/*userGroup* are given. Error if *supplier* specified does not exist. Error if *supplier* does not match the supplier of catalog used for task content (container in the case of workflow task).
`substitute`	`ENTITY_ITEM`		The substitute replacing the assign user on the given escalation date/time if task is not completed yet.	
`responsible`	`ENTITY_ITEM`		The responsible user.	

Properties of the `Task` object				
Field	**Datatype**	**ReadOnly**	**Parameter description**	**Remarks**
`creationUser`	`ENTITY_ITEM`	yes	The user who created the task.	Set automatically
`creationDate`	`DATETIME` (see page 25)	yes	The date/time the task has been created.	Set automatically
`escalationDate`	`DATETIME` (see page 25)		The date/time the task escalates and the delegate is assigned replacing the originally assigned user.	
`deadline`	`DATETIME` (see page 25)		The date/time the task is scheduled to be fulfilled. If deadline is reached, task will be assigned to responsible.	
`finishEstimate`	`DATETIME` (see page 25)		The date/time the assigned user plans to fulfill the task (pure information).	
`finishDate`	`DATETIME` (see page 25)		The date/time the task has been finished.	

Properties of the `Task` object				
Field	**Datatype**	**ReadOnly**	**Parameter description**	**Remarks**
`progress`	`Integer`		The progress of the task (pure information).	When creating the progress 0% is used. Error if given value does not exist in progress enum (0-5).
`priority`	`Integer`		The priority of that task (pure information).	When creating the priority "normal" is used. Error if given value does not exist in priority enum (0,1,2).
`notificationLevel`	`Integer`		The profile for email notification to be used with that task.	System wide default notification level is used when creating a task.
`count`	`Integer`	yes	Number of objects in task.	Set automatically
`displayOrder`	`Integer`		Display order for arranging task in UI.	Set automatically when creating a task.
`accepted`	`Boolean`	yes	Whether task has been accepted or not.	
`acceptanceDate`	`DATETIME` (see page 25)	yes	The date/time when the task has been accepted by an assignee.	Set automatically when accepting a task.

Properties of the `Task` object				
Field	**Datatype**	**ReadOnly**	**Parameter description**	**Remarks**
`workflowTaskId`	`Long`		Id of the workflow associated with the task. (Hint: this is not related to InfaBPM workflows!)	Deprecated field. This has been used to combine Tasks with the old Workflow engine which is no longer in use.
`complete`	`Boolean`		Calculated parameter reflecting completion status.	If finishDate contradicts the intended completion status, finishDate is updated.
`template`	`ENTITY_ITEM`		UI template the task is bound to	For flexible task UI
`workflowServiceEndpoint`	`String`		The endpoint which is to be triggered on Informatica BPM side when a workflow task is marked as finished.	
`workflowCorrelationId`	`String`		The correlation id of the workflow instance which is to be triggered on InfaBPM side when a workflow task is marked as finished.	
`workflowCommunicationMode`	`QUEUE/REST`		The communication mode which can define message queue or REST communication to BPM	

Properties of the `Task` object				
Field	**Datatype**	**ReadOnly**	**Parameter description**	**Remarks**
`workflowQueueId`	`String`		The queue id from the server.properties which is defined by the syntax "queue.[queueId].name" which is used for the response message	
`taskLangs`	`Map<String,String>`		The language specific data of a task.	**Only if feature 'Language specific Task data' is enabled**

Properties of the `TaskLang` object (only if feature 'Language specific Task data' is enabled)				
Field	**Datatype**	**ReadOnly**	**Parameter description**	**Remarks**
`language`	`String`		The language of the name and description. Valid values include all synonyms of `Enum.Language.WithLanguageIndependent`	
`name`	`String`		The name of the task. Ideally a meaningful short description of what is to do.	
`description`	`String`		Optional long description of the task.	

7.11.1.2 Result

An object reference of the new task.

Field	Data type	Description
`id`	Integer	The task id
`label`	String	The label of the task

7.11.2 Update Task

URL Pattern	`/manage/task/{task-id}`
Method	`POST`
Content types	`application/json`

Updates task properties (fields). To update the list of entities associated with the task use Update Task Content (see page 394) .

7.11.2.1 Content

The content has to be a Task JSON object (see above).

7.11.3 Get Task

URL Pattern	`/manage/task/{task-id}`
Method	`GET`

Media types	`application/json`
Result	Task object

7.11.4 Delete Task

URL Pattern	`/manage/task/{task-id}`
Method	`DELETE`
Media types	`application/json`
Result	'true'/'false' string whether object could be deleted

7.11.5 Update Task Content

URL Pattern	`/manage/task/{task-id}/content/{entity}`
Method	`POST`
Parameters	`contentMode: add, remove, set - default: set` `queryMode: static, dynamic - default: static.`
Content types	`application/json`
Media types	`application/json`
Result	ENTITY_ITEM object of the updated task.

A task with static contents has a list of *entity items* (Products, Items etc) associated with it while a task with dynamic contents has an associated query. Setting task contents changes task *dynamic* property accordingly. Static contents can be adjusted by adding and removing items with the corresponding *contentMode* and static *queryMode*. Adding to and removing from a dynamic task is not possible. The corresponding task will be updated (which can be time consuming) before returning the result.

7.11.5.1 Content

The content has to be a valid ReportQuery JSON object (see above). Content of tasks created for supplier must contain objects from catalogs of assigned supplier only.

7.11.6 Get Task Content

URL Pattern	`/manage/task/{task-id}/content`
Method	`GET`
Parameters	`updateTask: updates the task if set to true, by default - false.`
Content types	`-`
Media types	`application/json`
Result	report result object

Returns report result object, which contains all information from internal report result + entity identifier.

If `updateTask` is set, corresponding task will be updated (which can be time consuming) before returning the result.

List Management API (see page 65) should be used to obtain actual report items, for example

```
http://localhost:1501/rest/V1.0/list/Article/byReportId?
reportId=<report_id>&datasource=PCM_MASTER
```

7.11.7 Workflow callback

If the `workflowServiceEnpoint` and `workflowCorrelationId` parameters are set, the application server will send a callback message to the Informatica BPM server as soon as the task gets marked as complete.

The callback message posts e.g. following XML body:

```
<taskCompletedRequest>
    <taskId>10146</taskId>
    <task>
        <taskType>SingleTask</taskType>
        <name>Test task callback</name>
        <priority>1</priority>
        <notificationLevel>1</notificationLevel>
        <creationDate>2015-10-21T14:38:40:580+0200</creationDate>
        <escalationDate/>
        <deadline/>
        <finishEstimate/>
        <finishDate/>
        <progress>0</progress>
        <dynamic>false</dynamic>
        <count>2</count>
        <entity>Article</entity>
        <displayOrder>3125</displayOrder>
        <creationUser>restuser</creationUser>
        <user>Administrator</user>
        <responsible>restuser</responsible>
        <accepted>true</accepted>
        <complete>true</complete>
    </task>
</taskCompletedRequest>
```

The <user> tag will carry the user who accepted the task before completion.

7.11.8 Information Page

A short version of the API description is available at `/manage/task/info`.

7.11.9 Examples

7.11.9.1 Create a task for all items in the catalog *TOOLS*

Rest Client Java Code

```
    EntityItemReference catalog = EntityItemReferenceFactory.createByIdentifier(
"TOOLS" );
    ReportQuery reportQuery = new ReportQuery( "byCatalog" );
    reportQuery.addParameterValue( "catalog", catalog );

    Task task = new Task();
    task.setName( "Verify all tools" );
    task.setTaskType( TaskType.SINGLE.classifier() );
    task.setEntity( "Article" );
    TaskCreationProfile taskProfile = new TaskCreationProfile();
    taskProfile.setTask( task );
    taskProfile.setContent( reportQuery );

    TaskRequest taskRequest = getRestClient().createTaskRequest();
    EntityItemReference newTask = taskRequest.createTask( taskProfile );
    Long taskId = Long.valueOf( newTask.getId() );
```

JSON

```
//POST to http://localhost:1501/rest/V1.0/manage/task
{
  "task":{"name":"Verify all tools.",
          "taskType":"SingleTask",
          "entity":"Article"
         },
  "content":{"identifier":"byCatalog",
  "parameterList":[{"key":"Catalog","value":"'TOOLS'"}]
            }
}
```

7.11.9.2 Create a task for supplier

Use POST request like http://<server>:<port>/rest/V1.0/manage/task

Create a task for Supplier

```
{
  "task":{
           "taskType" : "SingleTask",
           "name" : "Task for supplier created via Service API",
           "supplier" : "SUPPLIER_FOR_TASKS",
           "responsible" : "Administrator"
         }},
  "content":{
```

```
              "identifier":"byCatalog",
              "parameterList":
[{"key":"Catalog","value":"'CATALOG_OF_SUPPLIER_FOR_TASKS'"}]
            }
}
```

7.11.9.3 Update a task: Change name and description

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" "Content-Type: application/json" -X
POST http://localhost:1501/rest/V1.0/manage/task/1/
```

JSON

```
{
 "name":"Check short descriptions",
 "description" : "Check short descriptions in German and English"
}
```

7.11.9.4 Update a task's content: Add another item

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" "Content-Type: application/json" -X
POST http://localhost:1501/rest/V1.0/manage/task/1/content/Article?
contentMode=add&queryMode=static
```

JSON

```
{
  "identifier": "byItems",
  "parameterList": [
            {
               "key": "items",
               "value": "'I-3'@1"
            }
  ]
}
```

7.12 REST Workflow API

The Rest Workflow API allows to manage workflows, workflow status and process status. This API is used for the Informatica BPM workflow engine integration.

We strongly recommend to read the Business Process Management overview article in the knowledge base prior to this as it gives a good general overview on the workflow integration.

- Create / Update Workflow Details (see page 399)
- Get Workflow Details (see page 405)
- Enter Workflow Process Status (see page 407)
- Leave Workflow Process Status (see page 409)
- End Workflow Process (see page 411)
- Get Workflow Process Status of an entity item per process (see page 412)
- Add error message to the BPM perspective protocoll view (see page 414)

7.12.1 Create / Update Workflow Details

Creates / updates a workflow definition.

7.12.1.1 General Info

URL Pattern	/manage/workflow
Method	POST
Parameters	-
Content types	application/json
Media types	application/json
Result	Workflow creation / update optional: Workflow status creation / update

7.12.1.2 Content

The content has to be a JSON object which includes the properties listed below.

Workflow

Field	Required	Default	Datatype	Parameter description	Example
`identifier`	yes	n/a	String	Unique identifier of the workflow. Has to include a version number in case of changes in the list of available workflow states	Workflow1 Workflow1-V2
`label`	no	n/a	String	Short description of the workflow and its purpose.	
`status`	no	n/a	List of Status	List of status definitions	
`version`	no, but should be given	0.0	Decimal	This gives a version to the current workflow stored. If a version is stored once, it cannot be modified. To add or delete new status entries e.g. you have to send the workflow data with a newer version!	1.0

Status

Field	Required	Default	Datatype	Parameter description	Example
`status`	yes	n/a	`String`	Unique identifier of the status.	Status1
`displayOrder`	no	n/a	`Integer`	Position of the status within the workflow. If omitted, the order of the status in the request list is used	1

Field	Required	Default	Datatype	Parameter description	Example
`workflowTask`	no	n/a	`Task`	Workflow task that should be generated for this workflow status. See the REST Workflow Task (see page 416) API for details	<pre>{ "container": "1", "containerEntityId": "MasterCatalog", "entity": "Article" }</pre>

Field	Required	Default	Datatype	Parameter description	Example
`rejectDecision`	no	n/a	`WorkflowDecision`	WorkflowDecision containing all reject possibilities and also the content for the corresponding reject dialog that will be shown to the user. Relevant for approval workflows where an user can approve/reject at a specific point in the workflow process.	{ "label": "Title of the dialog", "singleChoice": false, "decisions": [{ "label": "Classification", "id": "executeClassification" }, { "label": "Spanish translation", "id": "executeSpanishTranslation" }] }

WorkflowDecision

Field	Required	Default	Datatype	Parameter description	Example
`label`	yes	n/a	`String`	The text for the title of the dialog	
`singleChoice`	no	false	`Boolean`	Defines if the children workflow decisions are interpreted in the dialog as radio button group (true), or check boxes (false)	
`id`	no	n/a	`String`	This identifier will be transmitted to the Informatica BPM if the corresponding decision is marked as rejected by the user. The existence of this identifier can than be used on Informatica BPM side to enter a different path in the workflow process for example.	
`decisions`	no	n/a	`ArrayList<WorkflowDecision>`	Possibility of defining a hierarchical reject dialog, combining for example radio button groups with sub ordinary check boxes.	

Result

Properties

Field	Data type	Description
`List`	`ResultSummary[]`	Counters indicating the number of created, updated and failed entities. NOTE: Successful Workflow status creations are reported as updated entities

7.12.1.3 Example

Create workflow

Request

POST http://localhost:1512/rest/V1.0/manage/workflow

```
{
    "identifier": "Workflow1",
    "label": "First Workflow",
    "status": [
        { "status": "Status1",
          "workflowTask" : { "container" : "1", "containerEntityId" :
"MasterCatalog", "entity" : "Article" }
        }
        { "status": "Status2" },
    ]
}
```

Result

```
{
    createdCounter: 2
    updatedCounter: 2
    failedCounter: 0
}
```

Create workflow with tasks for supplier and P360 user

Use POST request like http://<server>:<port>/rest/V1.0/manage/workflow

Create workflow with tasks for supplier and P360 user

```
{
  "identifier": "Workflow with Supplier Task",
  "label": "Workflow with Supplier Task",
  "status": [
             {
               "status": "Enrich data",
               "displayOrder": 1,
               "workflowTask": [
                                {
                                  "name":"Enrich data: Task for Supplier",
                                  "entity":"Article",
```

```
                                    "supplier": "SUPPLIER_FOR_TASKS",
                                    "workflowServiceEndpoint" :
"Enrich_Data_Completed",
                                    "container" : { "id" :
"'CATALOG_OF_SUPPLIER_FOR_TASKS'" }
                                  }
                                 ]
              },
              {
                "status": "Approve supplier data",
                "displayOrder": 2,
                "workflowTask": [
                                  {
                                    "name":"Approve supplier data: Task for P360
User",
                                    "entity":"Article",
                                    "user": "P360_USER",
                                    "workflowServiceEndpoint" :
"Approve_Data_Completed",
                                    "container" : { "id" :
"'CATALOG_OF_SUPPLIER_FOR_TASKS'" }
                                  }
                                 ]
              }
             ]
}
```

7.12.2 Get Workflow Details

Get a workflow definition.

7.12.2.1 General Info

URL Pattern	/manage/workflow/{workflowIdentifier}
Method	GET
Parameters	-
Content types	application/json
Media types	application/json

Result	Workflow details Workflow status list

7.12.2.2 Parameters

Workflow

Field	Required	Default	Datatype	Parameter description	Example
`workflowIdentifier`	yes	n/a	String	Unique identifier of the workflow. Has to include a version number in case of changes in the list of available workflow states	Workflow1 Workflow1-V2
`resolveWorkflowTasks`	no	false	boolean	If set to true, the workflow task information is resolved, too	
`includeObsoletes`	no	false	boolean	If true, status entries which are not available in the newest version of the workflow, will be resolved, too. Otherwise obsolete status entries will not be shown	

Example

Request

GET http://localhost:1512/rest/V1.0/manage/workflow/Workflow1

Result

```
{
   "identifier": "Workflow1",
   "label": "First Workflow",
   "status": [
       { "status": "Status1", "displayOrder": 1 },
       { "status": "Status2", "displayOrder": 2 }
   ]
```

```
}
```

7.12.3 Enter Workflow Process Status

Enters a specific status within the workflow process. If a WorkflowTask is related to the status, this call adds the entity to it.

> ⚠ Starting from 10.5.0.01 EBF3 customers still using the *deprecated* request with only query parameters & no JSON/XML payload will need to explicitly add the request header - "**Content-Type: application/octet-stream**" in the requests.

7.12.3.1 General Info

URL Pattern	/manage/workflow/status/enter
Method	POST
Parameters	-
Content types	application/json
Media types	application/json
Result	success result

7.12.3.2 Content

The content has to be a JSON object which includes the properties listed below.

Field	Required	Default	Datatype	Parameter description	Example
`processId`	yes	n/a	`String`	Unique identifier of the workflow instance. Within Informatica BPM workflow instances are called processes.	'1234567'
`workflowId`	yes	n/a	`String`	Unique identifier of the workflow itself	'workflowXYZ'
`status`	yes	n/a	`String`	A status name	'stateA', 'stateB'
`entity`	yes	n/a	`String`	The entity name	'Article', 'Variant' or 'Product2G'
`itemId`	yes	n/a	`String`	The itemids in the typical service API syntax	'1234@1' (see page 399)
`resetAcceptUser`	no	true	`boolean`	If the item was previously accepted by an user, the user is reset and the item available to the usergroup again (since 8.0.01)	'true'
`hint`	no	n/a	`String`	A comment field, which can be sent to the server, when this status has been entered	

7.12.3.3 **Example**

Request

POST http://localhost:1512/rest/V1.0/manage/workflow/status/enter

```
{
   "processId": "4711",
   "workflowId": "Workflow1",
   "status": "Status1",
   "entity": "Article",
   "hint" : "Comment for Status1",
   "itemId": [ "123@1" ]
}
```

Result

```
{
  "successful":true,
  "canNotFindItem":false,
  "canNotFindWorkflowId":false,
  "workflowTaskNotFound":false
}
```

7.12.4 Leave Workflow Process Status

Leaves a specific status within the workflow process. This call is only required in case no Workflow Task is assigned to the status. In this case PIM leaves the workflow process status implicitly.



Starting from 10.5.0.01 EBF3 customers still using the *deprecated* request with only query parameters & no JSON/XML payload will need to explicitly add the request header - "**Content-Type: application/octet-stream**" in the requests.

7.12.4.1 General Info

URL Pattern	/manage/workflow/status/leave
Method	POST
Parameters	-
Content types	application/json

Media types	application/json
Result	success result

7.12.4.2 Content

The content has to be a JSON object which includes the properties listed below.

Field	Required	Default	Datatype	Parameter description	Example
`processId`	yes	n/a	`String`	Unique identifier of the workflow instance	'1234567'
`status`	yes	n/a	`String`	A status name	'stateA', 'stateB'
`entity`	yes	n/a	`String`	The entity name	'Article', 'Variant' or 'Product2G'
`itemId`	yes	n/a	`String`	The itemids in the typical service API syntax	'1234@1' (see page 399)
`hint`	no	n/a	`String`	A comment field, which can be sent to the server, when this status has been left	

7.12.4.3 **Example**

Request

POST http://localhost:1512/rest/V1.0/manage/workflow/status/leave

```
{
   "processId": "4711",
   "status": "Status1",
   "entity": "Article",
   "hint" : "Comment for Status1",
   "itemId": [ "123@1" ]
}
```

Result

```
{
  "successful" : true,
  "canNotFindItem" : false
}
```

7.12.5 End Workflow Process

Ends a workflow process and soft deletes all related status entries

7.12.5.1 General Info

URL Pattern	/manage/workflow/process
Method	DELETE
Parameters	-
Content types	application/json
Media types	application/json
Result	-

7.12.5.2 Parameters

Field	Required	Default	Datatype	Parameter description	Example
`processId`	yes	n/a	`String`	Unique identifier of the workflow instance	'1234567'

7.12.5.3 **Example**

Request

DELETE

```
http://localhost:1512/rest/V1.0/manage/workflow/process?processId=9375
```

7.12.6 Get Workflow Process Status of an entity item per process

Resolves information about the current status entries related to a given workflow process

7.12.6.1 General Info

URL Pattern	/manage/workflow/status
Method	GET
Parameters	-
Content types	application/json
Media types	application/json

Result	success result and status information

7.12.6.2 Parameters

Field	Required	Default	Datatype	Parameter description	Example
`processId`	yes	n/a	`String`	Unique identifier of the workflow instance	'1234567'
`entity`	yes	n/a	`String`	The entity name	'Article', 'Variant' or 'Product2G'
`itemId`	yes	n/a	`String`	The itemids in the typical service API syntax	'1234@1' (see page 399)

7.12.6.3 **Example**

Request

GET

```
 http://localhost:1512/rest/V1.0/manage/workflow/status?
itemId=30025@1&entity=Article&processId=9375
```

Result


```
{
  "noItemFound":false,
  "statusList":[
    {
      "status":"teststatusX",
      "visits":1,
      "duration":0,
      "startTime":"2015-04-17T16:41:41:950+0200",
```

```
      "endTime":"2015-04-17T16:41:41:957+0200"
    }]
}
```

7.12.7 Add error message to the BPM perspective protocoll view

This can be used within catch exception events within a workflow, to add information to the bpm perspectives protocoll log view

7.12.7.1 General Info

URL Pattern	/manage/workflow/message
Method	PUT
Parameters	-
Content types	application/json
Media types	application/json
Result	HTTP 201

7.12.7.2 Parameters

Field	Required	Default	Datatype	Parameter description	Example
`message`	yes	n/a	`String`	Error message	'Workflow 1 - Process 1234 - Assign task to user xyz failed'

Field	Required	Default	Datatype	Parameter description	Example
`severity`	no	ERROR	`String`	The entity name	ERROR, INFO, WARNING, CANCEL, OK

7.12.8 REST Workflow Task

This page describes the data structure of a workflow task as it's used in combination with the REST Workflow API (see page 399). A workflow task has a limited number of attributes compared to a static task as most of it's attributes are defined by the workflow status to which it belongs.

- Example (see page 416)
- WorkflowTask (see page 417)
- Task Callback (see page 419)
  - Callback Payload for Finished state (see page 420)
  - Callback Payload when the process must be terminated (see page 424)

7.12.8.1 Example

This is an example workflow which has multiple workflow tasks. In this example only typical and mandatory attributes are provided for the workflow task. Please see the table below for the full list of attributes which can be provided here.

```
<rest:RESTRequest
    xmlns:rest="http://schemas.activebpel.org/REST/2007/12/01/aeREST.xsd">
    <rest:method>POST</rest:method>
    <rest:pathInfo>rest/V1.0/manage/workflow</rest:pathInfo>
    <rest:headers>
        <rest:header name="authorization" value="{ $Authorization }"/>
        <rest:header name="bpm-correlation-id" value="{ $processId }"/>
        <rest:header name="Content-Type" value="application/json"/>
        <rest:header name="accept" value="application/json"/>
    </rest:headers>
    <rest:payload contentType="application/json">
{{
   "identifier": "{ $workflowName }",
   "label": "{$workflowName}",
   "version": "{$workflowVersion}",
   "status": [
      {{ "status": "Classification",
         "workflowTask" :  {{ "container" : "'{$catalogIdentifier}'", "entity" :
"Article", "template" : "Item classification UI", "workflowServiceEndpoint" :
"ApprovalWorkflowTaskClassification-Finish", "userGroup" : "AllRights" }}  }},
      {{ "status": "TranslateSpanish",
         "workflowTask" :  {{ "container" : "'{$catalogIdentifier}'", "entity" :
"Article", "template" : "Item translation UI", "workflowServiceEndpoint" :
"ApprovalWorkflowTaskTranslateSpanish-Finish", "userGroup" : "AllRights" }}  }},
      {{ "status": "TranslateItalian",
         "workflowTask" :  {{ "container" : "'{$catalogIdentifier}'", "entity" :
"Article", "template" : "Item translation UI", "workflowServiceEndpoint" :
"ApprovalWorkflowTaskTranslateItalian-Finish", "userGroup" : "AllRights" }}  }},
      {{ "status": "TranslateFrench",
         "workflowTask" :  {{ "container" : "'{$catalogIdentifier}'", "entity" :
"Article", "template" : "Item translation UI", "workflowServiceEndpoint" :
"ApprovalWorkflowTaskTranslateFrench-Finish", "userGroup" : "AllRights" }}  }},
```

```
      {{ "status": "Approve",
         "workflowTask" :  {{ "container" : "'{$catalogIdentifier}'", "entity" :
"Article", "template" : "Item approve UI", "workflowServiceEndpoint" :
"ApprovalWorkflow-Finish", "userGroup" : "AllRights" }},
        "rejectDecision": {{ "label": "Title of Reject Dialog", "singleChoice": true,
            "decisions": [{{ "label": "Classification", "id":
"executeClassification" }},
                          {{ "label": "Translation", "singleChoice": false,
"decisions":
                             [{{ "label": "Translate texts into French", "id":
"executeTranslationFrench" }},
                              {{ "label": "Translate texts into Spanish", "id":
"executeTranslationSpanish" }},
                              {{ "label": "Translate texts into Italian", "id":
"executeTranslationItalian" }}
                             ]
                          }}
                        ]
          }}
      }}
     ]
}}
</rest:payload>
    <rest:locales>
        <rest:locale country="US" lang="en">en_US</rest:locale>
    </rest:locales>
</rest:RESTRequest>
```

7.12.8.2 WorkflowTask

Field	Datatype	Description
`container`	`ENTITY_ITEM`	The container, items of the workflow task are contained in. In case of Products, Variants or Items this would be a Catalog for example.
`entity`	`String`	Entity identifier of the task's content type.

Field	Datatype	Description
`workflowServiceEndpoint`	`String`	The endpoint which is to be triggered on Informatica BPM side when a workflow task is marked as finished.
`workflowCommunicationMode`	`QUEUE/REST`	The communication mode which can define message queue or REST communication to BPM
`workflowQueueId`	`String`	The queue id from the server.properties which is defined by the syntax "queue.[queueId].name" which is used for the response message
`userGroup`	`ENTITY_ITEM`	The user group the task will be assigned to. Error if specified *userGroup* does not exist. At least one of the parameters *user* or *userGroup* or *supplier* have to be given. If none of them is given the creating user (see below "creationUser") will be used as assigned user by default. If *user* and *userGroup* are given the assignee *user* has priority. Error if *supplier* and *user*/*userGroup* are given.
`user`	`ENTITY_ITEM`	The user the task will be assigned to. Error if specified *user* does not exist. At least one of the parameters *user* or *userGroup* or *supplier* have to be given. If none of them is given the creating user (see below "creationUser") will be used as assigned user by default. If *user* and *userGroup* are given the assignee *user* has priority. Error if *supplier* and *user*/*userGroup* are given.

Field	Datatype	Description
`supplier`	`ENTITY_ITEM`	The supplier the task will be assigned to. Error if specified *supplier* does not exist. Error if *supplier* does not match the supplier of catalog used for task content (container in the case of workflow task). At least one of the parameters *user* or *userGroup* or *supplier* have to be given. If none of them is given the creating user (see below "creationUser") will be used as assigned user by default. Error if *supplier* and *user*/*userGroup* are given.
`template`	`ENTITY_ITEM`	Identifier of the UI template the task is bound to (aka Flexible Task UI). Error in case the template for this identifier does not exist
`maximumDuration`	`Long`	The maximum amount of time in milliseconds an item residing in a workflow task can be worked on (after accepting) until the deadline for this item is reached. No functional impact as of now (only for information purposes).
`name`	`String`	The name of the task. Ideally a meaningful short description of what is to do.
`description`	`String`	Optional long description of the task.
`priority`	`Integer`	The priority of that task (pure information). When creating the priority "normal" is used. Error if given value does not exist in priority enumeration (0,1,2).

7.12.8.3 Task Callback

As soon as the user decides that the task is finished for a specific list of items of a workflow task, Informatica BPM will be called from Product 360 so BPM can move the affected workflow instances to the next step.

For this, you need to provide the `workflowServiceEndpoint` when the workflow task is created, and the `processId` when the item will enter the corresponding workflow status. Informatica Product 360 will call BPM for each workflow instance of this status so the workflow can be moved forward to the next action. Please note that Informatica BPM will be called for each process instance for this workflow.

Additionally to this pure "signal" we provide some payload data on the current process status the item was in when the user finished the task for the item. This way the workflow can decide what to do next without having to fetch this again from Product 360.

Callback Payload for Finished state

Here is an example of the payload which is given in the callback. Note that either user, or userGroup or supplier is given here. (TODO: correct!?)

Note: This extended callback payload is available with 8.1.1.00.05 and after.

```
<rest:RESTRequest xmlns:rest="http://schemas.activebpel.org/REST/2007/12/01/
aeREST.xsd">
   <rest:method>POST</rest:method>
   <rest:headers>
      <rest:header name="bpm-correlation-id" value="8002"/>
      <rest:header name="connection" value="keep-alive"/>
      <rest:header name="host" value="localhost:8080"/>
      <rest:header name="Content-Length" value="1547"/>
      <rest:header name="accept" value="text/html, image/gif, image/jpeg, *; q=.2, */
*; q=.2"/>
      <rest:header name="Content-Type" value="text/xml"/>
   </rest:headers>
   <rest:payload contentType="text/xml">
      <decisions>
         <decision id="p360.bpm.finish"/>
         <processStatusInfo>
            <processIdentifier>8002</processIdentifier>
            <itemEntity>Article</itemEntity>
            <item>2401@1</item>
            <statusEntry>
               <attribute>
                  <name>workflow</name>
                  <value>905</value>
               </attribute>
               <attribute>
                  <name>status</name>
                  <value>Test Status1</value>
               </attribute>
               <attribute>
                  <name>startTime</name>
                  <value>2018-10-02T14:50:38:440+0200</value>
               </attribute>
               <attribute>
                  <name>lastStartTime</name>
                  <value>2018-10-02T14:50:38:443+0200</value>
```

```
</attribute>
<attribute>
   <name>user</name>
   <value>100</value>
</attribute>
<attribute>
   <name>userGroup</name>
</attribute>
<attribute>
   <name>supplier</name>
</attribute>
<attribute>
   <name>endTime</name>
   <value>2018-10-02T14:51:16:486+0200</value>
</attribute>
<attribute>
   <name>duration</name>
   <value>0</value>
</attribute>
<attribute>
   <name>visitCounter</name>
   <value>1</value>
</attribute>
<attribute>
   <name>errorMessage</name>
   <value>No error</value>
</attribute>
<attribute>
   <name>acceptDate</name>
   <value>2018-10-02T14:51:08:521+0200</value>
</attribute>
<attribute>
   <name>deadline</name>
   <value>2018-10-02T14:51:08:440+0200</value>
</attribute>
<attribute>
   <name>creationDate</name>
   <value>2018-10-02T14:50:38:480+0200</value>
</attribute>
<attribute>
   <name>creationUser</name>
   <value>300</value>
</attribute>
<attribute>
   <name>modificationDate</name>
   <value>2018-10-02T14:51:08:520+0200</value>
</attribute>
<attribute>
   <name>modificationUser</name>
   <value>100</value>
</attribute>
<attribute>
   <name>hint</name>
```

```
               </attribute>
            </statusEntry>
         </processStatusInfo>
      </decisions>
   </rest:payload>
   <rest:ssl>false</rest:ssl>
   <rest:contextPath>/active-bpel</rest:contextPath>
   <rest:requestURI>/active-bpel/services/REST/TestStatus1-Finished</rest:requestURI>
   <rest:locales>
      <rest:locale country="US" lang="en">en_US</rest:locale>
   </rest:locales>
</rest:RESTRequest>
```

Field	Datatype	Description
`processIdentifier`	`String`	The process identifier, aka `correlationId` of the workflow process
itemEntity	String	The repository entity of the item which has been finished (e.g. Article, or Product2G etc.)
`item`	`ENTITY_ITEM`	The object for which the workflow task has been finished in the String Syntax of entity items
`statusEntry`	`StatusEntry[]`	The status entry object of the status the user just finished. Note: Although the model defines an array here, it can actually be only a single status entry each status represents a task the user finished working on.

The `StatusEntry` object

Field	DataType	Description
`workflow`	`ENTITY_ITEM (in String syntax)`	The workflow object this status entry belongs to

The `StatusEntry` object		
Field	**DataType**	**Description**
`status`	`String`	The identifier of the status
`user`	`ENTITY_ITEM (in String syntax)`	The user which finished the work on the item. It's either a supplier, or a user, never both.
`supplier`	`ENTITY_ITEM (in String syntax)`	The supplier which finished the work on the item. It's either a supplier, or a user, never both.
`errorMessage`	`String`	
`hint`	`String`	Optional hint provided by the user when finishing the work. Example would be an approval task in which the approving user gives a hint as to what is not correct.
`startTime`	`Timestamp`	The first time the item started to be part of the workflow task. In other words, the first time the item entered this workflow state
`lastStartTime`	`Timestamp`	The last time the item started to be part of the workflow task. In other words, the last time the item entered this workflow state
`endTime`	`Timestamp`	The last time when the item finished this workflow task. In other words, the timestamp the item left this workflow state the very last

The `StatusEntry` object		
Field	**DataType**	**Description**
`acceptDate`	`Timestamp`	The time the user accepted the workflow task for this item the last time
`deadline`	`Timestamp`	The calculated deadline of the item in this workflow task. Based on the time the item entered the task (`startTime`) and the given `maximumDuration` for this status. (TODO: is this correct?)
`visitCounter`	`Long`	The number of times this state has been entered
`duration`	`Long`	The total duration the item was part of this workflow state. This means for each recurring enter/leave the time is summed up here.

Typically a workflow is designed to not trust the users very much. In case a data quality result will trigger the item to be part of a manual task, it's normal that the workflow will check the data quality again after the task has been finished. In case the data quality is still not correct, the workflow will loop back and enter the same workflow task again. So a user which "just finishes" a task, without actually solving the data quality issue of the item, will get this item back.

In other words, the same item can be multiple time in the same status of the same workflow instance. Therefore we do record how many times the item was in this state (`visitCounter`) and we sum up the total `duration` of the item in this state (duration).

Callback Payload when the process must be terminated

In case a workflow id deleted in Product360 while there are still instances running, or because of other scenarios, Product 360 will contact BPM with a predefined terminate process endpoint. The identifier of this endpoint is `P360-TerminateProcessInstance` .

Even in this case, in which the user did not finish a task by himself, but the system terminated the workflows we provide as much information as possible to the workflow.

Field	Datatype	Description
`processIdentifier`	`String`	The process identifier, aka `correlationId` of the workflow process
`item`	`ENTITY_ITEM`	The object for which the workflow task has been finished
`statusEntry`	`StatusEntry[]`	An array of status entry objects of all the status the item was currently in at the time the workflow got terminated. In contrary to the payload when the user finishes his work for a specific status (above) this payload contains the entries of all status - as all of them terminated at once. For details on the StatusEntry object please see above

8 REST Enumeration API

The Enumeration API provides read-only access to all enumerations in Product 360. The enum entry keys are used in field values as well as for the qualification of fields.

- All Enumerations (see page 426)
  - Result (see page 426)
- Meta Data and Enumeration Entries (see page 427)
  - Result (see page 428)
- Examples (see page 429)
  - Retrieving all enumerations (see page 429)
  - Retrieving the entries of enumeration Enum.Languages (see page 430)
  - Retrieving the entries of enumeration Enum.SupplierWithMainSupplier (see page 431)

8.1 All Enumerations

Returns the list of all available enumerations.

URL Pattern	/enum/info
Method	GET
Parameters	No parameters are available
Media type	text/html, application/json, application/xml
Result	A list of all enumerations defined in the repository

8.1.1 Result

For each enumeration the following properties are returned:

Key	Data type	Description
identifier	`String`	Unique identifier of the enumeration
dataType	`String`	Data type of the enumeration entry's key, see also REST Datatypes (see page 25).
name	`String`	Localized name of the enumeration
description	`String`	Localized description of the enumeration

8.2 Meta Data and Enumeration Entries

Returns the entries of the specified enumeration.

URL Pattern	/enum/{enumeration-identifier}
Method	GET
Parameters	<table><tr><th>Parameter</th><th>Required</th><th>Default</th><th>Datatype</th><th>Description</th></tr><tr><td>container</td><td>no</td><td>none</td><td>ENTITY_ITEM</td><td>In case the enum requires a container it must be specified with this parameter and the ENTITY_ITEM syntax. So either with the internal ID like `container=4711` or the identifier like `container='MyLookup'` If the parameter is missing but the enumeration needs one, a corresponding status and error message is returned.</td></tr></table>

Media type	`text/html, application/json, application/xml`
Result	The meta data of the enumeration (like the localized name) and all entries of the enumeration

8.2.1 Result

The following properties are returned for an enumeration:

Enumeration properties			
	Field	**Data type**	**Description**
	identifier	`String`	Unique identifier of this enumeration
	name	`String`	Localized name of this enumeration
	description	`String`	Localized description of this enumeration
	dataType	`String`	Data type of the enumeration entry's key, see also REST Datatypes (see page 25).
	entries	`Array`	List of all entries of this enumeration

Each enumeration entry has the following properties:

Properties of a member of `entries`			
	Field	**Data type**	**Description**
	label	String	Unique label of this enumeration entry

	key	String or Object	Unique key of this enumeration entry, not necessarily human readable. In case the key is of type ENTITY_ITEM, the key is represented as proxy object containing the properties `id`, `url` and `label`. If XML is specified as output format, a proxy object is embedded in the tag `object`. When creating a qualified field, this value should be used. If it is a proxy object, the value of the property `id` should be used.
	externalCode	String	External code of the enumeration entry.
	keySynonyms	Array of Strings	List synonyms for the entry

8.3 Examples

8.3.1 Retrieving all enumerations

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/enum/info
```

The following JSON object is returned:

```
{
   "enumerations": [
        {
            "identifier": "Enum.Language",
            "dataType": "INTEGER",
            "name": "All languages",
            "description": ""
        },
    ...
]}
```

Rest Client Java Code

```
EnumerationInfoRequest enumInfoRequest = restClient.createEnumerationInfoRequest();
EnumerationInfos allEnumInfos = enumInfoRequest.getAllEnumerations();
for (EnumerationInfo enumInfo : allEnumInfos)
{
        // ...
 }
```

8.3.2 Retrieving the entries of enumeration Enum.Languages

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/enum/Enum.Language
```

The following JSON object is returned:

```
{
    "identifier": "Enum.Language",
    "name": "All languages",
    "description": "",
    "dataType": "INTEGER",
    "entries": [
        {
            "label": "Italian",
            "key": "16",
            "externalCode": "ita",
            "synonyms": [
                "italian",
                "it",
                "ita",
                "ita_it"
            ]
        },
        ...
    ]
}
```

Rest Client Java Code

```
EnumerationRequest enumRequest = restClient.createEnumerationRequest();
Enumeration enumeration = enumRequest.getEnumeration( "Enum.Language" );
```

```
List< EnumerationEntry > enumEntries = enumeration.getEntries();
```

8.3.3 Retrieving the entries of enumeration Enum.SupplierWithMainSupplier

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/enum/Enum.SupplierWithMainSupplier
```

The following JSON object is returned:

(Note that the keys are in this case returned as proxy objects and the special object *<main supplier>* does not really exist but is a placeholder for the main supplier as specified within the HPM.)


```
{
    "identifier": "Enum.SupplierWithMainSupplier",
    "name": "Suppliers (inc. main supplier)",
    "description": "",
    "dataType": "ENTITY_ITEM",
    "entries": [
        {
            "label": "<Main supplier>",
            "key": {
                "id": "-10",
                "label": ""
            },
            "externalCode": "",
            "synonyms": []
        },
        {
            "label": "TestProvider",
            "key": {
                "id": "101",
                "label": "TestProvider"
            },
            "externalCode": "TestProvider",
            "synonyms": []
        },
        {
            "label": "Heiler Product Manager",
            "key": {
                "id": "3",
                "label": "Heiler Product Manager"
            },
            "externalCode": "Heiler Product Manager",
            "synonyms": []
        }
    ]
```

```
35 }
```

9 REST Media API

The REST Media API provides access to all kinds of media assets in the Product Manager.

- Single Media Asset File (see page 433)
- Preview of Media Asset File (see page 433)
- Media Asset File Location (see page 434)
- Upload single Media Asset File (see page 434)
- Examples (see page 435)
  - Retrieve single media asset file in 36dpi JPEG quality with identifier D123456 (see page 435)
  - Retrieve media asset file preview (see page 436)
  - Retrieve media asset file location (see page 436)
  - Upload media asset file to media asset server (see page 436)

9.1 Single Media Asset File

URL Pattern	`/media/{quality/derivativeDefinition}/{identifier}`	
Method	GET	
Parameters	`lastModified`	DATETIME
Content types	application/octet-stream	
Returns	the media asset file as binary data stream	

9.2 Preview of Media Asset File

URL Pattern	`/media/{identifier}/preview`	
Method	GET	
Parameters	`size`	All allowed values: small | medium | large

Content types	image/jpg	
Returns	the media asset preview as binary jpeg	

9.3 Media Asset File Location

URL Pattern	`/media/{quality/derivativeDefinition}/{identifier}`
Method	GET
Parameters	-
Content types	text/plain
Returns	the URL to the media asset file

9.4 Upload single Media Asset File

To upload a local image to the media asset server, the following two steps should be performed:

1. call the REST File API (see page 339) to upload a local file to the PIM storage, which returns a result contains id for the uploaded file.
2. call the following REST API to add the uploaded file to the media manager. The path parameter "fileId" is the id of the returned file object of the first step.

URL Pattern	`/media/{fileId}`	
Method	POST	
Parameters	`categoryId`	optional parameter indicates the id of the category
Content types	text/plain	

Returns	the identifier of the uploaded media asset file	

The query parameter "categoryId" is optional, it indicates the id of the category to which the added media asset file should belong. If it is not given, the media asset file will be assigned to the default category.

9.5 Examples

9.5.1 Retrieve single media asset file in 36dpi JPEG quality with identifier D123456

Rest Call

```
curl -u rest:heiler -H "Accept: application/octet-stream" -X GET http://
localhost:1501/rest/V1.0/media/JPEG 36dpi/D123456
```

Rest Client Java Code

```
MediaRequest mediaAssetRequest = restClient.createMediaRequest();

InputStream result = mediaAssetRequest.getMediaAsset( "D123456", "JPEG 36dpi",
null );
```

Return media asset file only if modified after given time (otherwise empty stream is returned):

Rest Call

```
curl -u rest:heiler -H "Accept:application/octet-stream" -X GET http://
localhost:1501/rest/V1.0/media/JPEG 36dpi/D123456?lastModified=2012-09-26T15:33:12
```

Rest Client Java Code

```
MediaRequest mediaRequest = restClient.createMediaRequest();

Timestamp lastModified = new Timestamp( calendar.getTime().getTime() );
```

```
InputStream result = mediaRequest.getMediaAsset( "D123456", "JPEG 36dpi",
lastModified );
```

9.5.2 Retrieve media asset file preview

Rest Call

```
curl -u rest:heiler -H "Accept: image/jpg" -X GET http://localhost:1501/rest/V1.0/
media/D123456/preview?size=small
```

Rest Client Java Code

```
MediaRequest mediaRequest = restClient.createMediaRequest();

PreviewSize size = PreviewSize.BIG;

InputStream result = mediaAssetRequest.getMediaAssetPreview( "D123456", size );
```

9.5.3 Retrieve media asset file location

```
curl -u rest:heiler -H "Accept: text/plain" -X GET http://localhost:1501/rest/V1.0/
media/JPEG 36dpi/D123456
```

Rest Client Java Code

```
MediaRequest mediaRequest = restClient.createMediaRequest();

URL mediaAssetUrl = mediaRequest.getMediaAssetLocation( "D123456", "JPEG 36dpi");
```

9.5.4 Upload media asset file to media asset server

call REST File API (see page 339) to upload a local file to the PIM storage

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type:application/octet-
stream"  --data-binary "@Test.jpg" -X POST http://localhost:1501/rest/V1.0/manage/
file?originalFilename=Test.jpg
```

Rest Client Java Code

```
FileUploadRequest fileUploadRequest = restClient.createFileUploadRequest();

FileReference result = fileUploadRequest.uploadFile( "Test.jpg", inputStream );
```

An object including an ID and an URL is returned:

```
{
  "id": "feb3cd45-d27e-4700-912e-26014512a6e3",
  "originalFilename": "Test.jpg"
}
```

Then call following Rest Media API to add the uploaded file to the media asset server(in the default category) .

Rest Call

```
curl -u rest:heiler -H "Accept: text/plain" -X POST http://localhost:1501/rest/V1.0/
media/feb3cd45-d27e-4700-912e-26014512a6e3
```

Rest Client Java Code

```
MediaRequest mediaRequest = restClient.createMediaRequest();

String mediaAssetIdentifier = mediaRequest.addMediaAsset( "feb3cd45-
d27e-4700-912e-26014512a6e3");
```

Or call the same API with query parameter "categoryId" to add the uploaded file in desired category

```
curl -u rest:heiler -H "Accept: text/plain" -X POST http://localhost:1501/rest/V1.0/
media/feb3cd45-d27e-4700-912e-26014512a6e3?
categoryId=00990000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000
```

Rest Client Java Code

```
MediaRequest mediaRequest = restClient.createMediaRequest();

String mediaAssetIdentifier = mediaRequest.addMediaAsset( "feb3cd45-
d27e-4700-912e-26014512a6e3",
```

```
"00990000000000
```

9.6 REST List API for MediaAssetFile entity

The REST List API for MediaAssetFile entity provides searching, reporting, navigation, deletion and finally result listing of MediaAssetFile like other general objects in system.

Since PIM 8.0 the MediaAssetFile entity is redefined for the media asset object, but currently it is only supported by PIM Core with Media Manager.

The data access for the MediaAssetFile entity is not per PIM database, but per the connector API, so that the corresponding persistence logic like fragments, mediators and reports are adjusted in a special way, therefore it is meaning to introduce the REST List API for that with this additional chapter.

- Adjustments for MediaAssetFile entity (see page 439)
  - Internal id (see page 439)
  - Special reports (see page 440)
    - byCategory (see page 440)
    - bySearch (see page 440)
      - Search with query (see page 441)
      - Search with mediaQuery (see page 441)
- Examples (see page 441)
  - Retrieve media asset file with "byIdentifiers" report for external id (see page 441)
  - Retrieve media asset file with "byItems" report for internal id (see page 442)
  - Retrieve media asset files which belong(be assigned) to a category and are used in PIM core (see page 443)
  - Retrieve all media asset files which belong to a category or its child categories (see page 444)
  - Update media asset file(Root Entity) (see page 445)
  - Update media asset file language special attribute(Sub Entity) (see page 446)
  - Delete media asset file with identifier D11000100112532 (see page 448)
  - Create media asset file (see page 448)

9.6.1 Adjustments for MediaAssetFile entity

The REST List API access for general object is already described in REST List API (see page 55), which is also available for the MediaAssetFile entity with some adjustments.

9.6.1.1 Internal id

Each PIM entity needs an unique internal id which should be a long value and usually comes from the database. For the MediaAssetFile entity it is converted from the media asset identifier (PKOM_PNR) of Media Manager in the way which replaces the first character "D" with the "9".

For example the media asset with identifier D110001123456 has then an internal id 9110001123456.

9.6.1.2 Special reports

byCategory

Search media asset files which belong (be assigned) to a category

Identifier	Name	Description	Mandatory	Data type	Collection	Entity	Enumeration
categoryId	Category ID	Category id defined in Media Manager	true	STRING	false		
usageFilter	Usage filter	Filter all/used/unused media asset files	false	INTEGER	false		Media asset files usage filter

bySearch

Identifier	Name	Description	Mandatory	Data type	Collection	Entity	Enumeration
query available since 8.0.03 Hotfixes 1	Search query	The query condition for the search. For details of the syntax to be used, refer to the documentation. If this parameter is not specified, then the parameter "mediaQuery" should be set.	false	STRING	false		
mediaQuery	Special media search query	Special search query for media asset file. If this parameter is not specified, then the parameter "query" should be set.	false	STRING	true		

Identifier	Name	Description	Mandatory	Data type	Collection	Entity	Enumeration
usageFilter	Usage filter	Filter all/used/unused media asset files	false	INTEGER	false		Media asset files usage filter

Search with query



This search is available since the 8.0.03 Hotfixes 1

For detailed information about the search with the parameter "query" please visit the page Search query for MediaAssetFile entity (see page 455).

Search with mediaQuery

For detailed information about the search with the parameter "mediaQuery" please visit the page Media search query for MediaAssetFile entity (see page 449).

9.6.2 Examples

9.6.2.1 Retrieve media asset file with "byIdentifiers" report for external id

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/byIdentifiers?identifiers=D11000100112129&fields=MediaAssetFile.Identifier,MediaAssetFileAttribute.Filename,MediaAssetFileAttributeLang.Memo(en),MediaAssetFileAttributeLang.Memo(german)
```

A list model result that contains the desired media asset files and attributes is returned:

```
{
    "cacheId": "20150813_172919_0",
    "entityIdentifier": "MediaAssetFile",
```

```
    "totalSize": 1,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 1,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "911000100112129",
                "label": "Media asset file 911000100112129"
            },
            "values": [
                "D11000100112129",
                "Lighthouse.jpg",
                "Lighthouse_memo_english",
                "Lighthouse_memo_german"
            ]
        }
    ]
}
```


9.6.2.2 Retrieve media asset file with "byItems" report for internal id

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H -X GET http://localhost:1512/
rest/V1.0/list/MediaAssetFile/byItems?
items=911000100112129,911000100112211&fields=MediaAssetFile.Identifier,MediaAssetFile
Attribute.Filename,MediaAssetFileAttributeLang.Memo(en),MediaAssetFileAttributeLang.M
emo(german)
```

A list model result that contains the desired media asset files and attributes is returned:


```
{
    "cacheId": "20150812_164055_0",
    "entityIdentifier": "MediaAssetFile",
    "totalSize": 2,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 2,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "911000100112129",
```

```
                "label": "Media asset file 911000100112129"
            },
            "values": [
                "D11000100112129",
                "Lighthouse.jpg",
                "Lighthouse_memo_english",
                "Lighthouse_memo_german"
            ]
        },
        {
            "object": {
                "id": "911000100112211",
                "label": "Media asset file 911000100112211"
            },
            "values": [
                "D11000100112211",
                "Koala.jpg",
                "Koala_memo_english",
                "Koala_memo_german"
            ]
        }
    ]
}
```


9.6.2.3 Retrieve media asset files which belong(be assigned) to a category and are used in PIM core

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/byCategory?usageFilter=used&CategoryId=00990000000000
```

```
            "object": {
                "id": "9110000100112048",
                "label": "Media asset file 9110000100112048"
            },
            "values": [
                "D11000100112048",
                "text.txt"
            ]
        },
       ...
    ]
}
```

9.6.2.4 Retrieve all media asset files which belong to a category or its child categories

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/byCategory?CategoryId=0099%&fields=MediaAssetFile.Identifier,MediaAssetFileAttribute.Filename
```

The following JSON object is returned:

```
{
  "cacheId": "20160204_181155_0",
  "entityIdentifier": "MediaAssetFile",
  "totalSize": 191,
  "startIndex": 0,
  "pageSize": 100,
  "rowCount": 100,
  "columnCount": 0,
  "columns": [],
  "rows": [
    {
      "object": {
        "id": "9110000100111676",
        "label": "Media-Asset-Datei 9110000100111676",
        "entityId": 2500
      },
      "values": [
        "D11000100111676",
        "testtest.jpg"
      ]
    }
    ...
    ]
}
```

9.6.2.5 Update media asset file(Root Entity)

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1512/rest/V1.0/list/MediaAssetFile
```

The following JSON object is provided as content:

```
{
  "columns": [
    {
      "identifier": "MediaAssetFileAttribute.AgencyId"
    },
    {
      "identifier": "MediaAssetFileAttribute.Level"
    }
    ,
    {
      "identifier": "MediaAssetFileAttribute.State"
    }
    ,
    {
      "identifier": "MediaAssetFileAttribute.Status"
    }
    ,
    {
      "identifier": "MediaAssetFileAttribute.Finished"
    }
    ,
    {
      "identifier": "MediaAssetFileAttribute.InProgress"
    }
    ,
    {
      "identifier": "MediaAssetFileAttribute.Class"
    }
    ,
    {
      "identifier": "MediaAssetFileAttribute.Categories"
    }
  ],
  "rows": [
    {
      "object": {
        "id": "'D11000100112038'"
      },
      "values": [
```

```
        "111", 0, 1, "s", false, true, 0,
["00990000000000
```

```
  "columns": [
    {
      "identifier": "MediaAssetFileAttributeLang.Name"
    }
  ],
  "rows": [
    {
      "object": {
        "id": "911000100112038"
      },
         "qualification": {
        "language": "English"
      },
      "values": [
        "updatedName en"
      ]
    },
    {
      "object": {
        "id": "'D11000100112013'"
      },
         "qualification": {
        "language": "de"
      },
      "values": [
        "updated Name (deu)"
      ]
    }
  ]
}
```

The following JSON object is returned:

```
{
    "counters": {
        "errors": 0,
        "warnings": 0,
        "createdObjects": 0,
        "updatedObjects": 2,
        "objectsWithErrors": 0,
        "objectsWithWarnings": 0
    },
    "entries": [],
    "objects": [
        {
            "row": 0,
            "object": {
                "id": "911000100112038",
                "label": "Media asset file 911000100112038"
            },
```

```
            "status": [
                "UPDATED"
            ]
        },
        {
            "row": 1,
            "object": {
                "id": "911000100112013",
                "label": "Media asset file 911000100112013"
            },
            "status": [
                "UPDATED"
            ]
        }
    ]
}
```


9.6.2.7 Delete media asset file with identifier D11000100112532

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type: application/x-www-form-urlencoded" -X DELETE http://localhost:1512/rest/V1.0/list/MediaAssetFile/byIdentifiers?identifiers=D11000100112532
```

or

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Content-Type: application/x-www-form-urlencoded" -X DELETE http://localhost:1512/rest/V1.0/list/MediaAssetFile/byItems?Items=911000100112532
```

9.6.2.8 Create media asset file

Please visit the page Upload single Media Asset File.

9.6.3 Media search query for MediaAssetFile entity

This search with the parameter "mediaQuery" for MediaAssetFile entity is available since the Version 8.0.00.00.

The report "bySearch" for the entity MediaAssetFile provides searching and finally result listing of the data entity MediaAssetFile. Its specific parameter "mediaQuery" is used to search the result directly with the search fields defined in Media Manager, while the specific parameter "query" is for the usual search with the Field entity defined in PIM repository, just like other general objects in system.

This chapter introduces the report "bySearch" with the parameter "mediaQuery".

Currently it is only supported by PIM Core with Media Manager and the data access is not per PIM database, but per the connector API.

- Syntax for mediaQuery (see page 449)
- Examples (see page 454)
  - Search all media asset files whose filename contain "test" (see page 454)

9.6.3.1 Syntax for mediaQuery

The value for the parameter **mediaQuery** contains three components: FIELD, OPERATOR, CONTENT.

Syntax

```
mediaQuery: <FIELD> <OPERATOR> <CONTENT>
```

The following table indicates all possible value for FIELD, OPERATOR and corresponding CONTENT.

FIELD	OPERATOR	CONTENT	Example search content	Corresponding PIM Desktop search fieldname (en)
*	contain	Character	My search value.	Quick search
F_IMGKOHI.IMHI_IHIE_ID	is	Category ID	03110000%	Category

FIELD	OPERATOR	CONTENT	Example search content	Corresponding PIM Desktop search fieldname (en)
F_IMGKOMP.PKOM_BEZ	is/contain/ begin/ empty	Character	My search value (If 'empty' operator is used no search value is needed)	Name
FULLTEXT	contains	Character	My search value	(Text search) Document
FILEName	is/contain/ begin/ empty	Character	My search value (If 'empty' operator is used no search value is needed)	File name
F_ATTRIBUT.ATTR_NAME	is/contain/ begin/ empty	Character	My custom defined state (If 'empty' operator is used no search value is needed)	State
F_IMGKOMP.PKOM_STATUS	is	-/s/a	s -> Locked a -> Archived - -> Free	Status
F_IMGKOMP.PKOM_TYPE	is	9 values	docus images videos sounds fonts internet profiles common drawings	Medias type

FIELD	OPERATOR	CONTENT	Example search content	Corresponding PIM Desktop search fieldname (en)
F_IMGKOMP.PKOM_FERTIG	is	True/False	false	Finished flag
F_IMGKOMP.PKOM_INARBEIT	is	True/False	true	In progress
F_IMGKOMP.PKOM_PSIZE	equal/less/greater	Number	10000000	Size in bytes
F_IMGKOMP.PKOM_RESOL	equal/less/greater	Number	96	Resolution
F_IMGKOMP.PKOM_COLS	equal/less/greater	Number	24	Color depth
F_IMGMEMO.PIMG_MEMO	contain/begin	Character	My search value (If 'empty' operator is used no search value is needed)	Memo
F_IMGKOMP.PIMG_FARBRAUM	is/contain/begin	Character	rgb	Color space
F_IMGKOMP.PIMG_VERS_NR	equal/less/greater	Number	2	Version number
F_IMGKOMP.PIMG_ANG_AM	is/before/after	Date	20.11.2008 (dd.MM.yyyy)	Media asset created
F_IMGKOMP.PIMG_LASTMOD_TIMESTAMP	is/before/after	Date	20.11.2008 (dd.MM.yyyy)	Media asset last modification date

FIELD	OPERATOR	CONTENT	Example search content	Corresponding PIM Desktop search fieldname (en)
F_IMGKOMP.PKOM_LASTDATE	is/before/after	Date	20.11.2008 (dd.MM.yyyy)	Last modification date of the source file of the media asset. Last file modification date.
F_IMGKOMP.PKOM_PNR	is/contain/begin	Character	D030001315183	Id
F_IMGKOMP.PIMG_PREVIEW_STATE	equal/less	4 values	Special case only this 4 combinations are allowed: No preview: OPERATOR 'less' with '5' No read only preview: OPERATOR 'less' with '9' Error creating preview: OPERATOR 'equal' with '0 or 6' Preview generation queued: OPERATOR 'equal' with CONTENT '3 or 4 or 7 or 8'	Preview version
NUMBER_OF_GROUP_ASSIGNMENT	equal/greater	Number	1	Number of group assignments
F_IMGITEM.IMI_ITEMxx (text type property field)	is/contain/begin	Character	My search value	Itemfield type text
F_IMGITEM.IMI_ITEMxx (list type property field)	is	List values	My selected list value	Itemfield type list

FIELD	OPERATOR	CONTENT	Example search content	Corresponding PIM Desktop search fieldname (en)
F_IMGITEM.IMI_ITEMxx (bool type property field)	is	True/ False	true	Itemfield type bool
F_IMGITEM.IMI_ITEMxx (date type property field)	equal/less/ greater	Date	09.08.2013	Itemfield type date
F_IMGITEM.IMI_ITEMxx (int type property field)	equal/less/ greater/ lessequal/ greaterequal	Number	4	Itemfield type int
F_IMGITEM.IMI_ITEMxx (real type property field)	equal/less/ greater/ lessequal/ greaterequal	Number	3.5	Itemfield type real
F_IMGITEM.IMI_ITEMxx (multiselect list type property field)	contain	Listvalues	My selected list value	Itemfield type multiselect list

9.6.3.2 Examples

Search all media asset files whose filename contain "test"

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?mediaQuery= FILEName contain test &fields=MediaAssetFile.Identifier,MediaAssetFileAttribute.Filename
```

The following JSON object is returned:


```
{
    "cacheId": "20150812_171149_0",
    "entityIdentifier": "MediaAssetFile",
    "totalSize": 79,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 79,
    "columnCount": 0,
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "911000107072",
                "label": "Media asset file 911000107072"
            },
            "values": [
                "D11000107072",
                "test2.png"
            ]
        },
       ...
    ]
}
```

9.6.4 Search query for MediaAssetFile entity

This search with the parameter "query" for MediaAssetFile entity is available since the 8.0.03 Hotfixes 1

The report "bySearch" for the entity MediaAssetFile provides searching and finally result listing of the data entity MediaAssetFile. Its specific parameter "mediaQuery" is used to search the result directly with the search fields defined in Media Manager, while the specific parameter "query" is for the usual search with the Field entity defined in PIM repository, just like other general objects in system.

This chapter introduces the report "bySearch" with the parameter "query".

Currently it is only supported by PIM Core with Media Manager and the data access is not per PIM database, but per the connector API.

- Syntax for query (see page 455)
- Examples (see page 460)
  - Retrieve all media asset files whose filename contain "test" (see page 460)
  - Retrieve all media asset files which belong to a category or its child categories (see page 461)
  - Retrieve all media asset files whose version is greater or equals 2 (see page 462)
  - Retrieve all media asset files who are changed on the 2015-11-12 or later (see page 463)
  - Retrieve all media asset files whose medias type equals "images" (see page 464)
  - Retrieve all media asset files whose english memo contains "test" (see page 465)
  - Retrieve all media asset files whose english property field(of MultiSelectList type) contains "First" (see page 466)
  - Retrieve all media asset files whose name start with "test" or whose english memo equals "test" (see page 467)

9.6.4.1 Syntax for query

The syntax for the search with the parameter "query" is same as the other general objects in PIM, please visit the following page for the corresponding information REST Search Query Language (see page 38).

While the data access of the search query is not per PIM database, but per the connector API, there are several limitations by search with the parameter "query":

- Search value is always case insensitive, it means that upper and lower case differences will be ignored, so that the result from "equals" is same as the one from "equalsIC", also for "contains" and "containsIC", "startsWith" and "startsWithIC".
- Search with mixed language in one query is not allowed. For example, a search with combination of the different language specifical attributes like following is not possible:

ILLEGAL REST CALL

illegal REST call with mixed language

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://
localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?
query=MediaAssetFileAttributeLang.Memo(English) startsWith "test" or
```

```
MediaAssetFileAttributeLang.Memo(German) startsWith
"test"&fields=MediaAssetFile.Identifier
```

The following table indicates search information for all searchable MediaAssetFile fields defined in repository

ENTITY FIELD	OPERATOR	CONTENT	Example search content	Notice
MediaAssetFile.Identifier	=, != / <>, equals(IC), contains(IC), startsWith(IC), in, is empty	Media asset file identifier (STRING)	D1100010561	
MediaAssetFileAttribute.Categories	=, != / <>, equals(IC)	• exact category id (**120-digit-STRING**) • id for category and all sub categories (STRING with suffix %)	• "0099000100010... • "0099%"	
MediaAssetFileAttribute.Filename	=, != / <>, equals(IC), contains(IC), startsWith(IC), in, is empty	File name(String)	test.jpg	
MediaAssetFileAttribute.Resolution	=, != / <>, <, <=, <, >=, equals	Resolution value (INTEGER)	100	

ENTITY FIELD	OPERATOR	CONTENT	Example search content	Notice
MediaAssetFileAttribute.ColorDepth	=, != / <>, <, <=, <, >=, equals	Color depth value (INTEGER)	24	
MediaAssetFileAttribute.ColorSpace	=, != / <>, equals(IC), contains(IC), startsWith(IC), in, is empty	Color space value (STRING)	RGB	
MediaAssetFileAttribute.ColorProfile	=, != / <>, equals(IC), contains(IC), startsWith(IC), in, is empty	Color profile value (STRING)	sRGB	
MediaAssetFileAttribute.Version	=, != / <>, <, <=, <, >=, equals	Version value (INTEGER)	2	
MediaAssetFileAttribute.CreationDate	=, != / <>, <, <=, <, >=, equals	Creation date value (DATETIME)	2011-10-21T10:00:00	Only the date of the search content is considered for search, time information will be ignored.
MediaAssetFileAttribute.LastModified	=, != / <>, <, <=, <, >=, equals	Last modified value (DATETIME)	2011-10-21T10:00:00	Only the date of the search content is considered for search, time information will be ignored.

ENTITY FIELD	OPERATOR	CONTENT	Example search content	Notice
MediaAssetFileAttribute.Type	=, != / <>, equals(IC), in	Only the following string values are allowed: • docus • images • videos • sounds • fonts • internet • profiles • common • drawings	images	The allowed search contents are the labels of the Media Manager Enum **MediaDataTypes** whose id and the detailed medias type name are stored as Class for this entity field in PIM. E.g. a search with the "query=MediaAssetFileAttribute.Type = images" may return media asset files with field value "2#####JPEG" and "2#####Graphics Interchange Format", in which "2" is the id of the **MediaDataTypes** enum entry, "JPEG" and "Graphics Interchange Format" are detailed medias type names.
MediaAssetFileAttribute.State	=, equals, is empty	entry of Enum.MediaAssetFileState	"Job is finished" 2	
MediaAssetFileAttribute.Status	=, equals	entry of Enum.MediaAssetFileStatus	Free	
MediaAssetFileAttribute.Finished	=, equals	True False	True	
MediaAssetFileAttribute.InProgress	=, equals	True False	False	

ENTITY FIELD	OPERATOR	CONTENT	Example search content	Notice
MediaAssetFileAttributeLang.Name	=, != / <>, equals(IC), contains(IC), startsWith(IC), in, is empty	Name value (STRING)	nameTest	
MediaAssetFileAttributeLang.Memo	=, != / <>, equals(IC), contains(IC), startsWith(IC), is empty	Name value (STRING)	memoTest	
Property Field of Boolean type	=, equals	True False	True	
Property Field of Text type	=, != / <>, equals(IC), contains(IC), startsWith(IC), in, is empty	Text value (STRING)	test	
Property Field of Integer type	=, != / <>, <, <=, <, >=, equals, in	Integer value (INTEGER)	2	
Property Field of Decimal type	=, != / <>, <, <=, <, >=, equals	Decimal value (DECIMAL)	11.0	
Property Field of Date type	=, != / <>, <, <=, <, >=, equals	Date value (DATE)	2016-05-03	

ENTITY FIELD	OPERATOR	CONTENT	Example search content	Notice
Property Field of SelectionList type	=, equals	entry of the corresponding enumeration (STRING)	one	
Property Field of MultiSelectionList type	=, equals, contains	entry of the corresponding enumeration (STRING)	First	
Property Field of Long Text type	=, != / <>, equals(IC), contains(IC), startsWith(IC), is empty	Long text value (STRING)	long test test	

9.6.4.2 Examples

Retrieve all media asset files whose filename contain "test"

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?query=MediaAssetFileAttribute.Filename contains test&fields=MediaAssetFile.Identifier,MediaAssetFileAttribute.Filename
```

The following JSON object is returned:


```
{
    "cacheId": "20160204_110314_0",
    "entityIdentifier": "MediaAssetFile",
    "totalSize": 79,
    "startIndex": 0,
    "pageSize": 100,
    "rowCount": 79,
    "columnCount": 0,
```

```
    "columns": [],
    "rows": [
        {
            "object": {
                "id": "911000107072",
                "label": "Media asset file 911000107072"
            },
            "values": [
                "D11000107072",
                "test2.png"
            ]
        },
       ...
    ]
}
```

Retrieve all media asset files which belong to a category or its child categories

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?query=MediaAssetFileAttribute.Categories = "0099%"&fields=MediaAssetFile.Identifier,MediaAssetFileAttribute.Filename, MediaAssetFileAttribute.Categories
```

The following JSON object is returned:

```
{
  "cacheId": "20160204_172920_0",
  "entityIdentifier": "MediaAssetFile",
  "totalSize": 191,
  "startIndex": 0,
  "pageSize": 100,
  "rowCount": 100,
  "columnCount": 0,
  "columns": [],
  "rows": [
    {
      "object": {
        "id": "911000100111676",
        "label": "Media asset file 911000100111676",
        "entityId": 2500
      },
      "values": [
        "D11000100111676",
        "test1.jpg",
        [

"00990002000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
```

```
00000000000000000000000000000000000#####testFolder#####/test2/
testFolder#####2#####false"
        ]
      ]
    },
    {
      "object": {
        "id": "9110000100112015",
        "label": "Media asset file 9110000100112015",
        "entityId": 2500
      },
      "values": [
        "D110000100112015",
        "test2.jpg",
        [

"00990001000100000000000000000000000000000000000
```

```
        "label": "Media asset file 911000107070",
        "entityId": 2500
      },
      "values": [
        "D11000107070",
        "2"
      ]
    },
    {
      "object": {
        "id": "9110001070102",
        "label": "Media asset file 9110001070102",
        "entityId": 2500
      },
      "values": [
        "D110001070102",
        "4"
      ]
    },
    ...
  ]
}
```


Retrieve all media asset files who are changed on the 2015-11-12 or later

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?query=MediaAssetFileAttribute.LastModified >= 2015-11-12T10:00:00&fields=MediaAssetFile.Identifier,MediaAssetFileAttribute.LastModified
```

The following JSON object is returned:


```
{
  "cacheId": "20160204_173754_0",
  "entityIdentifier": "MediaAssetFile",
  "totalSize": 181,
  "startIndex": 0,
  "pageSize": 100,
  "rowCount": 100,
  "columnCount": 0,
  "columns": [],
  "rows": [
    {
      "object": {
        "id": "911000100112281",
        "label": "Media asset file 911000100112281",
        "entityId": 2500
```

```
      },
      "values": [
        "D11000100112281",
        "2015-11-12T18:59:56:623+0100"
      ]
    },
    {
      "object": {
        "id": "911000100112290",
        "label": "Media asset file 911000100112290",
        "entityId": 2500
      },
      "values": [
        "D11000100112290",
        "2015-11-13T12:44:44:543+0100"
      ]
    },
    ...
  ]
}
```

Retrieve all media asset files whose medias type equals "images"

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?query=MediaAssetFileAttribute.Type = images&fields=MediaAssetFile.Identifier,MediaAssetFileAttribute.Filename, MediaAssetFileAttribute.Type
```

The following JSON object is returned:

```
{
  "cacheId": "20160204_174021_0",
  "entityIdentifier": "MediaAssetFile",
  "totalSize": 7535,
  "startIndex": 0,
  "pageSize": 100,
  "rowCount": 100,
  "columnCount": 0,
  "columns": [],
  "rows": [
    {
      "object": {
        "id": "91100010561",
        "label": "Media asset file 91100010561",
        "entityId": 2500
      },
      "values": [
```

```
        "D1100010561",
        "background.jpg",
        "2#####JPEG"
      ]
    },
    {
      "object": {
        "id": "91100010702",
        "label": "Media asset file 91100010702",
        "entityId": 2500
      },
      "values": [
        "D1100010702",
        "joerg_bundesjogi_2.png",
        "2#####Portable Network Graphics"
      ]
    },
    ...
  ]
}
```

Retrieve all media asset files whose english memo contains "test"

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?query=MediaAssetFileAttributeLang.Memo(English) contains "test"&fields=MediaAssetFile.Identifier,MediaAssetFileAttributeLang.Memo(English),MediaAssetFileAttributeLang.Memo(German)
```

The following JSON object is returned:

```
{
  "cacheId": "20160204_174304_0",
  "entityIdentifier": "MediaAssetFile",
  "totalSize": 5,
  "startIndex": 0,
  "pageSize": 100,
  "rowCount": 5,
  "columnCount": 0,
  "columns": [],
  "rows": [
    {
      "object": {
        "id": "911000107063",
        "label": "Media asset file 911000107063",
        "entityId": 2500
      },
```

```
      "values": [
        "D11000107063",
        "test",
        "no"
      ]
    },
    {
      "object": {
        "id": "91100010701194",
        "label": "Media asset file 91100010701194",
        "entityId": 2500
      },
      "values": [
        "D1100010701194",
        "memo_engtest",
        "memo_ger"
      ]
    },
    ...
  ]
}
```


Retrieve all media asset files whose english property field(of MultiSelectList type) contains "First"

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/list/MediaAssetFile/bySearch?query=MediaAssetFileAttributeLang.MetaDataMultiSelectionList(en) contains "First"&fields=MediaAssetFile.Identifier,MediaAssetFileAttributeLang.MetaDataMultiSelectionList(en),MediaAssetFileAttributeLang.MetaDataMultiSelectionList(de)
```

The following JSON object is returned:


```
{
  "cacheId": "20160204_174522_0",
  "entityIdentifier": "MediaAssetFile",
  "totalSize": 11,
  "startIndex": 0,
  "pageSize": 100,
  "rowCount": 11,
  "columnCount": 0,
  "columns": [],
  "rows": [
    {
      "object": {
        "id": "91100010702",
        "label": "Media asset file 91100010702",
        "entityId": 2500
```

```
      },
      "values": [
        "D1100010702",
        [
          "Second",
          "First"
        ],
        [
          "Fuenfte",
          "Dritte",
          "Zweite"
        ]
      ]
    },
    {
      "object": {
        "id": "911000107066",
        "label": "Media asset file 911000107066",
        "entityId": 2500
      },
      "values": [
        "D11000107066",
        [
          "First",
          "Third"
        ],
        []
      ]
    },
    ...
  ]
}
```

Retrieve all media asset files whose name start with "test" or whose english memo equals "test"

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/
V1.0/list/MediaAssetFile/bySearch?query=MediaAssetFileAttribute.Filename startsWith
"test" or (MediaAssetFileAttributeLang.Memo(en) =
test )&fields=MediaAssetFile.Identifier,
MediaAssetFileAttribute.Filename,MediaAssetFileAttributeLang.Memo(en)
```

The following JSON object is returned:

```
{
  "cacheId": "20160204_175436_0",
  "entityIdentifier": "MediaAssetFile",
  "totalSize": 47,
```

```
  "startIndex": 0,
  "pageSize": 100,
  "rowCount": 47,
  "columnCount": 0,
  "columns": [],
  "rows": [
    {
      "object": {
        "id": "911000107063",
        "label": "Media-Asset-Datei 911000107063",
        "entityId": 2500
      },
      "values": [
        "D11000107063",
        "garfield_and_odie.jpg",
        "test"
      ]
    },
    {
      "object": {
        "id": "911000107072",
        "label": "Media-Asset-Datei 911000107072",
        "entityId": 2500
      },
      "values": [
        "D11000107072",
        "test2.png",
        "memo"
      ]
    },
    ...
  ]
}
```

9.7 REST Media Asset Category API

The REST Media Asset Category API provides access to media asset categories in the PIM Core.

Currently It is only supported by PIM Core with Media Manager.

- Get Media Asset Category (see page 469)
- Get top Media Asset Categories (see page 469)
- Get direct child Media Asset Categories (see page 470)
- Add Media Asset Category (see page 470)
- Update Media Asset Category (see page 471)
- Delete Media Asset Category (see page 471)
- Examples (see page 472)
  - Retrieve single media asset category with identifier (see page 472)
  - Retrieve all top media asset categories (see page 472)
  - Retrieve all direct child media asset categories of the category with identifier (see page 473)
  - Add a top media asset category (see page 474)
  - Add a child media asset category under the category with identifier (see page 475)
  - Update the name of the media asset category with identifier (see page 475)
  - Delete a media asset category with identifier (see page 476)

9.7.1 Get Media Asset Category

URL Pattern	/media/categories/{identifier}
Method	GET
Parameters	-
Media types	application/xml, application/json
Returns	the media asset category as MediaAssetCategory object

9.7.2 Get top Media Asset Categories

URL Pattern	/media/categories/

Method	GET
Parameters	-
Media types	application/xml, application/json
Returns	all top media asset categories as a List of MediaAssetCategory objects

9.7.3 Get direct child Media Asset Categories

URL Pattern	/media/categories/{identifier}/categories
Method	GET
Parameters	-
Media types	application/xml, application/json
Returns	all direct child media asset categories as a List of MediaAssetCategory objects

9.7.4 Add Media Asset Category

URL Pattern	/media/categories
Method	POST
Parameters	parentId
Media types	application/xml, application/json

Returns	created media asset category as a MediaAssetCategory object

The query parameter "parentId" is optional: if it is not set, then an new top category will be created, otherweis a child category will be created under the category with parentId.

9.7.5 Update Media Asset Category

URL Pattern	/media/categories/{identifier}
Method	PUT
Parameters	-
Media types	application/xml, application/json
Returns	updated media asset category as a MediaAssetCategory object

9.7.6 Delete Media Asset Category

URL Pattern	/media/categories/{identifier}
Method	DELETE
Parameters	-
Media types	application/xml, application/json
Returns	string if media asset category is successfully deleted

9.7.7 Examples

9.7.7.1 Retrieve single media asset category with identifier

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -H "Accept-Language: de-DE" -X GET
http://localhost:1501/rest/V1.0/media/categories/
000100000000000
```

The following JSON objec is returned:


```
[
    {
        "id":
"000100000000000
```

Rest Client Java Code

```
MediaAssetCategoryRequest mediaAssetCategoryRequest =
restClient.createMediaRequest().createCategoryRequest();
String categoryId =
"000100000000000
```

```
categoryAttributes.setName( "newTopCategory" );
MediaAssetCategory topCategory =
mediaAssetCategoryRequest.addCategory( categoryAttributes );
```

9.7.7.5 Add a child media asset category under the category with identifier

Rest Call

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1501/rest/
V1.0/media/categories/?
parentId=0001000000000
```

```
{
    "id":
"000100000000000
```

10 REST Meta API

The REST Meta API (see page 477)REST Meta API (see page 477) provides meta-information on all available Product Manager objects. It provides access not only to the objects, but also to their field including names, data types, constraints, etc. Using this API you can build powerful generic clients which may work with different installations of the Product Manager with different Repository configurations.

- All Root Entities (see page 477)
  - Result (see page 478)
- Metadata for a Root Entity (see page 478)
  - Parameters (see page 479)
  - Result (see page 479)
- All Fields of a Root Entity (see page 480)
  - Result (see page 480)
- Metadata for a Sub Entity (see page 481)
  - Parameters (see page 482)
  - Result (see page 482)
- Metadata for a Field (see page 482)
  - Parameters (see page 483)
  - Result (see page 483)
- Metadata for a Qualifier (see page 485)
  - Result (see page 486)
- Examples (see page 487)
  - Retrieving all root entities (see page 487)
  - Retrieving meta data of the root entity Article (see page 487)
  - Retrieving all fields of the root entity Article (see page 489)
  - Retrieving meta data of the sub entity ArticlePriceValuePurchase (see page 489)
  - Retrieving metadata for the field ArticleLang.DescriptionLong (see page 490)
  - Retrieving meta data of the field ArticlePriceVa.DescriptionLong with qualification (see page 491)
  - Retrieving meta data of the qualifier Language for entity ArticleLang (see page 492)

10.1 All Root Entities

Returns all root entities defined in the repository.

URL Pattern	/meta
Method	GET
Parameters	No parameters are available
Media type	text/html, application/json, application/xml

Result	All root entities defined in the repository

10.1.1 Result

A list of all root entities supporting the REST API is returned. Each entry contains the following properties:

Properties for `entities` element			
	Field	**Data type**	**Description**
	identifier	String	Unique identifier of the entity
	name	String	Localized name of the entity
	description	String	Localized description of the entity

10.2 Metadata for a Root Entity

Returns the complete metadata for a root entity including all fields and all sub entities with their qualifiers, fields and sub entities.

URL Pattern	/meta/{root-entity-identifier}
Method	GET
Parameters	visibleOnly
Media type	text/html, application/json, application/xml
Result	The metadata for a root entity, like all direct fields, all direct sub entities etc.

10.2.1 Parameters

Available parameters

	Parameter	Required	Default	Datatype	Parameter description
	visibleOnly	no	false	boolean	Limits the result of the fields depending on the qualification permissions of the authentificated user. If false, all fields will be returned, even when the user doesn't have the read permission on this field.

10.2.2 Result

The following entity properties are returned:

Properties of the returned object

	Field	Data type	Description
	identifier	String	Unique identifier
	name	String	Localized name
	qualifiedName	String	Qualified and localized name (in case of a root entity always identical to *name*)
	description	String	Localized description
	qualifiers	Array	List of qualifiers, always empty in case of a root entity). The available properties for each field are described at qualifier properties (see page 486).

	fields	Array	List of direct fields. The available properties for each field are described at field properties (see page 483).
	entities	Array	List of direct sub entities. The available properties are identical to those of the root entity.

Remarks

- If the media type `text/html` is requested, only the direct fields and sub entities are returned to ensure the HTML page has a manageable size.

10.3 All Fields of a Root Entity

Returns all fields of a root entity including fields of sub entities. The fields are grouped by category.

URL Pattern	/meta/{root-entity-identifier}/fields
Method	GET
Parameters	No parameters are available
Media type	text/html, application/json, application/xml
Result	All fields (including fields of sub entities) of the specified entity grouped by category. The returned structure contains a list of categories whereby the corresponding fields are included for each category.

10.3.1 Result

The following properties for each field are returned:

Properties of the each `categories` element

	Field	Data type	Description
	identifier	String	Unique identifier of the category
	qualifiedName	String	Localized name of category.
	fields	Array	Localized description of the field

Properties of the each `fields` element

	Field	Data type	Description
	resource	URL	Link to the meta data of the field
	identifier	String	Unique identifier of the field
	qualifiedName	String	Qualified and localized name of the field. The default qualifiers are used.
	description	String	Localized description of the field

10.4 Metadata for a Sub Entity

Returns the metadata for a sub entity. The returned metadata includes a list of qualifiers, a list of direct fields, and a list of direct sub entities.

URL Pattern	/meta/{root-entity-identifier}/{sub-entity-identifier}
Method	GET
Parameters	visibleOnly

Media type	text/html, application/json, application/xml
Result	The metadata for a sub entity. The returned metadata includes information like all direct fields, all sub entities etc. The sub entity does not need to be a direct sub entity of the root entity.

10.4.1 Parameters

Available parameters

	Parameter	Required	Default	Datatype	Parameter description
	visibleOnly	no	false	boolean	Limits the result of the fields depending on the qualification permissions of the authentificated user. If false, all fields will be returned, even when the user doesn't have the read permission on this field.

10.4.2 Result

The returned object has the same properties as the result object for a root entity. See Properties of a root entity (see page 479).

10.5 Metadata for a Field

Returns the available metadata for a field.

URL Pattern	/meta/{root-entity-identifier}/{field-identifier}
Method	GET
Parameters	qualification visibleOnly

Media type	text/html, application/json, application/xml
Result	The metadata for a field. The returned metadata includes information like the data type, the enumeration (if available), the cardinality, the field qualifiers etc. If the parameter *qualification* is set, the qualified name is generated using the specified qualification.

10.5.1 Parameters

Available parameters

	Parameter	Required	Default	Datatype	Parameter description
	qualification	no		String	Sets the qualification which is used to generate the content of the property `qualifiedName`. The format is described in REST Field Qualification (see page 31) whereby only the qualification part has to be specified (including the parentheses). Qualification example for the field *ArticlePriceValueSales.Amount*: `(1,nrp,USD,US,2010-08-06,1.5)`
	visibleOnly	no		boolean	Limits the result of the fields depending on the qualification permissions of the authentificated user. If false, all fields will be returned, even when the user doesn't have the read permission on this field.

10.5.2 Result

The following field properties are returned:

Properties

	Field	Data type	Description

	identifier	String	Unique identifier of this field
	name	String	Localized name of the field
	qualifiedName	String	Qualified and localized name. The default qualifiers are used if not specified explicitly in the `qualification` parameter.
	category	String	Category this field belongs to
	description	String	Localized description of this field
	dataType	String	Data type of this field. The possible data types are limited and documented here
	enumeration	Object	Proxy object to the enumeration containing all possible values for this field. The proxy contains the properties `name` and `identifier`.
	proposalEnum	Object	Proxy object to the enumeration containing a proposal list of possible values (other values are allowed too). The proxy contains the properties `name` and `identifier`.
	lowerBound	Integer	Defines the cardinality (lower bound). 0 means the field is optional, 1 means it is mandatory.
	upperBound	Integer	Defines the cardinality (upper bound). 1 means the field is a single value field, -1 means, this field is a list of values.
	minLength	Integer	Defines the minimum length. Only meaningful if the data type is String.

	maxLength	Integer	Defines the maximun length. Only meaningful if the data type is String.
	richtext	Boolean	Defines if this field can hold formatting markers. Only meaningful if the data type is String.
	scale	Integer	Defines the number of digits after the decimal separator. Only meaningful if the data type is Decimal.
	pictureClause	String	Defines how the value should be formatted. If specified, the default picture clauses which come from the corresponding datatype is overwritten. See Repository documentation for details.
	value	String	Default value or fixed value in case this field is not editable
	visible	Boolean	Defines if this field is visible for the authentificated user or not
	editable	Boolean	Defines if this field can be edited by the authentificated user or not
	multiline	Boolean	Defines if carriage return line feeds are allowed in the string, also adjusts the visualization of the field. Only meaningful if the data type is String.
	qualifiers	Array of objects	All qualifiers of this field. The object contains the properties name, description, data type and enumeration. This property is not provided when this field description is embedded in an entity description.

10.6 Metadata for a Qualifier

Returns the available metadata for a qualifier.

URL Pattern	/meta/{root-entity-identifier}/{sub-entity-identifier}/{qualifier-identifier}

Method	GET
Parameters	No parameters are available
Media type	text/html, application/json, application/xml
Result	The metadata for a qualifier. The returned metadata includes information like the data type, the enumeration (if available), etc

10.6.1 Result

The following qualifier properties are returned:

Properties			
	Field	**Data type**	**Description**
	name	String	Localized name.
	qualifiedName	String	Localized and qualified name.
	qualificationFilterIdentifier	String	Identifier used in the property qualification when executing a REST List API Write for Sub Entities (see page 120).
	description	String	Localized description of this field.
	dataType	String	Data type of this qualifier. The possible data types are limited and documented here
	enumeration	Object	Proxy to the enumeration containing all allowed values for this qualifier.

	proposalEnum	Object	Proxy to the enumeration containing a proposal list of possible values (other values are allowed too)

10.7 Examples

10.7.1 Retrieving all root entities

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/meta
```

The following JSON object is returned:

```
{
    "entities": [
        {
            "identifier": "Article",
            "name": "Item",
            "description": ""
        },
    ...
    ]
}
```

10.7.2 Retrieving meta data of the root entity *Article*

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/meta/Article
```

The following JSON object is returned:

```
 {
    "identifier": "Article",
    "name": "Item",
    "qualifiedName": "Item",
    "description": "",
    "qualifiers": [],
    "fields": [
        {
            "identifier": "Article.Id",
            "name": "Internal item number",
```

```
            "qualifiedName": "Internal item number",
            "category": "",
            "description": "",
            "dataType": "INTEGER",
            "lowerBound": "0",
            "upperBound": "1",
            "minLength": "0",
            "maxLength": "0",
            "richtext": "false",
            "rangeMin": "",
            "rangeMax": "",
            "scale": "0",
            "pictureClause": "",
            "value": "",
            "editable": "false",
            "multiline": "false",
            "searchable": "false"
        },
        ...
     ],
    "entities": [
        {
            "identifier": "ArticleLogistic",
            "name": "Logistical data",
            "qualifiedName": "Logistical data",
            "description": "",
            "qualifiers": [
                {
                    "name": "Country",
                    "qualifiedName": "Country",
                    "qualificationFilterIdentifier": "territory",
                    "description": "",
                    "dataType": "STRING",
                    "enumeration": {
                        "name": "Countries",
                        "identifier": "Enum.Territory"
                    },
                    "value": "DE"
                }
            ],
            "fields": [ ... ]
       }
   ]
}
```

10.7.3 Retrieving all fields of the root entity *Article*

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/meta/Article/fields
```

The following JSON object is returned:

```
{
    "categories": [
        {
            "identifier": "master-data",
            "qualifiedName": "Header data",
            "fields": [
                {
                    "identifier": "Article.AclProxy",
                    "qualifiedName": "Object rights",
                    "description": "Defines the rights for an object"
                },
                {
                    "identifier": "Article.AclFlag",
                    "qualifiedName": "Object right type",
                    "description": "Type of object right"
                },
             ....
       }
    ]
}
```

10.7.4 Retrieving meta data of the sub entity *ArticlePriceValuePurchase*

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/meta/Article/ArticlePriceValuePurchase
```

The following JSON object is returned:

```
{
    "identifier": "ArticlePriceValuePurchase",
    "name": "Price tier",
    "qualifiedName": "Price tier",
    "description": "",
    "qualifiers": [
        {
            "name": "Supplier",
```

```
            "description": "Name of the supplier",
            "dataType": "ENTITY_ITEM",
            "enumeration": {
                "name": "Suppliers (inc. main supplier)",
                "identifier": "Enum.SupplierWithMainSupplier"
            }
        },
        ...
    ],
    "fields": [
        {
            "identifier": "ArticlePriceValuePurchase.Amount",
            "name": "Price (from 1.0000)",
            "description": "Purchase price level"
        },
        ...
    ],
    "entities": []
}
```

10.7.5 Retrieving metadata for the field ArticleLang.DescriptionLong

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/meta/Article/ArticleLang.DescriptionLong
```

The following JSON object is returned:

```
{
    "identifier": "ArticleLang.DescriptionLong",
    "name": "Long description",
    "qualifiedName": "Long description (German)",
    "category": "Texts",
    "description": "Detailed description of item",
    "dataType": "STRING",
    "lowerBound": "0",
    "upperBound": "1",
    "minLength": "0",
    "maxLength": "2147483647",
    "richtext": "true",
    "rangeMin": "",
    "rangeMax": "",
    "scale": "0",
    "pictureClause": "",
    "value": "",
    "editable": "true",
    "multiline": "true",
    "searchable": "true",
    "qualifiers": [
        {
```

```
            "name": "Language",
            "description": "Unique identifier of the language in which the
item has been recorded",
            "dataType": "INTEGER",
            "enumeration": {
                "name": "All languages",
                "identifier": "Enum.Language"
            }
        }
    ]
}
```

10.7.6 Retrieving meta data of the field ArticlePriceVa.DescriptionLong with qualification

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/
V1.0/meta/Article/ArticlePriceValueSales.Amount?
qualification=(1,nrp,USD,US,2012-08-21,1.0)
```

The following JSON object is returned:

```
{
    "identifier": "ArticlePriceValueSales.Amount",
    "name": "Price",
    "qualifiedName": "Non-binding price recommendation (from 1)",
    "category": "Prices",
    "description": "Selling price level",
    "dataType": "DECIMAL",
    "lowerBound": "0",
    "upperBound": "1",
    "minLength": "0",
    "maxLength": "0",
    "richtext": "false",
    "rangeMin": "0",
    "rangeMax": "9999999999.999999",
    "scale": "2",
    "pictureClause": "",
    "value": "",
    "editable": "true",
    "multiline": "false",
    "searchable": "true",
    "qualifiers": [
        {
            "name": "Customer",
            "description": "Unique number of purchasing company",
            "dataType": "ENTITY_ITEM",
            "enumeration": {
                "name": "Customers",
```

```
                "identifier": "Enum.Buyer"
            }
        },
        ...
    ]
}
```

10.7.7 Retrieving meta data of the qualifier *Language* for entity *ArticleLang*

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/meta/Article/ArticleLang/ArticleLangType.LK.Language
```

The following JSON object is returned:

```
{
    "name": "Language",
    "qualifiedName": "Language",
    "qualificationFilterIdentifier": "language",
    "description": "Unique identifier of the language in which the item has been recorded",
    "dataType": "INTEGER",
    "enumeration": {
        "name": "All languages",
        "identifier": "Enum.Language"
    },
    "value": "7"
}
```

10.8 REST Meta Media API

e

The REST Meta Media API provides access to some meta functionalities for media area.

It is only supported when the Informatica Media Manager is used.

- Derivative definition (see page 493)
  - Get single derivative definition (see page 494)
    - Content (see page 494)
  - Get all derivative definitions (see page 495)
- Media Asset File Property field definition (see page 495)
  - Get single property field definition for one supported language (see page 495)
    - Content (see page 496)
  - Get single property field definition for all supported languages (see page 499)
  - Get all property field definitions for all supported languages (see page 499)
- Examples (see page 500)
  - Retrieve single derivative definition with identifier (see page 500)
    - REST call for JSON (see page 500)
    - REST call for XML (see page 500)
    - REST Client Java (see page 501)
  - Retrieve all derivative definitions (see page 501)
    - REST call for JSON (see page 501)
    - REST call for XML (see page 502)
    - REST Client Java (see page 503)
  - Retrieve single property field definition for one supported language (see page 503)
    - REST call for JSON (see page 503)
    - REST call for XML (see page 504)
    - REST Client Java (see page 504)
  - Retrieve single property field definition for all supported languages (see page 505)
    - REST call for JSON (see page 505)
    - REST call for XML (see page 505)
    - REST Client Java (see page 506)
  - Retrieve property field definitions for all supported languages (see page 507)
    - REST call for JSON (see page 507)
    - REST call for XML (see page 508)
    - REST Client Java (see page 508)

10.8.1 Derivative definition

The following API methods provides the read/list access to the derivative definition which are defined in the Informatica Media Manager. The call result can be used to fetch the corresponding derivative of the desired media asset file, for more details please visit the page REST Media API (see page 433).

Derivative definition is a predefined schema with which the master asset (original image) can be converted to a derivative for certain rules (e.g. extension, resolution).

10.8.1.1 Get single derivative definition

URL Pattern	/meta/media/derivativeDefinitions/{id}
Method	GET
Path Parameters	id: The id of the desired derivaitve definition, must be an integer value
Media types	application/xml, application/json
Result	Derivative definition as a MediaAssetDerivativeDefinition object

Content

The content has to be a JSON/XML object which includes the properties listed below.

MediaAssetDerivativeDefinition

Field	Required	Datatype	Parameter description	Example
id	yes	INTEGER	Unique id of the derivatrive definition	1
shortDescription	no	STRING	Short description of the derivatrive definition	JPEG 36dpi
longDescription	no	STRING	Long description of the derivatrive definition	Derivative with JPEG extension and 36 dpi resolution
volumeId	no	INTEGER	Id of the volumn which is defined in the Media Manager to indicate the file storage where the corresponding derivatives are stored	0

10.8.1.2 Get all derivative definitions

URL Pattern	/meta/media/derivativeDefinitions
Method	GET
Parameters	-
Media types	application/xml, application/json
Result	all derivative definitions as a List of MediaAssetDerivativeDefinition objects

10.8.2 Media Asset File Property field definition

The following API methods provides the read/list access to the media asset file property field definition which are defined in the Informatica Media Manager.

> Property field definition is a schema which defines the corresponding settings for a media asset file property field, e.g. number, type, default value.

10.8.2.1 Get single property field definition for one supported language

URL Pattern	/meta/media/fields/{field-identifier}/{locale}
Method	GET
Path Parameters	field-identifier: field identifier for the property field definition which seems like 'F_IMGITEM.IMI_ITEM3' locale: The locale for which the desired property field definition is defined in the Media Manager
Media types	text/html, application/xml, application/json

Result	The metadata of a field for property field definition. The returned metadata includes information like the identifier, locale, data type etc.

Content

The content has to be a HTML/JSON/XML object which includes the properties listed below.

Field properties for property field definition

Field	Required	Data type	Parameter description	Example
identifier	yes	STRING	id of the corresponding property field definition	F_IMGITEM.IMI_ITEM3
locale	yes	STRING	locale for which the corresponding property field definition is defined	en_US
name	no	STRING	name of the property file definition	License Free
category	no	STRING	fix value as "General information(Media asset file attributes)"	General information(Media asset file attributes)
dataType	no	Data Type	data type of the property field definition which is transfered from the original type in the Media Manager	STRING
lowerBound	no	INTEGER	Possible values are 0 or 1, where 0 means the property field is completely optional, and 1 indicates it defines a mandatory property field	0
upperBound	no	INTEGER	The value can be 1 or -1, where -1 means an unlimited amount of objects, and 1 exactly one child object. Currently only the "Multiple selection list" type property file definitoin has a -1 upperBound	1

Field	Required	Data type	Parameter description	Example
maxLength	no	INTEGER	currently only available for the common "Text" type property field definition	3000
rangeMin	no	STRING	only available for "Integer" and "Dezimal number" type property field definition	-1.0
rangeMax	no	STRING	only available for "Integer" and "Dezimal number" type property field definition	9.9
scale	no	INTEGER	The precision of decimal data types. Defines the number of digits after the comma.	2
value	no	STRING	default value	"test"
searchable	no	BOOLEAN	True if the search functionalities can be used to search for this property field content, false if not	TRUE
readLevel	no	INTEGER	read access level which defined in the Media Manager	0
readWriteLevel	no	INTEGER	read/write access level which defined in the Media Manager	0
allowedValues	no	List of STRING	currently only available for "Selection list" and "Multiple selection list"	["One", "Two", "Three"]

Field	Required	Data type	Parameter description	Example
regularExpression	no	STRING	only available for the common "Text" type property field definition(not for "Long text")	*

Following table indicates the mapping from the original property filed defintion type to the corresponding Product 360 DataType inclusive addtional notes

Property field definiton type in Media Manager	DataType in Product 360	additional notes
Boolean	BOOLEAN	
Date	DATE	
Decimal number	DECIMAL	rangeMin: it is calculated from the "minimum value", "places" and "decimal places" which are defined in the Media Manager rangeMax: it is calculated from the "maximum value", "places" and "decimal places" which are defined in the Media Manager
Integer	INTEGER	rangeMin: the "minimum value" defined in the Media Manager rangeMax: the "maximum value" defined in the Media Manager
Multiple selection list	STRING	upperBound: -1 allowedValues is defined
Selection list	STRING	allowedValues is defined

Property field definiton type in Media Manager	DataType in Product 360	additional notes
Text	STRING	number: [1, 100]
Time	TIME	
Long Text	STRING	nummer: [101, 110]

10.8.2.2 Get single property field definition for all supported languages

URL Pattern	/meta/media/fields/{field-identifier}
Method	GET
Parameters	field-identifier: field identifier for the property field definition which seems like 'F_IMGITEM.IMI_ITEM3'
Media types	text/html, application/xml, application/json
Result	All property field definitions as a List of fields grouped by one category. The returned structure contains one category with all fields

10.8.2.3 Get all property field definitions for all supported languages

URL Pattern	/meta/media/fields
Method	GET
Parameters	-

Media types	text/html, application/xml, application/json
Result	All property field definitions as a List of fields grouped by one category. The returned structure contains one category with all fields

10.8.3 Examples

10.8.3.1 Retrieve single derivative definition with identifier

REST call for JSON

Rest Call for JSON

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/meta/media/derivativeDefinitions/1
```

A MediaAssetDerivtiveDefinition JSON object including id and other attributes of the desired derivative definition is returned:

Returned JSON object

```
{
  "id": 1,
  "shortDescription": "JPEG 36dpi",
  "longDescription": "Derivative with JPEG extension and 36 dpi resolution",
  "volumeId": 0
}
```

REST call for XML

Rest Call for XML

```
curl -u rest:heiler -H "Accept: application/xml" -X GET http://localhost:1512/rest/V1.0/meta/media/derivativeDefinitions/1
```

A MediaAssetDerivativeDefinition XML object including id and other attributes of the desired derivative definition is returned:

Returned XML object

```
<mediaAssetDerivativeDefinition>
    <id>1</id>
    <shortDescription>JPEG 36dpi</shortDescription>
    <longDescription>Derivative with JPEG extension and 36 dpi resolution</
longDescription>
    <volumeId>0</volumeId>
</mediaAssetDerivativeDefinition>
```

REST Client Java

Rest Client Java Code

```
MetaMediaRequest metaMediaRequest =
restClient.createMetaRequest().createMediaRequest();
MediaAssetDerivativeDefinition derivativeDefinition =
metaMediaRequest.getDerivativeDefinition( 1 );
```

10.8.3.2 Retrieve all derivative definitions

REST call for JSON

Rest Call for JSON

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/
V1.0/meta/media/derivativeDefinitions
```

The following JSON objec is returned:

Returned JSON object

```
[
  {
    "id": 1,
    "shortDescription": "JPEG 36dpi",
    "longDescription": "Derivative with JPEG extension and 36 dpi resolution",
    "volumeId": 0
  },
  {
    "id": 2,
    "shortDescription": "BMP 90dpi",
```

```
    "longDescription": "Derivative with BMP extension and 90 dpi resolution",
    "volumeId": 0
  },
  {
    "id": 3,
    "shortDescription": "PNG 48dpi",
    "longDescription": "Derivative with PNG extension and 48 dpi resolution",
    "volumeId": 0
  },
  ...
 ]
```

REST call for XML

Rest Call for XML

```
curl -u rest:heiler -H "Accept: application/xml" -X GET http://localhost:1512/rest/
V1.0/meta/media/derivativeDefinitions
```

The following XML objec is returned:

Returned XML object

```
<mediaAssetDerivativeDefinitions>
    <mediaAssetDerivativeDefinition>
        <id>1</id>
        <shortDescription>JPEG 36dpi</shortDescription>
        <longDescription>Derivative with JPEG extension and 36 dpi resolution</
longDescription>
        <volumeId>0</volumeId>
    </mediaAssetDerivativeDefinition>
    <mediaAssetDerivativeDefinition>
        <id>2</id>
        <shortDescription>BMP 90dpi</shortDescription>
        <longDescription>Derivative with BMP extension and 90 dpi resolution</
longDescription>
        <volumeId>0</volumeId>
    </mediaAssetDerivativeDefinition>
    <mediaAssetDerivativeDefinition>
        <id>3</id>
        <shortDescription>PNG 48dpi</shortDescription>
        <longDescription>Derivative with PNG extension and 48 dpi resolution</
longDescription>
        <volumeId>0</volumeId>
    </mediaAssetDerivativeDefinition>
    ...
</mediaAssetDerivativeDefinitions>
```

REST Client Java

Rest Client Java Code

```
MetaMediaRequest metaMediaRequest =
restClient.createMetaRequest().createMediaRequest();
List< MediaAssetDerivativeDefinition > derivativeDefinitions =
metaMediaRequest.getDerivativeDefinitions();
```

10.8.3.3 Retrieve single property field definition for one supported language

REST call for JSON

Rest Call for JSON

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/
V1.0/meta/media/fields/F_IMGITEM.IMI_ITEM3/en_US
```

A MediaAssetFilePropertyFieldDefinition JSON object including language id and a list of PropertyFieldSingleDefinition is returned:

Returned JSON object

```
{
  "identifier": "F_IMGITEM.IMI_ITEM3",
  "locale": "en_US",
  "name": "Multiple selelction list english",
  "category": "General information(Media asset file attributes)",
  "dataType": "STRING",
  "lowerBound": "0",
  "upperBound": "-1",
  "maxLength": "0",
  "scale": "0",
  "value": "",
  "searchable": "false",
  "readLevel": "0",
  "readWriteLevel": "0",
  "allowedValues": "[First, Second, Third]"
}
```

REST call for XML

Rest Call for XML

```
curl -u rest:heiler -H "Accept: application/xml" -X GET http://localhost:1512/rest/V1.0/meta/media/fields/F_IMGITEM.IMI_ITEM3/en_US
```

A MediaAssetFilePropertyFieldDefinition JSON object including language id and a list of PropertyFieldSingleDefinition is returned:

Returned XML object

```
<?xml version="1.0" encoding="UTF-8"?>
<field>
    <identifier>F_IMGITEM.IMI_ITEM3</identifier>
    <locale>en_US</locale>
    <name>Multiple selelction list english</name>
    <category>General information(Media asset file attributes)</category>
    <dataType>STRING</dataType>
    <lowerBound>0</lowerBound>
    <upperBound>-1</upperBound>
    <maxLength>0</maxLength>
    <rangeMin></rangeMin>
    <rangeMax></rangeMax>
    <scale>0</scale>
    <value></value>
    <searchable>false</searchable>
    <readLevel>0</readLevel>
    <readWriteLevel>0</readWriteLevel>
    <allowedValues>[First, Second, Third]</allowedValues>
    <regularExpression></regularExpression>
</field>
```

REST Client Java

Rest Client Java Code

```
MetaMediaRequest metaMediaRequest =
restClient.createMetaRequest().createMediaRequest();
PropertyFieldDefinitionField propertyFieldDefinition =
metaMediaRequest.getPropertyFieldDefinition( "F_IMGITEM.IMI_ITEM3", "en_US" );
```

10.8.3.4 Retrieve single property field definition for all supported languages

REST call for JSON

Rest Call for JSON

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/meta/media/fields/F_IMGITEM.IMI_ITEM3
```

A List of MediaAssetFilePropertyFieldDefinition JSON object is returned:

Returned JSON object

```
{
  "categories": [
    {
      "identifier": "media-asset-file-attributes-general",
      "qualifiedName": "General information(Media asset file attributes)",
      "fields": [
        {
          "identifier": "F_IMGITEM.IMI_ITEM3",
          "qualifiedName": "Multiple selelction list english",
          "description": "Property field definition 'F_IMGITEM.IMI_ITEM3' for the locale 'en_US'"
        },
        {
          "identifier": "F_IMGITEM.IMI_ITEM3",
          "qualifiedName": "MultiSelectList Deutsch",
          "description": "Property field definition 'F_IMGITEM.IMI_ITEM3' for the locale 'de_DE'"
        }
      ]
    }
  ]
}
```

REST call for XML

Rest Call for XML

```
curl -u rest:heiler -H "Accept: application/xml" -X GET http://localhost:1512/rest/V1.0/meta/media/fields/F_IMGITEM.IMI_ITEM3
```

The following XML objec is returned:

Returned XML object

```
<?xml version="1.0" encoding="UTF-8"?>
<fieldsByCategory>
    <categories>
        <category>
            <identifier>media-asset-file-attributes-general</identifier>
            <qualifiedName>General information(Media asset file attributes)</
qualifiedName>
            <fields>
                <field>
                    <identifier>F_IMGITEM.IMI_ITEM3</identifier>
                    <qualifiedName>Multiple selelction list test</qualifiedName>
                    <description>Property field definition 'F_IMGITEM.IMI_ITEM3' for
the locale 'en_US'</description>
                </field>
                <field>
                    <identifier>F_IMGITEM.IMI_ITEM3</identifier>
                    <qualifiedName>MultiSelectList DE</qualifiedName>
                    <description>Property field definition 'F_IMGITEM.IMI_ITEM3' for
the locale 'de_DE'</description>
                </field>
            </fields>
        </category>
    </categories>
```

REST Client Java

Rest Client Java Code

```
MetaMediaRequest metaMediaRequest =
restClient.createMetaRequest().createMediaRequest();
RepositoryCategories categories = metaMediaRequest.getPropertyFieldDefinitions(
"F_IMGITEM.IMI_ITEM3" );
RepositoryCategory generalAttributeCategory = categories.getCategories()
                                                        .get( 0 );
List< RepositoryField > propertyFieldDefinitions =
generalAttributeCategory.getFields();
```

10.8.3.5 Retrieve property field definitions for all supported languages

REST call for JSON

Rest Call for JSON

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1512/rest/V1.0/meta/media/fields
```

A List of MediaAssetFilePropertyFieldDefinition JSON object is returned:

Returned JSON object

```
{
  "categories": [
    {
      "identifier": "media-asset-file-attributes-general",
      "qualifiedName": "General information(Media asset file attributes)",
      "fields": [
        {
          "identifier": "F_IMGITEM.IMI_ITEM1",
          "qualifiedName": "Licensefree",
          "description": "Property field definition 'F_IMGITEM.IMI_ITEM1' for the locale 'en_US'"
        },
        ...
        {
          "identifier": "F_IMGITEM.IMI_ITEM1",
          "qualifiedName": "Lizenzfrei",
          "description": "Property field definition 'F_IMGITEM.IMI_ITEM1' for the locale 'de_DE'"
        },
       ...
      ]
    }
  ]
}
```

REST call for XML

Rest Call for XML

```
curl -u rest:heiler -H "Accept: application/xml" -X GET http://localhost:1512/rest/V1.0/meta/media/fields
```

The following XML objec is returned:

Returned XML object

```
<?xml version="1.0" encoding="UTF-8"?>
<fieldsByCategory>
    <categories>
        <category>
            <identifier>media-asset-file-attributes-general</identifier>
            <qualifiedName>General information(Media asset file attributes)</qualifiedName>
            <fields>
                <field>
                    <identifier>F_IMGITEM.IMI_ITEM1</identifier>
                    <qualifiedName>Licensefree</qualifiedName>
                    <description>Property field definition 'F_IMGITEM.IMI_ITEM1' for the locale 'en_US'</description>
                </field>
                ...
                <field>
                    <identifier>F_IMGITEM.IMI_ITEM1</identifier>
                    <qualifiedName>Lizenzfrei</qualifiedName>
                    <description>Property field definition 'F_IMGITEM.IMI_ITEM1' for the locale 'de_DE'</description>
                </field>
                ...
            </fields>
        </category>
    </categories>
```

REST Client Java

Rest Client Java Code

```
MetaMediaRequest metaMediaRequest =
restClient.createMetaRequest().createMediaRequest();
RepositoryCategories categories = metaMediaRequest.getAllPropertyFieldDefinitions();
RepositoryCategory generalAttributeCategory = categories.getCategories().get( 0 );
```

```
List< RepositoryField > allPropertyFieldDefinitions  =
generalAttributeCategory.getFields();
```

11 REST Security API

The REST Security API provides access to securities of the Product Manager.

- List all Security ACL APIs (see page 510)
  - Result (see page 510)
- Read ACL object (see page 511)
  - Content (see page 511)
- Create ACL object (see page 513)
- Examples (see page 515)
  - Read ACL object for an id (see page 515)
    - REST call for JSON (see page 515)
    - REST call for XML (see page 517)
    - REST Client Java (see page 518)
  - Create ACL object (see page 518)
    - REST call for JSON (see page 518)
    - REST call for XML (see page 519)
    - REST Client Java (see page 520)

11.1 List all Security ACL APIs

Returns the list of all available REST APIs for acl security.

URL Pattern	/security/acl/info
Method	GET
Parameters	-
Media type	text/html, application/json, application/xml
Result	A list of all available REST APIs for acl security

11.1.1 Result

A list of all available REST APIs for acl security.

Description	URL Pattern	Method	Parameters	Result	Content
Read ACL Object	/security/acl/{aclId}	GET	-	The ACL object in JSON or XML format	-
Create ACL Object	/security/acl	POST	-	The ACL object, that was created or already existed, in JSON or XML format.	ACL object in JSON or XML format

11.2 Read ACL object

Returns the ACL object of the specified acl id.

URL Pattern	/security/acl/{acl-id}
Method	GET
Parameters	-
Media type	text/html, application/json, application/xml
Result	The ACL object

11.2.1 Content

The content as returned ACL object has to be a JSON/XML object which includes the properties listed below.

ACL properties			
	Field	**Data type**	**Description**

	id	Long	id of the ACL
	entries	List of AclEntry	all AclEntries of the ACL

Each AclEntry has following properties:

Properties of a member of AclEntry			
	Field	**Data type**	**Description**
	principal	PrincipalItemReference	An EntityItemReference to principal (user, user group or party)
	fullPermission	boolean	A boolean value indicates whether the principal has the Full permission object right, default value is false
	deletePermission	boolean	A boolean value indicates whether the principal has the Delete permission object right, default value is false
	writePermission	boolean	A boolean value indicates whether the principal has the Write permission object right, default value is false
	readPermission	boolean	A boolean value indicates whether the principal has the Read permission object right, default value is false

Principal properties			
	Field	**Data type**	**Description**
	id	String	External identifier of the principal (user, user group or party)
	entityId	Short	Entity id of the principal (user, user group or party)

	deleted	boolean	A flag indicating whether the principal is deleted on the system, default value is false

11.3 Create ACL object

Creates a new ACL object if it does not exist for the desired AclEntries, otherwise returns the existing ACL object.

URL Pattern	/security/acl
Method	POST
Parameters	-
Media type	text/html, application/json, application/xml
Result	The created or existing ACL object

- Handling for deleted principal: following table indicates the different responses according to the provided content (which contains the deleted flag for principal) and the fact whether the corresponding principal is actually deleted or not.

Flag "deleted" of principal in content	Principal is deleted on the system	Result
false/true	false (principal existing)	AclEntry with that principal is considered for the created Acl on the system. The returned ACL contains the principal with "false" value for the "deleted" flag.
false	true (principal deleted)	An Exception is returned that the specified principal EntityItemReference does not exist.

Flag "deleted" of principal in content	Principal is deleted on the system	Result
true	true (principal deleted)	AclEntry with that principal is **NOT** considered for the created Acl on the system. The created and returned ACL does not contain the principal. If no existing principal is available an exception (HTTP 400) is thrown.

- The given permission combination of each AclEntry might be adjusted according to the ACL object right rule: all implicit permissions of the given ones will be also granted.

fullPermission	deletePermission	writePermission	readPermission	adjusted permissions of created object
true	true/false	true/false	true/false	`"fullPermission": true, "deletePermission": true, "writePermission": true, "readPermission": true`
false	true	true/false	true/false	`"fullPermission": false, "deletePermission": true, "writePermission": true, "readPermission": true`
false	false	true	true/false	`"fullPermission": false, "deletePermission": false, "writePermission": true, "readPermission": true`

fullPermission	deletePermission	writePermission	readPermission	adjusted permissions of created object
false	false	false	true	`"fullPermission": false, "deletePermission": false, "writePermission": false, "readPermission": true`
false	false	false	false	`"fullPermission": false, "deletePermission": false, "writePermission": false, "readPermission": false`

- When a principal of the type Party is used in the AclEntry, only a PrincipalItemReference with the id 'Heiler Product Manager' is allowed.
  If other id values are used, this will result in a not found principal in the system.

11.4 Examples

11.4.1 Read ACL object for an id

11.4.1.1 REST call for JSON

Rest Call for JSON

```
curl -u rest:heiler -H "Accept: application/json" -X GET http://localhost:1501/rest/V1.0/security/acl/3
```

The following JSON object is returned:

Returned JSON object

```
{
```

```
  "id": 3,
  "entries": [
    {
      "principal": {
        "id": "'Heiler Product Manager'",
        "entityId": 2800
        "deleted": false
      },
      "fullPermission": false,
      "deletePermission": false,
      "writePermission": false,
      "readPermission": true
    },
    {
      "principal": {
        "id": "'userGroup1'",
        "entityId": 2700,
        "deleted": false
      },
      "fullPermission": false,
      "deletePermission": true,
      "writePermission": true,
      "readPermission": true
    },
    {
      "principal": {
        "id": "'user1'",
        "entityId": 2600,
        "deleted": false
      },
      "fullPermission": true,
      "deletePermission": true,
      "writePermission": true,
      "readPermission": true
    },
    {
      "principal": {
        "id": "'deletedUser'",
        "entityId": 2600,
        "deleted": true
      },
      "fullPermission": false,
      "deletePermission": true,
      "writePermission": true,
      "readPermission": true
    }
  ]
}
```

11.4.1.2 REST call for XML

Rest Call for XML

```
curl -u rest:heiler -H "Accept: application/xml" -X GET http://localhost:1501/rest/V1.0/security/acl/3
```

The following XML object is returned:

Returned XML object

```
<acl>
    <id>3</id>
    <entries>
        <entry>
            <principal>
                <id>'Heiler Product Manager'</id>
                <entityId>2800</entityId>
                <deleted>false</deleted>
            </principal>
            <fullPermission>false</fullPermission>
            <deletePermission>false</deletePermission>
            <writePermission>false</writePermission>
            <readPermission>true</readPermission>
        </entry>
        <entry>
            <principal>
                <id>'userGroup1'</id>
                <entityId>2700</entityId>
                <deleted>false</deleted>
            </principal>
            <fullPermission>false</fullPermission>
            <deletePermission>true</deletePermission>
            <writePermission>true</writePermission>
            <readPermission>true</readPermission>
        </entry>
        <entry>
            <principal>
                <id>'user1'</id>
                <entityId>2600</entityId>
                <deleted>false</deleted>
            </principal>
            <fullPermission>true</fullPermission>
            <deletePermission>true</deletePermission>
            <writePermission>true</writePermission>
            <readPermission>true</readPermission>
        </entry>
```

```
        <entry>
            <principal>
                <id>'deletedUser'</id>
                <entityId>2600</entityId>
                <deleted>true</deleted>
            </principal>
            <fullPermission>false</fullPermission>
            <deletePermission>true</deletePermission>
            <writePermission>true</writePermission>
            <readPermission>true</readPermission>
        </entry>
    </entries>
</acl>
```

11.4.1.3 REST Client Java

Rest Client Java Code

```
SecurityRequest securityRequest = getRestClient().createSecurityRequest();
Long actualAclId = 3L;
ACL actualAcl = securityRequest.getAcl( actualAclId );
```

11.4.2 Create ACL object

11.4.2.1 REST call for JSON

Rest Call for JSON

```
curl -u rest:heiler -H "Accept: application/json" -X POST http://localhost:1501/rest/V1.0/security/acl
```

The following JSON object is provided as content:

JSON object as content

```
{
  "entries": [
    {
      "principal": {
        "id": "'user1'",
        "entityId": 2600
      },
      "fullPermission": true,
```

```
      "deletePermission": false,
      "writePermission": true,
      "readPermission": false
    }
  ]
}
```

The returned protocol looks like:

Returned JSON object

```
{
  "id": 4,
  "entries": [
    {
      "principal": {
        "id": "'user1'",
        "entityId": 2600,
        "deleted": false
      },
      "fullPermission": true,
      "deletePermission": true,
      "writePermission": true,
      "readPermission": true
    }
  ]
}
```

11.4.2.2 REST call for XML

Rest Call for XML

```
curl -u rest:heiler -H "Accept: application/xml" -X POST http://localhost:1501/rest/V1.0/security/acl
```

The following JSON object is provided as content:

XML object as content

```
<acl>
    <entries>
        <entry>
            <principal>
                <id>'userGroup1'</id>
                <entityId>2700</entityId>
```

```
            </principal>
            <fullPermission>false</fullPermission>
            <deletePermission>true</deletePermission>
            <writePermission>false</writePermission>
            <readPermission>false</readPermission>
        </entry>
    </entries>
</acl>
```

The returned protocol looks like:

Returned XML object

```
<acl>
    <id>5</id>
    <entries>
        <entry>
            <principal>
                <id>'userGroup1'</id>
                <entityId>2700</entityId>
                <deleted>false</deleted>
            </principal>
            <fullPermission>false</fullPermission>
            <deletePermission>true</deletePermission>
            <writePermission>true</writePermission>
            <readPermission>true</readPermission>
        </entry>
    </entries>
</acl>
```

11.4.2.3 REST Client Java

Rest Client Java Code

```
SecurityRequest securityRequest = getRestClient().createSecurityRequest();
//prepares ACL parameter for addAcl method
ACL aclToPersist = new ACL();
List< AclEntry > aclEntriesToPersist = new ArrayList< AclEntry >();
aclToPersist.setEntries( aclEntriesToPersist );

AclEntry aclEntryUser = new AclEntry();
PrincipalItemReference user1 = new
 PrincipalItemReference( EntityItemReferenceFactory.createByIdentifier( "User1" ));
user1.setEntityId( ( short ) 2600 );
user1.setDeleted( false );
aclEntryUser.setPrincipal( user1 );
aclEntryUser.setReadPermission( true );
aclEntriesToPersist.add( aclEntryUser );
```

```
//calls the method
ACL createdAcl = securityRequest.addAcl( aclToPersist );

//gets result
Long actualAclId = createdAcl.getId();
List< AclEntry > createdAclEntries = createdAcl.getEntries()
```

12 PUBLIC Server Health Status

12.1 Overview

In a multi-server environment, an external load balancer needs to determine the health of each server of a cluster. A server's health is one important aspect for selecting best server for incoming requests. For basic server monitoring, the application login /pim/webaccess page can be used, however, this page doesn't take the server startup and shutdown period into account. Especially in the scenario of graceful shutdown, where the servers first finishes all running jobs before terminating, it is important that the health status is reflecting the unavailability of the server for new incoming requests correctly.

12.1.1 Health Status Page

The health status page indicates if clients can connect to the server. You do not need any authentication to access this status page. The health status can be accessed with the URI `/public/V1.0/health`.

12.1.1.1 Example

Request:

```
curl -i -X GET http://localhost:1512/public/V1.0/health
```

Response (when clients can connect):

```
HTTP/1.1 200 OK
Content-Type: text/plain

Server state: RUNNING
```

Response Code	Description
200	Clients can connect to the sever.
503	The server is starting or shutting down and no client can connect to this server.

Server state will be displayed in the response body.

12.1.2 Info REST API

For a more sophisticated health check, load balancers can also query the REST endpoint /rest/V1.0/manage/system/info. In this case, consumers of this API need to be able to provide authentication and parse the HTTP response.

12.1.2.1 Example

```
curl -X GET http://localhost:1512/rest/V1.0/manage/system/info -H 'Authorization:
Basic cmVzdDpoZWlsZXI='
```

```
HTTP/1.1 200 OK
{
    "state": "RUNNING",
    "runLevel": "STD_READY",
    "identifier": "pim-server1",
    "runtimeInfo": {
        "freeMemory": 747502440,
        "allocatedMemory": 1790967808,
        "maximumMemory": 1908932608,
        "availableProcessors": 8
    },
    "connectedNodes": [],
    "userInfo": {
        "loginName": "portal"
    }
}
```

Copyright

© Copyright Informatica LLC 1993, 2025

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.